Build an AI Agent (From Scratch)

Ghi chú học tập

LLM • Tool Use • MCP • ReAct • RAG • Memory • Planning • Reflection • Context Engineering

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 7 tháng 11 năm 2026

Mục lục

Tổng quan **2**

1 AI agent là gì? **4**
1.1 Thời đại của AI agent . . . 4
1.2 Hiểu về LLM agent . . . 5
1.3 Workflow và agent . . . 9
1.4 Những tác vụ cần đến agent . . . 11
1.5 Context engineering . . . 13
1.6 Kiến thức cần có để đọc sách . . . 16
1.7 Tóm tắt . . . 17

2 Bộ não của AI agent: LLM **18**
2.1 Chọn LLM cho agent . . . 18
2.2 Cơ bản về LLM API để xây dựng agent . . . 21
2.3 Nâng cao trí thông minh của agent: prompt engineering . . . 26
2.4 Trải nghiệm giới hạn của LLM với GAIA . . . 29
2.5 Tóm tắt . . . 36

3 Cho agent hành động: Tool use **38**
3.1 Tool cho LLM . . . 39
3.2 LLM dùng tool như thế nào . . . 40
3.3 Xây dựng tool và tool definition cho LLM . . . 49
3.4 MCP: Chuẩn hoá tool . . . 58
3.5 Tóm tắt . . . 66

4 Xây dựng ReAct agent cơ bản **67**
4.1 ReAct agent hoạt động như thế nào . . . 68
4.2 Tổng quan kiến trúc agent . . . 69
4.3 ExecutionContext: nơi lưu trữ trung tâm của agent . . . 72
4.4 Tool abstraction . . . 74
4.5 Tầng giao tiếp với LLM . . . 79
4.6 Cài đặt agent . . . 83
4.7 Thêm structured output . . . 88
4.8 Kiểm tra với benchmark GAIA . . . 92
4.9 Tóm tắt . . . 95

5 Xây dựng knowledge base với RAG **96**
5.1 Vấn đề khi dùng dữ liệu nội bộ . . . 97
5.2 Các loại phương pháp tìm kiếm . . . 98
5.3 Thực hành vector search . . . 103
5.4 Mở rộng agent bằng callback . . . 118
5.5 Tóm tắt . . . 125

6 Thêm memory cho agent **126**
6.1 Giải phẫu memory của agent . . . 126
6.2 Quản lý context trong lúc thực thi . . . 130
6.3 Thực thi liên tục: Session và quản lý trạng thái . . . 141
6.4 Long-term memory: tích luỹ tri thức xuyên session . . . 158
6.5 Tóm tắt . . . 168

7 Planning và reflection cho tác vụ phức tạp **169**
7.1 Cho agent thời gian để suy nghĩ . . . 170
7.2 Planning: Định hướng . . . 171
7.3 Reflection: Kiểm tra và điều chỉnh . . . 175
7.4 Kết hợp planning và reflection . . . 178
7.5 Tóm tắt . . . 180

Tổng quan

Tài liệu này là ghi chú học tập tiếng Việt cho cuốn *Build an AI Agent (From Scratch)* của Jungjun Hur và Younghee Song (Manning, bản MEAP). Bản ghi chú bám theo mục lục sách, giữ đủ các ý, ví dụ, con số và code listing. Thuật ngữ chuyên ngành giữ tiếng Anh. Hình được dùng lại từ sách gốc; chú thích mỗi hình ghi số trang trong file PDF gốc để tiện tra lại.

Lời chào của tác giả

Để đọc sách hiệu quả, bạn không cần là chuyên gia AI hay machine learning. Chỉ cần nắm cơ bản Python (viết hàm và class đơn giản), biết gọi và dùng API, và quen một chút với Git và GitHub.

Khi mới tìm hiểu AI agent, tác giả thấy choáng ngợp: tuần nào cũng có agent framework, bài báo nghiên cứu hay tool mới. Dù là kỹ sư, tác giả vẫn khó theo kịp. Tác giả nhận ra rằng nếu chỉ dừng ở vai trò người dùng framework thì sẽ không bao giờ thật sự hiểu chúng hoạt động thế nào, hay nên tin framework nào. Từ đó tác giả quyết định đào sâu, và con đường đó dẫn tới cuốn sách này.

Trọng tâm của sách là một dự án thực hành: cùng nhau xây một agent framework đơn giản nhưng mang tính giáo dục tên là `scratch_agents`, rồi dùng nó để tạo các agent giải bài toán từng bước. Trong quá trình xây framework, bạn sẽ thấy bản chất của agent nằm ở việc đưa *đúng ngữ cảnh* cho LLM để trí thông minh của nó được dùng hết. Sách gọi quá trình này là **Context Engineering**. Agent framework về bản chất được thiết kế để xử lý tốt thách thức này, và qua sách bạn sẽ học cách thiết kế, hiện thực và thử nghiệm ý tưởng đó trong thực tế.

Sách không nhằm đưa thêm một tool để học thuộc. Mục tiêu là giúp bạn hiểu cơ chế bên trong agent framework để vượt qua mức dùng bề mặt. Cuối hành trình, bạn không chỉ dùng được framework có sẵn mà còn biết mở chúng ra, hiểu các đánh đổi (trade-off), và tự xây framework riêng khi cần.

Tác giả mong sách giúp bạn lọc nhiễu trong lĩnh vực thay đổi nhanh này, và có sự tự tin, rõ ràng khi làm việc với AI agent. Góp ý và câu hỏi có thể gửi lên diễn đàn liveBook.

— Younghee Song & Jungjun Hur

Mục lục sách gốc

Bản MEAP hiện có 7 chương đầu (phần 1 và phần 2, trừ chương 8 và 9). Các chương còn lại chưa phát hành nên không có trong ghi chú này.

Phần 1: Xây LLM agent đầu tiên

1. What is an AI agent? (Chương 1)
2. The brain of AI agents: LLMs (Chương 2)
3. Enabling actions: Tool use (Chương 3)
4. Implementing a basic ReAct agent (Chương 4)

Phần 2: Phát triển năng lực nâng cao cho agent

5. Building knowledge bases with RAG (Chương 5)
6. Adding memory to your agent (Chương 6)
7. Planning and reflection for complex tasks (Chương 7)

8. Creating code agents that write their own tools *(chưa phát hành)*
9. Coordinating multi-agent systems *(chưa phát hành)*

Phần 3: Triển khai agent lên production

10. Evaluating performance and monitoring agent behavior *(chưa phát hành)*

Chương 1

AI agent là gì?

Chương này bao gồm

- Bức tranh toàn cảnh về AI agent hiện nay.
- LLM đóng vai trò lõi ra quyết định của agent.
- Workflow và agent khác nhau thế nào, khi nào dùng loại nào.
- Benchmark GAIA để đo hiệu năng agent.
- Context engineering để xây agent hiệu quả.

Có thể bạn đã nghe tới các framework xây agent như LangGraph, CrewAI, AutoGen hay OpenAI Agents. Các framework này giúp dựng agent nhanh, nhưng đồng thời che đi những gì thật sự diễn ra bên trong. Sách chọn cách khác: tự xây agent từ đầu, hiểu từng thành phần trước khi dựa vào bất kỳ framework nào.

Vì sao phải xây từ đầu? Vì phát triển agent về cơ bản là gỡ lỗi những lần agent thất bại. Khi agent trả lời sai hay bị kẹt trong vòng lặp, bạn cần biết chính xác điều gì đã hỏng. LLM (Large Language Model) hiểu sai context? Tool trả về kết quả không như mong đợi? Thiếu một thông tin quan trọng? Nếu không hiểu agent vận hành bên trong ra sao thì rất khó chẩn đoán những vấn đề này, dù bạn dùng công cụ nào. Khi tự xây từng thành phần, bạn sẽ hình thành mô hình tư duy (mental model) cần thiết để xử lý sự cố cho mọi hệ thống agent, dù do bạn tự viết hay tiếp quản từ người khác.

Chương này đặt nền móng cho mọi phần phía sau. Ta bắt đầu bằng việc khảo sát các loại AI agent, từ trợ lý cá nhân tới công cụ lập trình chuyên biệt, để hiểu mình đang hướng tới cái gì. Sau đó ta xem LLM đóng vai "bộ não" của agent như thế nào và điều gì phân biệt một agent thực sự với một workflow đơn giản. Tiếp theo, chương giới thiệu GAIA, benchmark được dùng xuyên suốt sách để đo tiến độ, và tìm hiểu context engineering, lĩnh vực quyết định agent thành công hay thất bại. Cuối chương, bạn sẽ có mô hình tư duy rõ ràng về agent là gì, khi nào nên dùng, và các nguyên tắc dẫn dắt quá trình hiện thực.

1.1 Thời đại của AI agent

AI agent đang thay đổi nhanh chóng cách chúng ta tương tác với công nghệ. Từ trợ lý cá nhân giúp việc hằng ngày tới các hệ thống phức tạp xử lý công việc chuyên môn, agent đang trở thành một phần không thể thiếu của cả sản phẩm tiêu dùng lẫn giải pháp doanh nghiệp. Trước khi đi vào cách xây dựng, hãy điểm qua các loại AI agent để hiểu mình đang hướng tới điều gì.

Personal AI Agent (agent cá nhân) là dạng AI agent quen thuộc nhất hiện nay. Các dịch vụ như ChatGPT, Claude và Gemini khởi đầu là chatbot hội thoại: bạn hỏi, chúng trả lời. Nhưng các hệ thống này đã tiến hoá rất nhiều. Agent cá nhân hiện đại có thể phân tích tài liệu được tải lên, tìm kiếm web để lấy thông tin mới, tạo hình ảnh, viết và chạy code, thậm chí hỗ trợ quyết định mua sắm. Chúng là trợ lý

đa năng, thích ứng với bất cứ việc gì bạn giao, và dần học được sở thích cũng như phong cách giao tiếp của bạn theo thời gian.

Customer-Facing Agent (agent phục vụ khách hàng) hoạt động thay mặt doanh nghiệp, tương tác trực tiếp với khách hàng theo thời gian thực. Những agent này vượt xa chatbot FAQ đơn giản. Chúng có thể truy cập chính sách công ty, lấy dữ liệu khách hàng, xử lý giao dịch và ra quyết định dựa trên quy tắc nghiệp vụ. Khi bạn làm việc với một nhân viên hỗ trợ kiểm tra trạng thái đơn hàng, xử lý hoàn tiền hay giúp khắc phục sự cố sản phẩm, ngày càng có nhiều khả năng bạn đang giao tiếp với một AI agent. Các hệ thống này phải cân bằng giữa việc hữu ích và việc tuân thủ chặt chẽ quy định của công ty cũng như yêu cầu pháp lý.

Specialized Agent (agent chuyên biệt) giải quyết các tác vụ đặc thù theo lĩnh vực, đòi hỏi chuyên môn sâu hoặc thời gian xử lý dài. Các coding agent như Claude Code, Cursor và Codex CLI có thể duyệt codebase, hiện thực tính năng, sửa bug và refactor code, những việc trước đây tốn hàng giờ của lập trình viên. Các agent Deep Research có thể điều tra toàn diện trên hàng trăm nguồn rồi tổng hợp kết quả thành báo cáo chi tiết. Vì các tác vụ này thường mất nhiều thời gian, agent chuyên biệt hay chạy bất đồng bộ (asynchronous) và báo cho người dùng khi có kết quả. Khi được kết nối với dữ liệu nội bộ của doanh nghiệp, các agent này trở thành giải pháp chuyên ngành (vertical solution) may đo cho từng ngành hoặc từng use case cụ thể.

Dù ứng dụng và giao diện khác nhau, mọi agent trên đều có chung một nền tảng: chúng được vận hành bởi Large Language Model. Hiểu LLM hoạt động thế nào và chúng mang lại năng lực gì cho agent là điều thiết yếu với bất kỳ ai muốn xây agent hiệu quả. Trong sách, "AI agent" và "LLM agent" được dùng thay thế cho nhau, vì gần như mọi AI agent hiện đại đều chạy bằng LLM.

1.2 Hiểu về LLM agent

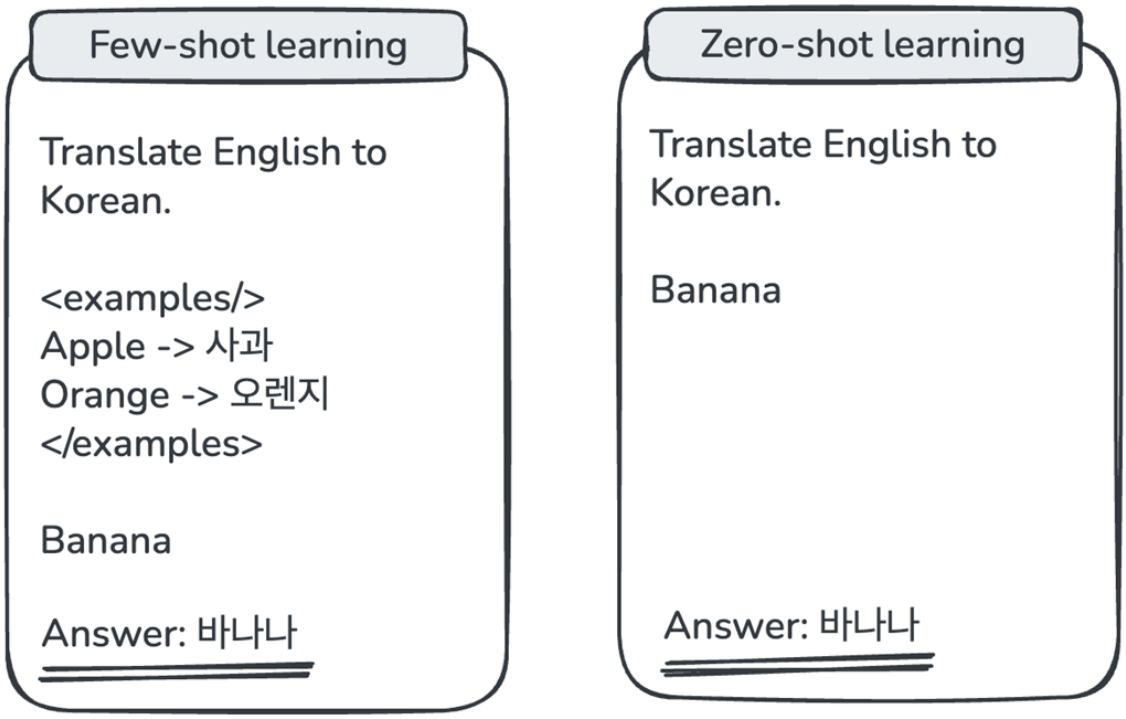
Mọi agent vừa điểm qua đều có một điểm chung đáng chú ý: chúng dựa vào LLM làm lõi ra quyết định. Nhưng làm sao một hệ thống được thiết kế để đoán từ tiếp theo lại có thể tự hoàn thành các tác vụ nhiều bước? Trước hết ta xem điều gì khiến LLM đặc biệt phù hợp để vận hành agent, rồi xét ba thành phần thiết yếu biến một LLM thành agent. Phần này giúp bạn hiểu vòng lặp agent (agent loop), nền tảng cho mọi thứ ta sẽ xây trong sách.

1.2.1 LLM là gì?

LLM (Large Language Model) là mô hình ngôn ngữ được huấn luyện trên gần như toàn bộ văn bản công khai có trên internet. Dù LLM hiện đại đã tiến hoá thành mô hình đa phương thức (multimodal) xử lý được ảnh, âm thanh và video, sách vẫn dùng thuật ngữ "LLM" cho mọi biến thể này, vì chúng có chung kiến trúc nền tảng.

Nguyên lý cốt lõi của LLM rất đơn giản: học cách đoán từ tiếp theo bằng cách xử lý một lượng văn bản khổng lồ. Với câu "Vì đang nghỉ hè nên tôi không đi _____", hầu hết mọi người sẽ điền "học". Nhờ huấn luyện ở quy mô lớn dựa trên nguyên lý đơn giản này, LLM hình thành hiểu biết về cấu trúc, ngữ cảnh và ý nghĩa của ngôn ngữ. LLM xử lý văn bản bằng cách chia nó thành các token, thường là từ hoặc mảnh của từ. Mô hình đoán token tiếp theo dựa trên tất cả các token đứng trước.

Các cách tiếp cận trước đây như hệ thống dựa trên luật (rule-based) hay học tăng cường (reinforcement learning) cũng từng cố tạo ra agent thông minh. Điều làm LLM khác biệt là **khả năng tổng quát hoá** (generalization): xử lý được nhiều loại tác vụ mà không cần huấn luyện lại cho từng tác vụ. Hình 1.1 minh hoạ điều này. Trước tiên, LLM dịch từ "banana" nhờ một chỉ dẫn và hai ví dụ (apple, orange); đó là few-shot learning. Còn với zero-shot learning, LLM làm đúng chỉ với chỉ dẫn, không cần ví dụ nào.



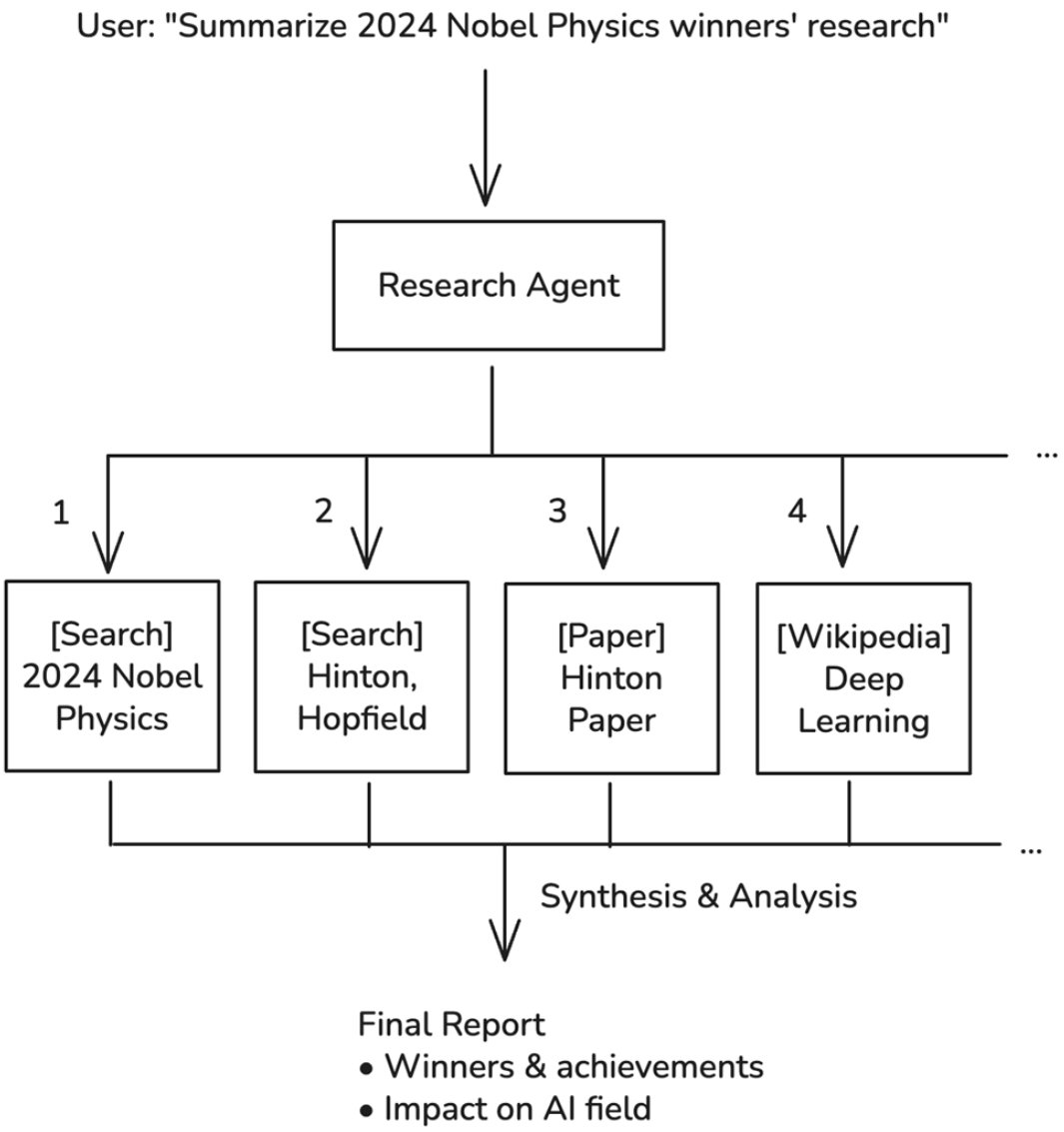
Hình 1.1: Ví dụ về khả năng tổng quát hoá của mô hình ngôn ngữ. *(PDF gốc tr. 9)*

Gần đây xuất hiện các "reasoning model" (mô hình suy luận), phân tích tác vụ hoặc lập kế hoạch trước khi đưa ra kết quả. Reasoning model không trả lời ngay mà trước hết sinh các thinking token bắt đầu bằng `<think>`. Những tiến bộ này giúp LLM agent xử lý hiệu quả các bài toán phức tạp và chưa từng gặp.

1.2.2 LLM agent là gì?

Một ví dụ tiêu biểu của LLM agent là research agent (agent nghiên cứu). Khi người dùng hỏi "Tóm tắt công trình nghiên cứu của những người đoạt giải Nobel Vật lý 2024", agent thu thập thông tin từ nhiều nguồn, phân tích kết quả và tạo ra một báo cáo đầy đủ.

Như Hình 1.2 cho thấy, research agent không chỉ đơn giản sinh câu trả lời từ dữ liệu huấn luyện. Nó tìm thông tin về giải Nobel 2024, xác định những người đoạt giải (Geoffrey Hinton và John Hopfield), tìm hiểu các bài báo khoa học và tra Wikipedia. Sau đó nó tổng hợp tất cả thành một báo cáo mạch lạc, trình bày thành tựu của những người đoạt giải và ảnh hưởng của họ tới AI.



Hình 1.2: Yêu cầu của người dùng đi qua research agent, agent rẽ nhánh thành nhiều lượt tìm kiếm rồi tổng hợp. *(PDF gốc tr. 11)*

Định nghĩa cốt lõi: LLM + Tool + Loop

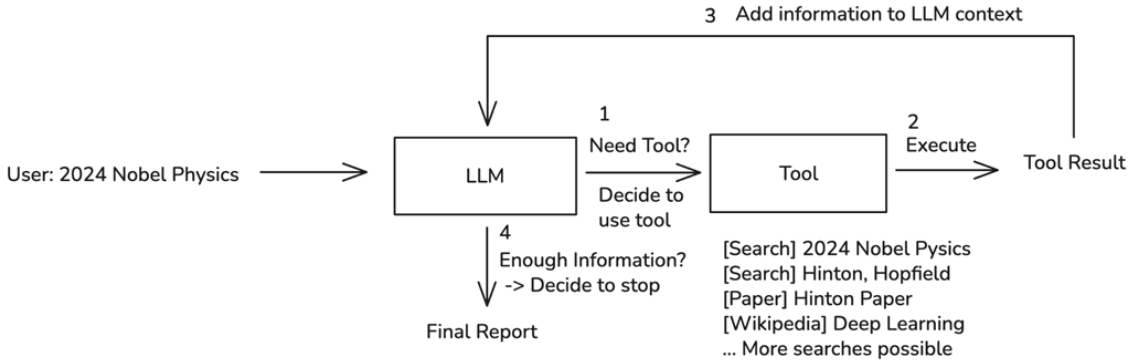
Ví dụ trên cho thấy bản chất của agent. **LLM agent là một chương trình tự quyết định cần thực hiện hành động gì và khi nào dừng, dựa trên context hiện tại và mục tiêu.** Về cốt lõi, nó gồm ba thành phần:

- **LLM** là bộ não của agent. Nó hiểu tình huống hiện tại và quyết định bước tiếp theo.
- **Tool** là phương tiện tương tác với thế giới bên ngoài: tìm kiếm web, chạy code, truy cập cơ sở dữ liệu.
- **Loop** (vòng lặp) là cấu trúc lặp lại quá trình này cho tới khi đạt mục tiêu.

LLM là "bộ não" nghĩa là nó không chỉ sinh văn bản mà còn quyết định dùng tool nào và khi nào dừng. Đây chính là điều phân biệt LLM agent với một LLM thuần tuý hay phần mềm truyền thống.

Agent loop hoạt động thế nào

Cần có Loop vì khó biết trước tác vụ sẽ cần những tool nào hay phải qua bao nhiêu bước. Hình 1.3 thể hiện tác vụ nghiên cứu về những người đoạt giải Nobel 2024 trong Hình 1.2, nhưng nhìn dưới góc độ Agent Loop.



Hình 1.3: Vòng lặp ra quyết định của LLM agent là quá trình lặp giữa việc LLM ra quyết định và sử dụng tool. *(PDF gốc tr. 12)*

Quá trình diễn ra theo một vòng lặp liên tục:

1. **LLM đánh giá context hiện tại** và xác định có cần tool hay không. Nếu cần thêm thông tin, nó quyết định dùng tool nào dựa trên những gì còn thiếu.
2. **Tool được chọn sẽ được thực thi**, chẳng hạn tìm "2024 Nobel Physics", tra "Hinton, Hopfield", tìm bài báo khoa học, hay truy cập Wikipedia để lấy kiến thức nền.
3. **Kết quả của tool được thêm trở lại vào context của LLM**, làm giàu hiểu biết của nó về chủ đề. Context tích luỹ này cho phép LLM ra quyết định sáng suốt hơn ở các vòng lặp sau.
4. **LLM quyết định tiếp tục hay dừng.** Nếu thấy đã thu thập đủ thông tin để trả lời đầy đủ câu hỏi của người dùng, nó sinh báo cáo cuối cùng. Nếu chưa, nó quay lại bước 1 cho một vòng lặp mới.

Quá trình lặp gồm suy luận, hành động và quan sát này giúp agent xử lý các tác vụ có độ phức tạp không đoán trước được, từ câu hỏi đơn giản chỉ cần một lần tìm kiếm tới nghiên cứu phức tạp cần hàng chục nguồn thông tin.

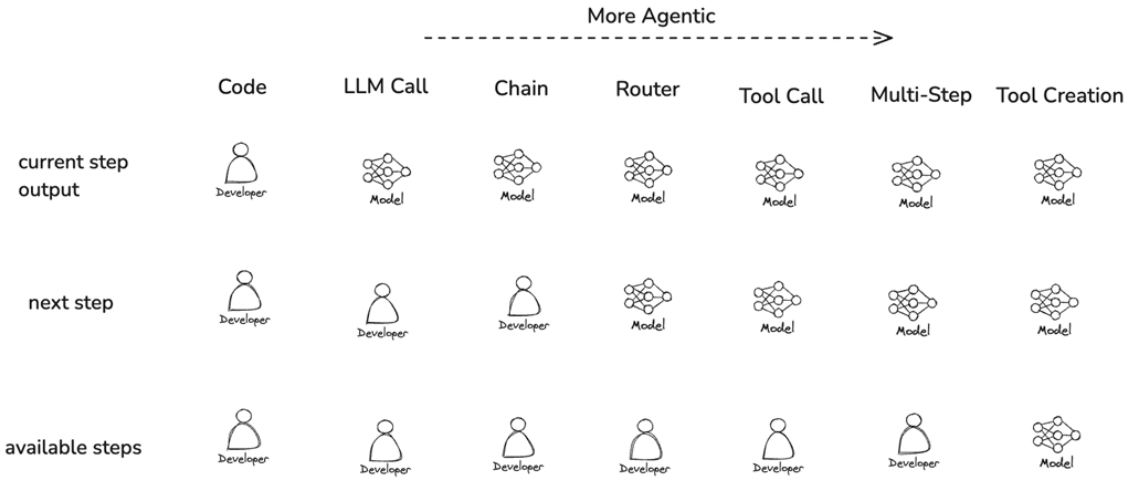
LLM agent khác LLM và phần mềm truyền thống thế nào

Phần mềm truyền thống đòi hỏi lập trình viên định nghĩa trước mọi hành động. Đường thực thi là cố định, và các tương tác với bên ngoài được hiện thực trực tiếp trong code. Một LLM có thể sinh văn bản, nhưng tự nó không tương tác được với thế giới bên ngoài hay thực hiện tác vụ nhiều bước.

LLM agent kết hợp và vượt lên cả hai. LLM quyết định hành động một cách linh hoạt dựa trên context, tương tác với thế giới bên ngoài qua tool, và lặp lại quá trình này cho tới khi đạt mục tiêu. Một tác vụ nghiên cứu kết thúc sau một lần tìm web hay cần nhiều bài báo khoa học và bài Wikipedia là điều không biết trước. Agent tự xác định con đường của mình dựa trên độ phức tạp của câu hỏi và thông tin phát hiện được dọc đường.

1.3 Workflow và agent

Khi tích hợp LLM vào ứng dụng, lập trình viên đối mặt với một lựa chọn kiến trúc căn bản: LLM nên được kiểm soát luồng thực thi tới mức nào? Quyết định này ảnh hưởng tới mọi thứ, từ độ tin cậy của hệ thống tới chi phí vận hành. Hình 1.4 minh hoạ bảy mức độ tự chủ (agency) khác nhau, tăng dần từ trái sang phải.



Hình 1.4: Các mức độ tự chủ (agency) tăng dần trong ứng dụng LLM. *(PDF gốc tr. 14)*

Ba hàng trong hình thể hiện ba khía cạnh của quyền kiểm soát: ai tạo ra output hiện tại (current step), ai quyết định điều gì xảy ra tiếp theo (next step), và ai định nghĩa các lựa chọn có sẵn (available steps). Càng sang phải, LLM càng nắm nhiều quyền kiểm soát trên từng khía cạnh này.

Bảy mức này có thể chia thành hai cách tiếp cận kiến trúc lớn: **workflow** và **agent**. Với workflow, lập trình viên định nghĩa trước luồng thực thi và dùng LLM để thực hiện các bước cụ thể trong cấu trúc đó. Với agent, LLM tự xác định quy trình của mình, quyết định thực hiện hành động nào và khi nào dừng. Hiểu sự khác biệt này là điều cần thiết để chọn đúng cách tiếp cận cho use case của bạn.

1.3.1 Workflow: luồng do lập trình viên định nghĩa

Workflow là hệ thống trong đó lập trình viên thiết kế tường minh trình tự các thao tác, còn LLM thực hiện các bước cụ thể bên trong cấu trúc định sẵn đó. Đặc điểm then chốt là tính dự đoán được (predictability): cùng một input, hệ thống luôn đi cùng một đường qua workflow.

LLM call

Mẫu cơ bản nhất là một lần gọi LLM duy nhất: một prompt đi vào, một response đi ra. Cách này giao một tác vụ cụ thể cho mô hình, mọi thứ khác vẫn do lập trình viên kiểm soát. Ví dụ: phân loại văn bản, tóm tắt, hoặc trả lời một câu hỏi đơn giản. Dù đơn giản, một lần gọi được thiết kế tốt có thể xử lý những tác vụ phức tạp đến bất ngờ.

Chain

Chain nối nhiều lần gọi LLM theo một trình tự định sẵn, output của bước này thành input của bước sau. Lập trình viên thiết kế luồng này từ trước, LLM thực hiện tuần tự từng bước.

Mẫu này dựa trên một nguyên lý cơ bản: LLM làm tốt nhất khi tác vụ tập trung và được định nghĩa rõ. Thay vì yêu cầu mô hình "phân tích tài liệu này và tạo bài thuyết trình", một chain có thể tách việc này thành các bước riêng: trích ý chính, sắp xếp theo chủ đề, sinh nội dung slide, rồi chỉnh sửa cho rõ ràng. Mỗi bước đơn giản hơn và dễ thành công hơn so với làm tất cả cùng lúc.

Router

Router đưa vào logic điều kiện, trong đó LLM quyết định đi theo nhánh định sẵn nào tiếp theo. Với một câu hỏi của người dùng, mô hình có thể phân loại và chuyển nó tới bộ xử lý phù hợp: câu hỏi về thanh toán đi vào một workflow, hỗ trợ kỹ thuật đi vào workflow khác.

Dù LLM có ra quyết định ở đây, nó chỉ chọn trong một tập lựa chọn cố định mà lập trình viên đã định nghĩa tường minh. Các nhánh có sẵn, và những gì xảy ra trên mỗi nhánh, vẫn do lập trình viên kiểm soát. Vì vậy router là một mẫu workflow chứ không phải mẫu agent. LLM tác động tới việc chọn đường nhưng không điều khiển cả hành trình.

1.3.2 Agent: luồng do LLM điều khiển

Agent là hệ thống trong đó LLM tự điều khiển quy trình và cách dùng tool của mình một cách linh hoạt, nắm quyền kiểm soát cách hoàn thành tác vụ. Thay vì đi theo con đường định trước, agent quyết định làm gì tiếp theo dựa trên context hiện tại và tiến độ hướng tới mục tiêu.

Tool use với vòng lặp nhiều bước

Mẫu định nghĩa nên agent kết hợp hai năng lực: khả năng dùng tool bên ngoài và quyền tự chủ tiếp tục hoạt động cho tới khi xong tác vụ.

Vì LLM chỉ sinh được văn bản, chúng không thể trực tiếp tương tác với hệ thống bên ngoài, truy cập thông tin thời gian thực hay tính toán chính xác. Tool lấp khoảng trống này bằng cách đưa các chức năng bên ngoài (tìm kiếm web, chạy code, truy vấn cơ sở dữ liệu, gọi API) ra dưới dạng hàm có thể gọi. LLM xem các tool có sẵn, quyết định tool nào có ích, và nếu có thì chỉ rõ gọi tool nào với tham số gì.

Điều biến tool use thành agent là vòng lặp. Thay vì gọi tool một lần rồi dừng, agent hoạt động theo chu kỳ: đánh giá trạng thái hiện tại, quyết định hành động, quan sát kết quả, và lặp lại cho tới khi xong tác vụ. Chính LLM quyết định khi nào tiếp tục và khi nào dừng.

Vòng lặp nhiều bước này là cơ chế cốt lõi giúp agent xử lý các tác vụ có độ phức tạp không đoán trước được. Một câu hỏi đơn giản có thể xong sau một bước; một tác vụ nghiên cứu phức tạp có thể cần hàng chục vòng lặp qua nhiều tool.

Tool creation

Ở mức tự chủ cao nhất, agent không chỉ chọn trong các tool có sẵn mà còn tạo tool mới. Việc này thường gồm sinh code để hiện thực một chức năng chưa có trong bộ tool định sẵn.

Ví dụ, nếu agent cần xử lý dữ liệu theo một định dạng đặc thù mà chưa tool nào xử lý được, nó có thể viết một hàm parse riêng, chạy hàm đó, rồi dùng kết quả để tiếp tục tác vụ. Năng lực này cho phép agent tự mở rộng khả năng của mình một cách linh hoạt, thích ứng với các yêu cầu không được lường trước khi thiết kế hệ thống.

1.3.3 Kết hợp workflow và agent trong thực tế

Trong hệ thống production, ranh giới giữa workflow và agent không phải lúc nào cũng rạch ròi. Trên thực tế, các kiến trúc hiệu quả nhất thường kết hợp cả hai: dùng workflow để có cấu trúc và tính dự đoán được, đồng thời nhúng agent vào những điểm cụ thể cần sự linh hoạt.

Agent làm node trong workflow

Một mẫu phổ biến là thiết kế workflow tổng thể với các giai đoạn rõ ràng, nhưng hiện thực một hoặc vài giai đoạn trong đó bằng agent. Xét một pipeline xử lý tài liệu:

1. **Tiếp nhận tài liệu** (workflow): kiểm tra định dạng, trích metadata.

2. **Phân tích nội dung** (agent): tìm hiểu bối cảnh, thu thập thông tin liên quan, tổng hợp kết quả.
3. **Kiểm tra chất lượng** (workflow): đối chiếu với các quy tắc tuân thủ (compliance), định dạng output.
4. **Bàn giao** (workflow): chuyển tới đích phù hợp.

Agent hoạt động tự do trong giai đoạn được giao: gọi tool nhiều lần, lần theo các chuỗi truy vấn, thích ứng với những gì nó phát hiện. Nhưng quy trình tổng thể vẫn giữ tính dự đoán được của workflow. Nếu agent thất bại hoặc cho kết quả bất ngờ, workflow có thể bắt được ở giai đoạn kiểm tra chất lượng.

Vì sao nên kết hợp?

Cách tiếp cận lai này có một số ưu điểm. Thứ nhất, nó khoanh vùng độ phức tạp. Hành vi của agent vốn khó dự đoán hơn, nên giới hạn nơi agent hoạt động giúp toàn hệ thống dễ suy xét và gỡ lỗi hơn. Thứ hai, nó tối ưu chi phí. Vòng lặp agent có thể tốn kém do gọi LLM nhiều lần, nên chỉ dùng chúng ở nơi thật sự cần sự linh hoạt sẽ giữ chi phí vận hành trong tầm kiểm soát. Thứ ba, nó cho phép suy giảm có kiểm soát (graceful degradation). Nếu một thành phần agent thất bại, cấu trúc workflow cho phép có hành vi dự phòng (fallback) hoặc chuyển cho con người xử lý (human escalation) tại các điểm định sẵn.

Lựa chọn kiến trúc không phải là "workflow hay agent" mà là "ở chỗ nào trong hệ thống này, hành vi agent mang lại đủ giá trị để bù cho chi phí của nó?". Cách đặt vấn đề thực dụng này sẽ hữu ích khi bạn thiết kế ứng dụng chạy bằng LLM của riêng mình.

1.4 Những tác vụ cần đến agent

Trước khi lao vào phát triển agent, ta cần đặt đúng câu hỏi theo đúng thứ tự. Hai quyết định then chốt xác định agent có phải giải pháp phù hợp hay không:

1. Tác vụ này có cần LLM không?
2. Nếu cần LLM, tác vụ có cần agent không, hay một workflow là đủ?

Trả lời "không" ở bất kỳ bước nào đồng nghĩa với việc chọn cách tiếp cận đơn giản và tiết kiệm hơn. Hãy xem tiêu chí cho từng quyết định, rồi xem một benchmark gom các tác vụ mà agent thật sự vượt trội.

1.4.1 Tác vụ cần LLM

Trước khi cân nhắc dùng LLM agent, bạn cần xác định bản thân tác vụ có cần LLM hay không. Câu hỏi nền tảng này quan trọng vì LLM mang theo chi phí phụ đáng kể về tính toán và nguy cơ sai sót. Chi phí phụ đó có thể là thừa nếu tác vụ giải được bằng cách đơn giản và tất định (deterministic) hơn. Chẳng hạn, nếu tác vụ chỉ là xử lý dữ liệu cơ bản với input và output dự đoán được, logic lập trình truyền thống sẽ nhanh hơn, rẻ hơn và đáng tin cậy hơn bất kỳ giải pháp dựa trên LLM nào.

Có hai tiêu chí chính để ra quyết định này:

Tác vụ liên quan tới dữ liệu phi cấu trúc. Nếu tác vụ cần phân tích dữ liệu phi cấu trúc như văn bản, hình ảnh hay âm thanh, đó là ứng viên sáng giá để dùng LLM. Lập trình truyền thống rất giỏi với dữ liệu có cấu trúc và schema rõ ràng, nhưng gặp khó với sự mơ hồ vốn có của ngôn ngữ tự nhiên hay nội dung hình ảnh. Các mô hình xử lý nhiều định dạng dữ liệu thường được gọi là Multimodal LLM (hay LMM), nhưng về cơ bản chúng dựa trên LLM, nên sách gọi chung tất cả là LLM.

Độ đa dạng của input. Nếu input của người dùng bị giới hạn hoặc các tác vụ được yêu cầu có phạm vi hẹp, dùng một mô hình ML chuyên biệt hoặc hệ thống dựa trên luật có thể tiết kiệm hơn. Tương tự, nếu số lượng tác vụ tiềm năng ít và được định nghĩa trước, để lập trình viên hard-code logic sẽ hiệu quả hơn dùng LLM cả về chi phí lẫn độ trễ. LLM toả sáng khi input khó đoán và đa dạng, đòi hỏi diễn giải linh hoạt mà không thể lường trước.

1.4.2 Điều kiện để dùng agent

Khi đã xác định cần LLM, câu hỏi tiếp theo là dùng nó thế nào. Một lần gọi LLM có đủ không, hay tác vụ cần suy luận lặp và tool use, những thứ định nghĩa nên agent?

Xét câu hỏi sau:

> *"If Eliud Kipchoge could maintain his marathon world record pace indefinitely, how long would it take him to run from Earth to the Moon?"*
> (Nếu Eliud Kipchoge có thể giữ mãi tốc độ kỷ lục thế giới marathon của mình, anh ấy sẽ mất bao lâu để chạy từ Trái Đất tới Mặt Trăng?)

Trả lời câu hỏi này cần nhiều bước. Trước hết, phải tìm xem Kipchoge là ai và kỷ lục thế giới marathon của anh ấy là bao nhiêu. Sau đó, tra Wikipedia để lấy khoảng cách từ Trái Đất tới Mặt Trăng. Cuối cùng, tính toán bằng cách chia quãng đường cho tốc độ để ra đáp án.

Ai cũng có thể làm việc này bằng tay. Tuy nhiên, việc qua lại giữa nhiều website để thu thập thông tin, đổi đơn vị và tính toán vừa tốn thời gian vừa dễ sai. Loại tác vụ nghiên cứu nhiều bước như thế này chính là nơi agent phát huy thế mạnh.

Khi sự hào hứng với agent tăng lên, có một cái bẫy phổ biến: áp dụng agent cho mọi tác vụ chỉ vì chúng mới và mạnh. Nhưng cũng như bạn không cần một web framework phức tạp để làm một website tĩnh đơn giản, chọn đúng công cụ là chìa khoá để xây hệ thống hiệu quả.

Agent đi kèm những đánh đổi cố hữu. Thứ nhất, chi phí cao hơn: nhiều lần gọi LLM nhân chi phí API lên so với một request đơn lẻ. Thứ hai, độ trễ tăng: thời gian phản hồi cộng dồn qua mỗi bước suy luận. Thứ ba, lỗi lan truyền: sai sót ở các bước đầu kéo theo dây chuyền qua toàn bộ quá trình.

Xét một câu hỏi chăm sóc khách hàng có thể xử lý bằng một lần gọi LLM tốn $0.01. Xử lý cùng câu hỏi đó bằng một agent cần 10 lần gọi sẽ tốn $0.10, chênh lệch 10 lần. Khi bạn xử lý hàng nghìn request mỗi ngày, đây không chỉ là quyết định kỹ thuật mà còn là quyết định kinh doanh.

Vậy khi nào nên dùng agent? Bạn có thể quyết định dựa trên ba tiêu chí sau:

Độ phức tạp của tác vụ. Nếu khó đoán trước tác vụ cần bao nhiêu bước, agent có lợi thế. "Tìm dân số của vùng A" là việc đơn giản. Ngược lại, "Phân tích LLM agent sẽ định hình tương lai ra sao" đòi hỏi thu thập nhiều loại thông tin và tổng hợp các mối quan hệ phức tạp. Đó là tác vụ khó định nghĩa trước bằng logic tĩnh.

Giá trị của tác vụ. Vì agent cần nhiều lần gọi và ra quyết định linh hoạt, chúng tốn kém hơn và phản hồi chậm hơn. Do đó, giá trị của việc hoàn thành tác vụ phải lớn hơn chi phí và độ trễ phát sinh thêm.

Cái giá của lỗi và khả năng phát hiện lỗi. LLM dễ ra quyết định sai, và rủi ro tăng theo số lần gọi. Nếu lỗi dẫn tới hậu quả nghiêm trọng, agent có thể không phải lựa chọn tốt nhất. Khả năng phát hiện lỗi cũng quan trọng: trong các lĩnh vực đòi hỏi kiến thức chuyên sâu, người dùng hay lập trình viên có thể thậm chí không nhận ra agent đã cho kết quả sai.

1.4.3 GAIA: phòng tập cho agent

Câu hỏi về Kipchoge ở trên thực ra lấy từ một benchmark tên là GAIA. Năm 2023, Meta và HuggingFace phát hành GAIA (General AI Assistants), một bộ dữ liệu thu thập có hệ thống các tác vụ cần đến agent.

GAIA gồm các cặp câu hỏi–đáp án. Mỗi câu hỏi đòi hỏi suy luận nhiều bước, tìm kiếm web, tính toán, v.v. Chúng khó giải bằng một lần gọi LLM, và cũng không dễ định nghĩa trước một workflow rõ ràng. Đây là những bài toán tự nhiên cần đến cách tiếp cận agentic.

GAIA ra đời năm 2023, và từ đó tới nay năng lực mô hình đã cải thiện đáng kể, đồng thời xuất hiện các benchmark khó hơn. Tuy vậy, các bài trong GAIA trải từ dễ tới thật sự khó ngay cả với mô hình hiện tại, nên rất phù hợp để học phát triển agent. Sách dùng GAIA vì ba lý do cụ thể.

Đáp án rõ ràng cho phản hồi nhanh. Cốt lõi của phát triển agent là "nhận ra khi nào nó thất bại và giảm những lần thất bại đó". Khi đáp án rõ ràng, bạn biết ngay mình có đang đi đúng hướng không. Bạn có thể lặp nhanh qua các chu kỳ thử nghiệm và cải tiến mà không vướng tiêu chí đánh giá mơ hồ.

Tối ưu để thực hành chu trình phát triển agent. Quá trình xây agent diễn ra như sau: trước hết, xác định agent có giúp giải một bài toán cụ thể không. Nếu có tiềm năng, phát triển một prototype. Sau

đó quan sát khi nào agent thất bại, phân tích nguyên nhân và cải tiến. Rồi lặp lại chu trình quan sát–phân tích–cải tiến này. GAIA có độ khó phù hợp để trải nghiệm trọn vẹn chu trình này từ đầu tới cuối.

Không cần kiến thức chuyên ngành. Phần lớn bài trong GAIA xoay quanh tìm kiếm web và tổng hợp thông tin. Ai trong chúng ta cũng làm những việc kiểu "tìm kiếm, thu thập thông tin và sắp xếp lại" trong đời sống hằng ngày. Bạn có thể hiểu trực quan bài toán và cách giải mà không cần kiến thức chuyên sâu về y khoa hay luật.

Xuyên suốt sách, ta sẽ dùng GAIA để theo dõi thay đổi hiệu năng của agent khi bổ sung kỹ thuật mới ở mỗi chương. Bạn sẽ tận mắt thấy từng thành phần như tool use, memory và planning thực sự cải thiện khả năng giải bài của agent ra sao.

GAIA cho ta cách đo hiệu năng agent. Nhưng chỉ đo thôi thì không cải thiện được agent. Điều gì thực sự giúp agent giải các bài này tốt hơn? Câu trả lời nằm ở cách ta thiết kế (engineer) context mà LLM nhận được.

1.5 Context engineering

Trước khi đi vào phát triển agent, hãy làm rõ hai thuật ngữ hay bị dùng lẫn. **Prompt** là văn bản input mà người dùng gửi cho LLM. Nó có thể chia tiếp thành system prompt, quy định mô hình nên phản hồi và cư xử thế nào (chẳng hạn "You are a friendly travel guide"), và user prompt (hay message), chứa yêu cầu thực tế của người dùng.

Context là khái niệm rộng hơn, bao gồm mọi thông tin LLM tham chiếu khi sinh response: system prompt, message của người dùng, lịch sử hội thoại trước đó, kết quả chạy tool, tài liệu được truy xuất, và nhiều thứ khác. Theo nghĩa này, prompt chỉ là một thành phần của context.

Hãy coi context là bộ nhớ làm việc (working memory) của LLM. Cũng như bạn không giải được bài toán nếu người ta chỉ cho bạn một nửa phương trình, LLM không hoàn thành được tác vụ nếu context thiếu thông tin quan trọng.

LLM đoán token tiếp theo chỉ dựa trên thông tin nằm trong context window. Mô hình sinh văn bản bằng cách dùng các mẫu (pattern) học được khi huấn luyện để khai thác thông tin trong context. Sự thật đơn giản này là lý do context engineering cực kỳ quan trọng trong phát triển agent.

1.5.1 Vì sao agent thất bại

Thất bại của agent có thể quy về hai nguyên nhân chính. Nguyên nhân thứ nhất là **mô hình chưa đủ thông minh**. Bao gồm các trường hợp mô hình không giải được bài toán phức tạp hoặc mắc lỗi suy luận logic. Cách khắc phục là dùng mô hình mạnh hơn hoặc chờ năng lực mô hình tiến bộ.

Nguyên nhân thứ hai là **thiếu thông tin cần thiết**. Dù mô hình đủ thông minh, nó cũng không thể cho đáp án đúng nếu thông tin cần để hoàn thành tác vụ không có trong context. Ví dụ, nếu ai đó hỏi "Chính sách nghỉ phép của công ty mình thế nào?" mà tài liệu chính sách công ty không có trong context, thì ngay cả mô hình mạnh nhất cũng không trả lời chính xác được.

Điều thú vị là một phần đáng kể thất bại của agent trong thực tế đến từ nguyên nhân thứ hai: thiếu thông tin. LLM chỉ thông minh tới mức context mà nó được cung cấp cho phép. Đó chính là lý do "đưa thông tin gì vào context" là yếu tố then chốt quyết định hiệu năng agent.

1.5.2 Từ prompt engineering tới context engineering

Thời kỳ đầu áp dụng LLM, prompt engineering là trọng tâm chính. Người làm thực tế tập trung vào cách viết system prompt để mô hình trả lời chính xác hơn và những chỉ dẫn nào cho ra output đúng định dạng mong muốn. Các kỹ thuật như "Let's think step by step" và "You are an expert" xuất hiện trong giai đoạn này.

Tuy nhiên, tình hình thay đổi khi agent bắt đầu dùng tool. Khi agent tìm kiếm web, kết quả được thêm vào context. Khi nó chạy code, kết quả thực thi được thêm vào. Khi nó truy vấn cơ sở dữ liệu, kết quả truy vấn được thêm vào. Context bắt đầu thay đổi động, vượt ra ngoài các system prompt tĩnh.

Sự chuyển dịch này đặt ra một câu hỏi mới: "Làm sao để context chỉ giữ những thông tin thiết yếu cho việc hoàn thành tác vụ?" Cách trả lời câu hỏi này chính là context engineering.



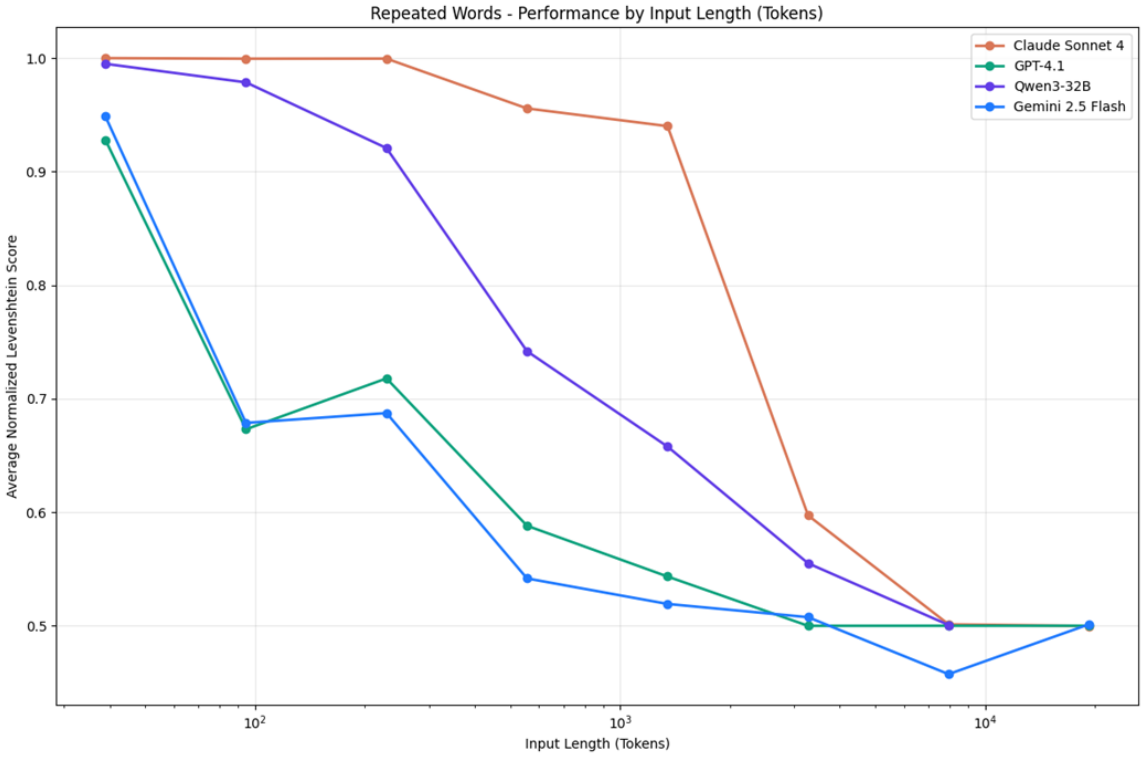
Hình 1.5: LLM chỉ có thể cho response chính xác, chất lượng cao khi context được cung cấp đủ thông tin. *(PDF gốc tr. 23)*

Context engineering là lĩnh vực cung cấp thông tin mà LLM cần để thực hiện tác vụ, đúng lúc và đúng dạng. Nó vượt ra ngoài việc viết prompt tốt, hướng tới thiết kế toàn diện: kết quả chạy tool nên được xử lý ra sao, giữ lại bao nhiêu lịch sử hội thoại, và khi nào cần truy xuất tri thức bên ngoài.

1.5.3 Context lớn hơn không phải lúc nào cũng tốt hơn

Context window của các LLM gần đây đã tăng rất mạnh. Hầu hết mô hình hiện hỗ trợ context window từ hàng trăm nghìn tới một triệu token. Vậy nhồi toàn bộ context window bằng mọi dữ liệu có được có cho response tốt nhất không?

Tiếc là không. Các nghiên cứu gần đây cho thấy hiệu năng mô hình giảm khi độ dài context tăng. Hiện tượng này đôi khi được gọi là **Context Rot**. Hiệu ứng "Lost in the Middle" được ghi nhận đặc biệt rõ: mô hình có xu hướng bỏ sót thông tin quan trọng nằm ở giữa context dài.



Hình 1.6: Dù context window lớn, input dài hơn vẫn có thể làm giảm hiệu năng mô hình (Nguồn: `https://research.trychroma.com/context-rot`). *(PDF gốc tr. 25)*

Điều này cho một nhận định quan trọng: để tối đa hoá hiệu năng mô hình, ta không nên chỉ đơn giản đưa thêm thông tin mà cần chọn lọc, chỉ đưa thông tin thật sự liên quan. Khi context đầy thông tin không cần thiết, các chi tiết quan trọng có thể bị chôn vùi, hoặc sự chú ý (attention) của mô hình bị phân tán.

1.5.4 Năm chiến lược context engineering

Context engineering có thể chia thành năm chiến lược lớn. Mỗi chiến lược được trình bày chi tiết ở các chương khác nhau của sách.

Generation (sinh): Dùng output do LLM sinh ra bên trong context. Bao gồm sinh kế hoạch cho tác vụ phức tạp, hoặc phản tư (reflection) về công việc đã xong để điều chỉnh chiến lược. Không chỉ truy xuất thông tin bên ngoài, chiến lược này cho phép LLM tự cấu trúc và phát triển context.

Retrieval (truy xuất): Lấy thông tin cần thiết từ nguồn bên ngoài và thêm vào context. Bao gồm tìm kiếm web, truy vấn cơ sở dữ liệu, đọc file, và truy xuất tài liệu tương tự từ vector database. Đây là chiến lược cốt lõi giúp agent tiếp cận thông tin mới nhất hoặc tri thức chuyên ngành không có trong dữ liệu huấn luyện.

Write (ghi): Ghi thông tin từ context ra bộ lưu trữ bên ngoài. Bao gồm lưu nội dung hội thoại quan trọng vào bộ nhớ dài hạn (long-term memory), ghi kết quả tính toán trung gian vào scratchpad, hoặc lưu code đã sinh ra file. Cách này vượt qua giới hạn của context window và cho phép lưu trữ thông tin lâu dài.

Reduce (thu gọn): Thu nhỏ kích thước context. Bao gồm tóm tắt hội thoại cũ, xoá thông tin không cần thiết, hoặc lọc thông tin theo mức độ quan trọng. Đây là điều thiết yếu để ngăn Context Rot và giúp mô hình tập trung chú ý vào thông tin then chốt.

Isolate (cô lập): Tách tác vụ hoặc tool ra các môi trường riêng. Bao gồm chạy code phức tạp trong môi trường sandbox, hoặc giao các lĩnh vực chuyên môn khác nhau cho các agent riêng. Cách này ngăn một context đơn lẻ trở nên quá phức tạp và cho phép xây context tối ưu cho từng tác vụ.

Bảng 1.1: Năm nhóm chiến lược chính của context engineering

Chiến lược	Mô tả	Chương liên quan
Generation	Xây context bằng output do LLM sinh ra	Chương 7 (Planning & Reflection)
Retrieval	Đưa thông tin từ nguồn bên ngoài vào context	Chương 3 (Tools), Chương 5 (Knowledge Base), Chương 6 (Memory)
Write	Lưu từ context ra bộ lưu trữ bên ngoài	Chương 6 (Memory), Chương 8 (Workspace)
Reduce	Nén và xoá bớt context	Chương 6 (Memory)
Isolate	Tách tác vụ/tool ra các môi trường riêng	Chương 8 (Workspace), Chương 9 (Multi-Agent)

Chương 6 (Memory) là điểm hội tụ của ba chiến lược: Retrieval, Write và Reduce. Lý do là hệ thống memory bao gồm truy xuất trải nghiệm trong quá khứ (Retrieval), lưu trải nghiệm mới (Write), và tóm tắt hoặc xoá ký ức cũ (Reduce).

1.5.5 Hành trình của cuốn sách

Xuyên suốt sách, ta sẽ lần lượt áp dụng từng chiến lược context engineering vào agent qua mỗi chương. Bằng benchmark GAIA, ta sẽ kiểm chứng định lượng tác động của từng chiến lược lên hiệu năng agent.

Chương 2 tới 4 là hành trình hiện thực agent loop cơ bản. Ở Chương 2, ta học cách dùng LLM và thử xem chỉ riêng LLM có giải được các bài trong GAIA hay không. Một số bài giải được chỉ với LLM, nhưng nhiều bài cần tool như tìm kiếm web hay đọc file. Chương 3 xử lý vấn đề này: giới thiệu cách xây tool, cách để LLM gọi tool, và tìm hiểu MCP (Model Context Protocol) như một cách tích hợp tool mà không phải tự xây từ đầu. Chương 4 kết hợp các yếu tố này để hiện thực một agent loop lặp lại việc dùng tool để giải bài, không phải dưới dạng một vòng lặp đơn giản mà bằng cách xây một class `Agent` được trừu tượng hoá.

Chương 5 tới 9 tăng cường context engineering bằng cách bổ sung các yếu tố mới vào agent loop cơ bản. Chương 5 thêm năng lực truy xuất bằng cách cung cấp các nguồn dữ liệu mà agent có thể truy cập. Chương 6 hiện thực việc quản lý lịch sử hội thoại trong lúc thực hiện tác vụ, cùng long-term memory cho phép tận dụng những hiểu biết rút ra từ tác vụ hiện tại cho các tác vụ sau.

Chương 7 tìm hiểu cách lập kế hoạch cho hướng thực hiện tác vụ và điều chỉnh hướng đi khi gặp input dữ liệu dài hoặc phát sinh lỗi giữa chừng. Chương 8 giới thiệu cô lập context (context isolation) bằng cách cung cấp một workspace nơi agent có thể chạy code và làm việc. Chương 9 xem xét cách thu gọn và cô lập context bằng cách tạo nhiều agent chuyên biệt.



Hình 1.7: Tổng quan hành trình xuyên suốt cuốn sách. *(PDF gốc tr. 28)*

Chương 10 giới thiệu các phương pháp giám sát (monitoring) và đánh giá agent. Trong suốt hành trình, ta dùng GAIA, một bộ dữ liệu có sẵn đáp án. Tuy nhiên, khi phát triển agent thực tế, những bộ dữ liệu tiện lợi như vậy thường không có. Ta sẽ xem các bộ dữ liệu thường dùng và tìm hiểu phương pháp đánh giá cho trường hợp không có bộ dữ liệu phù hợp.

1.6 Kiến thức cần có để đọc sách

Sách trình bày toàn bộ quá trình hiện thực LLM agent từ đầu bằng Python thuần. Để hiểu đầy đủ và theo kịp nội dung cũng như các bài thực hành, bạn đọc cần đáp ứng các điều kiện cơ bản sau.

Chuẩn bị cần thiết

1. Kiến thức Python cơ bản. Mọi ví dụ và bài tập trong sách đều viết bằng Python. Vì vậy bạn đọc nên quen với các kiến thức Python nền tảng sau:

- Cấu trúc dữ liệu cơ bản như biến, kiểu dữ liệu, list và dictionary.
- Cấu trúc điều khiển như câu lệnh `if` và vòng lặp `for`/`while`.
- Định nghĩa và gọi hàm.
- Khái niệm lập trình hướng đối tượng (OOP) cơ bản, như class.

Nếu chưa quen các chủ đề này, bạn nên học trước một tài liệu Python cho người mới bắt đầu.

2. Thiết lập môi trường phát triển. Để làm bài thực hành hiệu quả, cần có môi trường phát triển Python trên máy:

- Cài phiên bản Python mới nhất trên máy (khuyến nghị: Python 3.12 trở lên).
- Chuẩn bị một code editor hoặc IDE (khuyến nghị: VS Code).
- Nên biết cách cài package Python bằng `pip` hoặc `uv`.

3. API key và dịch vụ bên ngoài.

- OpenAI API key để truy cập các mô hình GPT (LLM chính dùng trong các ví dụ).
- Tavily API key cho chức năng tìm kiếm web trong research agent.
- Hiểu cơ bản về khái niệm REST API và HTTP request.

4. Chi phí.

- OpenAI: tổng chi phí làm hết các bài thực hành dưới $10.
- Tavily Search API: gói miễn phí cho tới 1.000 lượt tìm kiếm mỗi tháng, đủ cho mọi bài thực hành trong sách.
- Sách sẽ đưa ra mẹo phát triển và kiểm thử tiết kiệm chi phí xuyên suốt các chương.

Khi đã kiểm tra xong các mục chuẩn bị trên, bạn đã sẵn sàng bắt đầu hành trình phát triển LLM agent cùng cuốn sách. Chương 2 sẽ xây nền hiểu biết về LLM, thứ vận hành mọi thứ ta sẽ xây trong sách.

1.7 Tóm tắt

- AI agent trải trên một phổ rộng, từ trợ lý cá nhân như ChatGPT và Claude tới agent phục vụ khách hàng và công cụ chuyên biệt như Claude Code và Cursor. Tất cả có chung một nền tảng: LLM là lõi ra quyết định.
- LLM agent gồm ba thành phần: LLM (bộ não), tool (phương tiện tương tác với thế giới bên ngoài) và loop (quá trình lặp cho tới khi đạt mục tiêu). LLM quyết định dùng tool nào và khi nào dừng.
- Workflow là luồng thực thi do lập trình viên định nghĩa, trong đó LLM thực hiện các bước cụ thể. Agent là luồng do LLM điều khiển, trong đó mô hình tự xác định quy trình của mình một cách linh hoạt. Hệ thống production thường kết hợp cả hai cách tiếp cận.
- Dùng agent khi tác vụ cần nhiều bước không đoán trước được, mang lại đủ giá trị để bù chi phí, và cho phép phát hiện lỗi.
- Benchmark GAIA cung cấp các bài thực hành lý tưởng cho phát triển agent.
- Context engineering là lĩnh vực cung cấp đúng thông tin vào đúng lúc. Năm chiến lược (Generation, Retrieval, Write, Reduce, Isolate) tạo thành khung để xây agent hiệu quả xuyên suốt sách.

Chương 2

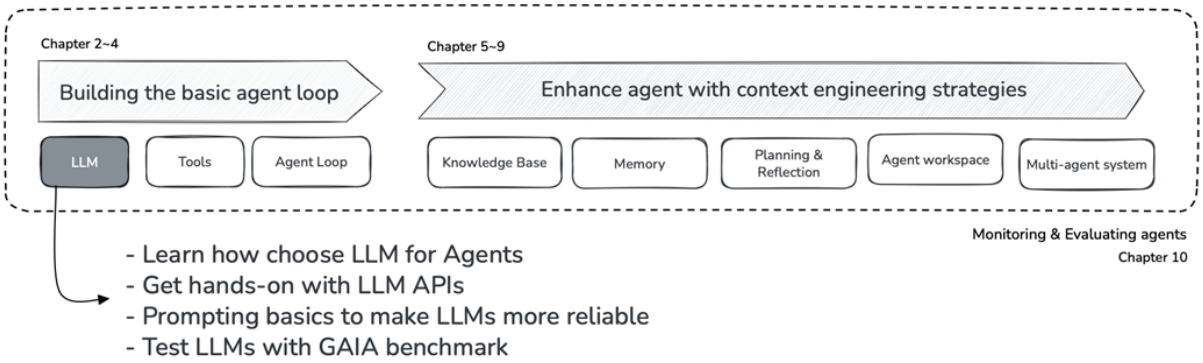
Bộ não của AI agent: LLM

Chương này bao gồm:

- Chọn LLM phù hợp để phát triển agent.
- Dùng API của LLM thông qua LiteLLM.
- Prompt engineering cho agent.
- Trải nghiệm giới hạn của LLM qua benchmark GAIA.

Xuyên suốt cuốn sách, chúng ta sẽ xây dựng một **Research Agent**: một hệ thống có thể hiểu các yêu cầu như "khảo sát các công trình gần đây về X" hay "rút ra các phát hiện chính từ những file PDF này", thu thập thông tin từ nhiều nguồn và tổng hợp kết quả thành câu trả lời đầy đủ. Trung tâm của agent này là LLM, đóng vai trò bộ não ra quyết định. Nhưng trước khi xây được một agent hiệu quả, ta cần trả lời vài câu hỏi nền tảng: nên dùng LLM nào? Giao tiếp với nó bằng code ra sao? Và quan trọng nhất: tự thân một LLM làm được gì, và nó hụt hơi ở đâu?

Chương này giải quyết thẳng các câu hỏi đó. Ta bắt đầu bằng cách chọn LLM cho việc phát triển agent, sau đó thực hành với API qua LiteLLM để làm việc liền mạch với nhiều nhà cung cấp. Tiếp theo, ta xem xét các nguyên tắc prompt engineering giúp biến một LLM đa dụng thành một agent đáng tin cậy. Cuối cùng, ta đưa LLM vào thử thách bằng benchmark GAIA để trực tiếp thấy giới hạn của chúng trước các bài toán thực tế. Thí nghiệm này sẽ cho thấy rõ vì sao agent cần tool, làm nền cho mọi thứ ta xây dựng từ chương 3 trở đi.



Hình 2.1: Hành trình qua cuốn sách – trọng tâm là Chương 2 *(PDF gốc tr. 32)*

2.1 Chọn LLM cho agent

Khi bắt đầu phát triển agent, câu hỏi đầu tiên bạn gặp là "Nên dùng LLM nào?". Lựa chọn này ảnh hưởng trực tiếp tới hiệu năng, chi phí và tốc độ phát triển agent. Hãy xem cách chọn LLM từ góc nhìn của người

phát triển agent.

2.1.1 Bắt đầu với closed LLM

LLM chia thành hai nhóm lớn. **Open LLM** là các mô hình có thể tải trọng số (weights) về, chạy cục bộ và fine-tune; Llama của Meta, Mistral của Mistral AI, Qwen của Alibaba và DeepSeek thuộc nhóm này. **Closed LLM** chỉ truy cập được qua API; GPT của OpenAI, Claude của Anthropic và Gemini của Google là các ví dụ hàng đầu. Cuốn sách này tập trung xây dựng agent bằng closed LLM. Dưới đây là lý do closed LLM là lựa chọn tốt nhất khi mới bắt đầu phát triển agent.

Thử nghiệm và kiểm chứng nhanh. Cốt lõi của phát triển agent là kiểm chứng ý tưởng thật nhanh rồi lặp lại. Với closed LLM, bạn bắt đầu ngay chỉ với một API key. Không cần chuẩn bị server GPU hay triển khai hạ tầng mô hình, bạn có thể tập trung thẳng vào việc hiện thực logic của agent. Vì dùng cùng một môi trường từ prototype tới production, các hành vi đã kiểm chứng lúc phát triển sẽ chạy y hệt trong dịch vụ thật.

Khi cần hiệu năng cao nhất. Hiệu năng của agent rốt cuộc phụ thuộc vào trí thông minh của LLM. Để thực hiện tác vụ phức tạp, agent phải hiểu đúng ý định người dùng, chọn tool phù hợp và đi tới kết quả qua suy luận nhiều bước. Các mô hình mạnh nhất hiện nay vẫn là closed LLM. Dù open LLM tiến bộ rất nhanh, closed LLM vẫn giữ lợi thế về suy luận phức tạp và độ chính xác khi dùng tool. Khi "bộ não" của agent thông minh, cả hệ thống mới vận hành đúng.

Tính năng sẵn sàng cho production. Phát triển agent hiện đại đòi hỏi nhiều hơn việc sinh văn bản đơn thuần. Các nhà cung cấp closed LLM cung cấp những tính năng thiết yếu cho agent qua API ổn định:

- **Tool Calling**: gọi tool và API bên ngoài một cách tự nhiên.
- **Structured Output**: sinh phản hồi theo định dạng có cấu trúc như JSON.
- **Tối ưu chi phí**: dùng token hiệu quả nhờ prompt caching và xử lý theo lô (batch processing).

Các khả năng này có thể bổ sung cho open LLM, nhưng phải tự hiện thực hoặc dùng thư viện bên thứ ba. Với closed LLM, bạn dùng được ngay các tính năng đã được kiểm nghiệm kỹ trong thực tế.

2.1.2 Mở rộng sang open LLM

Khi đã kiểm chứng thiết kế agent bằng closed LLM, open LLM trở nên đáng cân nhắc ở giai đoạn tiếp theo.

Tối ưu chi phí. Khi lưu lượng lớn, tự host (self-hosting) có thể rẻ hơn trả phí API. Các tác vụ đơn giản như phân loại hay trích xuất có thể giao cho các open model nhỏ hơn mà vẫn đủ tốt. Điều này cho phép một chiến lược: phần suy luận cốt lõi giữ ở closed LLM hiệu năng cao, còn các tác vụ phụ trợ được phân sang open LLM.

Ổn định và quyền kiểm soát. Phụ thuộc vào API bên ngoài nghĩa là sự cố của họ cũng thành sự cố của bạn. Open LLM tự host loại bỏ sự phụ thuộc này. Ngoài ra, khi quy định pháp lý yêu cầu dữ liệu nhạy cảm không được rời khỏi hạ tầng của bạn, open LLM có thể là lựa chọn duy nhất.

Một cách tiếp cận thực tế. Chuyển sang open LLM nên làm dần dần. Trước tiên, kiểm chứng thiết kế và hành vi của agent bằng closed LLM. Sau đó port cùng thiết kế đó sang open LLM và kiểm tra xem mức suy giảm hiệu năng có nằm trong ngưỡng chấp nhận được không. Một cấu hình lai (hybrid), giữ closed LLM cho suy luận cốt lõi và chỉ thay các tác vụ đơn giản bằng open LLM, cũng là một chiến lược tốt.

2.1.3 Các khả năng thiết yếu của LLM cho agent

Dù chọn LLM nào, một số khả năng là bắt buộc cho việc phát triển agent.

Tool calling (function calling). Để tương tác với thế giới bên ngoài, agent phải gọi được tool. Khả năng LLM chọn và gọi đúng các tool như tìm kiếm web, thực thi code hay truy vấn cơ sở dữ liệu là điều cốt lõi làm nên một agent. Khả năng này được trình bày chi tiết ở chương 3.

Structured output. Đầu ra của LLM thường phải tích hợp với các hệ thống khác chứ không chỉ để con người đọc. Khả năng sinh đầu ra có cấu trúc khớp với JSON hoặc một schema cụ thể là thiết yếu để tích hợp agent vào hệ thống phần mềm thật.

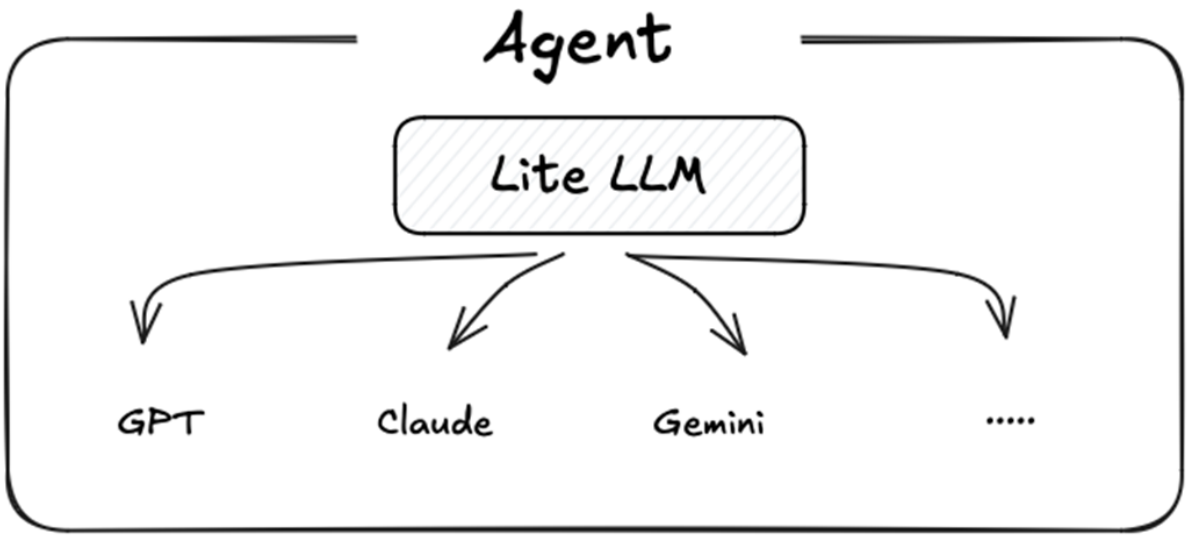
Hỗ trợ context dài. Agent tích luỹ đủ loại thông tin trong context: lịch sử hội thoại, kết quả thực thi tool, tài liệu truy xuất được, v.v. Để xử lý hiệu quả lượng thông tin này cần một context window đủ dài. Phần lớn LLM gần đây hỗ trợ từ vài trăm nghìn tới một triệu token, nhưng context dài hơn không phải lúc nào cũng tốt hơn. Chìa khoá của context engineering là cung cấp đúng thông tin, đúng lúc, đúng dạng.

Thiết kế để linh hoạt đổi mô hình trong một thị trường thay đổi nhanh. Thị trường LLM biến đổi với tốc độ chóng mặt. Mô hình mới ra mắt hằng tháng, bảng xếp hạng hiệu năng thay đổi thường xuyên, cấu trúc giá liên tục được cập nhật. Thứ lý tưởng hôm nay có thể lỗi thời ngày mai. Vì vậy cần thiết kế để linh hoạt đổi mô hình ngay từ đầu.

Để cập nhật thông tin, có thể dựa vào các nguồn đáng tin cậy:

- **Artificial Analysis** (`https://artificialanalysis.ai`): so sánh toàn diện về chất lượng, tốc độ và chi phí.
- **LMArena Leaderboard** (`https://lmarena.ai/leaderboard`): xếp hạng theo mức ưa thích của người dùng dựa trên tương tác thực tế.
- **Thông báo chính thức từ nhà cung cấp**: phát hành mô hình mới và cập nhật giá.

Đây cũng là lý do cuốn sách dùng LiteLLM xuyên suốt. Với cùng một đoạn code, bạn có thể dễ dàng đổi qua lại giữa mô hình của các nhà cung cấp khác nhau, nhờ đó thử nghiệm và chuyển đổi nhanh khi có mô hình mới.



Hình 2.2: LiteLLM, một lớp bọc (wrapper) thống nhất cho phép gọi nhiều nhà cung cấp LLM theo cùng một kiểu *(PDF gốc tr. 36)*

Lựa chọn của cuốn sách: OpenAI API. Xuyên suốt cuốn sách, chủ yếu dùng các mô hình GPT của OpenAI. Với ví dụ đơn giản, dùng `gpt-5-mini` để giảm chi phí; còn phần hiện thực agent xoay quanh `gpt-5`. OpenAI API được dùng rộng rãi, có nhiều tài liệu học tập, hỗ trợ ổn định Tool Calling và Structured Output, và có thư viện Python dễ dùng. Nhờ vậy ta tập trung được vào chính logic của agent.

Tuy nhiên, code được thiết kế để tránh bị khoá vào một nhà cung cấp (vendor lock-in). Dùng LiteLLM (giới thiệu ở mục 2.2), cùng một đoạn code có thể dễ dàng chuyển giữa OpenAI, Anthropic, Google, hoặc thậm chí open LLM tự host.

Ghi chú về reasoning model

Có thể bạn đã nghe về các "reasoning model" như dòng o của OpenAI: chúng tiêu tốn thêm thinking token để giải quyết bài toán phức tạp từng bước một. Các mô hình này giỏi ở tác vụ cần phân tích sâu nhưng đi kèm độ trễ và chi phí cao hơn.
Tuy vậy, ranh giới này ngày càng mờ đi. Claude của Anthropic hiện có các mô hình lai, chuyển được giữa chế độ thường và chế độ suy luận. `gpt-5` của OpenAI còn đi xa hơn: tự động quyết định có suy luận hay không dựa trên truy vấn, nên người dùng không cần chọn giữa các loại mô hình nữa. Xu hướng hướng tới các mô hình hợp nhất, tự điều chỉnh độ sâu suy luận theo tác vụ, giúp người phát triển agent chọn lựa đơn giản hơn.

2.2 Cơ bản về LLM API để xây dựng agent

Giờ hãy bắt tay vào dùng LLM API. Trong mục này, ta thiết lập môi trường phát triển, học cơ bản về API của OpenAI, rồi dùng LiteLLM để hợp nhất nhiều nhà cung cấp dưới một giao diện duy nhất. Ta cũng đề cập quản lý hội thoại, structured output và gọi bất đồng bộ (asynchronous) – những khả năng thiết yếu cho phát triển agent.

2.2.1 Thiết lập môi trường phát triển

Toàn bộ code ví dụ có trên GitHub tại `https://github.com/shangrilar/ai-agent-from-scratch`. Bắt đầu bằng cách clone repository:

```
git clone https://github.com/shangrilar/ai-agent-from-scratch
cd ai-agent-from-scratch
```

Cuốn sách dùng `uv` làm trình quản lý package Python. `uv` nhanh hơn `pip` và xử lý cả quản lý môi trường ảo lẫn giải quyết phụ thuộc trong một bước. Nếu chưa cài `uv`, có thể cài bằng lệnh sau:

```
# macOS/Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Repository có sẵn file `pyproject.toml`, nên chỉ một lệnh là tạo môi trường ảo và cài mọi phụ thuộc. Cài xong, mở Jupyter Lab để chạy theo các ví dụ:

```
uv sync
uv run jupyter lab
```

Tiếp theo, cần cấu hình API key. Tạo file `.env` ở thư mục gốc dự án và thêm các API key vào đó. Xem Phụ lục ("API Key Setup") để biết cách lấy API key từ từng nhà cung cấp. Tối thiểu bạn cần một OpenAI API key để chạy các ví dụ trong sách. Bạn cũng cần một HuggingFace token (`HF_TOKEN`) để truy cập bộ dữ liệu benchmark GAIA ở mục 2.4.

Để dùng các biến môi trường này trong Python, nạp file `.env` bằng thư viện `python-dotenv` (đã có trong phụ thuộc của dự án). Vì các notebook nằm trong thư mục con như `chapter_02/`, ta dùng `find_dotenv()` để tìm file `.env` ở thư mục gốc dự án.

Listing 2.1. Nạp biến môi trường

```
from dotenv import load_dotenv, find_dotenv
load_dotenv(find_dotenv())
```

Hàm `find_dotenv()` tìm ngược lên các thư mục cha để thấy file `.env`, nên nó hoạt động bất kể bạn chạy code từ thư mục con nào. Sau khi nạp, các LLM client sẽ tự đọc API key từ các biến môi trường này.

2.2.2 Làm quen với OpenAI API

Chat Completions API của OpenAI đã trở thành chuẩn chung của ngành. Phần lớn nhà cung cấp LLM có giao diện tương tự, nên hiểu OpenAI API thì làm việc với các nhà cung cấp khác cũng dễ. Hãy khởi tạo OpenAI client và gửi một request đơn giản.

Listing 2.2. Gọi OpenAI API cơ bản

```
from openai import OpenAI

client = OpenAI()
response = client.chat.completions.create(
    model="gpt-5-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"}
    ]
)
print(response.choices[0].message.content)
```

Hai tham số quan trọng nhất của `chat.completions.create` là `model` và `messages`. Tham số `model` chỉ định mô hình được dùng, còn `messages` chứa cuộc hội thoại.

Tham số `messages` là danh sách các message object, mỗi object có `role` và `content`. Có ba role:

- `developer` (trước đây là `system`): system prompt, định nghĩa hành vi tổng thể của mô hình.
- `user`: message từ người dùng.
- `assistant`: phản hồi do mô hình sinh ra.

Văn bản phản hồi thực nằm ở `response.choices[0].message.content`. `choices` là một list vì bạn có thể yêu cầu nhiều phản hồi bằng tham số `n`, dù thường chỉ dùng phản hồi đầu tiên.

Một tham số hay dùng là `temperature`, có giá trị từ 0 đến 2. Giá trị thấp cho phản hồi mang tính tất định và nhất quán hơn; giá trị cao cho đầu ra sáng tạo và đa dạng hơn. Dùng 0–0.3 cho tác vụ cần độ chính xác như trích xuất dữ liệu hay phân loại, và 0.7–1.0 cho viết sáng tạo hay brainstorming. Tham số `max_tokens` giới hạn số token tối đa được sinh ra.

Các nhà cung cấp khác hoạt động tương tự. Ví dụ, để dùng Claude của Anthropic:

Listing 2.3. Gọi Anthropic API

```
from anthropic import Anthropic

client = Anthropic()
response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "What is the capital of France?"}
    ]
)
print(response.content[0].text)
```

Dù OpenAI API và Anthropic API khá giống nhau, vẫn có những khác biệt nhỏ: tên method (`chat.completions.create` so với `messages.create`), cấu trúc phản hồi (`message.content` so với `content[0].text`), và tham số bắt buộc (`max_tokens` có bắt buộc hay không). Quản lý các khác biệt này giữa nhiều nhà cung cấp làm việc phát triển trở nên rườm rà. Mục tiếp theo cho thấy cách giải quyết vấn đề này.

2.2.3 Hợp nhất các nhà cung cấp bằng LiteLLM

Khi phát triển agent, bạn thường phải so sánh các mô hình hoặc chuyển đổi giữa chúng theo yêu cầu chi phí và hiệu năng. Nếu mỗi nhà cung cấp có một giao diện riêng, việc này rất tẻ nhạt. LiteLLM giải quyết vấn đề đó: đây là thư viện cho phép gọi hơn 100 LLM qua một giao diện thống nhất.

Listing 2.4. Gọi API thống nhất bằng LiteLLM

```
from litellm import completion

response = completion(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "Hello!"}]
)

response = completion(
    model="claude-sonnet-4-5",
    messages=[{"role": "user", "content": "Hello!"}]
)
```

Một hàm `completion` duy nhất xử lý mọi nhà cung cấp. Chỉ cần đổi tên mô hình, và định dạng phản hồi được hợp nhất theo định dạng của OpenAI. API key của từng nhà cung cấp được tự động đọc từ biến môi trường.

2.2.4 Quản lý hội thoại: xử lý API stateless

Khi dùng các dịch vụ web như ChatGPT hay Claude, chúng có vẻ nhớ được các cuộc hội thoại trước. Tuy nhiên, LLM API là *stateless*: mỗi lần gọi API là độc lập và không có ký ức gì về các lần gọi trước. Hãy kiểm chứng bằng code.

Listing 2.5. Minh hoạ tính stateless của API

```
from litellm import completion

# First call
response1 = completion(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "My name is Jungjun."}]
)
print(response1.choices[0].message.content)

# Second call
response2 = completion(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "What is my name?"}]
)
print(response2.choices[0].message.content)
```

Dù đã tự giới thiệu ở lần gọi đầu, lần gọi thứ hai không nhớ gì về điều đó. Để duy trì mạch hội thoại, người phát triển phải tự quản lý lịch sử hội thoại.

Listing 2.6. Quản lý lịch sử hội thoại

```
from litellm import completion

messages = []

# First exchange
messages.append({"role": "user", "content": "My name is Jungjun."})
response1 = completion(model="gpt-5-mini", messages=messages)
assistant_message1 = response1.choices[0].message.content
messages.append({"role": "assistant", "content": assistant_message1})
print(assistant_message1)

# Second exchange - includes previous conversation history
messages.append({"role": "user", "content": "What is my name?"})
response2 = completion(model="gpt-5-mini", messages=messages)
assistant_message2 = response2.choices[0].message.content
print(assistant_message2)
```

Ta tích luỹ toàn bộ nội dung hội thoại trong list `messages` và truyền cả lịch sử ở mỗi lần gọi. Message của người dùng được thêm với role `user`, phản hồi của mô hình với role `assistant`. Nhờ vậy mô hình hiểu được context trước đó và duy trì cuộc hội thoại mạch lạc.

Cách làm này càng quan trọng hơn khi phát triển agent. Agent không chỉ cần thêm lịch sử hội thoại mà còn cả kết quả gọi tool, kết quả tìm kiếm và các bước suy luận trung gian vào context. Hội thoại càng dài, lượng token dùng càng tăng, khiến chi phí cao hơn và phản hồi chậm hơn. Các kỹ thuật quản lý độ dài context hiệu quả được trình bày chi tiết ở chương 6.

2.2.5 Structured output

Văn bản ngôn ngữ tự nhiên do LLM sinh ra rất dễ đọc với con người nhưng bất tiện cho chương trình xử lý. Để gọi tool, agent phải xuất ra tool nào cần gọi với tham số nào theo một định dạng có cấu trúc. Structured Output là tính năng khiến LLM sinh phản hồi theo một định dạng định trước, chẳng hạn JSON.

Có thể định nghĩa định dạng đầu ra mong muốn bằng thư viện Pydantic của Python. Pydantic là thư viện kiểm tra hợp lệ dữ liệu (data validation), cho phép định nghĩa cấu trúc dữ liệu mong đợi dưới dạng class. Kế thừa từ `BaseModel` sẽ tạo ra một schema chỉ rõ tên trường và kiểu của chúng. Pydantic sau đó tự động kiểm tra mọi dữ liệu theo schema này và báo lỗi rõ ràng nếu dữ liệu không khớp. Nhờ vậy dễ áp đặt "hợp đồng dữ liệu" và bắt sớm đầu vào sai định dạng. Trong ví dụ dưới đây, `ExtractedInfo` định nghĩa ba trường: `name` và `email` là chuỗi bắt buộc, còn `phone` là chuỗi tuỳ chọn, mặc định là `None` nếu không có.

Listing 2.7. Structured output với Pydantic

```
from pydantic import BaseModel
from litellm import completion

class ExtractedInfo(BaseModel):
    name: str
    email: str
    phone: str | None = None

response = completion(
    model="gpt-5-mini",
    messages=[{
        "role": "user",
        "content": "My name is John Smith, my email is john@example.com, and my
        ↪  phone is 555-1234."
    }],
    response_format=ExtractedInfo
```

```
)
result = response.choices[0].message.content
```

Khi truyền một Pydantic model vào `response_format`, LLM sinh JSON khớp với schema đó. Bạn có thể dùng trực tiếp đối tượng đã parse từ thuộc tính `content` của phản hồi.

Structured output đóng vai trò then chốt trong phát triển agent. Trong tool calling (chương 3), LLM phải xuất ra tool cần gọi và tham số theo định dạng có cấu trúc. Chuyển ý định người dùng thành hành động phù hợp cũng là một ứng dụng của structured output.

2.2.6 Gọi bất đồng bộ

Khi phát triển agent, bạn thường phải xử lý nhiều request LLM cùng lúc: chấm hàng chục bài toán trong benchmark, so sánh phản hồi của nhiều mô hình, hoặc chạy nhiều agent đồng thời trong hệ multi-agent.

Nếu gửi request tuần tự, tổng thời gian bằng tổng thời gian của từng request. Với thực thi bất đồng bộ, bạn khởi động nhiều request cùng lúc và xử lý từng phản hồi khi nó về tới.

Python xử lý lập trình bất đồng bộ bằng cú pháp `async/await` và thư viện `asyncio`. LiteLLM hỗ trợ gọi bất đồng bộ qua hàm `acompletion`.

Listing 2.8. Gọi LLM bất đồng bộ

```
import asyncio
from litellm import acompletion

async def get_response(prompt: str) -> str:
    response = await acompletion(
        model="gpt-5-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

prompts = [
    "What is 2 + 2?",
    "What is the capital of Japan?",
    "Who wrote Romeo and Juliet?"
]

# Execute all requests concurrently
tasks = [get_response(p) for p in prompts]
results = await asyncio.gather(*tasks)

for prompt, result in zip(prompts, results):
    print(f"Q: {prompt}")
    print(f"A: {result}\n")
```

Hàm định nghĩa bằng `async def` được gọi là coroutine. Từ khoá `await` chờ một thao tác bất đồng bộ hoàn tất, đồng thời cho phép các tác vụ khác chạy trong lúc chờ. `asyncio.gather` thực thi nhiều coroutine đồng thời và trả về tất cả kết quả cùng lúc.

Nếu gửi tuần tự ba request, mỗi request mất 1 giây, tổng thời gian là 3 giây. Gửi bất đồng bộ thì nhận đủ phản hồi trong khoảng 1 giây, tức thời gian của request lâu nhất.

Quản lý request đồng thời. Khi gửi nhiều request cùng lúc, nảy sinh hai vấn đề: giới hạn tốc độ (rate limit) của nhà cung cấp API, và lỗi tạm thời do sự cố mạng hoặc server quá tải. Tham số `num_retries` của LiteLLM xử lý lỗi tạm thời bằng cách tự động thử lại với exponential backoff. Với rate limit, ta dùng `asyncio.Semaphore` của Python để giới hạn số request chạy đồng thời.

Điểm mấu chốt là Semaphore kiểm soát truy cập ở cấp khối `async with`. Bằng cách bọc chính lời gọi `acompletion`, ta đảm bảo dù có bao nhiêu task đang chạy, chỉ một số lượng giới hạn lời gọi API thật diễn ra cùng lúc.

Listing 2.9. Request đồng thời có giới hạn tốc độ

```
import asyncio
from litellm import acompletion

# Limit to 10 concurrent requests
semaphore = asyncio.Semaphore(10)

async def call_llm(prompt: str) -> str:
    """LLM call with rate limiting and automatic retry."""
    async with semaphore:
        response = await acompletion(
            model="gpt-5-mini",
            messages=[{"role": "user", "content": prompt}],
            num_retries=3  # Automatic retry with exponential backoff
        )
        return response.choices[0].message.content

# Even with 100 concurrent tasks, only 10 API calls run at a time
prompts = [f"What is {i} + {i}?" for i in range(100)]
tasks = [call_llm(p) for p in prompts]
results = await asyncio.gather(*tasks, return_exceptions=True)
```

Hàm `call_llm` bọc `acompletion` bằng cả hai lớp bảo vệ. Semaphore đảm bảo dù tạo 100 task cùng lúc, chỉ 10 lời gọi API được thực thi đồng thời; khi một lời gọi xong, task đang chờ kế tiếp được chạy. Tham số `num_retries=3` bảo LiteLLM tự thử lại request thất bại tối đa 3 lần với exponential backoff, đặc biệt hữu ích để xử lý lỗi rate limit tạm thời.

Đối số `return_exceptions=True` trong `asyncio.gather` ngăn một lỗi đơn lẻ huỷ tất cả các task khác. Thay vào đó, exception được trả về như giá trị trong list kết quả, cho phép xử lý lỗi êm thấm mà vẫn lấy được kết quả từ các lời gọi thành công.

Mẫu này thiết yếu cho các đợt đánh giá benchmark như GAIA, khi cần xử lý hàng trăm bài toán hiệu quả mà không làm quá tải API.

2.3 Nâng cao trí thông minh của agent: prompt engineering

Khi đã chọn LLM và biết dùng API của nó, bạn cần chỉ dẫn để LLM hoạt động đúng như một agent. Đây là phạm vi của thiết kế prompt. Prompt chia thành hai loại chính. **User prompt** là các message người dùng gõ vào giao diện chat; chúng thay đổi mỗi lượt và nằm ngoài tầm kiểm soát của người phát triển. Ngược lại, **system prompt** là các chỉ dẫn do người phát triển định nghĩa, tồn tại suốt cuộc hội thoại, thiết lập tính cách, quyền hạn, ràng buộc và chính sách dùng tool của agent. Cuốn sách tập trung vào thiết kế system prompt. Một system prompt được soạn kỹ sẽ biến một LLM đa dụng thành agent đáng tin cậy cho các tác vụ cụ thể. Dù không kiểm soát được user prompt, system prompt cho phép ta xây dựng agent phản hồi nhất quán trước rất nhiều kiểu đầu vào của người dùng.

2.3.1 Vai trò của system prompt

System prompt đặc biệt quan trọng với agent. Agent hành động tự chủ qua nhiều bước, và system prompt định nghĩa các quy tắc hành vi dẫn dắt mọi quyết định đó.

Từ góc nhìn context engineering, system prompt là thông tin *luôn* có mặt trong context. Mỗi khi agent gọi tool, phân tích kết quả hay quyết định hành động tiếp theo, system prompt nằm ở đầu context

để định hướng phán đoán của agent. Vì thế chất lượng system prompt quyết định chất lượng hành vi tổng thể của agent.

System prompt của một agent production thực sự trông như thế nào? Anthropic công bố các system prompt của Claude (`https://docs.anthropic.com/en/release-notes/system-prompts`). Phân tích chúng sẽ thấy được các nguyên tắc cốt lõi khi thiết kế system prompt cho agent. Lưu ý: system prompt bàn ở đây là prompt được áp dụng tự động khi dùng Claude qua website claude.ai hoặc giao diện chat của ứng dụng di động. Khi gọi Claude qua API, người phát triển tự viết system prompt của mình, nên prompt này không áp dụng.

System prompt của Claude có bốn vai trò chính: định nghĩa danh tính sản phẩm, quy định định dạng và văn phong đầu ra, đặt ranh giới cho các hành vi bị cấm, và làm rõ giới hạn hiểu biết của mô hình.

Định nghĩa danh tính sản phẩm. Vai trò cơ bản nhất của system prompt là định nghĩa agent "là ai". System prompt của Claude có một phần thông tin sản phẩm giúp Claude trả lời chính xác các câu hỏi về bản thân. Ví dụ, nó nêu rằng mô hình hiện tại thuộc họ mô hình Claude 4.5, có ba phiên bản (Opus, Sonnet và Haiku), và có nhiều cách truy cập, gồm API và Claude Code.

Vì sao cần thông tin này? Dữ liệu huấn luyện của LLM có thể chứa thông tin về các phiên bản cũ. Cung cấp rõ thông tin sản phẩm hiện tại trong system prompt giúp ngăn mô hình đưa ra thông tin lỗi thời hoặc sai. Ngoài ra, chỉ dẫn mô hình hướng người dùng tới tài liệu chính thức với những tính năng nó không biết sẽ tránh cung cấp thông tin sai.

Điều tương tự áp dụng khi phát triển agent. Agent phải hiểu chính xác vai trò và khả năng của mình để nói rõ cho người dùng nó làm được gì và không làm được gì. Ví dụ, khi xây agent chăm sóc khách hàng, bạn nên định nghĩa trong system prompt những loại yêu cầu nó xử lý được và khi nào cần chuyển (escalate) cho con người.

Quy định định dạng và văn phong đầu ra. Một phần đáng kể system prompt của Claude dành cho chỉ dẫn về giọng điệu và định dạng. Đáng chú ý là các hướng dẫn chi tiết về bullet point và cách dùng danh sách. Prompt chỉ dẫn Claude tránh định dạng quá mức và trả lời tự nhiên bằng câu, đoạn văn trong các cuộc trò chuyện thông thường. Danh sách chỉ được dùng khi người dùng yêu cầu rõ ràng, hoặc khi thông tin có nhiều khía cạnh tới mức bullet point là cần thiết.

Vì sao cần chỉ dẫn chi tiết đến vậy? LLM có xu hướng làm theo các mẫu trong dữ liệu huấn luyện, và vì bullet point được dùng nhiều trong tài liệu kỹ thuật và blog, mô hình tự nhiên thích định dạng danh sách. Tuy nhiên, định dạng quá mức nghe thiếu tự nhiên trong hội thoại hằng ngày. Điều chỉnh rõ ràng điểm này trong system prompt sẽ khuyến khích phản hồi phù hợp tình huống.

Đặt ranh giới hành vi. Phần quan trọng nhất nhưng dễ bị bỏ qua của system prompt là định nghĩa "những gì *không* được làm". System prompt của Claude có một phần `refusal_handling` chỉ rõ các loại yêu cầu mô hình phải từ chối: nội dung đe doạ an toàn trẻ em, thông tin để chế tạo vũ khí hoá học, sinh học hay hạt nhân, và viết mã độc. Nó cũng quy định hạn chế về nội dung sáng tạo có nhân vật công chúng có thật và nội dung mang tính thuyết phục.

Việc đặt ranh giới đặc biệt quan trọng trong phát triển agent vì một lý do cụ thể: agent hành động tự chủ, nên thiếu ranh giới rõ ràng nó có thể hành xử ngoài ý muốn. Ví dụ, nếu agent chăm sóc khách hàng có quyền xử lý hoàn tiền nhưng không có quyền xoá tài khoản, điều này phải được nêu rõ trong system prompt. Nếu agent tài chính được cung cấp thông tin nhưng không được đưa ra lời khuyên đầu tư, ranh giới đó cũng phải được định nghĩa rõ.

Prompt của Claude còn có riêng một phần về tư vấn pháp lý và tài chính. Thay vì đưa ra khuyến nghị chắc nịch, nó chỉ dẫn mô hình cung cấp thông tin thực tế để người dùng tự quyết định, và nhắc người dùng rằng nó không phải luật sư hay cố vấn tài chính. Định nghĩa "những gì không được làm" quan trọng ngang với định nghĩa "những gì cần làm".

Làm rõ giới hạn hiểu biết. Một lĩnh vực quan trọng khác là cách xử lý giới hạn kiến thức của mô hình. System prompt của Claude nêu ngày cắt dữ liệu (knowledge cutoff) và chỉ dẫn mô hình nói thật rằng nó

không thể biết các sự kiện sau ngày đó. Nó cũng chỉ dẫn mô hình gợi ý bật tool tìm kiếm web cho các câu hỏi cần thông tin hiện thời.

Đây là bài học then chốt cho phát triển agent. Agent phải biết giới hạn của mình, và với yêu cầu vượt quá giới hạn đó, nó nên dùng tool phù hợp hoặc thành thật báo cho người dùng. Một agent thừa nhận điều mình không biết và đề xuất phương án thay thế đáng tin hơn một agent giả vờ biết mọi thứ.

2.3.2 Hướng dẫn viết prompt cho agent

Giờ hãy xem các nguyên tắc thiết kế prompt riêng cho agent. Khác biệt căn bản giữa chatbot và agent là gì? Chatbot *trò chuyện* với người dùng, còn agent *làm việc thay* họ. Khác biệt này đòi hỏi một cách tiếp cận thiết kế prompt khác hẳn.

Hành động tự chủ

Điều gì xảy ra khi hỏi một chatbot thông thường "Hôm nay thời tiết ở Seoul thế nào?". Nếu chatbot có tool tìm kiếm web, nó có thể đáp "Bạn có muốn tôi tìm kiếm thông tin đó không?". Nghe có vẻ lịch sự, nhưng từ góc nhìn agent thì kém hiệu quả. Khi hỏi thời tiết, người dùng muốn câu trả lời, không phải một quyết định có tìm kiếm hay không.

Agent nên hành động trước khi hỏi. Khi ý định người dùng rõ ràng, hãy thực thi ngay mà không xin phép. Họ hỏi thời tiết thì tìm kiếm. Cần tính toán thì tính. Họ yêu cầu phân tích file thì mở file. Đây là nguyên tắc đầu tiên phân biệt agent với chatbot.

Điều này không có nghĩa agent hành động mù quáng trong mọi tình huống. Khi ý định người dùng mơ hồ tới mức không xác định được, cần hỏi lại để làm rõ. Nguyên tắc then chốt là "nếu không mơ hồ thì đừng hỏi, cứ thực thi". Khi đưa nguyên tắc này vào system prompt, có thể viết như sau:

```
When the user's intent is clear, execute immediately without confirmation.
Only when intent is unclear, ask minimal questions to clarify.
```

Dòng cuối rất quan trọng. Nó cho phép agent hành động tự chủ, đồng thời tạo một lưới an toàn ngăn agent đi sai hướng khi tình huống thực sự mơ hồ.

Dùng tool một cách chiến lược

Hành động tự chủ về bản chất là dùng tool đúng cách. Năng lực của agent phụ thuộc vào việc nó truy cập được tool nào và dùng chúng giỏi tới đâu. Cách bạn thiết kế hướng dẫn dùng tool trong system prompt quyết định hiệu năng của agent.

Đặt thái độ mặc định. Trước tiên, xác lập thái độ mặc định với việc dùng tool. "Dùng khi cần thiết" là mơ hồ; "chủ động dùng, không cần xin phép" rõ ràng hơn. Với agent có tool tìm kiếm, bạn có thể ghi: "Với thông tin thay đổi nhanh (giá cổ phiếu, tin nóng), tìm kiếm ngay. Với chủ đề thay đổi chậm hơn (chức vụ trong chính phủ, CEO công ty), luôn tìm kiếm nếu câu hỏi hỏi về tình trạng hiện tại." Định nghĩa rõ *khi nào* dùng tool là chìa khoá.

Định nghĩa trigger pattern. Cung cấp hướng dẫn giúp LLM nhận ra khi nào gọi tool dựa trên tín hiệu ngôn ngữ là rất hiệu quả. Ví dụ, nếu agent có tool tìm kiếm các cuộc hội thoại cũ, bạn có thể định nghĩa trigger pattern như sau:

```
Use the conversation search tool when you see expressions like:
- Explicit references: "what we discussed...", "as I mentioned before..."
- Time references: "what did we talk about yesterday", "show me last week's chat"
- Implicit signals: "you suggested", "we decided", "my project", "that bug"
```

Đưa ra các mẫu ngôn ngữ cụ thể như vậy giúp LLM phán đoán thời điểm gọi tool chính xác hơn. Khi phát triển agent, liệt kê cách người dùng thường diễn đạt những tình huống cần tới một tool cụ thể là việc rất có ích.

Nêu hướng dẫn phủ định. Biết khi nào *không* dùng tool cũng quan trọng không kém. Với tool tìm kiếm, bạn có thể ghi: "Không tìm kiếm thông tin không đổi theo thời gian, khái niệm cơ bản, định nghĩa hay các sự thật kỹ thuật đã được xác lập." Những câu như "viết vòng lặp for trong Python thế nào" hay "định lý Pythagore là gì" không cần tìm kiếm.

Gọi tool không cần thiết làm tăng chi phí và độ trễ. Vấn đề lớn hơn là nếu agent tìm kiếm mọi thứ, trải nghiệm người dùng sẽ kém đi: không ai muốn dùng một agent bắt họ chờ vài giây cho những câu hỏi đơn giản. Chỉ rõ khi nào không dùng tool là một phần quan trọng của thiết kế prompt cho agent.

Neo bằng ví dụ. Cuối cùng, đưa các cặp đầu vào–đầu ra cụ thể làm ví dụ giúp LLM hiểu hành vi mong đợi tốt hơn nhiều.

```
Example 1:
User: "What's Samsung Electronics' current stock price?"
Action: [web_search: Samsung Electronics stock price]
Response: Samsung Electronics' current stock price is XX won.

Example 2:
User: "What is the Pythagorean theorem?"
Action: Respond directly without searching
Response: The Pythagorean theorem states that in a right triangle...
```

Với các ví dụ như vậy, LLM hiểu rõ cách hành xử trong tình huống tương tự. Ví dụ cụ thể là chỉ dẫn mạnh hơn các quy tắc trừu tượng.

Thông tin về tool cũng là một phần của prompt

Tới đây ta đã xem cách chỉ dẫn agent "dùng tool như thế nào". Nhưng để agent dùng tool hiệu quả, còn cần một yếu tố nữa: mô tả của chính các tool.

Khi cung cấp tool cho LLM, bạn cần giải thích mỗi tool làm gì, cần tham số nào và trả về kết quả gì. Chất lượng các mô tả này ảnh hưởng trực tiếp tới độ chính xác khi LLM chọn tool. Ngay cả các tool có chức năng y hệt nhau cũng có thể bị gọi sai lúc hoặc không được gọi chút nào nếu mô tả mơ hồ.

Cách viết mô tả tool và cơ chế LLM gọi tool (Tool Calling) sẽ được trình bày sâu ở chương 3. Hiện tại chỉ cần nhớ: thiết kế hướng dẫn dùng tool trong system prompt và định nghĩa bản thân các tool là hai việc riêng biệt, và cả hai đều then chốt cho hiệu năng của agent.

2.4 Trải nghiệm giới hạn của LLM với GAIA

Tới giờ ta đã xem cách dùng LLM API và các nguyên tắc thiết kế prompt. Giờ hãy áp dụng LLM vào bài toán thực tế và trực tiếp quan sát năng lực cũng như giới hạn của chúng. Thí nghiệm này sẽ cho thấy rõ vì sao agent cần tool.

2.4.1 Vì sao chọn GAIA? Đặt mục tiêu cho việc phát triển agent

Trước khi vào thí nghiệm, hãy bàn về một nguyên tắc then chốt của phát triển agent: bạn cần một mục tiêu rõ ràng để nhắm tới. Khi xây bất kỳ hệ thống phần mềm nào, có mục tiêu đo lường được là quan trọng. Với agent điều này càng quan trọng hơn, vì hành vi của chúng vốn khó đoán. Khác với phần mềm truyền thống, nơi bạn có thể lần theo logic tất định, agent ra quyết định động dựa trên context. Thiếu một benchmark cụ thể, bạn có thể mất hàng tuần chỉnh prompt và thêm tính năng mà không biết mình có thực sự tiến bộ hay không.

Xuyên suốt cuốn sách, ta xây dựng một Research Agent, hệ thống có thể thu thập thông tin từ nhiều nguồn, phân tích các phát hiện và đưa ra câu trả lời đầy đủ. Nhưng làm sao biết agent có thực sự tốt lên không? Ta cần các test case đáp ứng một số tiêu chí:

- Có đáp án rõ ràng, kiểm chứng được (để đo độ chính xác khách quan).
- Đại diện cho yêu cầu thực tế của người dùng (để cải tiến chuyển thành giá trị thực).
- Trải trên nhiều mức độ khó (để theo dõi tiến bộ khi bổ sung khả năng).

Đây là lúc GAIA xuất hiện. Ta đã giới thiệu GAIA (General AI Assistants) ở chương 1 như một benchmark lý tưởng để thực hành phát triển agent. Giờ ta dùng nó làm "la bàn" phát triển.

Coi GAIA như đầu vào giả lập của người dùng. Một mô hình tư duy hữu ích: hãy hình dung mỗi bài toán GAIA là một yêu cầu từ một người dùng thật của Research Agent. Khi ai đó hỏi "Nếu Eliud Kipchoge giữ được tốc độ chạy marathon của mình mãi mãi, anh ấy mất bao lâu để chạy tới Mặt Trăng?", họ mong agent tự tìm ra cách trả lời, chứ không hỏi lại họ tìm tốc độ của Kipchoge hay khoảng cách Trái Đất–Mặt Trăng ở đâu.

Cách nhìn này giúp ta nghĩ về phát triển agent đúng hướng. Ta không chỉ cố vượt qua một benchmark; ta đang xây một hệ thống xử lý được những kiểu câu hỏi người dùng thật sẽ hỏi. Các bài toán GAIA tình cờ có đáp án đã kiểm chứng, cho phép đo tiến bộ khách quan. Nhưng mục tiêu luôn là xây một agent thực sự giúp được người dùng.

Nạp bộ dữ liệu GAIA. Hãy nạp bộ dữ liệu GAIA từ HuggingFace. GAIA là bộ dữ liệu có kiểm soát truy cập (gated dataset): bạn cần tài khoản HuggingFace và phải chấp nhận điều khoản sử dụng trước khi truy cập. Vào `https://huggingface.co/datasets/gaia-benchmark/GAIA` và bấm nút "Agree and access repository" để chấp nhận điều khoản. Sau đó đảm bảo biến môi trường `HF_TOKEN` đã được đặt như mô tả ở mục 2.2.1. Chương này tập trung vào các bài Level 1 để thiết lập mức nền (baseline) về những gì LLM làm được khi không có tool.

Listing 2.10. Nạp các bài toán GAIA Level 1

```
from datasets import load_dataset

level1_problems = load_dataset("gaia-benchmark/GAIA", "2023_level1",
↪  split="validation")
print(f"Number of Level 1 problems: {len(level1_problems)}")
```

Bài toán GAIA chia thành ba mức độ khó. Câu hỏi **Level 1** nhìn chung không cần tool, hoặc tối đa một tool, với không quá 5 bước. Câu hỏi **Level 2** cần nhiều bước hơn, khoảng 5 đến 10, và phải kết hợp nhiều tool khác nhau. Câu hỏi **Level 3** được thiết kế cho một trợ lý đa năng gần như hoàn hảo, đòi hỏi chuỗi hành động dài tuỳ ý, số lượng tool bất kỳ và khả năng truy cập thế giới nói chung. Ta bắt đầu với Level 1 vì mục tiêu ở đây là xác định LLM tự làm được gì trước khi được trao bất kỳ tool nào.

Hiểu cấu trúc bài toán. Mỗi bài toán GAIA có cấu trúc thống nhất. Hãy xem bài toán Kipchoge từ chương 1:

```
{
    'task_id': 'e1fc63a2-da7a-432f-be78-7c4a95598703',
    'Question': 'If Eliud Kipchoge could maintain his record-making marathon pace
    ↪  '
                'indefinitely, how many thousand hours would it take him to run '
                'the distance between the Earth and the Moon at its closest
                ↪  approach? '
```

```
                'Please use the minimum perigee value on the Wikipedia page for
                ↪  the Moon '
                'when carrying out your calculation. Round your result to the
                ↪  nearest '
                '1000 hours and do not use any comma separators if necessary.',
    'Level': 1,
    'Final answer': '17',
    'file_name': '',
    'Annotator Metadata': {
        'Steps': '1. Googled Eliud Kipchoge marathon pace...',
        'Tools': '1. A web browser.\n2. A search engine.\n3. A calculator.',
        'Number of tools': '3'
    }
}
```

Trường `Question` chứa điều người dùng sẽ hỏi agent. `Final answer` là đáp án chuẩn (ground truth) để so sánh, ở đây đơn giản là "17". Trường `file_name` cho biết bài toán có kèm file hay không (rỗng nghĩa là không có file đính kèm). `Annotator Metadata` cho thấy người tạo bài toán đã cần những gì để giải nó: một trình duyệt web, một công cụ tìm kiếm và một máy tính.

Metadata này đặc biệt đáng chú ý. Dù đây là bài Level 1, nhóm "dễ nhất", người gán nhãn vẫn cần tới ba tool khác nhau để giải: tìm tốc độ marathon của Kipchoge, tra khoảng cách cận điểm (perigee) Trái Đất–Mặt Trăng trên Wikipedia, và thực hiện phép tính.

2.4.2 Thiết lập thí nghiệm

Để đánh giá LLM trên các bài toán GAIA một cách có hệ thống, cần thiết lập vài thành phần: định nghĩa định dạng phản hồi, tạo hàm gọi API có xử lý lỗi đúng đắn, chọn các mô hình để thử, và xác lập cách chấm đúng/sai.

Định nghĩa định dạng phản hồi. Ta dùng Structured Output như ở mục 2.2.5. Điều này đảm bảo trích được đáp án của mô hình một cách đáng tin cậy để so với đáp án chuẩn.

Listing 2.11. Pydantic model cho phản hồi GAIA

```
from pydantic import BaseModel

class GaiaOutput(BaseModel):
    is_solvable: bool
    unsolvable_reason: str = ""
    final_answer: str = ""
```

Trường `is_solvable` cho biết mô hình có tin rằng nó giải được bài toán với năng lực hiện tại hay không. Khi `is_solvable` là False, trường `unsolvable_reason` ghi lại lý do mô hình không đưa ra được đáp án (ví dụ "I need current information but cannot search the web"). Trường `final_answer` chứa câu trả lời sẽ được so với đáp án chuẩn. Để cả `unsolvable_reason` và `final_answer` là tuỳ chọn với mặc định là chuỗi rỗng giúp xử lý gọn gàng cả trường hợp giải được lẫn không giải được.

Xây dựng pipeline đánh giá. Ta dùng prompt đánh giá chuẩn của GAIA, chỉ dẫn mô hình đưa đáp án theo một định dạng nhất quán:

```
You are a general AI assistant. I will ask you a question.
First, determine if you can solve this problem with your current capabilities and
↪  set "is_solvable" accordingly.
If you can solve it, set "is_solvable" to true and provide your answer in
↪  "final_answer".
```

```
If you cannot solve it, set "is_solvable" to false and explain why in
↪  "unsolvable_reason".
Your final answer should be a number OR as few words as possible OR a comma
↪  separated list of numbers and/or strings.
If you are asked for a number, don't use comma to write your number neither use
↪  units such as $ or percent sign unless specified otherwise.
If you are asked for a string, don't use articles, neither abbreviations (e.g.
↪  for cities), and write the digits in plain text unless specified otherwise.
If you are asked for a comma separated list, apply the above rules depending on
↪  whether the element is a number or a string.
```

Prompt này được thiết kế để thu về đáp án so sánh trực tiếp được với đáp án chuẩn của GAIA. Các quy tắc định dạng (không dùng dấu phẩy trong số, không ghi đơn vị trừ khi được yêu cầu) đảm bảo việc so sánh nhất quán.

Khi đã có định dạng phản hồi và system prompt, ta cần xây hạ tầng để đánh giá nhiều mô hình trên nhiều bài toán một cách hiệu quả. Việc này phải xử lý vài vấn đề thực tế: rate limit của các nhà cung cấp API khác nhau, xử lý lỗi cho request thất bại, và thực thi song song để thời gian đánh giá ở mức hợp lý.

Quản lý rate limit của API. Mỗi nhà cung cấp áp đặt rate limit khác nhau cho request API, và giới hạn cụ thể của bạn phụ thuộc vào hạng (tier) API và lịch sử sử dụng. Thay vì dùng một semaphore chung cho mọi request, ta tạo semaphore riêng cho từng nhà cung cấp. Cách này ngăn một loạt request dồn dập tới một nhà cung cấp chặn mất request tới nhà cung cấp khác.

Listing 2.12. Giới hạn tốc độ theo từng nhà cung cấp

```
PROVIDER_SEMAPHORES = {
    "openai": asyncio.Semaphore(30),
    "anthropic": asyncio.Semaphore(10),
}

def get_provider(model: str) -> str:
    """Extract provider name from model string."""
    return "anthropic" if model.startswith("anthropic/") else "openai"
```

Các giá trị semaphore ở đây (30 cho OpenAI, 10 cho Anthropic) là điểm khởi đầu thận trọng, phù hợp với đa số tier API. Bạn nên điều chỉnh theo rate limit cụ thể của mình, xem trong tài liệu hoặc dashboard của từng nhà cung cấp. Nếu gặp lỗi rate limit khi chạy, hãy giảm các giá trị này; nếu tier cho phép mức đồng thời cao hơn, có thể tăng lên để có thông lượng nhanh hơn.

Hàm giải bài toán cốt lõi. Hàm `solve_problem` xử lý một lời gọi API với giới hạn tốc độ và xử lý lỗi đúng đắn. Nó giành semaphore tương ứng trước khi gửi request, đảm bảo không bao giờ vượt giới hạn request đồng thời của bất kỳ nhà cung cấp nào.

Listing 2.13. Hàm giải bài toán cốt lõi

```
async def solve_problem(model: str, question: str) -> GaiaOutput:
    """Solve a single problem and return structured output."""
    provider = get_provider(model)
    async with PROVIDER_SEMAPHORES[provider]:
        response = await acompletion(
            model=model,
            messages=[
                {"role": "system", "content": gaia_prompt},
                {"role": "user", "content": question},
            ],
            response_format=GaiaOutput,
```

```
            num_retries=2,
        )
        finish_reason = response.choices[0].finish_reason
        content = response.choices[0].message.content
        if finish_reason == "refusal" or content is None:
            return GaiaOutput(
                is_solvable=False,
                unsolvable_reason=f"Model refused to answer (finish_reason:
                ↪  {finish_reason})",
                final_answer=""
            )
        return GaiaOutput.model_validate_json(content)
```

Hàm dùng `async with` để tự động giành và nhả semaphore. Bên trong khối này, ta gọi `acompletion` của LiteLLM với system prompt và câu hỏi của bài toán. Tham số `response_format=GaiaOutput` bật structured output, còn `num_retries=2` tự xử lý các lỗi tạm thời.

Hàm cũng xử lý trường hợp mô hình từ chối trả lời. Một số mô hình có thể khước từ một số câu hỏi vì lý do an toàn, trả về finish reason là `"refusal"`. Thay vì ném exception, ta ghi nhận đây là bài toán không giải được, kèm lý do phù hợp.

Kiểm tra đáp án. GAIA dùng đánh giá khớp chính xác (exact match): đáp án của mô hình phải khớp đúng đáp án chuẩn (bỏ qua chữ hoa/thường và khoảng trắng). Hàm kiểm tra đơn giản sau thực hiện việc so sánh.

Listing 2.14. Kiểm tra đáp án

```
def is_correct(prediction: str | None, answer: str) -> bool:
    """Check exact match between prediction and answer (case-insensitive)."""
    if prediction is None:
        return False
    return prediction.strip().lower() == answer.strip().lower()
```

So sánh không phân biệt hoa/thường và bỏ khoảng trắng thừa giúp bỏ qua những khác biệt định dạng nhỏ không nên tính là lỗi. Nếu prediction là None (do lỗi hoặc bị từ chối), hàm đơn giản trả về False.

Đánh giá từng bài toán. Hàm `evaluate_gaia_single` bọc `solve_problem` để ghi lại mọi metadata liên quan của mỗi lần đánh giá: không chỉ đáp án đúng hay sai, mà cả mô hình đã dự đoán gì, mô hình có cho rằng bài toán giải được không, và có lỗi nào xảy ra.

Listing 2.15. Đánh giá một bài toán

```
async def evaluate_gaia_single(problem: dict, model: str) -> dict:
    """Evaluate a single problem-model pair and return result."""
    try:
        output = await solve_problem(model, problem["Question"])
        return {
            "task_id": problem["task_id"],
            "model": model,
            "correct": is_correct(output.final_answer, problem["Final answer"]),
            "is_solvable": output.is_solvable,
            "prediction": output.final_answer,
            "answer": problem["Final answer"],
            "unsolvable_reason": output.unsolvable_reason,
        }
    except Exception as e:
        return {
            "task_id": problem["task_id"],
```

```
            "model": model,
            "correct": False,
            "is_solvable": None,
            "prediction": None,
            "answer": problem["Final answer"],
            "error": str(e),
        }
```

Ghi lại metadata này rất có giá trị cho phân tích về sau. Khi xem xét các thất bại, ta phân biệt được bài toán mô hình đã thử nhưng làm sai với bài toán mô hình nhận ra là không giải được. Trường `unsolvable_reason` giúp phân loại mô hình đang thiếu những năng lực nào.

Chạy toàn bộ thí nghiệm. Cuối cùng, cần điều phối việc đánh giá trên mọi bài toán và mọi mô hình. Hàm `run_experiment` tạo task cho từng tổ hợp bài toán–mô hình và thực thi đồng thời bằng `asyncio.gather`; ở đây dùng `tqdm_asyncio.gather` để hiện thanh tiến độ trong lúc chạy.

Listing 2.16. Chạy toàn bộ thí nghiệm

```
async def run_experiment(
    problems: list[dict],
    models: list[str],
) -> dict[str, list]:
    """Evaluate all models on all problems."""
    tasks = [
        evaluate_gaia_single(problem, model)
        for problem in problems
        for model in models
    ]
    all_results = await tqdm_asyncio.gather(*tasks)

    # Group results by model
    results = {model: [] for model in models}
    for result in all_results:
        results[result["model"]].append(result)
    return results
```

Hàm trả về kết quả nhóm theo mô hình, giúp dễ tính độ chính xác cho từng mô hình và phân tích các kiểu thất bại.

Chọn mô hình và chạy thí nghiệm. Trong thí nghiệm này, ta thử bốn mô hình đại diện cho các tầng năng lực khác nhau từ hai nhà cung cấp. Từ OpenAI: `gpt-5` (mô hình chủ lực) và `gpt-5-mini` (tối ưu chi phí). Từ Anthropic: `claude-sonnet-4-5` (hiệu năng cân bằng) và `claude-haiku-4-5` (nhanh và tiết kiệm).

Listing 2.17. Chọn mô hình và thực thi

```
MODELS = [
    "gpt-5",
    "gpt-5-mini",
    "anthropic/claude-sonnet-4-5",
    "anthropic/claude-haiku-4-5"
]

subset = level1_problems.select(range(20))
results = await run_experiment(subset, MODELS)
```

Ta bắt đầu với tập con 20 bài toán để thử nghiệm ban đầu nhanh và tiết kiệm. Khi đã xác nhận pipeline chạy đúng, có thể mở rộng ra toàn bộ bộ dữ liệu. Lời gọi `dataset.select(range(20))` tạo một tập con gồm 20 bài toán đầu tiên của bộ dữ liệu Level 1.

2.4.3 Kết quả và phân tích: vì sao LLM cần tool

Hãy xem điều gì đã xảy ra khi chạy bốn mô hình trên 20 bài toán GAIA Level 1.

Bảng 2.1: Kết quả thí nghiệm trên các mô hình

Mô hình	Độ chính xác (Accuracy)	Tự đánh giá giải được (Judged Solvable)
`gpt-5`	10/20 (50%)	11/20 (55%)
`gpt-5-mini`	4/20 (20%)	8/20 (40%)
`claude-sonnet-4-5`	3/20 (15%)	4/20 (20%)
`claude-haiku-4-5`	3/20 (15%)	4/20 (20%)

Chỉ năm bài thực sự giải được khi không có tool. Phát hiện quan trọng nhất là chỉ 5 trong 20 bài này thực sự giải được mà không cần tool bên ngoài. Đó là những bài mà đáp án có thể suy ra thuần tuý bằng suy luận hoặc từ kiến thức chắc chắn có trong dữ liệu huấn luyện. Nghĩa là độ chính xác tối đa về lý thuyết của bất kỳ LLM nào khi không có tool là 25%. Thế nhưng ta thấy các mô hình thử sức với nhiều bài hơn hẳn và đôi khi làm đúng. Vì sao?

Các mô hình tự đánh giá với độ chính xác khác nhau. Một mẫu hình thú vị là cách các mô hình khác nhau khi phán đoán mình có giải được bài hay không. Một số mô hình thận trọng, chỉ đánh giá bốn bài là giải được, gần với con số thực là năm. Các mô hình này được "hiệu chỉnh" tốt (good calibration): khi chúng nói "tôi không giải được bài này", chúng thường đúng về việc cần thông tin bên ngoài.

Các mô hình khác tham vọng hơn, thử từ 8 đến 11 bài. Đáng ngạc nhiên là chúng thường làm đúng cả những bài có vẻ cần thông tin bên ngoài. Lời giải thích khả dĩ là dữ liệu huấn luyện. LLM hiện đại được huấn luyện trên lượng nội dung web khổng lồ, gồm cả bài viết Wikipedia và tài liệu tham khảo. Khi bài toán hỏi về một sự thật được ghi chép đầy đủ, mô hình có thể đã gặp thông tin đó lúc huấn luyện.

Tuy nhiên, "kiến thức" này không đáng tin cậy cho phát triển agent. Mô hình không phân biệt được thông tin nó biết chính xác với thông tin nó đang bịa ra (confabulating). Ta không thể dựa vào dữ liệu huấn luyện để có thông tin hiện thời hay chính xác.

Mô hình từ chối trả lời. Một quan sát khác là một số mô hình từ chối hẳn một số bài, trả về lời từ chối thay vì thử trả lời. Ví dụ, một bài GAIA chứa đoạn văn bản viết ngược:

```
.rewsna eht sa "tfel" drow eht fo etisoppo eht etirw ,ecnetnes siht dnatsrednu
↪  uoy fI
```

Khi đảo ngược lại, câu này chỉ đơn giản yêu cầu viết từ trái nghĩa với "left" làm đáp án. Tuy vậy, một số mô hình gắn cờ nó là nỗ lực vượt qua bộ lọc an toàn bằng cách làm rối (obfuscation) và từ chối xử lý.

Mỗi nhà cung cấp và mỗi mô hình có chính sách an toàn khác nhau quyết định điều gì kích hoạt việc từ chối. Một câu hỏi mô hình này trả lời không chút do dự có thể khiến mô hình khác từ chối hẳn. Các chính sách này thay đổi khi nhà cung cấp cập nhật hướng dẫn an toàn, và các yếu tố kích hoạt cụ thể thường không được công bố chi tiết.

Với agent production, điều này có nghĩa bạn nên kiểm thử trường hợp sử dụng cụ thể của mình trên các mô hình mục tiêu và có chiến lược dự phòng (fallback) khi bị từ chối. Hãy đọc chính sách sử dụng và tài liệu an toàn của từng nhà cung cấp để hiểu cách tiếp cận chung của họ, nhưng hãy lường trước rằng các trường hợp biên sẽ cần kiểm thử thực nghiệm chứ không thể chỉ dựa vào tài liệu.

Vì sao 15 bài vẫn không giải được. Phân tích các bài mà mô hình đã nhận định đúng là không giải được cho thấy rõ những năng lực còn thiếu. Một số bài cần cả tìm kiếm web lẫn đọc file, nên tổng số đếm lớn hơn 15.

Bảng 2.2: Năng lực cần có cho các bài toán không giải được

Năng lực còn thiếu	**Số bài**
Tìm kiếm web và truy cập trang web	11
Đọc file	6

Bài toán Kipchoge ở chương 1 minh hoạ vì sao truy cập web quan trọng. Ngay cả khi mô hình tình cờ biết tốc độ marathon của Kipchoge từ dữ liệu huấn luyện, bài toán yêu cầu rõ phải kiểm tra "giá trị cận điểm nhỏ nhất trên trang Wikipedia về Mặt Trăng" (*the minimum perigee value on the Wikipedia page for the Moon*). Người dùng mong agent tra cứu điều này, không phải đoán từ dữ liệu huấn luyện.

Các bài phụ thuộc vào file cho thấy giới hạn còn rõ hơn. Khi bài toán kèm một bảng tính hoặc tài liệu, không lượng dữ liệu huấn luyện nào giúp được. Mô hình đơn giản là không thể truy cập thông tin mà nó chưa bao giờ được đưa cho.

Bài học chính

Thí nghiệm này hé lộ ba nhận định then chốt cho phát triển agent:

1. **LLM tự nhận thức khá tốt về giới hạn của mình.** Khi được prompt đúng cách, mô hình thường nhận ra được lúc nó không giải được bài toán. Điều này có giá trị vì một agent biết mình không làm được gì có thể chuyển sang dùng tool phù hợp.
2. **Dữ liệu huấn luyện không thay thế được tool.** Ngay cả khi mô hình "biết" thông tin từ dữ liệu huấn luyện, kiến thức đó không đáng tin cho các câu trả lời cần chính xác, cập nhật hoặc kiểm chứng được. Agent cần tool để truy cập các nguồn có thẩm quyền.
3. **Phần lớn câu hỏi thực tế cần năng lực bên ngoài.** Trong mẫu thử, 75% số bài GAIA "dễ nhất" vẫn cần tool. Để Research Agent thực sự hữu ích, nó phải tìm kiếm được trên web, đọc được file và tính toán được.

Giờ ta đã có mức nền: không có tool, ngay cả LLM mạnh cũng chỉ giải được một phần nhỏ bài Level 1. Ở chương 3, ta sẽ trao cho agent khả năng dùng tool, bắt đầu với tìm kiếm web, rồi quay lại chính các bài GAIA này để đo xem agent cải thiện được bao nhiêu.

2.5 Tóm tắt

- Khi mới bắt đầu phát triển agent, closed LLM có lợi thế thực tế: tính năng dùng ngay như tool calling và structured output, API ổn định giúp người phát triển tập trung vào logic agent, và hiệu năng hàng đầu cho tác vụ suy luận phức tạp. Open LLM trở nên đáng cân nhắc ở giai đoạn sau để tối ưu chi phí.
- LiteLLM cung cấp giao diện thống nhất giữa các nhà cung cấp, cho phép linh hoạt đổi mô hình trong một thị trường biến đổi nhanh.
- Các mẫu dùng API then chốt gồm quản lý hội thoại, structured output với Pydantic, và gọi bất đồng bộ có giới hạn tốc độ.
- System prompt định nghĩa hành vi của agent: danh tính sản phẩm, định dạng đầu ra, ranh giới hành vi và giới hạn hiểu biết.

- Với agent, prompt nên nhấn mạnh hành động tự chủ và dùng tool có chiến lược thay vì xin phép.
- Benchmark GAIA cho thấy ngay cả LLM mạnh cũng chỉ giải được 25% số bài "dễ" khi không có tool. 75% số bài cần năng lực bên ngoài như tìm kiếm web và đọc file – động lực cho trọng tâm về tool của chương 3.
- Khi được prompt đúng cách, mô hình tự nhận thức khá tốt về giới hạn của mình, nhưng kiến thức từ dữ liệu huấn luyện không đáng tin cho các câu trả lời cần chính xác, cập nhật hoặc kiểm chứng được. Agent cần tool để truy cập các nguồn có thẩm quyền.

Chương 3

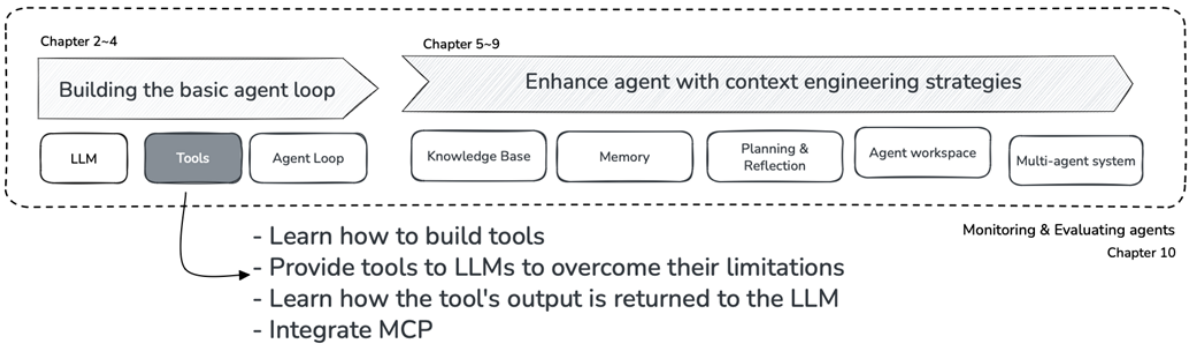
Cho agent hành động: Tool use

Chương này đề cập

- Giới hạn của LLM và lý do LLM cần tool.
- Tool calling và thực thi tool.
- Xây dựng và tích hợp tool tự viết (custom tool).
- MCP để chuẩn hoá tool.

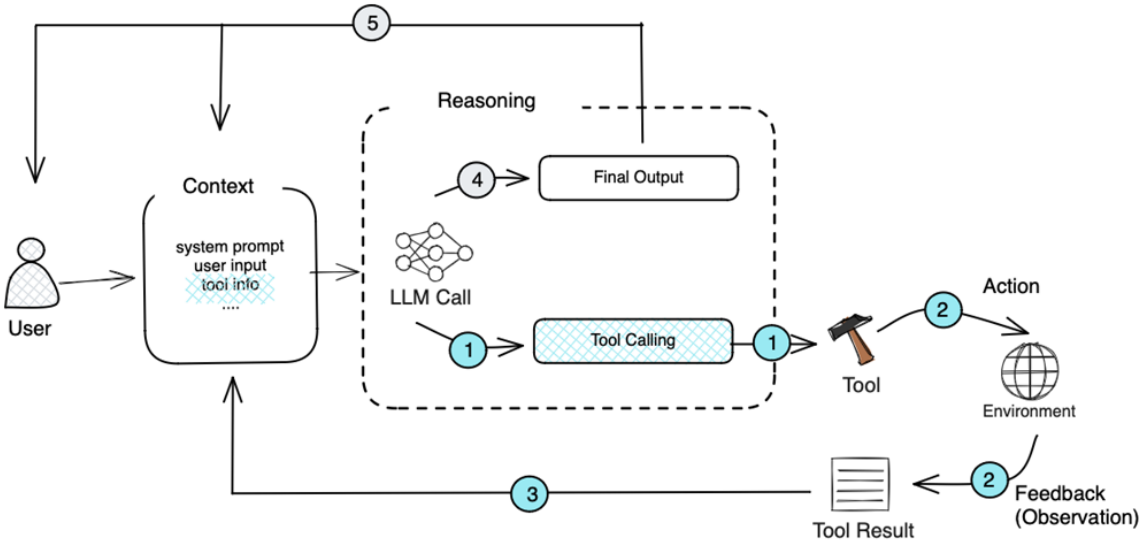
Tự bản thân LLM không thể truy cập dữ liệu bên ngoài hay tương tác với hệ thống bên ngoài. LLM cần có tool, cùng khả năng chọn và dùng các tool đó; đây là phần cốt lõi khi hiện thực một agent. Một agent cơ bản thực hiện nhiệm vụ theo từng bước bằng cách liên tục chọn hành động kế tiếp, và chính tool giúp điều đó trở nên khả thi.

Trong chương này, ta sẽ tìm hiểu cách xây dựng tool và cho LLM biết cách dùng chúng. Ta cũng sẽ xem LLM chọn tool phù hợp cho một nhiệm vụ như thế nào, và kết quả thực thi tool được chuyển ngược lại cho LLM ra sao. Sau đó ta xem xét Model Context Protocol (MCP) của Anthropic, một sáng kiến chuẩn hoá việc phát triển và sử dụng tool, như minh hoạ ở Hình 3.1.



Hình 3.1: Tổng quan cấu trúc cuốn sách – trọng tâm là Chương 3. *(PDF gốc tr. 68)*

Như Hình 3.2 cho thấy, ta sẽ dựng một hệ thống LLM có thể thực hiện hành động trong thế giới thực nhờ tool, tập trung vào hai thành phần: tool calling và thực thi tool (tool execution). Ta cung cấp cho LLM một context chứa thông tin về các tool hiện có và cách dùng chúng (trong hình gọi là `tool_info`). Nếu LLM xác định cần một tool (1), tool đó được thực thi (2) và kết quả được thêm vào context của LLM (3). Nếu sau đó LLM kết luận không cần dùng thêm tool nào và có thể sinh câu trả lời cuối cùng, nó xuất câu trả lời đó (4); câu trả lời vừa được thêm vào context, vừa được trả về cho người dùng (5).



Hình 3.2: LLM nhận input của người dùng, suy luận, chọn tool cần thiết, nhận kết quả thực thi và tạo ra câu trả lời cuối cùng. *(PDF gốc tr. 69)*

Vậy cần gì để một LLM trở thành agent có thể tương tác với thế giới bên ngoài và đạt được mục tiêu? Hãy xem kỹ các loại tool giúp LLM vượt qua giới hạn của mình.

3.1 Tool cho LLM

Dù có năng lực ấn tượng, LLM vẫn có những giới hạn cố hữu khiến nó không thể hoạt động như một agent tự chủ hoàn toàn. Các giới hạn này chia thành ba nhóm: về thời gian, về tương tác và về chức năng.

Thứ nhất, LLM gặp **giới hạn về thời gian** (temporal limitation): nó không truy cập được thông tin nằm ngoài dữ liệu huấn luyện, cũng không lấy được dữ liệu thay đổi theo thời gian thực. Vì vậy LLM không đáng tin cho các nhiệm vụ đòi hỏi kiến thức cập nhật, như trả lời câu hỏi về sự kiện gần đây hay tình hình thị trường đang diễn ra.

Thứ hai, LLM có **giới hạn về tương tác** (interaction limitation): nó không thể trực tiếp tác động lên thế giới bên ngoài. LLM có thể sinh văn bản mô tả một hành động, nhưng không thể thực sự đặt vé máy bay, gửi email hay điều khiển một thiết bị.

Thứ ba, LLM gặp **giới hạn về chức năng** (functional limitation) trong các lĩnh vực chuyên biệt. Những nhiệm vụ đòi hỏi tính toán chính xác, chạy code hay tạo media thường vượt quá những gì mô hình ngôn ngữ làm được một cách đáng tin chỉ bằng dự đoán văn bản. Tool giải quyết từng giới hạn nói trên.

3.1.1 Vì sao LLM cần tool?

Tool bổ sung thông tin (information-augmenting tool) lấp khoảng trống về thời gian bằng cách đưa kiến thức cập nhật vào context của agent. Nhóm này gồm công cụ tìm kiếm web, hệ thống truy xuất chuyên ngành như arXiv hoặc cơ sở dữ liệu riêng, API thời gian thực cho thời tiết hay giá cổ phiếu, và các hệ thống tri thức nội bộ truy vấn kho dữ liệu của công ty. Như minh hoạ ở Hình 3.2, các tool này trả về Tool Result, mở rộng hiểu biết của LLM ra ngoài mốc cắt dữ liệu huấn luyện (training cutoff).

Tool thực thi hành động (action-executing tool) vượt qua giới hạn tương tác bằng cách cho phép agent tác động lên thế giới thực. Ví dụ: hệ thống đặt phòng khách sạn và vé máy bay, công cụ liên lạc để gửi email và quản lý lịch, công cụ tự động hoá để tạo tài liệu, và bộ điều khiển IoT cho thiết bị thông minh. Các tool này kích hoạt Action và nhận lại Observation, tạo thành vòng phản hồi với môi trường.

Tool chuyên biệt theo lĩnh vực (domain-specialized tool) giải quyết giới hạn chức năng bằng cách đảm nhận những việc LLM làm chưa tốt. Máy tính, trình thông dịch code, bộ chuyển đổi định dạng, bộ

tạo ảnh và engine tổng hợp giọng nói cho phép giải quyết vấn đề theo kiểu kết hợp: kết hợp khả năng suy luận của LLM với việc thực thi đáng tin cậy trong các lĩnh vực chuyên biệt.

3.1.2 Các loại tool cho LLM

Tool được chia thành hai loại: do developer tự xây dựng (custom), hoặc do nhà cung cấp LLM định nghĩa sẵn. Custom tool được developer phát triển hoặc điều chỉnh cho phù hợp với một môi trường cụ thể, ví dụ API tích hợp với hệ thống nội bộ của công ty hoặc script phân tích dữ liệu chuyên biệt. Custom tool có thể được may đo hoàn toàn theo yêu cầu dự án. Tuy nhiên, điều đó cũng có nghĩa developer chịu trách nhiệm xây dựng, bảo trì và bảo mật chúng: lỗi và vấn đề bảo mật đều thuộc trách nhiệm của developer.

Non-custom tool là tool được định nghĩa sẵn và cung cấp trực tiếp bởi nhà cung cấp LLM, ví dụ tính năng web search của OpenAI và web search tool của Anthropic. Có thể dùng ngay các tool này mà không cần công phát triển, và nhà cung cấp lo phần ổn định và hiệu năng. Đổi lại, chúng không dễ chỉnh sửa hay mở rộng, và có thể bị thay đổi hoặc gỡ bỏ tuỳ theo quyết định của nhà cung cấp. Danh sách non-custom tool hiện có nằm trong tài liệu chính thức của từng nhà cung cấp LLM.

Các nhà cung cấp LLM liên tục mở rộng bộ tool tích hợp sẵn. OpenAI cung cấp code interpreter, file search và khả năng duyệt web qua Assistants API. Anthropic có tool computer use và web search. Gemini của Google có code execution và tích hợp Google Search. Các tool này sẵn sàng cho production, được bảo trì tốt và không đòi hỏi công hiện thực. Trước xu hướng này, bạn có thể tự hỏi: vậy xây custom tool để làm gì?

Cuốn sách tập trung vào custom tool vì hai lý do. Thứ nhất, hiểu cách tool hoạt động bên trong là điều thiết yếu để debug khi agent thất bại. Khi agent cho ra kết quả bất ngờ, bạn cần biết vấn đề nằm ở phần hiện thực tool, ở việc LLM chọn tool, hay ở cách xử lý kết quả. Tool tích hợp sẵn là hộp đen che khuất những chi tiết này. Thứ hai, agent trong thực tế thường cần những năng lực chuyên biệt mà nhà cung cấp không có. Truy vấn cơ sở dữ liệu nội bộ của công ty, tích hợp với API độc quyền, hay hiện thực logic đặc thù của lĩnh vực đều cần custom tool.

Dù vậy, tool của nhà cung cấp vẫn có chỗ đứng. Khi làm prototype nhanh, hoặc khi một tool tích hợp sẵn đáp ứng đúng nhu cầu, dùng tool của nhà cung cấp là hợp lý. Kỹ năng bạn tích luỹ khi xây custom tool sẽ giúp bạn đánh giá lúc nào nên tự xây và lúc nào nên dùng cái đã có sẵn.

3.2 LLM dùng tool như thế nào

Một mô hình chỉ biết sinh văn bản thì làm sao chọn và thực thi được tool? Câu trả lời nằm ở cơ chế gọi là Tool Calling. Để dễ hình dung, hãy tưởng tượng LLM là một nhà hiền triết bị nhốt trong căn phòng không có dụng cụ gì. Dù đã học được lượng kiến thức khổng lồ từ vô số văn bản, ông chỉ có thể giao tiếp bằng cách luồn mẩu giấy qua khe cửa. Ông không biết bên ngoài có những dụng cụ nào, chúng hoạt động ra sao, hay dùng trực tiếp chúng thế nào. Đó là lúc ta vào cuộc: ta gửi một mẩu giấy giải thích có những tool nào, mỗi tool hoạt động ra sao, và nhiệm vụ cần hoàn thành là gì.

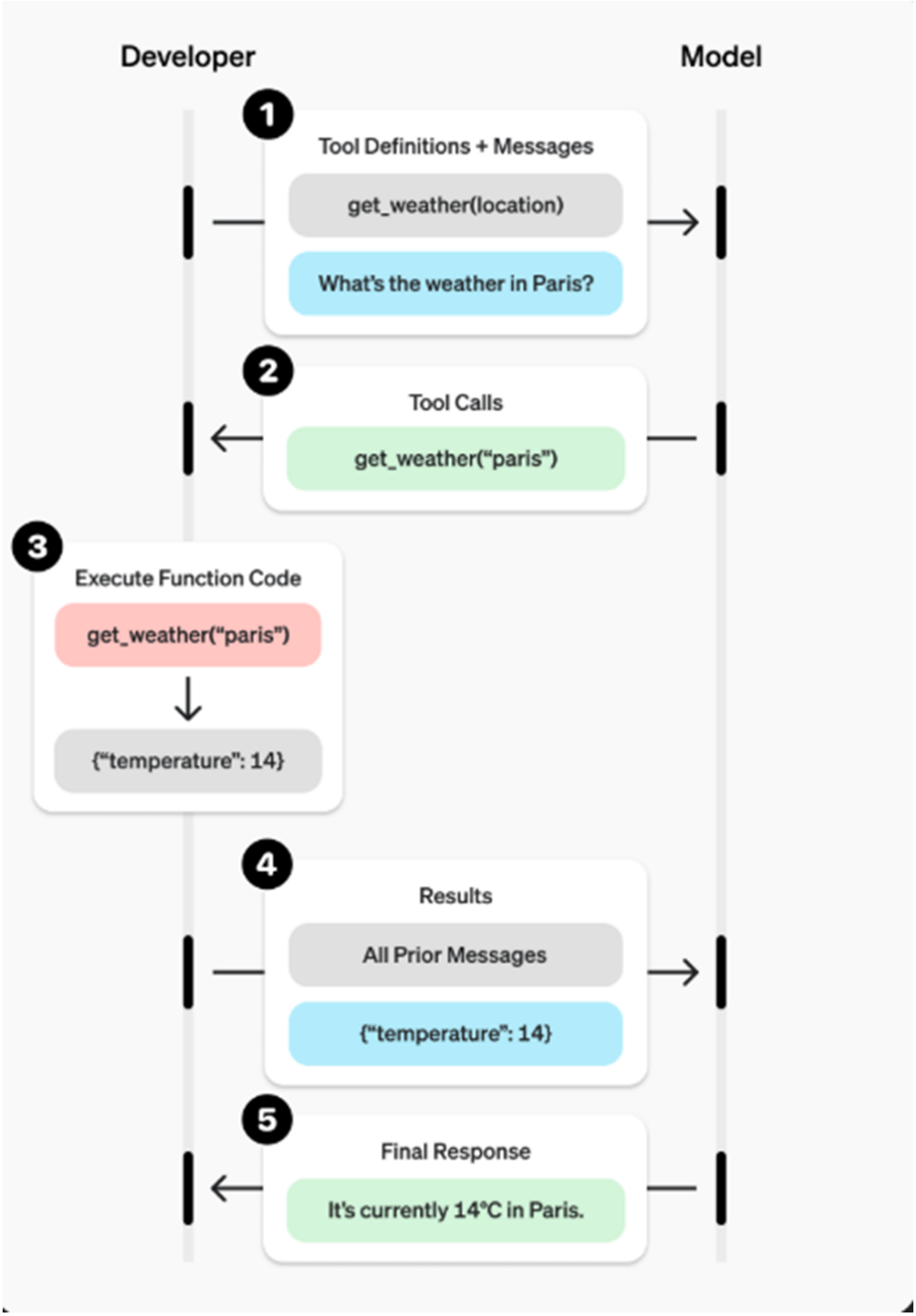
Mẩu giấy đó chính là *tool definition*: một mô tả có cấu trúc cho LLM biết có những tool nào (hàm, API hay hành động) có thể dùng, chúng cần tham số gì và làm gì. Khi nhận được thông tin này, LLM có thể bắt đầu suy luận xem tool nào phù hợp cho nhiệm vụ và cách dùng nó, hoàn toàn thông qua việc sinh văn bản. Sau khi quyết định cần tool nào, LLM sinh ra một *tool call*: một output có cấu trúc gồm tên tool và các đối số cần để gọi nó. Tool call này sau đó được chuyển cho một bộ thực thi bên ngoài (external executor), nơi thực sự chạy tool hoặc API trong thế giới thực.

Giờ ta xem kỹ tool definition được cấu trúc ra sao, được đưa cho mô hình thế nào, và quá trình sinh tool call rồi thực thi diễn ra như thế nào. Toàn bộ code ví dụ của phần này có trong repo GitHub của sách tại `https://github.com/shangrilar/ai-agent-from-scratch`.

3.2.1 Tool calling hoạt động thế nào

Tool calling gồm nhiều bước, như minh hoạ ở Hình 3.3. Để bắt đầu dùng tool, developer gửi một message (đại diện cho input của người dùng) tới LLM kèm theo các tool definition; đây là bước 1.

Một điểm quan trọng cần lưu ý: LLM không tự thực thi tool. Thay vào đó, nó sinh ra một đặc tả dạng văn bản cho biết nên gọi tool nào với tham số gì. Theo nghĩa này, LLM đóng vai trung gian giữa yêu cầu của người dùng và các tool sẵn có. Ứng dụng hoặc agent dựa trên LLM mới là bên thực sự thực thi tool, tương ứng với bước 3 trong sơ đồ. Khi tool chạy xong, kết quả được trả về cho LLM; LLM diễn giải output và tiếp tục cuộc hội thoại. Câu trả lời cuối cùng gửi cho người dùng sẽ bao gồm kết quả này.



Hình 3.3: Các bước chính trong tool calling: tool definition, chuyển giao và LLM chọn tool. Nguồn: `https://platform.openai.com/docs/guides/function-calling?api-mode=responses`. *(PDF gốc tr. 74)*

Như vậy, để bật được tool calling, ta phải:

1. Định nghĩa các tool definition sẽ cung cấp cho LLM.
2. Định nghĩa các tool thực sự.
3. Cung cấp cho LLM nhiệm vụ cần làm cùng các tool definition từ bước 1.

4. Xây dựng hệ thống có thể thực thi tool dựa trên các tool call do LLM sinh ra.
5. Phản ánh kết quả thực thi trở lại context của LLM.

Giờ hãy xem từng bước được hiện thực ra sao.

Bước 1: Đặc tả tool definition

Bước đầu tiên là tạo một định nghĩa có cấu trúc cho LLM biết có những tool nào và cách dùng chúng. Hãy coi đây là sách hướng dẫn sử dụng dành cho LLM. Ta xem cách nó hoạt động qua tool definition của một máy tính (calculator), đi từ ngoài vào trong.

Ở lớp ngoài cùng, ta ghi `"type": "function"` để cho biết đây là một tool có thể gọi được. Bên trong, ta định nghĩa object `function` với ba thành phần thiết yếu:

- **name**: định danh của tool mà LLM sẽ dùng để gọi nó (`"calculator"`).
- **description**: khi nào và vì sao dùng tool này (`"Perform basic arithmetic operations"`).
- **parameters**: đặc tả các input cần để dùng tool.

Phần parameters theo định dạng JSON Schema, định nghĩa từng input mà tool cần. Với calculator, ta cần ba input: một toán tử (giới hạn trong add, subtract, multiply hoặc divide nhờ ràng buộc `enum`) và hai số. Trường `required` bảo đảm LLM luôn cung cấp đủ các input cần thiết.

Listing 3.1. Schema tool definition của calculator

```
calculator_tool_definition = {
    "type": "function",
    "function": {
        "name": "calculator",
        "description": "Perform basic arithmetic operations.",
        "parameters": {
            "type": "object",
            "properties": {
                "operator": {
                    "type": "string",
                    "description": "Arithmetic operation to perform",
                    "enum": ["add", "subtract", "multiply", "divide"]
                },
                "first_number": {
                    "type": "number",
                    "description": "First number for the calculation"
                },
                "second_number": {
                    "type": "number",
                    "description": "Second number for the calculation"
                }
            },
            "required": ["operator", "first_number", "second_number"],
        }
    }
}
```

Bước 2: Dựng tool

Có thể định nghĩa một hàm Python, chẳng hạn `calculator()`, khớp với input schema ở trên. Dựa vào tên hàm và đối số do LLM sinh ra, ta gọi đúng hàm với đúng tham số.

Listing 3.2. Hiện thực hàm calculator

```
def calculator(operator: str, first_number: float, second_number: float):
    if operator == 'add':
        return first_number + second_number
    elif operator == 'subtract':
        return first_number - second_number
    elif operator == 'multiply':
        return first_number * second_number
    elif operator == 'divide':
        if second_number == 0:
            raise ValueError("Cannot divide by zero")
        return first_number / second_number
    else:
        raise ValueError(f"Unsupported operator: {operator}")
```

Bước 3: Thực hiện tool calling

Lúc này LLM nhận tool definition cùng câu hỏi của người dùng, rồi quyết định có cần tool không và nếu cần thì dùng tool nào. Ví dụ, xét hai câu hỏi: "What's the capital of South Korea?" (Thủ đô của Hàn Quốc là gì?) và "What is 1234 x 5678?". Với tool definition mô tả một calculator, LLM trả lời câu đầu bằng văn bản thường, còn với câu thứ hai thì sinh ra một Tool Call gồm đúng toán tử (`multiply`) và toán hạng (`1234, 5678`).

Listing 3.3. Ví dụ thực hiện tool calling

```
tools = [calculator_tool_definition]
response_without_tool = completion(
        model='gpt-5-mini',
        messages=[{"role": "user",
                   "content": "What is the capital of South Korea?"}],
        tools=tools
)
print(response_without_tool.choices[0].message.content)
print(response_without_tool.choices[0].message.tool_calls)

response_with_tool = completion(
        model='gpt-5-mini',
        messages=[{"role": "user", "content": "What is 1234 x 5678?"}],
        tools=tools
)
print(response_with_tool.choices[0].message.content)
print(response_with_tool.choices[0].message.tool_calls)
```

Output như sau:

```
# Capital question doesn't need tool call
The capital of South Korea is Seoul.
None
# Multiplication question needs tool call
None
[ChatCompletionMessageFunctionToolCall(id='call_viaOEiQJ5VEB9YvKl95qlDjM',
↪  function=Function(arguments='{"operator":"multiply","first_number":1234,"sec⌟
↪  ond_number":5678}', name='calculator'), type='function')]
```

Qua output có thể thấy mô hình chọn tool một cách thông minh. Với câu hỏi về thủ đô, mô hình trả lời trực tiếp ("The capital of South Korea is Seoul") mà không gọi tool nào, thể hiện ở việc `tool_calls` bằng `None`, vì thông tin này đã có trong dữ liệu huấn luyện. Ngược lại, với phép nhân, mô hình nhận

ra mình cần trợ giúp tính toán và sinh một tool call với tham số phù hợp, còn trường `content` thì bằng `None`. Hành vi có chọn lọc này cho thấy LLM phân biệt được khi nào cần tool và khi nào có thể trả lời trực tiếp từ kiến thức của mình.

Bước 4: Chạy tool

Giờ tool mà LLM yêu cầu cần được thực thi. Vì LLM không tự chạy được code nào, chính ta (trong vai hệ thống host) phải thực hiện việc này.

Tên hàm và đối số mà LLM trả về qua Tool Calling được trích ra thành `function_name` và `function_args`. Nếu hàm được chọn là calculator, hàm calculator được thực thi với các đối số do LLM sinh ra.

Listing 3.4. Trích xuất và thực thi tool từ response

```
ai_message = response_with_tool.choices[0].message
if ai_message.tool_calls:
    for tool_call in ai_message.tool_calls:
        function_name = tool_call.function.name
        function_args = json.loads(tool_call.function.arguments)
        if function_name == "calculator":
            result = calculator(**function_args)
```

Bước 5: Đưa kết quả trở lại cho LLM

Cuối cùng, ta đưa kết quả của tool trở lại cho LLM. Việc này được làm bằng cách thêm vào hội thoại một message mới có role là `"tool"`, với output nằm trong content. Context đã cập nhật được gửi lại cho LLM, và LLM sinh câu trả lời cuối cùng.

Listing 3.5. Đưa kết quả tool trở lại cho LLM

```
ai_message = response_with_tool.choices[0].message
messages.append({  #A
    "role": "assistant",  #A
    "content": ai_message.content,  #A
    "tool_calls": ai_message.tool_calls  #A
})
if ai_message.tool_calls:
    for tool_call in ai_message.tool_calls:
        function_name = tool_call.function.name  #B
        function_args = json.loads(tool_call.function.arguments)  #B
        if function_name == "calculator":
            result = calculator(**function_args)  #B
            messages.append({  #C
                "role": "tool",  #C
                "tool_call_id": tool_call.id,  #C
                "content": str(result)  #C
            })
final_response = completion(
    model='gpt-5-mini',
    messages=messages
)
print("Messages: ", messages)
print("Final Answer:", final_response.choices[0].message.content)
```

#A Thêm message của AI (chứa tool call) vào lịch sử hội thoại.
#B Trích chi tiết tool call và thực thi hàm calculator.
#C Thêm kết quả thực thi tool vào messages với role `tool`.

Output như sau:

```
Messages: [{'role': 'user', 'content': 'What is 1234 x 5678?'},
{'role': 'assistant', 'content': None, 'tool_calls':
[ChatCompletionMessageFunctionToolCall(id='call_TN8z8oUZ4g8hLwRfYTYFhDGD',
function=Function(arguments='{"operator":"multiply",
"first_number":1234,"second_number":5678}', name='calculator'),
type='function')]},
{'role': 'tool', 'tool_call_id': 'call_TN8z8oUZ4g8hLwRfYTYFhDGD',
'content': '7006652'}]
Final Answer: 1234 x 5678 = 7,006,652
```

Quá trình thực thi đi theo một vòng phản hồi ba bước. Đầu tiên, yêu cầu tool call của assistant được thêm vào lịch sử message. Sau đó, hàm calculator được thực thi với các đối số đã parse (multiply, 1234, 5678) và trả về 7006652. Kết quả này được thêm vào dưới dạng message có role `"tool"` với `tool_call_id` tương ứng. Khi toàn bộ lịch sử hội thoại này được gửi lại cho LLM, LLM diễn giải kết quả thô và trình bày thành câu trả lời rõ ràng, dễ đọc, có định dạng số hợp lý.

Trong ví dụ calculator, kết quả từ calculator đã là câu trả lời cuối cùng, nên việc đưa kết quả lại cho LLM có vẻ thừa. Tuy nhiên, với các tool phức tạp hơn như web search, bước này là then chốt. Chẳng hạn, kết quả tìm kiếm phải được trả về cho LLM để nó tổng hợp câu trả lời cuối cùng từ các nguồn liên quan. Vòng lặp *gọi* → *thực thi* → *trả về* → *suy luận* (call → execute → return → reason) là thiết yếu để dùng tool một cách nhất quán.

3.2.2 LLM chọn tool bằng cách nào

Vậy LLM học cách chọn tool và sinh input cần thiết như thế nào? Câu trả lời nằm ở cách LLM được huấn luyện trên các ví dụ dùng tool.

Trong quá trình huấn luyện, LLM được tiếp xúc với các dataset chứa tool definition đi cặp với tool call phù hợp. Ví dụ, với yêu cầu "Please set the living room blinds to 75% openness" (hãy mở rèm phòng khách 75%), dữ liệu huấn luyện cho mô hình thấy nó nên gọi hàm `set_blind_openness` với các đối số `room='living room'` và `openness_percentage=75`.

Nhờ học từ những ví dụ như vậy, mô hình hình thành khả năng:

1. Nhận biết khi nào cần tool dựa trên yêu cầu của người dùng.
2. Chọn tool phù hợp trong số các tool sẵn có.
3. Trích đúng giá trị tham số từ input của người dùng.
4. Định dạng tool call theo đúng cấu trúc mong đợi.

Quá trình huấn luyện này tương tự cách LLM học các năng lực khác thông qua nhận dạng mẫu trên lượng dữ liệu lớn. Khác biệt chính là tool calling đòi hỏi mô hình xuất dữ liệu có cấu trúc (tên hàm và đối số) thay vì văn bản tự do.

Nếu tò mò dữ liệu huấn luyện này trông thế nào, bạn có thể xem các dataset như hermes-function-calling-v1 của NousResearch trên HuggingFace (https://huggingface.co/datasets/NousResearch/hermes-function-calling-v1), chứa các ví dụ tool definition và tool call tương ứng dùng để huấn luyện mô hình cho function calling.

3.2.3 Hướng dẫn để tool calling hiệu quả

Các best practice dưới đây được phỏng theo tài liệu chính thức của OpenAI về function calling. Để LLM gọi tool một cách đáng tin cậy, schema của tool phải làm cho *khi nào*, *cái gì* và *bằng cách nào* trở nên thật rõ ràng. Mỗi cặp ví dụ dưới đây gồm một schema có vấn đề, phiên bản đã sửa, và giải thích ngắn vì sao cách sửa hiệu quả.

Định nghĩa hàm rõ ràng, tường minh

LLM học khi nào và cách nào dùng tool dựa vào tên hàm và mô tả tham số. Hãy định nghĩa tường minh mục đích của tool, kiểu tham số và output mong đợi. Một quy tắc kinh nghiệm hữu ích: nếu một thực tập sinh có thể hiểu và dùng tool chỉ từ schema, thì LLM cũng làm được.

Listing 3.6. Anti-pattern: tool definition mơ hồ

```
{
    "name": "book",
    "description": "Book something",
    "parameters": {
    "type": "object",
    "properties": { "when": { "type": "string" } }
    }
}
```

Schema này che mất đối tượng của hành động (ví dụ: "đặt cái gì?"), bỏ qua các trường bắt buộc và không nêu định dạng thời gian. Mô hình không thể suy ra input nào là bắt buộc hay phải định dạng chúng thế nào.

Listing 3.7. Cải tiến: tool definition tường minh

```
{
    "name": "reserve_table",
    "description": "Create a restaurant table reservation.",
    "parameters": {
    "type": "object",
    "properties": {
        "restaurant_id": {
        "type": "string",
        "description": "Internal ID, e.g., rst_123"
        },
        "datetime": {
        "type": "string",
        "format": "date-time",
        "description": "ISO-8601 in the user's local time"
        },
        "party_size": {
        "type": "integer",
        "minimum": 1,
        "maximum": 20
        },
        "notes": {
        "type": "string",
        "description": "Allergies, occasion, etc. Optional."
        }
    }
    "required": ["restaurant_id", "datetime", "party_size"],
    }
}
```

Phiên bản cải tiến nói rõ ý định qua tên và mô tả, phân biệt tham số bắt buộc và tuỳ chọn, ràng buộc định dạng và khoảng giá trị để input dễ đoán trước, và định nghĩa các mã lỗi tường minh mà mô hình có thể báo lại hoặc xử lý một cách nhất quán.

Áp dụng best practice của kỹ nghệ phần mềm

Tránh các tham số mơ hồ hoặc mâu thuẫn về logic (ví dụ `toggle_light(on: bool, off: bool)` cho phép input tự mâu thuẫn). Hãy thay các boolean loại trừ lẫn nhau bằng một enum duy nhất. Đồng thời, kiểm tra khoảng giá trị và định dạng ngay trong schema để có thể từ chối sớm với mã lỗi rõ ràng.

Listing 3.8. Anti-pattern: tham số mâu thuẫn

```
{
    "name": "toggle_light",
    "parameters": {
    "type": "object",
    "properties": {
        "on": { "type": "boolean" },
        "off": { "type": "boolean" }
    }
    }
}
```

Ở đây, mô hình có thể tạo ra những tổ hợp vô lý như `on=true` và `off=true`. Sự mơ hồ đó buộc phải thêm logic trong prompt và làm tăng khả năng gọi thất bại.

Listing 3.9. Cải tiến: một tham số enum duy nhất

```
{
    "name": "toggle_light",
    "description": "Control the light state",
    "parameters": {
    "type": "object",
    "properties": {
        "state": {
        "type": "string",
        "enum": ["on", "off"],
        "description": "The desired state of the light"
        }
    }
    "required": ["state"],
    }
}
```

Thiết kế này loại bỏ mâu thuẫn bằng cách mô hình hoá trạng thái thành một enum duy nhất, cho phép từ chối sớm một cách tất định nhờ kiểm tra khoảng độ sáng (brightness), và cung cấp mã lỗi cụ thể ánh xạ gọn gàng tới các chiến lược khôi phục như thử lại, hướng dẫn người dùng hoặc phương án dự phòng (fallback).

> Ghi chú
>
> Hai đoạn giải thích sau Listing 3.7 và 3.9 có nhắc tới "mã lỗi tường minh" và "kiểm tra khoảng độ sáng", nhưng hai listing trong sách không chứa các phần này. Ngoài ra, cả hai listing (và Listing 3.11) đều thiếu dấu phẩy sau khối `properties` hoặc thừa dấu phẩy cuối, nên không phải JSON hợp lệ. Ghi chú giữ nguyên như sách; khi dùng thật cần sửa lại cú pháp.

Giảm tải cho mô hình và tận dụng code

Đừng bắt mô hình cung cấp lại những định danh bạn đã có. Hãy để code ghi nhớ context, còn mô hình tập trung vào chính quyết định cần đưa ra.

Bắt mô hình sinh lại thông tin đã có sẵn là không hiệu quả. Ví dụ, nếu `order_id` đã lấy được ở bước trước, sẽ hiệu quả hơn nếu cung cấp một hàm không tham số như `submit_refund()` và tự động truyền `order_id` qua code, thay vì yêu cầu mô hình gọi một hàm nhận `order_id` làm input.

Listing 3.10. Anti-pattern: định danh do mô hình cung cấp

```
{
    "name": "submit_refund",
    "parameters": {
    "type": "object",
    "required": ["order_id", "reason"],
    "properties": {
        "order_id": {
        "type": "string",
        "description": "The order ID to refund"
        },
        "reason": {
        "type": "string",
        "description": "Reason for the refund"
        }
    }
    }
}
```

Cách này buộc mô hình phải nhớ và truyền đúng `order_id` ở mọi lần gọi, làm tăng rủi ro lỗi và lượng token tiêu thụ, nhất là trong các luồng nhiều bước nơi định danh có thể bị trôi lệch hoặc nhầm lẫn.

Listing 3.11. Cải tiến: chỉ tham số mang tính quyết định

```
{
    "name": "submit_refund",
    "parameters": {
    "type": "object",
    "required": ["reason"],
    "properties": {
        "reason": {
        "type": "string",
        }
    }
    }
}
```

Giờ mô hình chỉ cung cấp input mang tính quyết định (lý do hoàn tiền), còn code gọi hàm tự chèn đúng `order_id` từ context. Cách này giảm khả năng hoàn tiền nhầm đơn, giảm chi phí prompt và tăng tỉ lệ thành công trong các workflow nhiều lượt.

Giới hạn số lượng hàm

Theo hướng dẫn chung, hãy giữ số tool dưới 20. Dùng quá nhiều tool có thể dẫn tới chọn nhầm. Nếu nhiệm vụ cần nhiều tool hoặc logic phức tạp, hãy cân nhắc fine-tune mô hình để tăng độ chính xác của Tool Calling.

3.3 Xây dựng tool và tool definition cho LLM

Giờ đã hiểu tool calling hoạt động ra sao, ta hãy tự xây một tool thực sự từ đầu. Ta sẽ hiện thực một tool web search, một trong những năng lực thiết yếu nhất của bất kỳ research agent nào. Qua quá trình này, bạn sẽ tự trải nghiệm những gì cần làm để tạo ra một tool sẵn sàng cho production.

3.3.1 Hiện thực tool web search

Để tìm kiếm web, ta dùng Tavily thay vì các lựa chọn khác như Google Search API hay Bing API, vì Tavily được tối ưu cho LLM và trả về kết quả dễ xử lý hơn. Để dùng Tavily, trước hết bạn cần đăng ký tại https://www.tavily.com/, sau đó lấy API key tại https://app.tavily.com/home. Tavily cho phép tối đa 1.000 lượt gọi API miễn phí mỗi tháng.

Trước khi tiếp tục, cài Tavily Python client:

```
uv add tavily-python
```

Sau đó lưu API key vào file `.env` với biến `TAVILY_API_KEY`.

```
TAVILY_API_KEY=<Your Tavily API KEY>
```

Bắt đầu đơn giản

Ta bắt đầu với cách hiện thực đơn giản nhất có thể để kiểm tra mọi thứ hoạt động. Ở giai đoạn này, mục tiêu chỉ là xác nhận rằng ta kết nối được với API và lấy được kết quả. Cấu hình được giữ ở mức tối thiểu, chỉ thêm độ phức tạp khi cần.

Code theo một khuôn mẫu đơn giản. Đầu tiên, ta nạp biến môi trường bằng `load_dotenv()`, hàm này đọc API key từ file `.env`. Sau đó ta tạo một instance `TavilyClient` với key đó. Bản thân hàm `search_web` rất gọn: nhận một chuỗi query, gọi method `search()` của Tavily API và trả về kết quả. Ta đặt mặc định `max_results` là 2 để response ngắn gọn trong lúc phát triển và giảm chi phí API.

Listing 3.12. Hàm web search cơ bản dùng Tavily

```
import os
from tavily import TavilyClient
from dotenv import load_dotenv

load_dotenv()
tavily_client = TavilyClient(os.getenv("TAVILY_API_KEY"))

def search_web(query: str, max_results: int = 2) -> list:
    response = tavily_client.search(query, max_results=max_results)
    return response.get("results")
```

Hãy thử tìm kỷ lục marathon của Kipchoge để kiểm tra `search_web` có hoạt động không. Mỗi kết quả gồm title, URL, đoạn trích nội dung (content snippet) và điểm liên quan (relevance score). Kết quả đầu tiên là trang Wikipedia của anh, kết quả thứ hai là một bài báo của BBC. Từ các đoạn trích này, ta thấy thành tích marathon chính thức tốt nhất của anh là 2:01:09 (Berlin 2022). Tool cơ bản của ta đã hoạt động.

Listing 3.13. Kiểm thử hàm web search

```
search_web("Kipchoge's marathon world record")
```

Output như sau:

```
[
    {
        'title': 'Eliud Kipchoge - Wikipedia',
        'url': 'https://en.wikipedia.org/wiki/Eliud_Kipchoge',
        'content': 'Eliud Kipchoge is a Kenyan long-distance runner who
competes in the marathon and formerly specialized in the 5000 metres.
Kipchoge is the 2016 and 2020 Olympic marathon champion, and was the
```

```
world record holder in the marathon from 2018 to 2023, until that
record was broken by Kelvin Kiptum at the 2023 Chicago Marathon.
Kipchoge has run 4 of the 10 fastest marathons in history...
Personal bests Marathon: 2:01:09 (Berlin 2022)...',
        'score': 0.92125154,
        'raw_content': None
    },
    {
        'title': 'Eliud Kipchoge breaks two-hour marathon mark by 20 seconds',
        'url': 'https://www.bbc.co.uk/sport/athletics/50025543',
        'content': 'Eliud Kipchoge has become the first athlete to run a
marathon in under two hours, beating the mark by 20 seconds. The
Olympic champion - who holds the official marathon world record of
2:01:39, set in Berlin, Germany in 2018 - missed out by 25 seconds
in a previous attempt at the Italian Grand Prix circuit at Monza in
2017...',
        'score': 0.9014448,
        'raw_content': None
    }
]
```

Thêm tuỳ chọn tìm kiếm

Hàm cơ bản đã lấy được kết quả, nhưng sử dụng thực tế thường đòi hỏi kiểm soát nhiều hơn. Hãy xét các tình huống: người dùng chỉ muốn tin tức gần đây về một chủ đề, hoặc cần kết quả riêng cho quốc gia của họ, hoặc muốn nghiên cứu sâu hơn với nhiều kết quả hơn. Mỗi yêu cầu cần thêm tham số.

API của Tavily hỗ trợ những điều này qua các đối số tuỳ chọn. Ta mở rộng hàm để đưa ra những tuỳ chọn hay dùng nhất. Hàm có thêm bốn tham số mới. Tham số `topic` cho phép tập trung vào loại nội dung cụ thể: `"general"` để tìm kiếm web rộng, `"news"` cho bài viết gần đây, hoặc `"finance"` cho thông tin tài chính. Tham số `time_range` lọc kết quả theo độ mới, hữu ích khi người dùng cần thông tin hiện tại thay vì nội dung cũ.

Listing 3.14. Hàm web search với các tuỳ chọn bổ sung

```
def search_web(
    query: str,
    max_results: int = 2,
    topic: str = "general",
    time_range: str | None = None,
    country: str | None = None,
) -> list:
    response = tavily_client.search(
        query,
        max_results=max_results,
        topic=topic,
        time_range=time_range,
        country=country,
    )
    return response.get("results")
```

Nhưng đây mới chỉ là phần nổi. API của Tavily hỗ trợ gần 20 tham số, gồm lọc theo domain cụ thể (chỉ lấy hoặc loại trừ một số website), điều chỉnh độ sâu tìm kiếm (basic hay advanced), kèm ảnh trong kết quả, lấy nội dung HTML thô, và nhiều thứ khác. Mỗi tham số đòi hỏi hiểu mục đích, giá trị hợp lệ và cách nó tương tác với các tuỳ chọn khác. Càng đưa ra nhiều tham số, tool definition càng phức tạp và LLM càng khó dùng đúng.

Xử lý lỗi êm thấm

Đến giờ ta vẫn giả định mọi thứ chạy hoàn hảo. Nhưng chuyện gì xảy ra khi có sự cố? API key không hợp lệ trả về lỗi xác thực 401. Vượt hạn mức sử dụng hàng tháng gây lỗi rate limit 429. Sự cố mạng gây timeout kết nối. API có thể trả về dữ liệu sai định dạng, hoặc dịch vụ có thể tạm thời không khả dụng.

Cách tối giản là bắt mọi exception và trả về thông báo lỗi. Hiện thực dưới đây bọc lời gọi API trong khối try-except và trả về chuỗi lỗi nếu có gì trục trặc. Kiểu trả về giờ là `list | str`, cho biết hàm trả về hoặc danh sách kết quả, hoặc chuỗi thông báo lỗi. Cách này đơn giản và bảo đảm agent vẫn tiếp tục hoạt động ngay cả khi tìm kiếm thất bại.

Listing 3.15. Hàm web search có xử lý lỗi

```
def search_web(
    query: str,
    max_results: int = 5,
    topic: str = "general",
    time_range: str | None = None,
) -> list | str:
    try:
        response = tavily_client.search(
            query,
            max_results=max_results,
            topic=topic,
            time_range=time_range,
        )
        return response.get("results")
    except Exception as e:
        return f"Error: Search failed - {e}"
```

Tuy nhiên, cách bắt tất cả này đối xử với mọi lỗi như nhau. Trong hệ thống production, các lỗi khác nhau thường cần phản ứng khác nhau. Lỗi xác thực (401) có thể báo hiệu vấn đề cấu hình cần con người xử lý. Lỗi rate limit (429) có thể nên tự động thử lại với backoff. Lỗi timeout có thể nên thử lại với query đơn giản hơn. Lỗi mạng có thể cần xử lý khác với lỗi API.

Với mục đích học tập, cách xử lý lỗi đơn giản này là đủ. Nhưng khi xây tool cho agent production, nhiều khả năng bạn sẽ cần phân biệt các loại lỗi và xử lý từng loại cho phù hợp. Độ phức tạp của việc xử lý lỗi chặt chẽ là thêm một khía cạnh của phát triển tool, vượt xa việc chỉ gọi một API.

3.3.2 Chuyển thành tool definition

Đến đây ta đã tạo một tool web search. Trước đó trong chương, ta cũng đã xây hàm `calculator` để tính toán. Để LLM dùng được các tool này, ta cần định nghĩa chúng theo một định dạng tool definition chuẩn.

Vì các hàm ta viết đều đơn giản, chuyển chúng thành tool definition bằng tay không tốn nhiều thời gian. Tuy nhiên, trong quá trình phát triển, hàm có thể thay đổi thường xuyên hoặc trở nên phức tạp hơn, khiến việc cập nhật thủ công dễ sai và kém hiệu quả. Để giải quyết, ta có thể tạo một hàm tiện ích tự động chuyển một hàm Python thành tool definition.

Để sinh tool definition từ một hàm, ta cần trích tên hàm, docstring và chi tiết tham số. Trong Python, việc này làm được bằng module `inspect`.

Để minh hoạ, xét một hàm mẫu tên `example_tool`. Hàm này nhận hai input: `input_1` là chuỗi, và `input_2` là số nguyên có giá trị mặc định là 1.

Listing 3.16. Trích metadata của tool bằng inspect

```
import inspect

def example_tool(input_1:str, input_2:int=1):
```

```
    """docstring for example_tool"""
    return

print(f"function name: {example_tool.__name__}")
print(f"function docstring: {example_tool.__doc__}")
print(f"function signature: {inspect.signature(example_tool)}")
```

Output như sau:

```
function name: example_tool
function docstring: docstring for example_tool
function signature: (input_1: str, input_2: int = 1)
```

Ta lấy được tên hàm qua thuộc tính `__name__` và mô tả qua thuộc tính `__doc__`. Kiểu tham số và việc tham số có bắt buộc hay không được xác định bằng `inspect.signature`. Từ output, ta thấy `input_1` là chuỗi bắt buộc, còn `input_2` là số nguyên tuỳ chọn với giá trị mặc định 1.

Dùng các thông tin này, ta có thể hiện thực một hàm tiện ích như sau.

Listing 3.17. Bộ chuyển đổi hàm thành tool schema

```
def function_to_input_schema(func) -> dict:
    type_map = {
        str: "string",
        int: "integer",
        float: "number",
        bool: "boolean",
        list: "array",
        dict: "object",
        type(None): "null",
    }

    try:
        signature = inspect.signature(func)  #A
    except ValueError as e:
        raise ValueError(
            f"Failed to get signature for function {func.__name__}: {str(e)}"
        )

    parameters = {}
    for param in signature.parameters.values():
        try:
            param_type = type_map.get(param.annotation, "string")  #B
        except KeyError as e:
            raise KeyError(
                f"Unknown type annotation {param.annotation} for parameter
                ↪  {param.name}: {str(e)}"
            )
        parameters[param.name] = {"type": param_type} #B

    required = [
        param.name  #C
        for param in signature.parameters.values()  #C
        if param.default == inspect._empty  #C
    ]

    return {
            "type": "object",
            "properties": parameters,
            "required": required,
```

```
        }
```

#A Trích signature của hàm bằng module `inspect`.
#B Ánh xạ kiểu Python sang kiểu JSON Schema, mặc định là `string`.
#C Tham số không có giá trị mặc định được đánh dấu là bắt buộc.

Nhìn vào code mẫu, ta thấy `inspect.signature` được dùng để trích tham số của hàm và chuyển kiểu của chúng sang định dạng phù hợp cho tool definition. Code kiểm tra từng tham số có giá trị mặc định hay không để xác định nó có bắt buộc không, và lưu các đối số bắt buộc vào một list. Cuối cùng, nó sắp xếp toàn bộ thông tin theo định dạng tool definition chuẩn.

Để kiểm tra tiện ích `function_to_tool_definition` hoạt động đúng, ta thử chuyển đổi tool web search của mình.

Listing 3.18. Bộ chuyển đổi hàm thành tool schema (tool definition hoàn chỉnh)

```
def format_tool_definition(name: str, description: str, parameters:dict) -> dict:
    return {
        "type": "function",
        "function": {
            "name": name,
            "description": description,
            "parameters": parameters,
        },
    }

def function_to_tool_definition(func) -> dict:
    return format_tool_definition(
        func.__name__,
        func.__doc__ or "",
        function_to_input_schema(func)
    )

function_to_tool_definition(search_web)
# {'type': 'function', 'function': {'name': 'search_web',
# 'description': 'Search the web for the given query.',
# 'parameters': {'type': 'object', 'properties':
# {'query': {'type': 'string'}}, 'required': ['query']}}}
```

Schema thu được cần chứa tên và mô tả của hàm, cùng kiểu của từng tham số và việc tham số đó có bắt buộc hay không, đúng như mong đợi.

```
search_tool_definition = function_to_tool_definition(search_web)
print(search_tool_definition)
```

Output như sau:

```
{'type': 'function', 'function': {'name': 'search_web', 'description': 'Search
↪  the web for the given query.', 'parameters': {'type': 'object', 'properties':
↪  {'query': {'type': 'string'}, 'max_results':{'type': 'integer'}, 'topic':
↪  {'type': 'string'}, 'time_range': {'type': 'string'}}, 'required':
↪  ['query']}}}
```

Giờ ta có thể dùng tiện ích này để chuyển mọi tool đã tạo thành schema và chuẩn bị giải bài toán mục tiêu.

3.3.3 Thực thi tool từ đầu đến cuối

Giờ hãy ghép mọi thứ lại. Ta đã có một hàm web search chạy được và một tiện ích chuyển nó thành tool definition. Mảnh ghép cuối cùng là hạ tầng thực thi, nối các tool call của LLM với việc thực thi hàm thực tế.

Ta cần hai thành phần: một hàm thực thi tool dựa trên output của LLM, và một vòng điều khiển (control loop) quản lý cuộc hội thoại giữa LLM và các tool.

Xây hệ thống thực thi tool

Đầu tiên, ta định nghĩa một hàm tiện ích thực thi tool và trả về kết quả. Hàm này nhận một toolbox (một dictionary ánh xạ tên tool sang hàm) và một tool call từ LLM, rồi thực thi hàm phù hợp với các đối số được cung cấp. Hàm trích tên tool và đối số từ response của LLM, tìm hàm tương ứng trong toolbox và gọi nó với các đối số đã parse. Khuôn mẫu đơn giản này dùng được cho mọi tool, miễn là tool đã được đăng ký trong toolbox.

Listing 3.19. Hàm tiện ích thực thi tool

```
def tool_execution(tool_box, tool_call):
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

    tool_result = tool_box[function_name](**function_args)
    return tool_result
```

Tiếp theo, ta xây một vòng điều khiển quản lý tương tác giữa LLM và tool. Hàm này gửi câu hỏi của người dùng tới LLM kèm các tool definition sẵn có. Nếu LLM yêu cầu một tool call, ta thực thi nó và đưa kết quả trở lại context. Chu trình này tiếp diễn cho tới khi LLM trả về câu trả lời cuối cùng mà không yêu cầu tool nào.

Hãy lần theo những gì xảy ra trong vòng lặp này:

1. LLM nhận system prompt, câu hỏi của người dùng và các tool definition sẵn có.
2. Nếu LLM xác định cần thông tin bên ngoài, nó sinh một tool call.
3. Ta thêm message của assistant (chứa tool call) vào lịch sử hội thoại.
4. Ta thực thi tool được yêu cầu và thêm kết quả dưới dạng message `"tool"`.
5. Vòng lặp tiếp tục, gửi hội thoại đã cập nhật lại cho LLM.
6. Khi LLM đã có đủ thông tin để trả lời, nó trả về response không có tool call, và ta thoát vòng lặp.

Listing 3.20. Agent loop đơn giản để thực thi tool

```
from litellm import completion

def simple_agent_loop(system_prompt, question):
    tools = [search_web]
    tool_box = {tool.__name__: tool for tool in tools}
    tool_definitions = [function_to_tool_definition(tool) for tool in tools]

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": question}
    ]

    while True:
```

```
        response = completion(
            model="gpt-5-mini",
            messages=messages,
            tools=tool_definitions
        )

        assistant_message = response.choices[0].message

        if assistant_message.tool_calls:
            messages.append(assistant_message)
            for tool_call in assistant_message.tool_calls:
                tool_result = tool_execution(tool_box, tool_call)
                messages.append({
                    "role": "tool",
                    "content": str(tool_result),
                    "tool_call_id": tool_call.id
                })
        else:
            return assistant_message.content
```

Kiểm thử agent loop

Hãy kiểm thử hệ thống bằng một câu hỏi cần thông tin cập nhật.

Listing 3.21. Kiểm thử agent loop

```
system_prompt = """You are a helpful assistant.
Use the search tool when you need current information."""

result = simple_agent_loop(
    system_prompt,
    "Who won the 2025 Nobel Prize in Physics?"
)
print(result)
```

Output như sau:

```
The 2025 Nobel Prize in Physics was awarded jointly to John Clarke, Michel H.
↪  Devoret, and John M. Martinis. They were recognized for experiments
↪  demonstrating macroscopic quantum tunnelling and energy quantisation in
↪  electric circuits - work underpinning superconducting quantum technologies
↪  (announced Oct. 7, 2025)
```

LLM nhận ra nó cần thông tin cập nhật không có trong dữ liệu huấn luyện, gọi tool `search_web` với query phù hợp, nhận kết quả tìm kiếm và tổng hợp thành câu trả lời cuối cùng. Nếu không có tool, LLM sẽ phải thừa nhận là không biết, hoặc có nguy cơ đưa ra thông tin lỗi thời.

3.3.4 Những thách thức của custom tool

Xây tool web search ở Mục 3.3.1 khá đơn giản. Ta cài Tavily client, viết một hàm đơn giản và chuyển nó thành tool definition. Toàn bộ quá trình chỉ mất vài phút. Tuy nhiên, sự đơn giản này dễ gây ảo tưởng. Khi bạn phát triển nhiều tool hơn và mở rộng sang dự án thực tế, sẽ nảy sinh nhiều thách thức khiến việc phát triển custom tool khó hơn bạn tưởng.

Phải hiểu những API xa lạ

Tavily được thiết kế riêng cho ứng dụng LLM nên tích hợp rất dễ. API đơn giản, định dạng response đã được tối ưu cho RAG, và tài liệu ngắn gọn. Nhưng hầu hết các dịch vụ bạn muốn tích hợp đều không được xây dựng với LLM trong đầu.

Hãy thử xây một tool tìm kiếm trong Slack workspace của công ty, lấy file từ Google Drive, hoặc truy vấn cơ sở dữ liệu Salesforce. Mỗi dịch vụ có quy ước API, chính sách rate limit, kiểu phân trang (pagination) và định dạng response riêng. Trước khi viết dòng code nào, bạn phải bỏ ra hàng giờ đọc tài liệu, tìm hiểu mô hình dữ liệu và tìm cách biến đổi response thành thứ có ích cho LLM. Khoản đầu tư ban đầu này nhân lên với mỗi tool mới bạn cần xây.

Xử lý lỗi chặt chẽ

Hãy nhìn lại hàm `search_web`. Nó chỉ xử lý lỗi ở mức tối thiểu. Trong môi trường production, như vậy là không đủ. Tool thực tế phải xử lý timeout mạng một cách êm thấm, quản lý rate limit của API với chiến lược backoff phù hợp, đối phó với sự cố dịch vụ, kiểm tra input trước khi gửi request, và đưa ra thông báo lỗi có ý nghĩa giúp LLM (và cuối cùng là người dùng) hiểu chuyện gì đã xảy ra.

Hiện thực xử lý lỗi chặt chẽ thường cần nhiều code hơn cả phần chức năng chính. Một hàm web search đơn giản có thể chỉ 10 dòng, nhưng bản sẵn sàng cho production với xử lý lỗi, thử lại và logging đầy đủ có thể dễ dàng lên tới 50 dòng hoặc hơn.

Công sức bị lặp lại giữa các nhóm

Trong bất kỳ tổ chức nào xây AI agent, nhiều nhóm thường cần những năng lực giống nhau. Nhóm marketing muốn có tool web search cho agent tạo nội dung. Nhóm chăm sóc khách hàng cần web search cho FAQ bot. Nhóm nghiên cứu cần nó cho pipeline phân tích.

Không có cách chuẩn để chia sẻ tool, mỗi nhóm tự xây bản hiện thực của mình. Họ giải cùng những bài toán một cách độc lập, mắc những lỗi tương tự và duy trì các codebase riêng rẽ. Xét trên toàn tổ chức, sự trùng lặp này lãng phí đáng kể nguồn lực kỹ sư. Cùng một tool có thể tồn tại ở ba, bốn phiên bản hơi khác nhau trong công ty.

Tái sử dụng code khó hơn tưởng tượng

Bạn có thể nghĩ giải pháp rất đơn giản: một nhóm xây tool, các nhóm khác dùng lại. Trên thực tế, việc này hiếm khi suôn sẻ. Các nhóm cấu trúc tool definition khác nhau: nhóm dùng dictionary, nhóm dùng Pydantic model. Cách thực thi khác nhau giữa bản đồng bộ và bất đồng bộ. Định dạng trả về không nhất quán. Chất lượng tài liệu chênh lệch rất nhiều.

Khi thử dùng một tool của nhóm khác, bạn thường mất nhiều thời gian để điều chỉnh nó cho hợp với codebase của mình hơn là tự xây từ đầu. Dependency xung đột, interface không khớp, và những giả định ngầm trong code không áp dụng được cho trường hợp của bạn. Ma sát khi tích hợp thường lớn hơn lợi ích của việc tái sử dụng.

Gánh nặng bảo trì liên tục

Xây một tool không phải là việc làm một lần. API thay đổi theo thời gian, và tool phải thay đổi theo. Dịch vụ ngừng hỗ trợ endpoint, đổi định dạng response và thêm tham số bắt buộc mới. Lỗ hổng bảo mật trong dependency đòi hỏi cập nhật. Người dùng yêu cầu tính năng mới và báo những trường hợp biên bạn không lường trước.

Khi chỉ bảo trì một hai tool, việc này còn kiểm soát được. Nhưng agent thường cần nhiều tool: web search, thao tác file, truy vấn cơ sở dữ liệu, email, lịch và các tích hợp đặc thù theo lĩnh vực. Mỗi tool trở thành một trách nhiệm bảo trì. Khi bộ sưu tập tool lớn dần, tổng gánh nặng bảo trì trở nên rất lớn.

Nhu cầu chuẩn hoá

Những thách thức này không chỉ riêng với tool cho LLM. Ngành phần mềm từng giải các vấn đề tương tự bằng chuẩn hoá. REST API chuẩn hoá cách các web service giao tiếp. Package manager chuẩn hoá cách chia sẻ và quản lý phiên bản code. Container hoá chuẩn hoá cách triển khai ứng dụng.

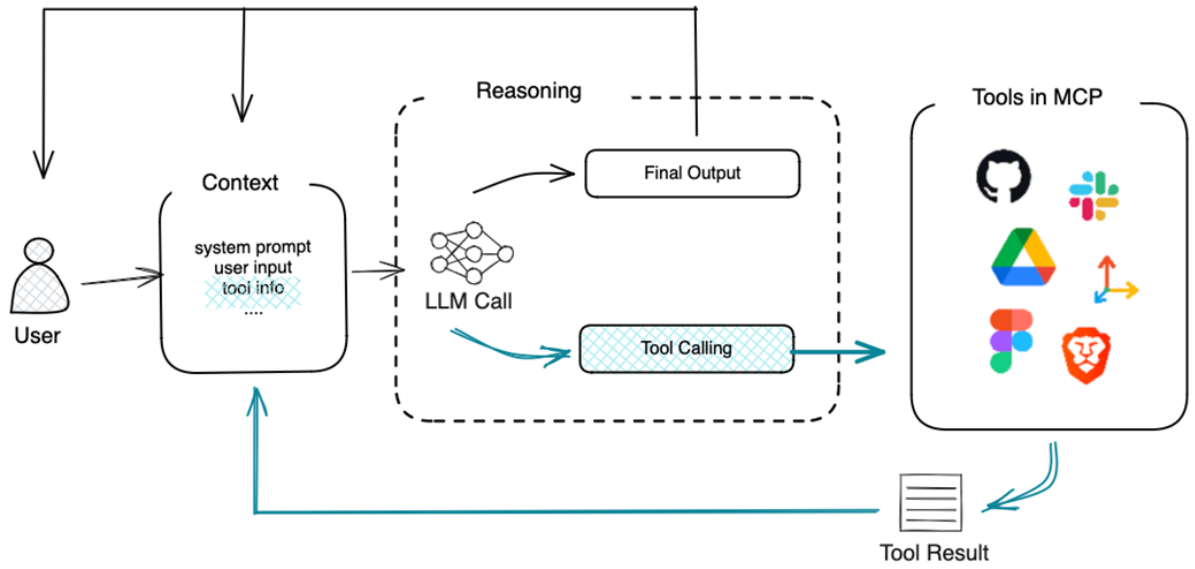
Hệ sinh thái tool cho LLM cũng cần một sự chuẩn hoá tương tự: một cách chung để định nghĩa tool, chia sẻ chúng giữa các dự án, và tích hợp chúng bất kể bạn dùng nhà cung cấp LLM nào. Đây chính là thứ MCP (Model Context Protocol) mang lại.

3.4 MCP: Chuẩn hoá tool

Như đã tự trải nghiệm ở Mục 3.3, phát triển tool cho LLM agent tốn thời gian, và sự thiếu nhất quán trong cách hiện thực khiến chúng khó tái sử dụng.

Tháng 11/2024, Anthropic giới thiệu MCP (Model Context Protocol) nhằm mang một chuyển biến tương tự vào thế giới tích hợp tool cho LLM. Nếu API đã tạo ra ngôn ngữ chung để developer giao tiếp với các dịch vụ, thì MCP tạo ra ngôn ngữ chung để LLM agent giao tiếp với tool. Giống như bạn có thể gọi bất kỳ REST API nào bằng cùng các khuôn mẫu HTTP, một agent tương thích MCP có thể dùng bất kỳ tool tương thích MCP nào mà không cần code tích hợp riêng.

Kể từ khi ra mắt, MCP ngày càng được chú ý trong suốt đầu năm 2025. Nó mở ra khả năng cho developer dùng các tool do công ty hoặc cộng đồng xây dựng mà không phải tự xây tất cả từ đầu. Như Hình 3.4 cho thấy, với MCP, agent có thể dễ dàng tích hợp tool từ các dịch vụ như Google Drive, Slack, Tavily, Brave Search, GitHub và Figma mà không cần viết lớp bọc (wrapper) tool riêng.



Hình 3.4: MCP giúp dễ dàng mở rộng bộ tool của agent bằng cách tích hợp tool của bên thứ ba. *(PDF gốc tr. 104)*

3.4.1 Cốt lõi của MCP: kiến trúc server–client

Về cốt lõi, MCP tách "bộ não" của agent khỏi "tay chân" của nó (các tool) thông qua kiến trúc ba thành phần: Host, Client và Server.

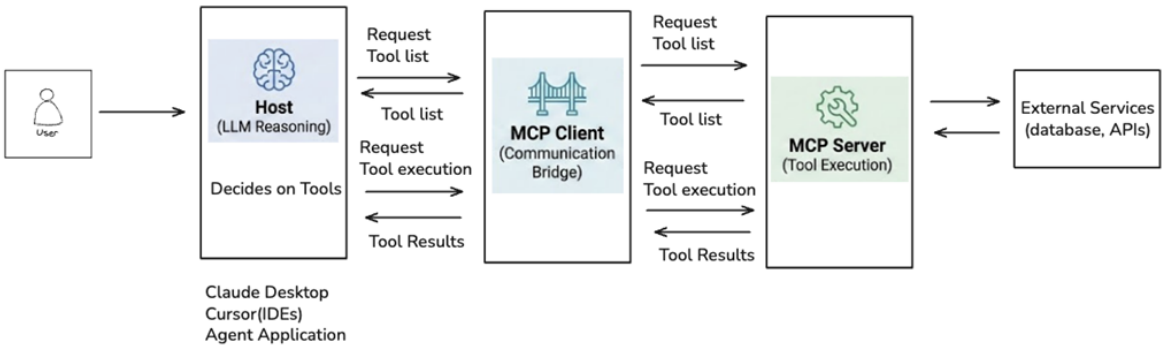
Hiểu ba thành phần

Hình 3.5 minh hoạ cách các thành phần này phối hợp với nhau. Hãy làm rõ vai trò của từng thành phần:

Host là ứng dụng tương tác với người dùng và đưa ra quyết định bằng LLM. Host nhận input của người dùng, suy luận và quyết định dùng tool nào, khi nào. Hãy coi nó là "bộ não" của hệ thống. Ví dụ: Claude Desktop, một IDE có tính năng AI, hoặc chính ứng dụng agent bạn đang xây.

MCP Client đóng vai cầu nối giữa Host và các MCP Server. Khi Host quyết định cần một tool, Client lo phần giao tiếp: yêu cầu server cung cấp các tool definition sẵn có và chuyển tiếp yêu cầu thực thi tool. Mỗi Client duy trì kết nối tới một MCP Server.

MCP Server chứa các tool thực sự và thực thi chúng khi được yêu cầu. Server đáp ứng hai loại yêu cầu: "bạn có những tool nào?" (trả về tool definition) và "hãy thực thi tool này với các đối số này" (chạy tool và trả kết quả). Server có thể kết nối tới các dịch vụ bên ngoài như cơ sở dữ liệu hay API.



Hình 3.5: Kiến trúc server–client của MCP. Host chứa LLM để suy luận và tương tác với người dùng, kết nối thông qua các MCP Client quản lý việc giao tiếp với server. MCP Server cung cấp tool definition và xử lý việc thực thi, có thể tích hợp thêm với các dịch vụ bên ngoài như cơ sở dữ liệu và API. *(PDF gốc tr. 105)*

Phạm vi của MCP rộng hơn tool

Thực ra MCP định nghĩa ba loại năng lực mà server có thể cung cấp: tool (các hàm LLM có thể gọi), prompt (các prompt template dùng lại được) và resource (dữ liệu LLM có thể truy cập). Trong sách này, ta chỉ tập trung vào tool vì đây là năng lực thiết yếu nhất để xây agent.

Hai interface được chuẩn hoá

Bất kể dùng transport nào, MCP chuẩn hoá hai interface chính:

- **Tool Discovery** (khám phá tool): Ở Mục 3.3, ta tự định nghĩa schema cho từng tool. Với MCP, client có thể yêu cầu các tool definition sẵn có, và server trả về chúng theo định dạng chuẩn. Người phát triển tool chỉ cần hiện thực schema một lần; mọi client tương thích MCP đều có thể tự động lấy và dùng nó.
- **Tool Execution** (thực thi tool): Trước đây, ta tự xây hàm `tool_execution` để nối output của LLM với lời gọi hàm thực tế. MCP cũng chuẩn hoá phần này. Client gửi yêu cầu thực thi tool tới server, server lo việc thực thi và trả kết quả theo định dạng nhất quán.

Cơ chế transport

MCP hỗ trợ ba cơ chế transport để client và server giao tiếp:

- **stdio (Standard I/O)** là cách đơn giản nhất: client khởi chạy server như một subprocess và giao tiếp qua luồng standard input/output. Vì mọi thứ chạy cục bộ trên cùng một máy nên không có chi phí mạng. Cách này lý tưởng cho phát triển cục bộ, và khi tool không cần chia sẻ giữa nhiều máy.

- **HTTP** cho phép giao tiếp từ xa qua mạng. Client gửi request qua HTTP, server stream response trở lại. Cách này phù hợp khi server cần chạy trên máy riêng hoặc được chia sẻ cho nhiều client.
- **WebSocket** cung cấp giao tiếp hai chiều đầy đủ, cho phép cả client lẫn server chủ động gửi message. Cách này hữu ích khi server cần chủ động đẩy cập nhật tới client.

Trong suốt chương này, ta dùng transport stdio vì nó dễ thiết lập nhất và đủ để học các khái niệm MCP. Khi cần tool server từ xa hoặc dùng chung trong production, bạn có thể chuyển sang HTTP hoặc WebSocket mà không phải sửa phần hiện thực tool.

3.4.2 Thực hành: chạy một MCP server

Trước khi đi vào chi tiết hiện thực, hãy trải nghiệm trực tiếp MCP bằng cách chạy một server có sẵn. Ta sẽ dùng Tavily MCP server chính thức, cung cấp đúng năng lực web search mà ta đã tự xây ở Mục 3.3, nhưng được đóng gói thành một MCP server dùng ngay được.

Các MCP server trong hệ sinh thái thường được phân phối dưới dạng npm package và chạy bằng `npx`. Nếu chưa cài Node.js, bạn cần cài trước. Với cả macOS và Windows, chỉ cần tải bộ cài cho hệ điều hành của bạn tại `https://nodejs.org/` và làm theo các bước cài đặt.

Kiểm tra cài đặt bằng cách xem phiên bản:

```
node --version
```

Nếu phiên bản Node hiện ra, việc cài đặt đã thành công.

Chạy Tavily MCP server

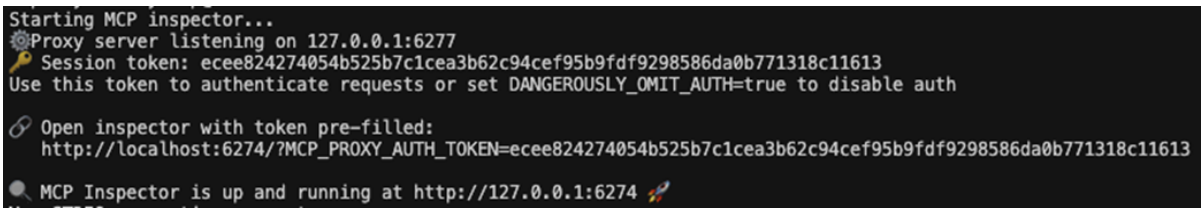
Khi đã có Node.js, ta có thể khởi chạy Tavily MCP server chỉ bằng một lệnh. Trước tiên, đặt Tavily API key làm biến môi trường:

```
export TAVILY_API_KEY=<your tavily api key>
```

Giờ khởi chạy server cùng MCP Inspector, một công cụ chạy trên trình duyệt để kiểm thử MCP server:

```
npx @modelcontextprotocol/inspector npx -y tavily-mcp@latest
```

Lệnh này làm nhiều việc cùng lúc. Lệnh `npx` chạy npm package mà không cần cài đặt global. Package `@modelcontextprotocol/inspector` cung cấp giao diện web để tương tác với MCP server. Cờ `-y` tự động xác nhận mọi câu hỏi. Cuối cùng, `tavily-mcp@latest` là package Tavily MCP server chính thức.



Hình 3.6: Log khởi động của MCP Inspector, cho thấy quá trình khởi tạo server và URL kết nối. *(PDF gốc tr. 108)*

Mở URL hiện trên terminal. Nếu URL có kèm session token, bạn có thể kết nối mà không cần cấu hình thêm.

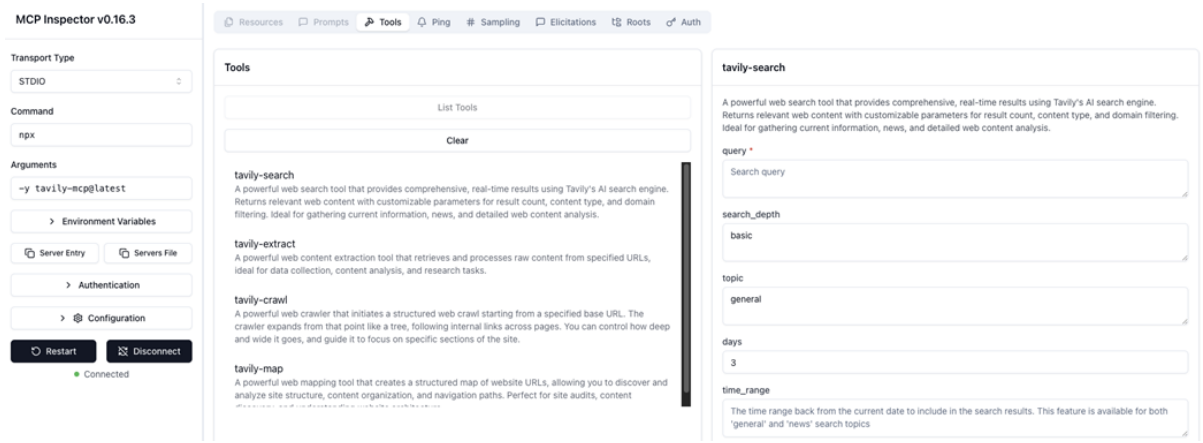
Khám phá server bằng MCP Inspector

Khi giao diện Inspector đã tải xong trên trình duyệt, bấm nút **Connect**. Sau khi kết nối thành công, bạn sẽ thấy vài tab xuất hiện: Resources, Prompts và Tools. Các tab này phản ánh những năng lực mà MCP server cung cấp.

Chọn tab **Tools** và bấm **List Tools**. Bạn sẽ thấy các tool mà Tavily MCP server cung cấp:

- **tavily-search**: chức năng tìm kiếm web (tương tự cái ta đã xây ở Mục 3.3).
- **tavily-extract**: trích nội dung từ các URL cụ thể.
- **tavily-crawl**: crawl website và trích dữ liệu có cấu trúc.
- **tavily-map**: tạo sơ đồ trang (site map).

Bấm vào **tavily-search** để mở chi tiết. Bạn sẽ thấy mô tả của tool và schema tham số, được định dạng theo chuẩn MCP. Thử nhập một query như "2025 Nobel Physics" và bấm **Run Tool**. Server thực thi tìm kiếm và trả kết quả ngay trên trình duyệt.



Hình 3.7: Giao diện MCP Inspector với Tavily MCP server chính thức, cung cấp bộ tool phong phú hơn gồm tavily-search, tavily-extract, tavily-crawl và tavily-map. *(PDF gốc tr. 109)*

Những gì ta vừa làm được

Hãy dừng lại một chút để thấy điều vừa diễn ra. Chỉ với một lệnh, ta đã:

1. Tải về và chạy một MCP server sẵn sàng cho production.
2. Kết nối tới nó bằng một interface client chuẩn.
3. Tự động khám phá các tool sẵn có.
4. Thực thi một tool và nhận kết quả.

So với Mục 3.3, nơi ta tốn khá nhiều công sức chỉ để hiện thực chức năng tìm kiếm cơ bản, Tavily MCP server có sẵn nhiều tool, xử lý lỗi chặt chẽ và định dạng response đã tối ưu, tất cả dùng được ngay mà không cần viết dòng code nào.

Đó là sức mạnh của chuẩn hoá. Người phát triển tool có thể tập trung xây server đáng tin cậy, nhiều tính năng. Người phát triển agent có thể tập trung vào suy luận và điều phối, vì biết rằng mọi tool tương thích MCP đều sẽ chạy được với hệ thống của mình.

Ở mục tiếp theo, ta xem phía bên kia của kiến trúc: cách xây một MCP client có thể khám phá và dùng các tool này bằng code ngay trong agent.

3.4.3 Tìm hiểu MCP client

Ở mục trước, ta tương tác với Tavily MCP server qua giao diện web của Inspector. Cách đó tiện cho kiểm thử, nhưng agent cần tương tác với MCP server bằng code. Đó là lúc MCP client vào cuộc.

MCP client là thứ agent dùng để khám phá và gọi tool từ MCP server. Nó lo các chi tiết của giao thức: thiết lập kết nối, yêu cầu tool definition, thực thi tool và parse kết quả. Hiểu cách client hoạt động là điều thiết yếu để tích hợp tool MCP vào agent của riêng bạn.

Cài MCP Python SDK để thực hành:

```
uv add mcp
```

Kết nối tới MCP server

Ta viết một client đơn giản kết nối tới Tavily MCP server, liệt kê các tool sẵn có và thực hiện một lần tìm kiếm.

Hãy lần theo những gì xảy ra trong đoạn code này:

1. **StdioServerParameters** đặc tả cách khởi chạy server. Command là `npx`, chạy npm package mà không cần cài global. Phần args gồm `-y` để tự động xác nhận câu hỏi và `tavily-mcp@latest` để dùng phiên bản mới nhất. Biến môi trường truyền API key và PATH của hệ thống.

2. **stdio_client** khởi chạy server như một subprocess. Câu lệnh `async with` bảo đảm dọn dẹp đúng cách khi xong việc. Nó trả về các luồng đọc và ghi mà session dùng để giao tiếp.

3. **ClientSession** cung cấp API mức cao. Sau khi gọi `initialize()`, ta có thể tương tác với server bằng các method chuẩn như `list_tools()` và `call_tool()`.

4. **session.list_tools()** yêu cầu server trả về mọi tool sẵn có. Server trả về các tool definition gồm tên, mô tả và schema tham số.

5. **session.call_tool()** thực thi một tool cụ thể. Ta truyền tên tool và một dictionary đối số. Server chạy tool và trả kết quả.

Listing 3.22. Kết nối tới một MCP server

```
import asyncio
import os
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from dotenv import load_dotenv

load_dotenv()

server_params = StdioServerParameters(
    command="npx",
    args=["-y", "tavily-mcp@latest"],
    env={
        "TAVILY_API_KEY": os.getenv("TAVILY_API_KEY"),
    }
)

async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        # List available tools
        tools_result = await session.list_tools()
```

```
        print("Available tools:")
        for tool in tools_result.tools:
            print(f"  - {tool.name}: {tool.description[:60]}...")

        result = await session.call_tool(
                "tavily-search",
                arguments={"query": "2025 Nobel Physics"}
            )
        print("Search Result:")
        print(result.content)
```

Chạy code này cho ra output tương tự như sau:

```
Available tools:
    - tavily-search: A powerful web search tool that provides comprehensive,
    ↪  real...
    - tavily-extract: A powerful web content extraction tool that retrieves and
    ↪  pr...
    - tavily-crawl: A powerful web crawler that initiates a structured web
    ↪  crawl...
    - tavily-map: A powerful web mapping tool that creates a structured map of...
Search Result:
[TextContent(type='text', text='Detailed Results:\n\nTitle: Press release: Nobel
↪  Prize in Physics 2025...
```

Chuyển tool MCP sang dạng LLM dùng được

Các tool do `list_tools()` trả về dùng định dạng schema của MCP. Để dùng chúng với LLM, ta cần chuyển sang định dạng mà nhà cung cấp LLM yêu cầu.

May mắn là ta đã xây nền móng cho việc này ở Mục 3.3.2. Hàm `format_tool_definition` nhận tên, mô tả và dictionary tham số, rồi trả về một tool definition đúng cấu trúc của OpenAI. Các object tool của MCP cung cấp đúng ba thông tin này qua các thuộc tính `name`, `description` và `inputSchema`. Nhờ vậy việc chuyển đổi rất đơn giản.

Listing 3.23. Chuyển tool MCP sang định dạng OpenAI

```
def mcp_tools_to_openai_format(mcp_tools) -> list[dict]:
    """Convert MCP tool definitions to OpenAI tool format."""
    return [
        format_tool_definition(
            name=tool.name,
            description=tool.description,
            parameters=tool.inputSchema,
        )
        for tool in mcp_tools.tools
    ]
```

Với hàm chuyển đổi này, ta có thể lấy tool từ bất kỳ MCP server nào và dùng ngay với API của OpenAI. Cách làm trong thực tế như sau.

Listing 3.24. Lấy tool theo định dạng OpenAI

```
async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        # List available tools
```

```
        tools_result = await session.list_tools()
        openai_format_tools = mcp_tools_to_openai_format(tools_result)
        for tool in openai_format_tools:
            print(tool)
```

3.4.4 Thực hành: hiện thực một MCP server

Đến giờ ta mới dùng các MCP server bên ngoài do người khác xây. Dù bạn có thể xây agent hiệu quả chỉ với các server do cộng đồng cung cấp, hiểu cách tự tạo server giúp bạn tự tin khi đi trong hệ sinh thái MCP. Bạn sẽ biết chính xác chuyện gì diễn ra bên dưới, nhờ đó dễ debug, mở rộng server có sẵn, hoặc xây custom tool cho nhu cầu riêng.

Tin vui là xây một MCP server đơn giản bất ngờ. Với thư viện FastMCP, bạn có thể biến một hàm Python có sẵn thành tool tương thích MCP chỉ bằng cách thêm một decorator. Hãy biến hàm `search_web` đã xây ở Mục 3.3 thành một MCP server hoạt động đầy đủ.

Xây server

Bắt đầu bằng việc cài các thư viện cần thiết:

```
uv add fastmcp
```

Giờ tạo một file mới tên `tavily_mcp_server.py`. Ta lấy hàm `search_web` có sẵn và bọc nó bằng hạ tầng MCP. Instance `FastMCP` tạo server với một cái tên dùng để định danh nó với client. Decorator `@mcp.tool()` đăng ký hàm làm một tool MCP. FastMCP tự động trích tên hàm, tham số và type hint để sinh tool schema. Docstring trở thành mô tả của tool, giúp LLM hiểu khi nào và cách nào dùng tool. Cuối cùng, `mcp.run(transport='stdio')` khởi chạy server, dùng standard input/output để giao tiếp.

Listing 3.25. Hiện thực custom Tavily MCP server

```
import os
from tavily import TavilyClient
from dotenv import load_dotenv
from mcp.server.fastmcp import FastMCP

load_dotenv()

tavily_client = TavilyClient(os.getenv("TAVILY_API_KEY"))
mcp = FastMCP("custom-tavily-search")

@mcp.tool()
def search_web(query: str, max_results: int = 5) -> str:
    """
    Search the web using Tavily API.

    Args:
        query: Search query string
        max_results: Maximum number of results to return (default:5)

    Returns:
        Search results as formatted string
    """
    try:
        response = tavily_client.search(
            query,
            max_results=max_results,
        )
        results = response.get("results", [])
```

```
        return "\n\n".join(
            f"Title: {r['title']}\nURL: {r['url']}\nContent: {r['content']}"
            for r in results
        )
    except Exception as e:
        return f"Error searching web: {str(e)}"

if __name__ == "__main__":
    mcp.run(transport='stdio')
```

Để ý rằng ta thêm rất ít code vào hàm ban đầu. Logic cốt lõi giữ nguyên; ta chỉ bọc nó bằng hạ tầng MCP.

Kết nối bằng client

Giờ hãy kiểm tra server hoạt động bằng cách kết nối tới nó với code client đã học ở Mục 3.4.3. Thay đổi duy nhất cần làm là sửa `StdioServerParameters` để trỏ tới custom server của ta thay vì package Tavily chính thức.

Listing 3.26. Kết nối tới custom MCP server

```
server_params = StdioServerParameters(
    command="uv",
    args=["run", "tavily_mcp_server.py"],
    env={
        "TAVILY_API_KEY": os.getenv("TAVILY_API_KEY"),
    }
)

async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        # List available tools
        tools_result = await session.list_tools()
        print("Available tools:")
        for tool in tools_result.tools:
            print(f"  - {tool.name}: {tool.description}")
```

Chạy code này cho ra output như sau:

```
Available tools:
    - search_web:
Search the web using Tavily API.

Args:
    query: Search query string
    max_results: Maximum number of results to return (default: 5)

Returns:
    Search results as formatted string
```

So với Mục 3.4.2, nơi server Tavily chính thức cung cấp bốn tool (tavily-search, tavily-extract, tavily-crawl, tavily-map), custom server của ta chỉ cung cấp một tool: `search_web`. Vậy mà code client gần như y hệt. Đó là sức mạnh của chuẩn hoá: dù bạn kết nối tới một server chính thức phức tạp hay một custom server đơn giản, interface phía client vẫn như nhau.

Những gì ta đã làm được

Trong mục này, ta đã đi trọn một chu trình MCP:

1. Chuyển hàm `search_web` tự viết thành một MCP server với rất ít thay đổi.
2. Kết nối tới server bằng đúng các khuôn mẫu client đã dùng với server Tavily chính thức.
3. Xác nhận việc khám phá tool và thực thi tool hoạt động đúng.

Điều này cho thấy giá trị cốt lõi của MCP: phần hiện thực tool và phần sử dụng tool được tách rời hoàn toàn. Người phát triển server tập trung xây tool đáng tin cậy. Người phát triển agent tập trung vào suy luận và điều phối. Giao thức lo mọi thứ ở giữa.

Khi đã nắm được tool và MCP, ta có đủ các khối xây dựng cho một agent hoàn chỉnh. Ở Chương 4, ta sẽ kết hợp các năng lực này thành một agent framework vững chắc, hiện thực lớp trừu tượng tool đúng nghĩa, xử lý lỗi, và vòng suy luận lặp biến một LLM có tool thành một agent thực thụ.

3.5 Tóm tắt

- Tool mở rộng năng lực của LLM, cho phép LLM truy cập API, thực hiện tính toán và lấy thông tin bên ngoài. Tool chia thành ba nhóm: bổ sung thông tin (information-augmenting), thực thi hành động (action-executing) và chuyên biệt theo lĩnh vực (domain-specialized).
- Tool calling là cơ chế qua đó LLM sinh output có cấu trúc chỉ rõ dùng tool nào với tham số gì. LLM đóng vai trung gian giữa yêu cầu của người dùng và các tool sẵn có, nhưng không tự thực thi tool.
- Việc thực thi tool diễn ra bên ngoài LLM. LLM sinh đặc tả, một hệ thống riêng thực thi đặc tả đó, và kết quả được đưa trở lại context của LLM để suy luận tiếp.
- Tool definition là các schema có cấu trúc mô tả tool sẵn có, tham số và output mong đợi của chúng. Định nghĩa rõ ràng, tường minh là điều thiết yếu để LLM chọn tool một cách đáng tin cậy.
- Xây custom tool gồm hiện thực hàm tool, tạo tool definition và xây hạ tầng thực thi. Dù đơn giản với các trường hợp nhỏ, custom tool tạo ra gánh nặng bảo trì và sự thiếu nhất quán khi dự án mở rộng.
- MCP (Model Context Protocol) chuẩn hoá việc phát triển tool thông qua kiến trúc server–client. Server chứa và thực thi tool, còn client khám phá và gọi tool. Sự tách biệt này cho phép tái sử dụng tool giữa các dự án và giữa các nhà cung cấp.

Chương 4

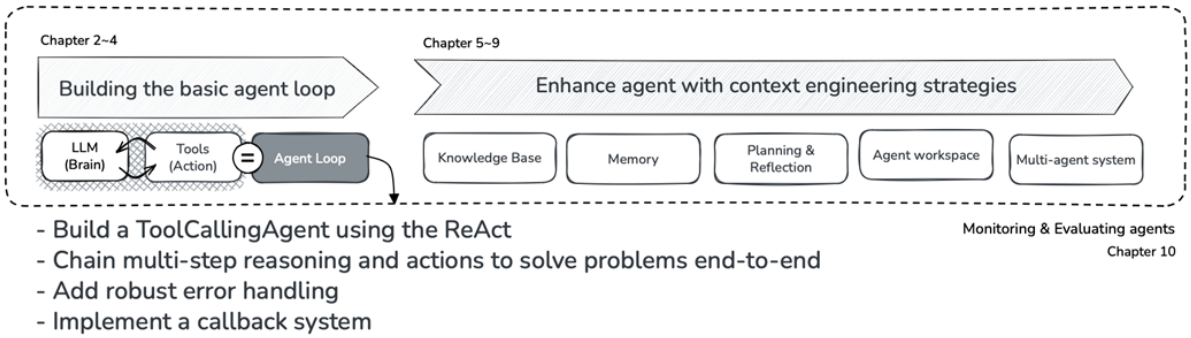
Xây dựng ReAct agent cơ bản

Nội dung chương

- ReAct, pattern cốt lõi đằng sau các AI agent hiện đại.
- Quản lý trạng thái của agent bằng ExecutionContext.
- Hợp nhất function cục bộ và MCP tool dưới một giao diện chung.
- Xây dựng tầng giao tiếp với LLM.
- Cài đặt ToolCallingAgent với vòng lặp think-act.
- Thêm structured output bằng Pydantic model.
- Kiểm tra hiệu năng agent trên benchmark GAIA.

Ở các chương trước, ta đã tìm hiểu cách LLM hoạt động và trao cho chúng khả năng dùng tool. Giờ là lúc ghép các mảnh đó thành một thứ mạnh hơn: một agent biết suy luận xem mình cần gì, hành động để lấy được thứ đó, và tiếp tục cho đến khi giải xong bài toán.

Như Hình 4.1 cho thấy, chương này hoàn thiện phần nền móng của vòng lặp agent cơ bản: xây dựng một Agent theo pattern ReAct, nối chuỗi nhiều bước suy luận và hành động để giải bài toán từ đầu đến cuối, đồng thời bổ sung cơ chế xử lý lỗi chắc chắn. ReAct (Reasoning + Acting) không phải là một framework mà là một cách thiết kế agent mô phỏng cách con người giải quyết vấn đề: đánh giá tình huống, quyết định cần thông tin hay hành động gì, thực hiện hành động đó, quan sát kết quả, rồi lặp lại.



Hình 4.1: Tổng quan cấu trúc sách, tập trung vào Chương 4. *(PDF gốc tr. 119)*

Kết thúc chương, bạn sẽ có một Agent chạy được: biết tìm kiếm web để lấy thông tin thời gian thực, nối nhiều bước suy luận để giải bài toán, xử lý lỗi một cách êm ái và trả về structured output theo bất kỳ định dạng nào bạn chỉ định. Ta sẽ thử nó trên benchmark GAIA. Agent này là nền tảng cho mọi thứ phía sau: hệ thống memory, chiến lược planning, truy xuất tri thức và cộng tác đa agent.

4.1 ReAct agent hoạt động như thế nào

Hãy nghĩ xem bạn sẽ trả lời câu hỏi sau thế nào: "Nếu vận động viên marathon Eliud Kipchoge có thể duy trì tốc độ kỷ lục thế giới của mình mãi mãi, anh ấy sẽ mất bao lâu để chạy tới Mặt Trăng?"

Bạn sẽ không cố trả lời ngay. Thay vào đó, bạn nghĩ: "Mình cần tốc độ marathon của Kipchoge và khoảng cách Trái Đất – Mặt Trăng." Sau đó bạn đi tìm các thông tin đó, tính kết quả và diễn đạt câu trả lời. Chu trình tự nhiên giữa suy nghĩ và hành động này chính là cách ReAct agent vận hành.

ReAct, viết tắt của Reasoning cộng Acting, là pattern nền tảng của hầu hết AI agent hiện đại. Ý tưởng cốt lõi rất đơn giản: thay vì cố trả lời mọi thứ trong một lần, agent luân phiên giữa việc suy luận xem mình cần gì và thực hiện hành động để lấy được thứ đó.

4.1.1 Chu trình think-act

Hãy xem một ReAct agent sẽ xử lý bài toán Kipchoge như thế nào:

```
Thought: I need to find Kipchoge's marathon world record pace.
Action: search_web("Kipchoge marathon world record")
Observation: Kipchoge's record is 2:01:09 for 42.195 km (Berlin 2022)
Thought: Now I need the Earth-Moon distance at closest approach.
Action: search_wikipedia("Moon perigee distance")
Observation: The Moon's perigee (closest approach) is 356,500 km
Thought: I can now calculate the time needed.
Action: calculator("356500 / (42.195 / 2.0186) * 1")
Observation: 17,034 hours
Thought: I have enough information to answer.
Final Answer: Approximately 17,000 hours
```

Hãy để ý pattern ở đây. Ở mỗi bước, agent nói rõ mình đang nghĩ gì, thực hiện một hành động cụ thể và quan sát kết quả. Chu trình này lặp lại cho đến khi agent thu thập đủ thông tin để đưa ra câu trả lời cuối cùng.

Sức mạnh của cách tiếp cận này nằm ở khả năng thích ứng. Nếu lần tìm kiếm đầu tiên trả về kết quả vô ích, agent có thể thử một truy vấn khác. Nếu phép tính có vẻ sai, nó có thể kiểm chứng bằng cách tìm thêm thông tin. Khác với một script cố định sẽ hỏng khi có điều bất ngờ xảy ra, ReAct agent điều chỉnh cách làm dựa trên những gì nó phát hiện ra.

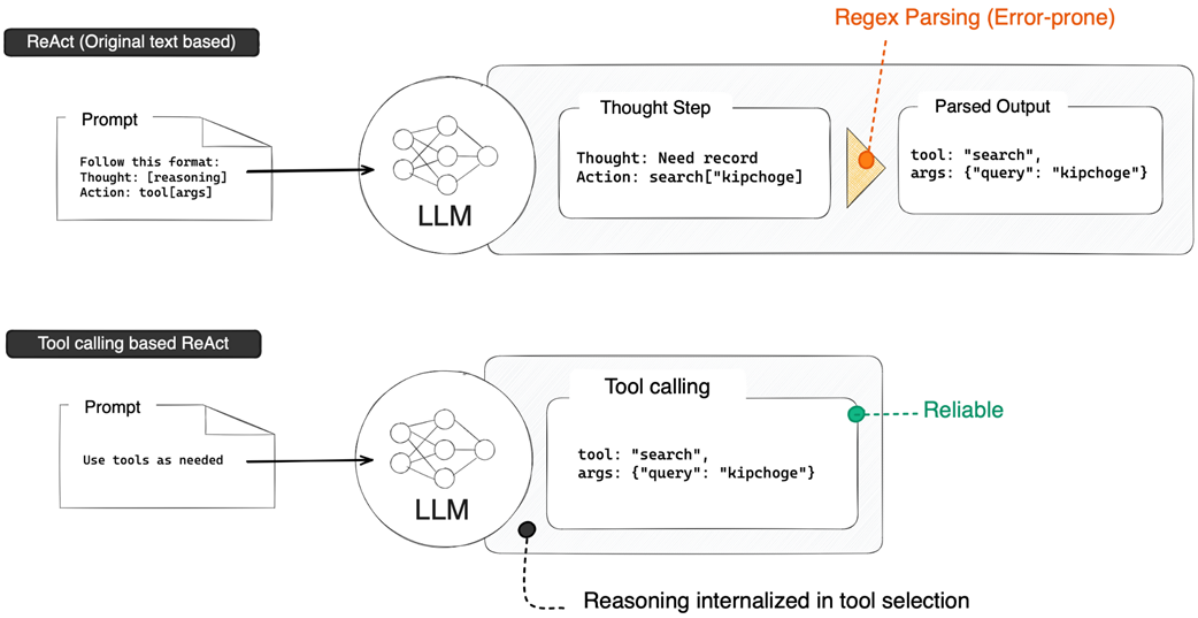
Điều này giống cách con người giải quyết vấn đề một cách tự nhiên. Ta không thực thi một chuỗi bước định sẵn. Ta nghĩ, hành động, quan sát kết quả và quyết định làm gì tiếp theo. LLM lo phần suy luận, còn tool lo phần hành động. Kết hợp lại, chúng tạo thành một hệ thống giải được những bài toán mà từng thành phần riêng lẻ không thể giải.

4.1.2 Từ phân tích văn bản (text parsing) đến tool calling

Các bản cài đặt ReAct đời đầu gặp một vấn đề khó chịu: chúng hoàn toàn dựa vào việc phân tích văn bản. Agent phải xuất phản hồi theo đúng một định dạng chính xác, chẳng hạn `Action: search["query"]`, và chỉ cần lệch một chút là hệ thống hỏng. Thiếu một dấu ngoặc, thừa một khoảng trắng hay dùng một từ khoá hơi khác đều gây lỗi parse. Lập trình viên đã tốn rất nhiều công sức soạn prompt "nài nỉ" LLM tuân theo các quy tắc định dạng chặt chẽ, vậy mà lỗi vẫn thường xuyên xảy ra.

Tool Calling loại bỏ sự mong manh này. Thay vì phải parse văn bản tự do, LLM sinh ra output có cấu trúc, chỉ rõ cần gọi tool nào với những đối số gì. Định dạng được API đảm bảo, chứ không phải trông chờ LLM làm theo hướng dẫn một cách hoàn hảo.

Hình 4.2 minh hoạ sự tiến hoá này. Bên trái, cách tiếp cận truyền thống yêu cầu văn bản "Thought" và "Action" tường minh, phải được parse. Bên phải, Tool Calling hiện đại xử lý việc này một cách tự nhiên ngay trong API.



Hình 4.2: Sự tiến hoá từ ReAct dựa trên văn bản sang ReAct dựa trên tool calling. Bước "Thought" tường minh đã được nội hoá vào quá trình chọn tool của LLM, loại bỏ nhu cầu parse văn bản dễ vỡ. *(PDF gốc tr. 122)*

Bạn có thể thắc mắc: nếu không còn bước "Thought" tường minh nữa thì đó có còn là ReAct không? Có. Việc suy luận không biến mất, nó chỉ chuyển vào bên trong LLM. Như đã thấy ở Chương 3, các LLM hiện đại được huấn luyện trên rất nhiều ví dụ tool calling, nhờ đó học được tool nào phù hợp với tình huống nào. Khi LLM chọn một tool và chỉ định đối số cho nó, nó đã thực hiện xong phần suy luận ở bên trong. Pattern vẫn giữ nguyên: suy luận xem cần gì, hành động để lấy nó, quan sát kết quả, lặp lại.

Đó là lý do các agent dựa trên Tool Calling vẫn được gọi là ReAct agent trong các framework lớn như LangGraph, Google ADK và HuggingFace SmolAgents. Các nguyên tắc cốt lõi vẫn còn nguyên: giải quyết vấn đề một cách thích ứng thông qua suy luận và hành động lặp đi lặp lại, với LLM tự chọn tool dựa trên context thay vì đi theo một script định sẵn.

Giờ ta đã sẵn sàng xây agent của riêng mình. Hãy xem toàn bộ kiến trúc của thứ ta sắp cài đặt, bắt đầu từ kết quả cuối cùng rồi đi ngược lại để hiểu từng thành phần cần xây.

4.2 Tổng quan kiến trúc agent

Trước khi đi vào chi tiết cài đặt, hãy xem ta đang hướng tới cái gì. Nắm được bức tranh tổng thể trước sẽ giúp bạn hiểu vì sao mỗi thành phần tồn tại và chúng phối hợp với nhau ra sao. Code của chương này có tại: https://github.com/shangrilar/ai-agent-from-scratch/tree/main/chapter_04_basic_agent.

4.2.1 Agent hoàn chỉnh

Đây là cách agent hoàn chỉnh của ta sẽ trông khi chạy.

Listing 4.1. Ví dụ sử dụng agent

```
from scratch_agents import Agent, LlmClient
from scratch_agents.tools import calculator, search_web

agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[calculator, search_web],
```

```
    instructions="You are a helpful assistant"
)

result = await agent.run("What is 1234 * 5678?")
```

Giao diện đơn giản này che giấu khá nhiều phức tạp. Agent nhận câu hỏi của người dùng, quyết định có dùng tool hay không, thực thi tool nếu cần và trả về câu trả lời cuối cùng. Với câu hỏi phép nhân ở trên, nó sẽ gọi tool `calculator` và trả về "7,006,652".

Nhưng điều gì diễn ra phía sau API gọn gàng này? Để xây một agent thực sự chạy được, ta cần hiểu luồng thông tin và các thành phần làm nên nó.

4.2.2 Luồng thông tin: thiết kế cốt lõi

Về mặt khái niệm, công việc của agent rất rõ ràng: nhận input của người dùng, hỏi LLM nên làm gì tiếp theo, thực thi tool nếu được yêu cầu, đưa kết quả trở lại cho LLM, và lặp lại cho đến khi xong. Chu trình này tiếp diễn cho tới khi LLM quyết định đã có đủ thông tin để đưa ra câu trả lời cuối cùng.

Vì sao chỉ có danh sách message là chưa đủ

Bạn có thể nghĩ rằng chỉ cần duy trì một danh sách message đơn giản (lịch sử hội thoại với LLM) là đủ. Dù sao thì đó cũng là cách ta đã làm ở Chương 2 khi quản lý hội thoại. Tuy nhiên, chạy một agent đòi hỏi theo dõi nhiều thông tin hơn thế:

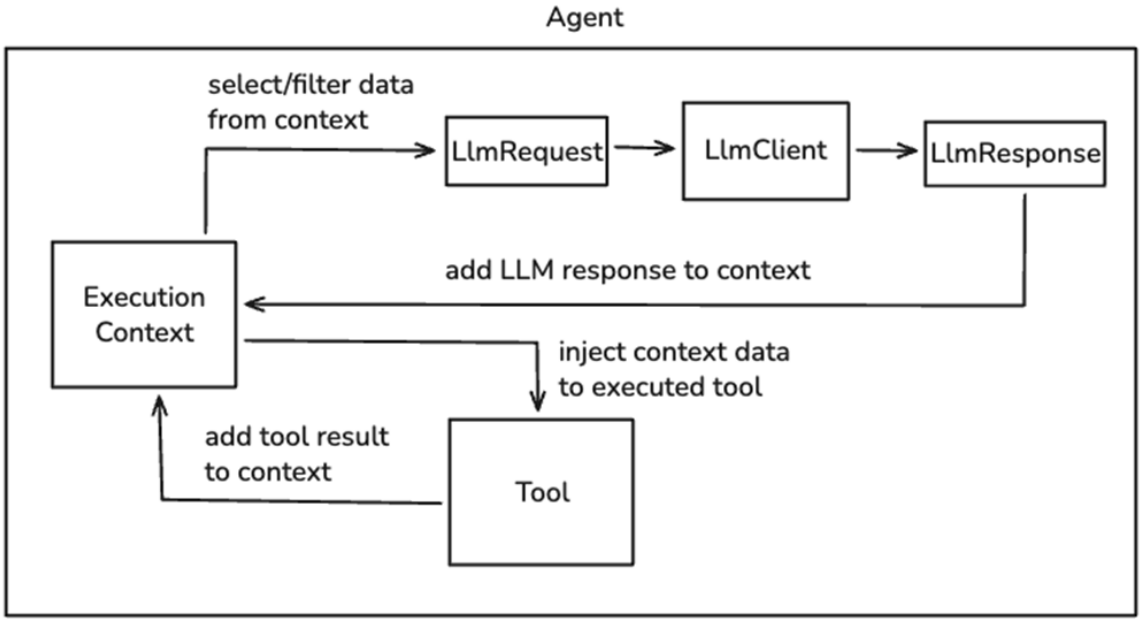
- **events**: lịch sử của mọi tương tác, gồm message của người dùng, phản hồi của LLM và kết quả thực thi tool.
- **current_step**: bộ đếm để tránh vòng lặp vô hạn (nếu agent cứ gọi tool mãi thì sao?).
- **execution_id**: định danh duy nhất phục vụ debug và logging (lần thực thi nào đã sinh ra lỗi này?).
- **state**: một "bảng nháp" cho dữ liệu động trong lúc thực thi, như tiến độ công việc, kết quả trung gian, hay sở thích của người dùng mà tool có thể cần.
- **final_result**: cờ cho biết khi nào việc thực thi đã hoàn tất.

Nếu rải các thông tin này ra nhiều biến khác nhau và truyền riêng lẻ cho từng method, code sẽ trở nên cồng kềnh. Mỗi method sẽ cần một danh sách tham số dài, và việc thêm thông tin mới (như memory ở Chương 6) sẽ buộc phải sửa chữ ký của mọi method.

Giải pháp là gom tất cả vào một container duy nhất. Mọi method đều nhận container này, và khi cần thêm thông tin mới, ta chỉ sửa định nghĩa của container. Container đó là **ExecutionContext**, nơi lưu trữ trung tâm cho toàn bộ trạng thái thực thi.

Thiết kế lấy ExecutionContext làm trung tâm

Hình 4.3 minh hoạ cách thông tin chảy qua agent, với ExecutionContext ở trung tâm.



Hình 4.3: Luồng thông tin trong kiến trúc agent. ExecutionContext nằm ở trung tâm, quản lý toàn bộ trạng thái thực thi. Các mũi tên thể hiện: (1) Context tới LlmRequest tới LlmClient cho các lời gọi LLM đi ra, (2) LlmResponse quay về Context cho các phản hồi đi vào, và (3) luồng hai chiều giữa Context và Tools khi thực thi tool. *(PDF gốc tr. 125)*

ExecutionContext là nơi lưu trữ trung tâm, nơi mọi thông tin hội tụ. Khi agent cần gọi LLM, nó trích thông tin liên quan từ ExecutionContext vào một LlmRequest. Khi LLM phản hồi, phản hồi đó chảy ngược về ExecutionContext. Khi tool thực thi, agent truyền ExecutionContext cho tool để tool truy cập được bất kỳ trạng thái thực thi nào nó cần, và kết quả của tool được ghi lại vào ExecutionContext.

Kiến trúc này mang lại hai lợi ích chính. Thứ nhất, nó đơn giản hoá chữ ký method vì mọi method chỉ cần nhận context. Thứ hai, nó cho phép **context propagation** (lan truyền context): agent có thể truyền ExecutionContext cho những tool cần truy cập trạng thái thực thi (ví dụ một tool kiểm tra quyền của người dùng trước khi hành động).

Hãy chú ý sự tách biệt giữa LlmRequest và ExecutionContext. ExecutionContext lưu mọi thứ, còn LlmRequest là một tập con được chọn lọc, chỉ chứa những gì LLM cần cho một lời gọi cụ thể. Sự tách biệt này chính là nơi context engineering diễn ra: quyết định đưa thông tin nào vào, bỏ thông tin nào ra, và định dạng nó thế nào để LLM hoạt động tốt nhất.

4.2.3 Các thành phần cần xây dựng

Khi kiến trúc đã rõ, đây là lộ trình cài đặt của ta. Mỗi thành phần có một vai trò cụ thể trong việc quản lý luồng thông tin vừa bàn.

Bảng 4.1: Các thành phần của agent và lộ trình cài đặt

Thứ tự	Thành phần	Mục	Vai trò
1	ExecutionContext	4.3	Nơi lưu trữ trung tâm, quản lý mọi thông tin trong quá trình thực thi
2	Tool Abstraction	4.4	Hợp nhất các tool từ Chương 3 dưới một giao diện nhất quán, có thể nhận Context
3	LLM Communication Layer	4.5	LlmRequest chọn thông tin gửi cho LLM; LlmClient xử lý lời gọi API; LlmResponse chuẩn hoá phản hồi
4	Agent	4.6	Bộ điều phối: tạo Context, điều phối luồng thông tin và cài đặt vòng lặp think-act

Ta sẽ xây từ nền móng lên. ExecutionContext đi trước vì mọi thứ khác phụ thuộc vào nó. Tool abstraction theo sau vì ta cần một cách nhất quán để định nghĩa và thực thi tool. Tầng giao tiếp LLM chuẩn hoá cách ta nói chuyện với các mô hình ngôn ngữ. Cuối cùng, class Agent ghép mọi thứ lại thành vòng lặp think-act để giải bài toán.

4.3 ExecutionContext: nơi lưu trữ trung tâm của agent

Ta cần ExecutionContext để tránh rải trạng thái ra nhiều biến và để đơn giản hoá cách các thành phần chia sẻ thông tin. Giờ hãy xác định *cần lưu gì* bằng cách xem điều gì thực sự xảy ra khi một agent chạy.

4.3.1 Điều gì xảy ra khi agent thực thi?

Nhớ lại bài toán Kipchoge ở mục 4.1. Khi agent giải bài toán này, một chuỗi sự kiện diễn ra:

1. Người dùng hỏi: “How long would it take Kipchoge to run to the Moon?”
2. LLM quyết định cần thông tin → yêu cầu tool `search_web`.
3. Tool thực thi và trả về: “Kipchoge’s record is 2:01:09 for 42.195km.”
4. LLM cần thêm thông tin → yêu cầu tool `search_wikipedia`.
5. Tool thực thi và trả về: “Moon’s perigee is 356,500 km.”
6. LLM giờ đã có thể tính → yêu cầu tool `calculator`.
7. Tool thực thi và trả về: “17034”.
8. LLM đưa ra câu trả lời cuối cùng: “Approximately 17,000 hours.”

Nhìn vào chuỗi này, ta nhận ra ba loại sự việc khác nhau:

- **Message** là các trao đổi văn bản trong hội thoại. Câu hỏi ban đầu của người dùng và câu trả lời cuối của LLM đều là message. Mỗi message có một role (user, assistant hoặc system) và nội dung văn bản.
- **Tool Call** xảy ra khi LLM quyết định dùng một tool. LLM chỉ định gọi tool nào và với đối số gì. Ở bước 2 phía trên, LLM yêu cầu `search_web` với truy vấn “Kipchoge marathon world record”.
- **Tool Result** ghi lại điều xảy ra khi tool thực thi, gồm output của tool và việc thực thi thành công hay thất bại. Bước 3 phía trên là một kết quả thành công chứa thành tích marathon của Kipchoge.

Hãy định nghĩa chúng thành các kiểu dữ liệu.

Listing 4.2. Các kiểu nội dung dùng cho giao tiếp của agent

```
class Message(BaseModel):
    """A text message in the conversation."""
    type: Literal["message"] = "message"
    role: Literal["system", "user", "assistant"]
    content: str

class ToolCall(BaseModel):
    """LLM's request to execute a tool."""
    type: Literal["tool_call"] = "tool_call"
    tool_call_id: str
    name: str
    arguments: dict

class ToolResult(BaseModel):
    """Result from tool execution."""
    type: Literal["tool_result"] = "tool_result"
    tool_call_id: str
    name: str
    status: Literal["success", "error"]
    content: list

ContentItem = Union[Message, ToolCall, ToolResult]
```

Trường `type` trong mỗi class đóng vai trò discriminator (trường phân biệt), giúp dễ dàng nhận ra ta đang xử lý loại nội dung nào. Trường `tool_call_id` liên kết một ToolResult ngược về ToolCall đã sinh ra nó, điều này rất quan trọng khi nhiều tool thực thi song song.

Các kiểu nội dung này ghi lại *điều gì* đã xảy ra, nhưng để debug và phân tích, ta còn cần biết *ai* tạo ra mỗi mẩu nội dung và *khi nào*. Ta bọc các content item cùng metadata bằng class Event.

Listing 4.3. Class Event dùng để theo dõi quá trình thực thi

```
class Event(BaseModel):
    """A recorded occurrence during agent execution."""
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    execution_id: str
    timestamp: float = Field(default_factory=lambda: datetime.now().timestamp())
    author: str  # "user" or agent name
    content: List[ContentItem] = Field(default_factory=list)
```

Trường `execution_id` gom mọi event của một lần chạy agent, cho phép ta lần theo toàn bộ một phiên giải bài toán. Trường `author` phân biệt input của người dùng với hành động của agent. Đây là cách một bước trong bài toán Kipchoge được biểu diễn dưới dạng Event:

```
Event(
    execution_id="abc-123",
    author="research_agent",
    content=[ToolCall(
        tool_call_id="call-1",
        name="search_web",
        arguments={"query": "Kipchoge marathon world record"}
    )]
)
```

Mỗi bước trong quá trình thực thi của agent trở thành một Event, tạo nên một audit trail (vết kiểm tra) đầy đủ, vô cùng hữu ích khi debug.

4.3.2 Cài đặt ExecutionContext

Khi đã biết cần lưu gì, việc cài đặt ExecutionContext khá đơn giản.

Listing 4.4. Cài đặt ExecutionContext

```
@dataclass
class ExecutionContext:
    """Central storage for all execution state."""
    execution_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    events: List[Event] = field(default_factory=list)
    current_step: int = 0
    state: Dict[str, Any] = field(default_factory=dict)
    final_result: Optional[str | BaseModel] = None

    def add_event(self, event: Event):
        """Append an event to the execution history."""
        self.events.append(event)

    def increment_step(self):
        """Move to the next execution step."""
        self.current_step += 1
```

Mỗi trường phục vụ một mục đích cụ thể:

- `execution_id`: định danh duy nhất cho phiên thực thi này, được sinh tự động.
- `events`: danh sách theo thứ tự thời gian của mọi Event xảy ra trong quá trình thực thi.
- `current_step`: bộ đếm để tránh vòng lặp vô hạn (agent dừng sau `max_steps`).
- `state`: kho key-value linh hoạt cho dữ liệu tuỳ biến mà tool có thể cần, như cấu hình API hay kết quả trung gian.
- `final_result`: giữ câu trả lời của agent khi việc thực thi hoàn tất.

ExecutionContext cũng có thể được truyền cho những tool cần truy cập trạng thái thực thi. Một tool có thể đọc từ `state` hoặc xem lại các event trước đó trong lịch sử thực thi. Để làm được điều này, ta cần một cách nhất quán để định nghĩa tool có thể (tuỳ chọn) nhận context. Giờ hãy cài đặt Tool Abstraction để hợp nhất các tool dạng function dưới một giao diện chung hỗ trợ khả năng này.

4.4 Tool abstraction

Ở Chương 3, ta xây tool dưới dạng các function Python đơn giản và tạo một tiện ích chuyển chúng thành tool definition cho LLM. Cách này rất hợp để học những điều cơ bản về tool calling. Nhưng khi chuẩn bị xây một framework agent hoàn chỉnh, vài thách thức thực tế xuất hiện mà cách tiếp cận dựa trên function không xử lý tốt.

4.4.1 Vì sao cần một giao diện tool thống nhất

Hãy xem lại những gì đã xây ở Chương 3. Ta tạo các function như `calculator()` và `search_web()`, rồi dùng `function_to_tool_definition()` để sinh schema mà LLM cần.

```
def search_web(query: str, max_results: int = 5) -> str:
    """Search the web for information."""
    return tavily_client.search(query, max_results=max_results)
```

```
search_tool_definition = function_to_tool_definition(search_web)
search_web(query="Python tutorials")
```

Cách này chạy được, nhưng function và metadata của nó tồn tại như hai thực thể tách rời. Khi xây agent, ta cần truyền definition cho LLM và ánh xạ tên ngược về function để thực thi, tức là phải tự tay giữ mọi thứ đồng bộ.

Với tool abstraction, mọi thứ được gói chung với nhau.

```
@tool
def search_web(query: str, max_results: int = 5) -> str:
    return tavily_client.search(query, max_results=max_results)

search_web.tool_definition
search_web.execute(query="Python tutorials")
```

Ta sẽ cài đặt điều này bằng hai class: `BaseTool` định nghĩa giao diện, và `FunctionTool` bọc các function có sẵn.

4.4.2 BaseTool: nền móng

Mọi tool trong framework của ta đều kế thừa từ BaseTool, một abstract base class định nghĩa thế nào là một tool. Hãy xem giao diện này cần cung cấp những gì.

Một tool phải có ba thuộc tính thiết yếu: tên (name) để định danh, mô tả (description) giải thích mục đích của nó, và schema `tool_definition` cho LLM biết cách dùng nó. Tool cũng phải thực thi được, điều mà ta biểu diễn bằng một method async `execute` nhận ExecutionContext cùng mọi đối số khác.

Phần khởi tạo cung cấp các giá trị mặc định hợp lý: nếu không truyền tên, nó dùng tên class; nếu không truyền mô tả, nó dùng docstring. Điều này giảm code lặp khi tạo các tool đơn giản. Property `tool_definition` trả về schema cho LLM biết tool nhận những tham số nào. Ta lưu nó trong `_tool_definition` để các subclass có thể sinh nó theo kiểu lazy nếu cần.

Listing 4.5. Abstract class BaseTool

```
class BaseTool(ABC):
    """Abstract base class for all tools."""
    def __init__(
        self,
        name: str = None,
        description: str = None,
        tool_definition: Dict[str, Any] = None,
    ):
        self.name = name or self.__class__.__name__
        self.description = description or self.__doc__ or ""
        self._tool_definition = tool_definition

    @property
    def tool_definition(self) -> Dict[str, Any] | None:
        return self._tool_definition

    @abstractmethod
    async def execute(self, context: ExecutionContext, **kwargs) -> Any:
        pass

    async def __call__(self, context: ExecutionContext, **kwargs) -> Any:
        return await self.execute(context, **kwargs)
```

Method `execute` là abstract vì mỗi tool phải tự cài đặt logic riêng. Lưu ý rằng nó luôn nhận `context` làm tham số đầu tiên. Đây là quyết định thiết kế then chốt cho phép context propagation: Agent truyền ExecutionContext cho mọi tool, và tool nào cần truy cập trạng thái thực thi thì dùng nó. Tool nào không cần context thì đơn giản là bỏ qua.

Cuối cùng, `__call__` giúp cú pháp gọn hơn. Thay vì viết `tool.execute(context, query="test")`, ta có thể viết `tool(context, query="test")`.

Khi đã định nghĩa BaseTool, ta có một "hợp đồng" mà mọi tool phải tuân theo. Ở mục tiếp theo, ta sẽ cài đặt FunctionTool để bọc các function đơn giản đã tạo ở Chương 3.

4.4.3 FunctionTool: bọc các function

FunctionTool đóng vai trò adapter, bọc các function có sẵn bằng giao diện BaseTool. Thách thức chính là context propagation: Agent truyền ExecutionContext cho mọi tool, nhưng phần lớn function không chờ tham số context. Chữ ký của chúng có dạng `search_web(query: str)`, chứ không phải `search_web(context: ExecutionContext, query: str)`.

Giải pháp là kiểm tra chữ ký của mỗi function lúc khởi tạo. Nếu function có tham số `context`, ta truyền nó vào. Nếu không, ta bỏ qua. Các function đơn giản chạy được mà không cần sửa, còn các function biết về context có thể truy cập trạng thái thực thi khi cần.

Trong lúc khởi tạo, ta kiểm tra function có nhận tham số `context` hay không. Method `execute` dùng thông tin này để quyết định có chuyển tiếp context hay không. Agent đối xử với mọi tool như nhau, truyền context cho tất cả, và mỗi tool tự quyết định có dùng nó hay không. Method `_generate_definition` tái sử dụng `function_to_input_schema` từ Chương 3 để tự động tạo tool schema từ type hint của function.

Listing 4.6. Cài đặt FunctionTool

```
class FunctionTool(BaseTool):
    """Wraps a Python function as a BaseTool."""
    def __init__(
        self,
        func: Callable,
        name: str = None,
        description: str = None,
        tool_definition: Dict[str, Any] = None
    ):
        self.func = func
        self.needs_context = 'context' in inspect.signature(func).parameters

        name = name or func.__name__
        description = description or (func.__doc__ or "").strip()
        tool_definition = tool_definition or self._generate_definition()

        super().__init__(
            name=name,
            description=description,
            tool_definition=tool_definition
        )

    async def execute(self, context: ExecutionContext, **kwargs) -> Any:
        """Execute the wrapped function."""
        if self.needs_context:
            result = self.func(context=context, **kwargs)
        else:
            result = self.func(**kwargs)

        # Handle both sync and async functions
        if inspect.iscoroutine(result):
```

```
            return await result
        return result

    def _generate_definition(self) -> Dict[str, Any]:
        """Generate tool definition from function signature."""
        parameters = function_to_input_schema(self.func)
        return format_tool_definition(self.name, self.description, parameters)
```

Đây là FunctionTool khi chạy. Tên, docstring và tham số của function được trích tự động để tạo thành một tool đầy đủ chức năng.

```
def calculator(expression: str) -> float:
    """Calculate mathematical expressions."""
    return eval(expression)

calc_tool = FunctionTool(calculator)
print(calc_tool.name)          # "calculator"
print(calc_tool.description)   # "Calculate mathematical expressions."
await calc_tool(context, expression="1234 * 5678")  # 7006652
```

Decorator @tool

Để tiện, ta cung cấp một decorator tạo ra các instance FunctionTool. Đoạn code sau tương đương với việc viết `calculator = FunctionTool(calculator)`, nhưng cú pháp gọn hơn:

```
@tool
def calculator(expression: str) -> float:
    """Calculate mathematical expressions."""
    return eval(expression)
```

Decorator này cũng nhận các tham số tuỳ chọn để ghi đè metadata của tool. Thay vì dùng tên và docstring của function, bạn có thể chỉ định giá trị riêng sẽ xuất hiện trong tool definition gửi cho LLM.

```
@tool(name="web_search", description="Search the internet for information")
def search_web(query: str) -> str:
    """Search the web."""
    return tavily_client.search(query)
```

Trong suốt cuốn sách, ta sẽ dùng `@tool` vì cú pháp gọn của nó. Phần cài đặt chỉ là một lớp bọc mỏng quanh FunctionTool; xem GitHub repository để biết chi tiết.

4.4.4 Tích hợp MCP tool

Ta đã tìm hiểu MCP và kết nối tới các MCP server để khám phá và thực thi tool. Giờ ta cần dùng các tool đó trong framework agent của mình. Thách thức là MCP tool nằm trên server bên ngoài và được thực thi qua `session.call_tool()`, trong khi Agent của ta chờ các instance BaseTool có method `execute()`.

Giải pháp là tạo một wrapper function cho mỗi MCP tool để gọi server, rồi bọc function đó bằng FunctionTool. Cách này tái sử dụng mọi thứ ta đã xây.

Function `load_mcp_tools` kết nối tới một MCP server, lấy về mọi tool có sẵn và chuyển từng tool thành một FunctionTool. Nó nhận các tham số kết nối (như lệnh khởi động server) và trả về danh sách tool sẵn sàng dùng với agent.

Listing 4.7. Nạp tool từ MCP server

```
async def load_mcp_tools(connection: dict) -> list[BaseTool]:
    """Load tools from an MCP server and convert to FunctionTools."""
    tools = []

    async with stdio_client(StdioServerParameters(**connection)) as (read,
    ↪  write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            mcp_tools = await session.list_tools()

            for mcp_tool in mcp_tools.tools:
                func_tool = _create_mcp_tool(mcp_tool, connection)
                tools.append(func_tool)

    return tools
```

Logic cốt lõi nằm trong `_create_mcp_tool`. Với mỗi MCP tool, nó tạo một wrapper function tên `call_mcp`, function này thiết lập kết nối tới server và thực thi tool với bất kỳ đối số nào được truyền vào. Wrapper function này sau đó được bọc bằng FunctionTool. Lưu ý rằng ta truyền `tool_definition` một cách tường minh thay vì để FunctionTool tự sinh, vì MCP server đã cung cấp sẵn schema đầy đủ qua `mcp_tool.inputSchema`.

Listing 4.8. Tạo FunctionTool từ MCP tool

```
def _create_mcp_tool(mcp_tool, connection: dict) -> FunctionTool:
    """Create a FunctionTool that wraps an MCP tool."""

    async def call_mcp(**kwargs):
        async with stdio_client(StdioServerParameters(**connection)) as (read,
        ↪  write):
            async with ClientSession(read, write) as session:
                await session.initialize()
                result = await session.call_tool(mcp_tool.name, kwargs)
                return _extract_text_content(result)

    tool_definition = {
        "type": "function",
        "function": {
            "name": mcp_tool.name,
            "description": mcp_tool.description,
            "parameters": mcp_tool.inputSchema,
        }
    }

    return FunctionTool(
        func=call_mcp,
        name=mcp_tool.name,
        description=mcp_tool.description,
        tool_definition=tool_definition
    )
```

Với adapter này, MCP tool tích hợp liền mạch vào agent. Ví dụ sau nạp tool từ Tavily MCP server và kết hợp chúng với một tool `calculator` cục bộ. Agent không biết và cũng không quan tâm tool nào là function cục bộ, tool nào gọi MCP server từ xa. Tất cả đều cài đặt cùng một giao diện BaseTool.

```
connection = {
    "command": "npx",
```

```
    "args": ["-y", "tavily-mcp@latest"],
    "env": {"TAVILY_API_KEY": os.getenv("TAVILY_API_KEY")}
}
mcp_tools = await load_mcp_tools(connection)

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[calculator, *mcp_tools],
    instructions="You are a helpful assistant."
)
```

Lưu ý về quản lý kết nối

Bản cài đặt này tạo một kết nối mới cho mỗi lần gọi tool. Cách này dễ hiểu nhưng tốn thêm chi phí. Trong hệ thống production thường xuyên dùng MCP tool, bạn có thể duy trì một session lâu dài (persistent session) để tránh thiết lập kết nối lặp đi lặp lại.

4.5 Tầng giao tiếp với LLM

Khi ExecutionContext đã lưu trạng thái thực thi và các tool đã được hợp nhất dưới BaseTool, ta cần thêm một mảnh nữa: cách giao tiếp với LLM. Tầng giao tiếp này bắc cầu giữa các cấu trúc dữ liệu nội bộ của ta và yêu cầu của API LLM.

4.5.1 Vì sao cần một tầng giao tiếp?

Hãy xem điều gì xảy ra khi agent cần gọi LLM. ExecutionContext chứa một danh sách Event, mỗi Event giữ các ContentItem như Message, ToolCall và ToolResult. Nhưng API LLM chờ một định dạng message cụ thể: danh sách các dictionary có role và content. Phải có ai đó dịch qua lại giữa hai cách biểu diễn này.

Ta cũng phải đối mặt với sự đa dạng của các nhà cung cấp. Dù dùng LiteLLM để che đi phần lớn khác biệt giữa các provider (như đã học ở Chương 2), ta vẫn cần cấu trúc request một cách nhất quán và parse response về một định dạng chuẩn. Không có tầng này, logic chuyển đổi sẽ rải khắp class Agent, khiến việc bảo trì và mở rộng khó hơn.

Giải pháp là ba thành phần phối hợp với nhau:

- **LlmRequest** đóng gói những gì ta muốn gửi: instruction, nội dung hội thoại và các tool có sẵn. Nó là "khu tập kết" nơi ta chọn lọc và sắp xếp thông tin trước khi gọi API.
- **LlmClient** xử lý việc giao tiếp API thực sự. Nó chuyển LlmRequest thành định dạng API LLM yêu cầu, thực hiện lời gọi, rồi chuyển response trở lại.
- **LlmResponse** chuẩn hoá những gì ta nhận về. Dù dùng provider nào, response cũng trở về cùng một định dạng mà Agent xử lý được.

Sự tách biệt này giữ cho mỗi thành phần tập trung. LlmRequest không biết gì về định dạng API. LlmClient không biết gì về ExecutionContext. Mỗi mảnh làm tốt một việc.

4.5.2 LlmRequest: chọn những gì sẽ gửi

LlmRequest là "cổng ra" từ agent tới LLM. Nó giữ mọi thứ cần cho một lần gọi LLM, sắp xếp thành bốn thành phần.

Listing 4.9. Class LlmRequest

```
class LlmRequest(BaseModel):
    """Request object for LLM calls."""
    instructions: List[str] = Field(default_factory=list)
    contents: List[ContentItem] = Field(default_factory=list)
    tools: List[BaseTool] = Field(default_factory=list)
    tool_choice: Optional[str] = None
```

Trường `instructions` giữ các mảnh system prompt. Thay vì một prompt nguyên khối, ta cho phép nhiều chuỗi instruction được ghép lại với nhau. Sự linh hoạt này hữu ích khi instruction đến từ nhiều nguồn khác nhau: instruction cơ bản của agent, hướng dẫn riêng cho từng tác vụ, hoặc context được sinh động.

Trường `contents` chứa lịch sử hội thoại dưới dạng các đối tượng ContentItem: Message từ user và assistant, ToolCall mà LLM đã yêu cầu, và ToolResult từ các lần thực thi. Đây là context cốt lõi mà LLM dùng để hiểu tình hình hiện tại.

Trường `tools` liệt kê các tool có sẵn dưới dạng instance BaseTool. LlmClient sẽ trích definition của chúng khi xây request API.

Trường `tool_choice` kiểm soát cách LLM chọn tool. Đặt nó là `"auto"` cho phép LLM tự do quyết định. Đặt là `"required"` buộc phải dùng tool, điều này trở nên quan trọng với structured output ở mục 4.6 (theo sách; phần structured output thực tế được trình bày ở mục 4.7).

Hãy để ý những gì LlmRequest *không* chứa: nó không tham chiếu tới ExecutionContext hay Event. Agent chịu trách nhiệm trích thông tin liên quan từ ExecutionContext và đóng gói vào LlmRequest. Sự tách biệt này là có chủ ý. Nó khiến LlmRequest chỉ là một container dữ liệu đơn giản, còn logic chọn context nằm trong Agent, đúng chỗ của nó.

Đây là nơi context engineering diễn ra. Ở mục 4.6, ta sẽ cài đặt `_prepare_llm_request()` trong class Agent, method quyết định thông tin nào từ ExecutionContext được đưa vào mỗi LlmRequest. Hiện tại, ta sẽ "làm phẳng" (flatten) mọi event thành contents. Ở Chương 6, đây sẽ trở thành điểm mở rộng cho các chiến lược memory và quản lý context.

4.5.3 LlmResponse: chuẩn hoá những gì nhận về

LlmResponse là "cổng vào", chuẩn hoá response của LLM bất kể provider nào sinh ra nó.

Listing 4.10. Class LlmResponse

```
class LlmResponse(BaseModel):
    """Response object from LLM calls."""
    content: List[ContentItem] = Field(default_factory=list)
    error_message: Optional[str] = None
    usage_metadata: Dict[str, Any] = Field(default_factory=dict)
```

Trường `content` chứa những gì LLM tạo ra, biểu diễn bằng các đối tượng ContentItem. Một response có thể gồm một Message chứa văn bản, một hay nhiều ToolCall yêu cầu thực thi tool, hoặc cả hai. Dùng chung các kiểu ContentItem đã định nghĩa ở mục 4.3 giúp mô hình dữ liệu nhất quán trong toàn hệ thống.

Trường `error_message` ghi lại lỗi. Khi lời gọi API thất bại do sự cố mạng, giới hạn tốc độ (rate limit) hay request không hợp lệ, ta lưu lỗi ở đây thay vì ném exception. Điều này cho phép Agent xử lý lỗi êm ái, chẳng hạn thử lại hoặc thông báo cho người dùng.

Trường `usage_metadata` theo dõi lượng token tiêu thụ. Biết mỗi lời gọi dùng bao nhiêu input token và output token giúp quản lý chi phí và debug. Hiện tại ta chỉ lưu thông tin này. Chương 10 sẽ bàn chi tiết về monitoring và theo dõi chi phí.

4.5.4 LlmClient: adapter cho các provider

LlmClient nối LlmRequest và LlmResponse với API LLM thực tế. Nhờ LiteLLM (đã giới thiệu ở Chương 2), ta có thể hỗ trợ nhiều provider chỉ với một bản cài đặt.

Listing 4.11. Class LlmClient

```
class LlmClient:
    """Client for LLM API calls using LiteLLM."""

    def __init__(self, model: str, **config):
        self.model = model
        self.config = config

    async def generate(self, request: LlmRequest) -> LlmResponse:
        """Generate a response from the LLM."""
        try:
            messages = self._build_messages(request)
            tools = [t.tool_definition for t in request.tools] if request.tools
            ↪  else None

            response = await acompletion(
                model=self.model,
                messages=messages,
                tools=tools,
                **({"tool_choice": request.tool_choice}
                    if request.tool_choice else {}),
                **self.config
            )

            return self._parse_response(response)
        except Exception as e:
            return LlmResponse(error_message=str(e))
```

Constructor nhận một định danh model và các tham số cấu hình tuỳ chọn như temperature hay `max_tokens`. Định danh model có dạng `"gpt-5-mini"` hoặc `"anthropic/claude-sonnet-4-5"`.

Method `generate` điều phối lời gọi API qua ba bước. Thứ nhất, nó xây danh sách messages từ request. Thứ hai, nó trích tool definition bằng một list comprehension đơn giản. Thứ ba, nó gọi `acompletion` của LiteLLM và parse response. Nếu có gì thất bại, nó trả về một LlmResponse chứa lỗi thay vì làm sập chương trình.

Xây dựng messages

Method `_build_messages` chuyển contents của LlmRequest thành định dạng message mà API LLM yêu cầu. Điểm phức tạp chính ở đây là API của OpenAI yêu cầu assistant message và các tool call của nó phải nằm chung trong một đối tượng message, trong khi ta lưu chúng thành các ContentItem riêng biệt.

Listing 4.12. Xây dựng messages cho API LLM

```
def _build_messages(self, request: LlmRequest) -> List[dict]:
    """Convert LlmRequest to API message format."""
    messages = []

    for instruction in request.instructions:
        messages.append({"role": "system", "content": instruction})

    for item in request.contents:
        if isinstance(item, Message):
```

```
            messages.append({"role": item.role, "content": item.content})

        elif isinstance(item, ToolCall):
            tool_call_dict = {
                "id": item.tool_call_id,
                "type": "function",
                "function": {
                    "name": item.name,
                    "arguments": json.dumps(item.arguments)
                }
            }
            # Append to previous assistant message if exists
            if messages and messages[-1]["role"] == "assistant":
                messages[-1].setdefault("tool_calls", []).append(tool_call_dict)
            else:
                messages.append({
                    "role": "assistant",
                    "content": None,
                    "tool_calls": [tool_call_dict]
                })

        elif isinstance(item, ToolResult):
            messages.append({
                "role": "tool",
                "tool_call_id": item.tool_call_id,
                "content": str(item.content[0]) if item.content else ""
            })

    return messages
```

Logic này xử lý mỗi loại ContentItem theo cách riêng. Message trở thành đối tượng message tiêu chuẩn. ToolCall được nối vào mảng `tool_calls` của assistant message ngay trước đó, vì agent của ta lưu chúng liên tiếp nhau từ cùng một response của LLM. Nếu một ToolCall xuất hiện mà không có assistant message đứng trước, ta tạo một message mới với content rỗng (null). ToolResult trở thành message có role `tool`, liên kết ngược về lời gọi gốc qua `tool_call_id`.

Parse response

Method `_parse_response` chuyển response của API trở lại định dạng chuẩn của ta. Method này xử lý cả response dạng văn bản lẫn tool call. Khi LLM trả về văn bản, ta bọc nó trong một Message. Khi nó yêu cầu tool, ta tạo các đối tượng ToolCall. Cả hai có thể xuất hiện trong cùng một response, và danh sách ContentItem của ta chứa được chúng một cách tự nhiên.

Listing 4.13. Parse response của API LLM

```
def _parse_response(self, response) -> LlmResponse:
    """Convert API response to LlmResponse."""
    choice = response.choices[0]
    content_items = []

    if choice.message.content:
        content_items.append(Message(
            role="assistant",
            content=choice.message.content
        ))

    if choice.message.tool_calls:
        for tc in choice.message.tool_calls:
```

```
            content_items.append(ToolCall(
                tool_call_id=tc.id,
                name=tc.function.name,
                arguments=json.loads(tc.function.arguments)
            ))

    return LlmResponse(
        content=content_items,
        usage_metadata={
            "input_tokens": response.usage.prompt_tokens,
            "output_tokens": response.usage.completion_tokens,
        }
    )
```

4.5.5 Ghép lại với nhau

Khi cả ba thành phần đã được định nghĩa, đây là cách chúng phối hợp. Ta tạo một LlmClient với định danh model, xây một LlmRequest chứa instruction và nội dung hội thoại, rồi gọi `generate()` để nhận LlmResponse. Danh sách content của response chứa output của LLM dưới dạng các đối tượng ContentItem, mà ta có thể duyệt qua để lấy câu trả lời.

Listing 4.14. Sử dụng tầng giao tiếp LLM

```
from scratch_agents import LlmClient, LlmRequest, Message

# Create client
client = LlmClient(model="gpt-5-mini")

# Build request
request = LlmRequest(
    instructions=["You are a helpful assistant."],
    contents=[Message(role="user", content="What is 2 + 2?")],
)

# Generate response
response = await client.generate(request)

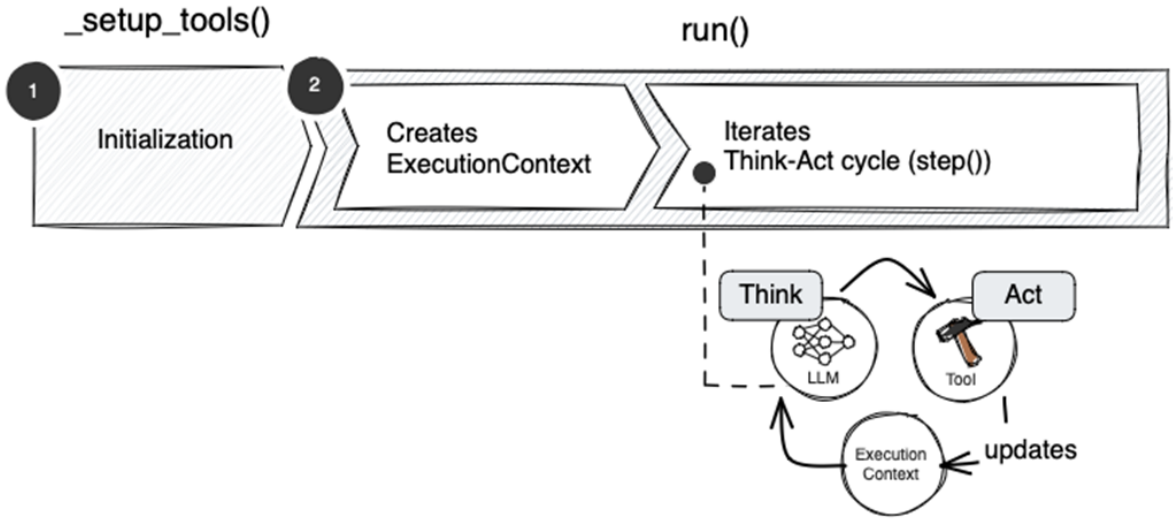
# Response contains the answer
for item in response.content:
    if isinstance(item, Message):
        print(item.content)  # "4"
```

Tầng giao tiếp này tách bạch các mối quan tâm một cách rõ ràng. LlmRequest đóng gói *cái gì* sẽ gửi. LlmClient xử lý *cách* gửi. LlmResponse chuẩn hoá những gì trả về.

4.6 Cài đặt agent

Với ExecutionContext lưu trạng thái, các tool được hợp nhất dưới BaseTool và LlmClient lo việc giao tiếp với LLM, ta đã sẵn sàng xây class Agent điều phối mọi thứ. Agent tập hợp mọi thành phần đã xây, điều phối luồng thông tin giữa LLM và tool để giải bài toán từng bước một.

Hình 4.4 cho thấy bức tranh đầy đủ về những gì ta sắp cài đặt. Agent khởi tạo bằng `_setup_tools()`, sau đó `run()` tạo một ExecutionContext và gọi `step()` lặp đi lặp lại cho đến khi hoàn tất. Mỗi lần `step()` thực hiện một chu trình think-act: `think()` gọi LLM để quyết định làm gì tiếp theo, và `act()` thực thi các tool được yêu cầu.



Hình 4.4: Lộ trình cài đặt agent, thể hiện năm method cốt lõi và luồng thực thi của chúng. Chuỗi đánh số minh hoạ cách từng thành phần phối hợp để giải bài toán. *(PDF gốc tr. 149)*

Ta sẽ cài đặt năm method này theo thứ tự hợp lý:

1. `_setup_tools()`: đăng ký các tool mà agent sẽ dùng, chuyển danh sách thành dictionary để tra cứu hiệu quả.
2. `run()`: điểm vào, nhận yêu cầu của người dùng, quản lý vòng lặp thực thi và trả về kết quả cuối cùng.
3. `step()`: thực hiện một chu trình think-act hoàn chỉnh, cập nhật ExecutionContext với mỗi tương tác.
4. `think()`: gọi LLM để phân tích tình hình và quyết định hành động tiếp theo.
5. `act()`: thực thi các tool mà LLM đã chọn và ghi lại kết quả.

Kết thúc mục này, bạn sẽ có một agent chạy được, giải được các bài toán nhiều bước. Ta cũng sẽ thêm khả năng structured output, cho phép agent trả về Pydantic model có kiểu thay vì văn bản tự do khi ứng dụng của bạn cần định dạng dữ liệu dự đoán được.

4.6.1 Cấu trúc class Agent

Hãy bắt đầu với constructor của class Agent và method `_setup_tools()`. Constructor nhận mọi thành phần đã xây ở các mục trước và lưu chúng lại để dùng trong quá trình thực thi.

Listing 4.15. Constructor của class Agent

```
class Agent:
    def __init__(
        self,
        model: LlmClient,
        tools: List[BaseTool] = None,
        instructions: str = "",
        max_steps: int = 10,
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.tools = self._setup_tools(tools or [])
```

Các tham số phục vụ những mục đích riêng:

- `model`: instance LlmClient xử lý giao tiếp với LLM.
- `tools`: danh sách các instance BaseTool mà agent có thể dùng.
- `instructions`: system prompt định nghĩa hành vi của agent.
- `max_steps`: giới hạn an toàn để tránh vòng lặp vô hạn khi agent cứ gọi tool mãi mà không đi đến kết luận.

Method `_setup_tools()` chuẩn bị danh sách tool để sử dụng. Hiện tại nó chỉ trả về danh sách nguyên vẹn, nhưng method này là điểm mở rộng cho các cải tiến sau này, như thêm các tool mặc định.

Listing 4.16. Thiết lập tool cho agent

```
def _setup_tools(self, tools: List[BaseTool]) -> List[BaseTool]:
    return tools
```

Khi cấu trúc cơ bản đã có, hãy cài đặt method `run()` điều phối toàn bộ quá trình thực thi.

4.6.2 Method run()

Method `run()` là điểm vào cho việc thực thi agent. Nó tạo môi trường thực thi, quản lý vòng lặp think-act và trả về kết quả.

Để người gọi truy cập được toàn bộ vết thực thi, ta định nghĩa `AgentResult`, gói output cuối cùng cùng với `ExecutionContext` của nó. Điều này cho phép xem xét từng bước agent đã làm, vô cùng hữu ích cho việc debug và phân tích.

Listing 4.17. Dataclass AgentResult

```
@dataclass
class AgentResult:
    """Result of an agent execution."""
    output: str | BaseModel
    context: ExecutionContext
```

Đầu tiên, method kiểm tra xem có ExecutionContext được truyền vào không, nếu không thì tạo mới. Sau đó nó bọc input của người dùng trong một Event và thêm vào context. Vòng lặp chính gọi `step()` lặp lại cho đến khi có kết quả cuối cùng hoặc chạm giới hạn số bước tối đa. Sau mỗi bước, ta kiểm tra event cuối cùng có phải là response cuối cùng không, và nếu phải thì trích kết quả.

Listing 4.18. Method run của Agent

```
async def run(
    self,
    user_input: str,
    context: ExecutionContext = None
) -> str:
    # Create or reuse context
    if context is None:
        context = ExecutionContext()

    # Add user input as the first event
    user_event = Event(
        execution_id=context.execution_id,
        author="user",
        content=[Message(role="user", content=user_input)]
```

```
    )
    context.add_event(user_event)

    # Execute steps until completion or max steps reached
    while not context.final_result and context.current_step < self.max_steps:
        await self.step(context)

        # Check if the last event is a final response
        last_event = context.events[-1]
        if self._is_final_response(last_event):
            context.final_result = self._extract_final_result(last_event)

    return AgentResult(output=context.final_result, context=context)
```

Các method hỗ trợ lo việc phát hiện hoàn tất và trích kết quả. Method `_is_final_response()` kiểm tra một event có phải câu trả lời cuối cùng hay không bằng cách xem nội dung của nó. Một event là cuối cùng khi nó không chứa tool call lẫn tool result, nghĩa là LLM đã đưa ra câu trả lời trực tiếp. Method `_extract_final_result()` duyệt qua nội dung của event để tìm message của assistant.

Listing 4.19. Các method hỗ trợ phát hiện hoàn tất

```
def _is_final_response(self, event: Event) -> bool:
    """Check if this event contains a final response."""
    has_tool_calls = any(isinstance(c, ToolCall) for c in event.content)
    has_tool_results = any(isinstance(c, ToolResult) for c in event.content)
    return not has_tool_calls and not has_tool_results

def _extract_final_result(self, event: Event) -> str:
    for item in event.content:
        if isinstance(item, Message) and item.role == "assistant":
            return item.content
    return None
```

Cách dùng rất đơn giản. Lấy câu trả lời qua `result.output`, và xem chi tiết thực thi qua `result.context` khi cần.

```
result = await agent.run("What is 1234 * 5678?")
print(result.output)                    # "7006652"
print(result.context.current_step)
```

4.6.3 Method step()

Method `step()` thực hiện một chu trình think-act hoàn chỉnh. Nó chuẩn bị request cho LLM, lấy quyết định của LLM và thực thi bất kỳ tool nào LLM yêu cầu.

Method bắt đầu bằng việc gọi `_prepare_llm_request()` để đóng gói context hiện tại thành định dạng phù hợp với LLM. Sau đó nó gọi `think()` để lấy response của LLM, bọc response đó trong một Event và ghi vào context. Nếu response chứa tool call, method gọi `act()` để thực thi chúng và ghi kết quả thành một Event khác. Cuối cùng, nó tăng bộ đếm bước.

Listing 4.20. Method step của Agent

```
async def step(self, context: ExecutionContext):
    # Prepare what to send to the LLM
    llm_request = self._prepare_llm_request(context)

    # Get LLM's decision
    llm_response = await self.think(llm_request)
```

```
    # Record LLM response as an event
    response_event = Event(
        execution_id=context.execution_id,
        author=self.name,
        content=llm_response.content,
    )
    context.add_event(response_event)

    # Execute tools if the LLM requested any
    tool_calls = [c for c in llm_response.content if isinstance(c, ToolCall)]
    if tool_calls:
        tool_results = await self.act(context, tool_calls)
        tool_event = Event(
            execution_id=context.execution_id,
            author=self.name,
            content=tool_results,
        )
        context.add_event(tool_event)

    context.increment_step()
```

Method `_prepare_llm_request()` trích thông tin từ ExecutionContext và đóng gói vào một LlmRequest. Nó làm phẳng mọi event thành một danh sách content item, kết hợp với instruction của agent và các tool có sẵn, rồi đặt tool choice là `"auto"` để LLM tự quyết định có dùng tool hay không.

Listing 4.21. Chuẩn bị LLM request từ context

```
def _prepare_llm_request(self, context: ExecutionContext) -> LlmRequest:
    # Flatten events into content items
    flat_contents = []
    for event in context.events:
        flat_contents.extend(event.content)

    return LlmRequest(
        instructions=[self.instructions] if self.instructions else [],
        contents=flat_contents,
        tools=self.tools,
        tool_choice="auto" if self.tools else None,
    )
```

Hãy để ý sự tách biệt giữa ExecutionContext và LlmRequest. ExecutionContext giữ toàn bộ lịch sử thực thi, còn LlmRequest chỉ chứa những gì ta chọn gửi cho LLM. Hiện tại ta gửi tất cả, nhưng sự tách biệt này chính là nơi context engineering diễn ra. Bạn có thể tóm tắt các message cũ, bỏ những tool result không liên quan, hoặc chèn thêm thông tin, tất cả chỉ bằng cách sửa cách xây LlmRequest mà không động đến context gốc.

4.6.4 Các method think() và act()

Hai method `think()` và `act()` xử lý hai thao tác cốt lõi trong mỗi bước: lấy quyết định của LLM và thực thi các tool được yêu cầu.

Method `think()` được cố ý giữ đơn giản. Nó nhận LlmRequest đã chuẩn bị, chuyển cho LlmClient và trả về bất kỳ response nào nhận được. Mọi phức tạp của việc xây request nằm trong `_prepare_llm_request()`, và mọi phức tạp của việc parse response nằm trong LlmClient. Điều này giữ cho `think()` chỉ có một trách nhiệm duy nhất.

Listing 4.22. Method think của Agent

```
async def think(self, llm_request: LlmRequest) -> LlmResponse:
    return await self.model.generate(llm_request)
```

Method `act()` thực thi các tool mà LLM yêu cầu. Trước tiên nó xây một dictionary ánh xạ tên tool tới instance tool để tra cứu hiệu quả. Sau đó nó duyệt qua từng ToolCall, lấy tool tương ứng, thực thi với các đối số được cung cấp và thu thập kết quả. Method truyền ExecutionContext vào mỗi lần thực thi tool, cho phép tool truy cập trạng thái thực thi nếu cần. Mỗi lần thực thi được bọc trong khối try-except, ghi nhận lỗi thành kết quả lỗi thay vì làm sập cả agent.

Listing 4.23. Method act của Agent

```
async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall]
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    results = []

    for tool_call in tool_calls:
        if tool_call.name not in tools_dict:
            raise ValueError(f"Tool '{tool_call.name}' not found")

        tool = tools_dict[tool_call.name]

        try:
            output = await tool(context, **tool_call.arguments)
            results.append(ToolResult(
                tool_call_id=tool_call.tool_call_id,
                name=tool_call.name,
                status="success",
                content=[output],
            ))
        except Exception as e:
            results.append(ToolResult(
                tool_call_id=tool_call.tool_call_id,
                name=tool_call.name,
                status="error",
                content=[str(e)],
            ))

    return results
```

Khi một lần thực thi tool thất bại, thông báo lỗi trở thành một phần của lịch sử hội thoại. LLM thấy lỗi này ở bước tiếp theo và có thể điều chỉnh cách làm, chẳng hạn thử một tool khác hoặc diễn đạt lại truy vấn. Cách xử lý lỗi êm ái này là một trong những ưu điểm của pattern ReAct: agent có thể phục hồi sau lỗi thay vì dừng hẳn.

4.7 Thêm structured output

Ở Chương 2, ta đã thử LLM trên các bài toán của benchmark GAIA bằng Structured Output. Ta định nghĩa một Pydantic model `GaiaOutput` với các trường như `is_solvable`, `unsolvable_reason` và `final_answer`, rồi truyền nó làm tham số `response_format` để đảm bảo LLM trả dữ liệu theo một cấu trúc dự đoán được. Cách này hoạt động tốt với các lời gọi LLM độc lập.

Tuy nhiên, Agent ta xây ở mục 4.6 chỉ trả về văn bản tự do. Khi LLM suy luận xong và đưa ra câu trả lời cuối, `_extract_final_result()` chỉ đơn giản trích nội dung văn bản từ assistant message. Điều này ổn cho các tương tác hội thoại, nhưng nhiều tình huống production cần dữ liệu có cấu trúc mà các hệ thống phía sau có thể parse một cách tin cậy.

Hãy xét một agent phân tích phản hồi của khách hàng. Thay vì trả về "The sentiment is positive with a confidence of 85%", bạn có thể cần nó trả về một đối tượng có cấu trúc với `sentiment: "positive"` và `confidence: 0.85`. Hoặc hình dung một research agent phải trả kết quả theo một định dạng cụ thể cho pipeline tạo báo cáo. Văn bản tự do sẽ đòi hỏi logic parse dễ vỡ, có thể hỏng bất cứ khi nào LLM diễn đạt khác đi.

Để giải quyết, ta sẽ thêm khả năng structured output cho Agent. Bài tập này phục vụ hai mục đích: bạn củng cố hiểu biết về kiến trúc Agent bằng cách sửa các method cốt lõi của nó, và bạn học được một pattern để mở rộng agent với tính năng mới.

4.7.1 Cách tiếp cận: dùng tool làm bộ định dạng output

Ý tưởng then chốt là ta có thể tận dụng hạ tầng tool calling sẵn có để ép buộc structured output. Thay vì để LLM sinh văn bản tự do rồi cố parse nó thành cấu trúc, ta tạo một tool đặc biệt nhận schema output mong muốn làm input. Khi agent cần đưa ra câu trả lời cuối cùng, nó gọi tool này với dữ liệu có cấu trúc đúng đắn, đảm bảo an toàn kiểu (type safety) và tuân thủ định dạng.

Cách tiếp cận này có vài ưu điểm. Thứ nhất, nó tái sử dụng cơ chế tool calling sẵn có, giữ cho phần cài đặt đơn giản và nhất quán. Thứ hai, nó tận dụng việc LLM đã được huấn luyện nhiều trên tool calling để đảm bảo định dạng đúng. LLM vốn đã biết cách sinh JSON hợp lệ khớp với một schema; ta chỉ cần đưa cho nó một schema khớp với cấu trúc output mong muốn.

Phần cài đặt cần sửa năm method, mỗi method đóng một vai trò cụ thể trong kiến trúc của Agent.

Method	Vai trò	Thay đổi
`__init__`	Lưu cấu hình	Thêm tham số `output_type`
`_setup_tools()`	Chuẩn bị các tool có sẵn	Tạo động tool `final_answer`
`_prepare_llm_request()`	Đóng gói context cho LLM	Buộc dùng tool khi cần structured output
`_is_final_response()`	Phát hiện hoàn tất	Kiểm tra tool `final_answer` đã thành công
`_extract_final_result()`	Trích câu trả lời	Trả về Pydantic model từ ToolResult

Hãy cài đặt từng thay đổi.

4.7.2 Sửa đổi agent

Thay đổi constructor

Đầu tiên, ta thêm tham số `output_type` vào constructor. Khi được cung cấp, nó phải là một class Pydantic model định nghĩa cấu trúc output mong muốn. Ta cũng thêm `output_tool_name` để theo dõi tên của tool được tạo động.

Listing 4.24. Constructor của Agent có structured output

```
class Agent:
    def __init__(
        self,
        model: LlmClient,
        tools: List[BaseTool] = None,
        instructions: str = "",
        max_steps: int = 10,
        output_type: Optional[Type[BaseModel]] = None,  # New parameter
    ):
        self.model = model
```

```
        self.instructions = instructions
        self.max_steps = max_steps
        self.output_type = output_type
        self.output_tool_name = None  # Will be set if output_type provided
        self.tools = self._setup_tools(tools or [])
```

Tạo tool động trong _setup_tools()

Logic then chốt nằm ở `_setup_tools()`. Khi `output_type` được chỉ định, ta tạo động một tool `final_answer` nhận Pydantic model đã chỉ định làm input. Tool này chỉ trả lại nguyên dữ liệu có cấu trúc nó nhận được, nhưng bằng cách biến nó thành tool, ta đảm bảo LLM phải định dạng response theo schema của ta.

Hãy để ý cách ta dùng closure của Python để "bắt" `self.output_type` vào tool definition. Decorator `@tool` đã cài đặt trước đó tự động xử lý việc validate Pydantic model, đảm bảo mọi lời gọi `final_answer` phải cung cấp dữ liệu khớp với schema.

Listing 4.25. Tạo động output tool

```
def _setup_tools(self, tools: List[BaseTool]) -> List[BaseTool]:
    if self.output_type is not None:
        @tool(
            name="final_answer",
            description="Return the final structured answer matching the required
            ↪  schema."
        )
        def final_answer(output: self.output_type) -> self.output_type:
            return output

        tools = list(tools)  # Create a copy to avoid modifying the original
        tools.append(final_answer)
        self.output_tool_name = "final_answer"

    return tools
```

Buộc dùng tool trong _prepare_llm_request()

Khi cần structured output, ta phải đảm bảo agent thực sự dùng tool `final_answer` thay vì đưa ra response dạng văn bản tự do. API của OpenAI và hầu hết provider khác hỗ trợ tham số `tool_choice` để buộc model dùng tool. Ta sửa `_prepare_llm_request()` để đặt tham số này cho phù hợp.

Logic rất đơn giản: nếu có output tool (nghĩa là cần structured output), ta đặt `tool_choice="required"` để buộc LLM dùng tool. Ngược lại, ta quay về hành vi ban đầu.

Listing 4.26. Buộc dùng tool cho structured output

```
def _prepare_llm_request(self, context: ExecutionContext) -> LlmRequest:
    flat_contents = []
    for event in context.events:
        flat_contents.extend(event.content)

    # Determine tool choice strategy
    if self.output_tool_name:
        tool_choice = "required"  # Force tool usage for structured output
    elif self.tools:
        tool_choice = "auto"
    else:
        tool_choice = None
```

```
    return LlmRequest(
        instructions=[self.instructions] if self.instructions else [],
        contents=flat_contents,
        tools=self.tools,
        tool_choice=tool_choice,
    )
```

Phát hiện hoàn tất trong _is_final_response()

Để phát hiện đúng lúc agent đã tạo ra structured output cuối cùng, ta sửa `_is_final_response()`. Khi cần structured output, ta tìm một kết quả thành công từ tool `final_answer`. Chỉ khi chính tool này trả về thành công thì ta mới coi công việc của agent là hoàn tất. Với response dạng văn bản tự do, ta giữ hành vi ban đầu.

Listing 4.27. Phát hiện hoàn tất với structured output

```
def _is_final_response(self, event: Event) -> bool:
    if self.output_tool_name:
        # For structured output: check if final_answer tool succeeded
        for item in event.content:
            if (isinstance(item, ToolResult)
                and item.name == self.output_tool_name
                and item.status == "success"):
                return True
        return False

    # Original logic for free-text responses
    has_tool_calls = any(isinstance(c, ToolCall) for c in event.content)
    has_tool_results = any(isinstance(c, ToolResult) for c in event.content)
    return not has_tool_calls and not has_tool_results
```

Trích kết quả có cấu trúc

Cuối cùng, ta cập nhật `_extract_final_result()` để trích dữ liệu có cấu trúc từ tool result. Khi có output tool bắt buộc được chỉ định, ta tìm lần thực thi thành công của nó và trả về instance Pydantic model đã được validate. Điều này cho code gọi một đối tượng có kiểu chặt chẽ để làm việc thay vì văn bản thô.

Listing 4.28. Trích kết quả có cấu trúc

```
def _extract_final_result(self, event: Event) -> str | BaseModel:
    if self.output_tool_name:
        # Extract structured output from final_answer tool result
        for item in event.content:
            if (isinstance(item, ToolResult)
                and item.name == self.output_tool_name
                and item.status == "success"
                and item.content):
                return item.content[0]

    # Original logic for free-text responses
    for item in event.content:
        if isinstance(item, Message) and item.role == "assistant":
            return item.content
    return None
```

4.7.3 Dùng structured output trong thực tế

Với các sửa đổi này, việc dùng structured output trở nên đơn giản. Đây là cách bạn có thể xây một agent phân tích phản hồi khách hàng và trả về dữ liệu sentiment có cấu trúc.

Listing 4.29. Dùng structured output trong thực tế

```
from pydantic import BaseModel
from typing import Literal, List

class SentimentAnalysis(BaseModel):
    sentiment: Literal["positive", "negative", "neutral"]
    confidence: float
    key_phrases: List[str]

agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[],
    instructions="Analyze the sentiment of the provided text.",
    output_type=SentimentAnalysis
)

result = await agent.run("This product exceeded my expectations! Highly
↪  recommend.")
print(f"Sentiment: {result.sentiment}")      # "positive"
print(f"Confidence: {result.confidence}")    # 0.92
print(f"Key phrases: {result.key_phrases}")  # ["exceeded expectations", "highly
↪  recommend"]
```

Agent xử lý input, suy luận về sentiment, và khi sẵn sàng trả lời thì gọi tool `final_answer` với một đối tượng `SentimentAnalysis` có cấu trúc đúng. Kiểu trả về chính là Pydantic model chứ không phải chuỗi, nên bạn có thể truy cập trực tiếp các trường với đầy đủ type safety.

Pattern này mở rộng tự nhiên sang các tình huống phức tạp hơn. Bạn có thể định nghĩa schema output cho bản tóm tắt nghiên cứu, kết quả trích xuất dữ liệu, hay bất kỳ định dạng có cấu trúc nào ứng dụng của bạn cần. Bằng cách tích hợp structured output qua cơ chế tool calling sẵn có, ta đã thêm type safety và đảm bảo định dạng mà không làm phức tạp logic cốt lõi của agent.

Ghi chú

Trong Listing 4.29 của sách, các trường được truy cập trực tiếp trên `result` (`result.sentiment`, …). Với `run()` trả về `AgentResult` như Listing 4.18, model nằm ở `result.output`, tức là `result.output.sentiment` (như cách Listing 4.34 dùng `result.output.final_answer`). Code ở trên được giữ nguyên như sách.

4.8 Kiểm tra với benchmark GAIA

Ở Chương 2, ta đã thử LLM trực tiếp trên các bài toán GAIA và quan sát những giới hạn của chúng. Phần lớn bài toán đòi hỏi thông tin mà LLM không có, dẫn đến độ chính xác thấp. Giờ hãy xem Agent của ta hoạt động ra sao khi được trang bị tool.

4.8.1 Từ LLM đến agent

Cách thiết lập đánh giá phần lớn giữ nguyên như ở Chương 2. Ta tái sử dụng `GaiaOutput`, `gaia_prompt`, `is_correct()` và cấu trúc của bộ chạy thí nghiệm. Khác biệt then chốt là thứ giải mỗi bài toán: thay vì gọi LLM trực tiếp, giờ ta dùng một Agent có tool.

Đầu tiên, ta nạp tool từ Tavily MCP server, thứ đã tìm hiểu ở Chương 3.

Listing 4.30. Nạp MCP tool cho GAIA

```
from scratch_agents.tools import load_mcp_tools

tavily_connection = {
    "command": "npx",
    "args": ["-y", "tavily-mcp@latest"],
    "env": {"TAVILY_API_KEY": os.getenv("TAVILY_API_KEY")}
}

mcp_tools = await load_mcp_tools(tavily_connection)
```

Tiếp theo, ta tạo một Agent được cấu hình cho việc đánh giá GAIA. Agent nhận các MCP tool để tìm kiếm web và dùng `GaiaOutput` làm kiểu structured output.

Listing 4.31. Tạo agent đánh giá GAIA

```
def create_gaia_agent(model: str, tools: list) -> Agent:
    return Agent(
        model=LlmClient(model=model),
        tools=tools,
        instructions=gaia_prompt,
        output_type=GaiaOutput,
        max_steps=15,
    )
```

Thay đổi cốt lõi nằm ở `solve_problem`. Ở Chương 2 ta gọi trực tiếp `acompletion()`, còn giờ ta gọi `agent.run()`. `AgentResult` trả về chứa cả output lẫn toàn bộ execution context.

Listing 4.32. Giải bài toán bằng agent

```
SEMAPHORE = asyncio.Semaphore(3)

async def solve_problem(agent: Agent, question: str) -> AgentResult:
    async with SEMAPHORE:
        return await agent.run(question)
```

Semaphore giờ giới hạn số lần thực thi Agent đồng thời thay vì từng lời gọi LLM riêng lẻ. Mỗi lần thực thi Agent có thể gồm nhiều lời gọi LLM nội bộ khi nó suy luận và dùng tool, nhưng ta giới hạn số bài toán được giải song song để tránh làm quá tải các API.

Phần còn lại của pipeline đánh giá, gồm `evaluate_single()` và `run_experiment()`, theo cùng cấu trúc như Chương 2. Điều chỉnh duy nhất là lấy câu trả lời qua `result.output` thay vì parse trực tiếp response của LLM. Xem GitHub repository để có bản cài đặt đầy đủ.

4.8.2 Kết quả

Thí nghiệm so sánh một agent có tool tìm kiếm web với một agent không có tool tìm kiếm web, trên mười bài toán GAIA cần truy cập web. Cả hai agent đều dùng gpt-5 làm model nền.

Listing 4.33. Chạy thí nghiệm GAIA

```
models = [
    "gpt-5",
]

results = await run_experiment(
```

```
    problems=list(problems),
    models=models,
    tools=mcp_tools,
)
```

Debug bằng execution trace

Khi bài toán thất bại, ta cần hiểu vì sao. Vì `AgentResult` chứa toàn bộ `ExecutionContext`, ta có thể xem chính xác điều gì đã xảy ra trong quá trình thực thi. Để trực quan hoá tiện lợi trong Jupyter notebook, ta cung cấp tiện ích `display_trace()`.

Listing 4.34. Debug bằng execution trace

```
from scratch_agents.utils import display_trace

result = await agent.run(
    "If Eliud Kipchoge could maintain his marathon pace, "
    "how many thousand hours to reach the Moon?"
)

print(f"Answer: {result.output.final_answer}")
print(f"Steps: {result.context.current_step}")

display_trace(result.context)
```

Trace này cho thấy quá trình suy luận của agent: nó nhận ra cần hai mẩu thông tin, tìm kiếm từng mẩu, rồi đưa ra câu trả lời cuối cùng. Khi debug các lần thất bại, bạn có thể chỉ ra chính xác chỗ nào đã sai.

Tác động của tool tìm kiếm web

Để định lượng giá trị của khả năng tìm kiếm web, các tác giả chạy một thí nghiệm có đối chứng, so sánh agent có và không có tool tìm kiếm trên các bài toán GAIA Level 1 cần truy cập web. Mười bài toán được chọn là những bài mà annotation chính thức của GAIA ghi rằng tìm kiếm web là tool bắt buộc.

Điều kiện	Số câu đúng	Tổng chi phí
Có tìm kiếm web	7/10	$1.89
Không có tìm kiếm web	2/10	$0.26

Kết quả cho thấy độ chính xác tăng 50 điểm phần trăm khi có tool tìm kiếm web. Không có khả năng tìm kiếm, agent chỉ trả lời được những câu hỏi mà thông tin tình cờ có sẵn trong dữ liệu huấn luyện của nó. Có tool tìm kiếm, agent đã lấy được thông tin cập nhật từ Wikipedia, bài báo học thuật và các nguồn web khác.

Xem xét các lần thất bại cho thấy vài pattern đáng chú ý. Agent không có tìm kiếm web hoặc thừa nhận không giải được bài toán (trả về `is_solvable=False`), hoặc bịa ra câu trả lời sai (hallucination). Ví dụ, khi được hỏi về số album phòng thu của Mercedes Sosa trong giai đoạn 2000–2009, agent có tìm kiếm đã tìm đúng 3 album từ Wikipedia, còn agent không có tìm kiếm thì đoán là 4.

Những thách thức còn lại

Dù tìm kiếm web cải thiện mạnh hiệu năng trên các tác vụ truy xuất thông tin, nhiều bài toán GAIA đòi hỏi thêm các khả năng khác. Một phần đáng kể bài toán Level 1 có kèm file như PDF, bảng tính, hình ảnh và file âm thanh mà agent phải xử lý để tìm ra câu trả lời. Agent hiện tại của ta chưa xử lý được các bài toán dựa trên file này.

Ở Chương 5, ta sẽ mở rộng agent với các tool xử lý file, có thể đọc PDF, parse bảng tính, phân tích hình ảnh và chuyển âm thanh thành văn bản. Điều này sẽ mở khoá các bài toán GAIA còn lại và đưa ta tới gần hơn một trợ lý AI thực sự đa năng.

4.9 Tóm tắt

- ReAct (Reasoning + Acting) là pattern nền tảng đằng sau các AI agent hiện đại, trong đó LLM luân phiên giữa việc suy luận xem mình cần gì và thực hiện hành động để lấy được thứ đó. Các bản cài đặt hiện đại dùng tool calling thay cho việc parse văn bản dễ vỡ.
- ExecutionContext tập trung toàn bộ trạng thái thực thi, gồm events, số bước, execution ID và kho state linh hoạt. Thiết kế này đơn giản hoá chữ ký method và cho phép context propagation tới những tool cần truy cập trạng thái thực thi.
- Tool abstraction qua BaseTool và FunctionTool cung cấp một giao diện nhất quán cho mọi tool. Decorator `@tool` bọc các function đơn giản, còn `load_mcp_tools()` tích hợp các MCP server bên ngoài, cho phép agent dùng liền mạch cả tool cục bộ lẫn tool từ xa.
- Tầng giao tiếp LLM tách bạch các mối quan tâm: LlmRequest đóng gói những gì cần gửi, LlmClient xử lý giao tiếp API qua LiteLLM, và LlmResponse chuẩn hoá những gì trả về. Sự tách biệt này là nơi context engineering diễn ra.
- Class Agent điều phối mọi thứ qua năm method cốt lõi: `_setup_tools()` chuẩn bị các tool có sẵn, `run()` quản lý vòng lặp thực thi, `step()` thực hiện một chu trình think-act, `think()` gọi LLM, và `act()` thực thi các tool được yêu cầu.
- Structured output cho phép agent trả về Pydantic model an toàn kiểu thay vì văn bản tự do. Bằng cách tạo động một tool `final_answer` từ schema output, ta tận dụng hạ tầng tool calling sẵn có để đảm bảo tuân thủ định dạng.

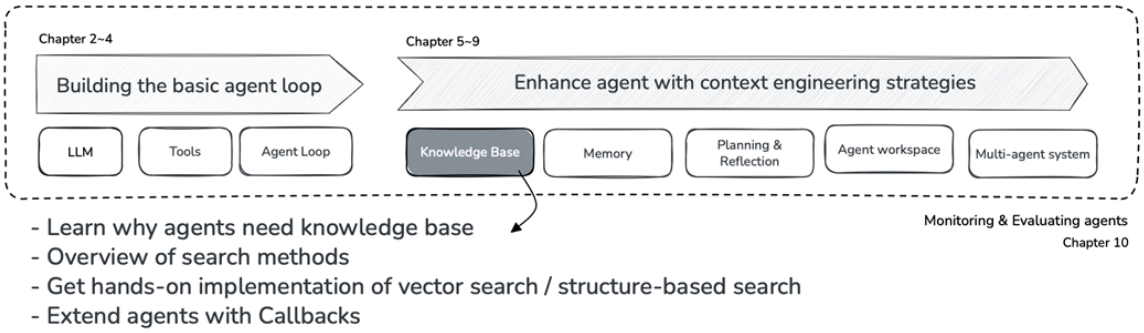
Chương 5

Xây dựng knowledge base với RAG

Chương này trình bày

- Vì sao agent cần knowledge base bên ngoài.
- Các phương pháp tìm kiếm khác nhau: keyword search, vector search, graph search và structure-based search.
- Tự cài đặt vector search từ đầu với embedding, chunking và tính độ tương đồng.
- Xây dựng các tool structure-based search để agent khám phá hệ thống file.
- Mở rộng khả năng của agent bằng Callback pattern.

Từ chương 2 đến chương 4, chúng ta đã dựng phần nền của một AI agent: kết nối với LLM, cài đặt tool use và tạo agent loop. Giờ chúng ta bước sang giai đoạn mới. Như Hình 5.1 cho thấy, từ đây ta sẽ nâng cấp agent cơ bản bằng các chiến lược context engineering. Chương này giải quyết nâng cấp đầu tiên: xây dựng knowledge base với RAG (Retrieval-Augmented Generation).



Hình 5.1: Tổng quan cấu trúc sách, Chương 5 là trọng tâm. *(PDF gốc tr. 170)*

Chúng ta bắt đầu với các thành phần cơ bản của vector search: embedding nắm bắt ý nghĩa văn bản như thế nào, vì sao cần chunking, và vector database vận hành ra sao. Ta sẽ cài đặt một hệ thống vector search thu nhỏ để trải nghiệm toàn bộ luồng xử lý. Tiếp theo, ta mở rộng sang structure-based search qua các bài toán có file zip của benchmark GAIA, trong đó agent duyệt cấu trúc thư mục và đọc file giống như một lập trình viên. Cuối cùng, ta giới thiệu Callback pattern để mở rộng hành vi của agent, cài đặt cơ chế phê duyệt Human in the Loop và tự động nén kết quả tìm kiếm.

Vì sách tập trung vào LLM agent, nó không đi sâu vào mọi phương pháp tìm kiếm trong RAG. Các thảo luận chi tiết về kỹ thuật keyword search (BM25, TF-IDF), cách cài đặt graph search (xây dựng knowledge graph, ngôn ngữ truy vấn), hay chiến lược kết hợp điểm số cho hybrid search đều nằm ngoài phạm vi. Trọng tâm của chương là cách agent dùng các tool knowledge base, với vector search và structure-based search qua các bài thực hành.

Bạn đọc muốn tìm hiểu thêm có thể tham khảo:

- *A Simple Guide to Retrieval Augmented Generation*: https://www.manning.com/books/a-simple-guide-to-retrieval-augmented-generation
- *Essential GraphRAG*: https://www.manning.com/books/essential-graphrag

5.1 Vấn đề khi dùng dữ liệu nội bộ

Ở chương 3, ta đã cài đặt web search qua Tavily API. Khi người dùng đặt câu hỏi, ta tìm thông tin liên quan trên web, thêm vào context và để LLM sinh câu trả lời. Bản chất của web search là gì? Đó là trích ra chỉ vài kết quả liên quan từ hàng tỉ trang web dựa trên một search query. Các công cụ tìm kiếm như Google hay Bing lo phần việc phức tạp này cho ta, nên chỉ cần một query đơn giản là có được thông tin cần thiết.

Tuy nhiên, trong môi trường làm việc thực tế có nhiều bài toán mà web search không giải được. Đó là những trường hợp cần dùng dữ liệu không công khai, như tài liệu nội bộ công ty, codebase của dự án hay file cá nhân. Các nguồn dữ liệu nội bộ này đòi hỏi cùng nguyên tắc như web search: phải tìm ra đúng những phần liên quan tới câu hỏi hiện tại trong khối dữ liệu khổng lồ rồi đưa chúng vào context. Ta sẽ cài đặt chức năng tìm kiếm để agent tận dụng hiệu quả dữ liệu nội bộ.

5.1.1 Trường hợp đơn giản: một file

Bắt đầu với tình huống đơn giản nhất. Benchmark GAIA có những bài toán kèm file đính kèm. Ví dụ, giả sử có một file CSV đi kèm câu hỏi "Tháng nào có doanh số cao nhất trong dữ liệu này?"

Cách giải ở đây rất thẳng: đọc file và đưa toàn bộ vào context. Nếu file chỉ vài trăm dòng, nó vừa vặn thoải mái trong context window của các LLM hiện đại. Ta có thể giải những bài như vậy chỉ bằng cách thêm một tool đọc file vào agent đã cài đặt ở chương 4.

5.1.2 Nếu có nhiều file thì sao?

Benchmark GAIA cũng có những bài toán đính kèm file zip thay vì một file đơn lẻ. Giải nén ra sẽ có nhiều file và thư mục. Vấn đề là ta không biết trước file nào chứa câu trả lời.

Ví dụ, giả sử bạn nhận được một kho mã kèm câu hỏi "Dự án này dùng loại database nào?" Giải nén file zip, bạn được cấu trúc sau:

```
project/
|-- README.md
|-- requirements.txt
|-- src/
|   |-- main.py
|   |-- database.py
|   `-- utils.py
|-- config/
|   `-- settings.json
`-- docs/
    `-- architecture.md
```

Câu trả lời có thể nằm trong `requirements.txt`, trong `database.py`, hoặc trong `settings.json`. Đọc tất cả các file rồi đưa vào context là kém hiệu quả, và sẽ trở nên bất khả thi khi số file tăng lên.

Điều cần trong tình huống này là *khám phá* (exploration). Trước hết nắm cấu trúc tổng thể, sau đó dùng tên file và tên thư mục làm manh mối để chọn những file có vẻ liên quan và xem nội dung của chúng. Nếu cần thì khám phá thêm các file khác. Đây chính là cách lập trình viên tìm hiểu một codebase mới. Các dự án code đặc biệt hợp với cách tiếp cận này vì chúng có cấu trúc có hệ thống. Những công cụ như Claude Code, Cursor và Gemini CLI hiểu code đúng theo cách này. Đó là structure-based search.

5.1.3 Nếu dữ liệu lớn hoặc quá nhiều thì sao?

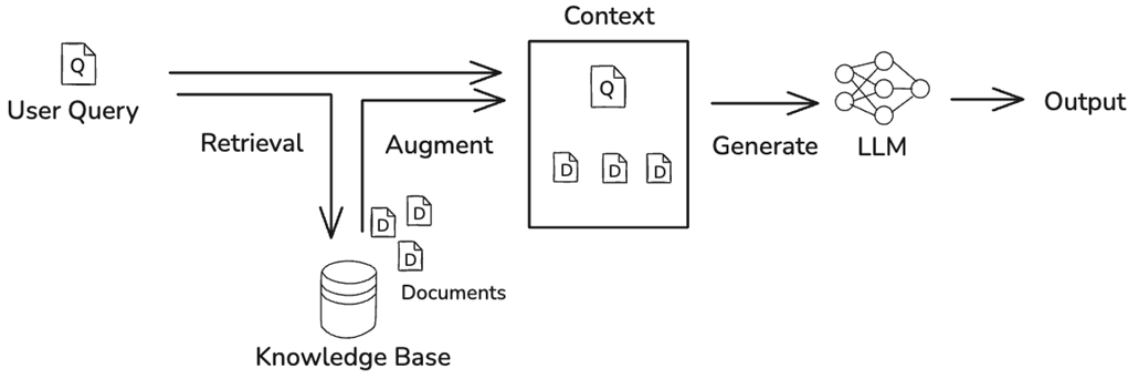
Có những tình huống mà việc khám phá theo cấu trúc không xử lý nổi. Thứ nhất, khi một file đơn lẻ quá lớn để nhét vừa context, chẳng hạn file log hàng chục nghìn dòng hay tài liệu lớn. Thứ hai, khi số lượng tài liệu quá nhiều. Nếu wiki công ty có hàng nghìn tài liệu, khám phá lần lượt từng cái là không thực tế. Hơn nữa, thông tin trong wiki công ty thường không được tổ chức có hệ thống.

Vì vậy, ta cần một cơ chế lấy ra có chọn lọc chỉ những phần cần thiết. Với câu hỏi về "cách kết nối tới database", thay vì đưa hàng chục nghìn dòng code vào context, ta nên chỉ tìm những phần liên quan đến database. Đó là lý do cần đến các công nghệ tìm kiếm như vector search hay keyword search.

5.2 Các loại phương pháp tìm kiếm

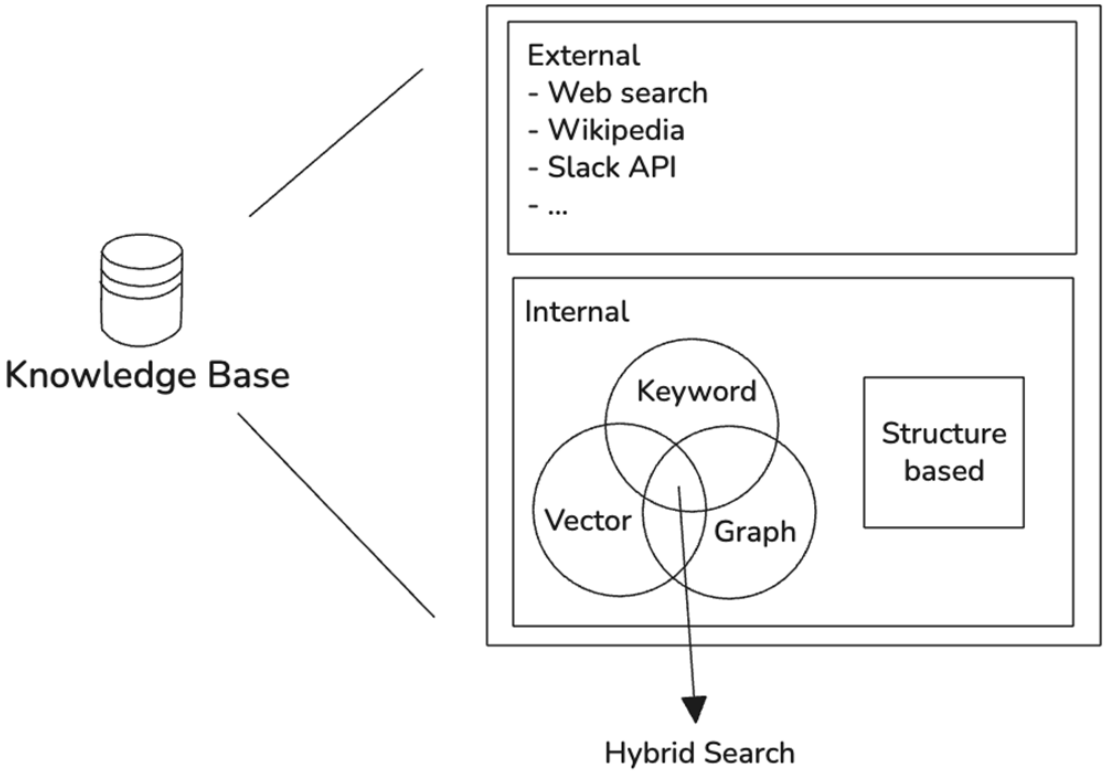
RAG (Retrieval-Augmented Generation) đúng như tên gọi: một phương pháp tìm kiếm (Retrieval) thông tin cần thiết, bổ sung (Augmented) nó vào đầu vào của LLM, rồi sinh (Generation) câu trả lời. Thay vì LLM chỉ trả lời dựa trên kiến thức của chính nó, nó tham khảo thông tin tìm được từ dữ liệu bên ngoài để soạn câu trả lời.

Hình 5.2 cho thấy luồng tổng thể của RAG. Khi có câu hỏi của người dùng, hệ thống trước hết tìm thông tin liên quan trong Knowledge Base. Kết quả tìm kiếm được chuyển cho LLM cùng câu hỏi gốc, và LLM sinh câu trả lời dựa trên thông tin tham khảo này. Qua quá trình đó, RAG vượt qua giới hạn kiến thức của LLM và tận dụng được thông tin mới nhất hoặc dữ liệu riêng tư.



Hình 5.2: Luồng tổng thể của RAG (Retrieval-Augmented Generation). Khi có câu hỏi của người dùng, hệ thống tìm thông tin liên quan trong Knowledge Base, rồi chuyển kết quả tìm kiếm cùng câu hỏi cho LLM để sinh câu trả lời. *(PDF gốc tr. 174)*

Chúng ta đã trải nghiệm RAG qua web search, nơi LLM giải quyết tác vụ bằng cách tìm thông tin liên quan với search query rồi thêm kết quả vào context để sinh câu trả lời. Theo nghĩa rộng này, RAG bao gồm mọi hình thức truy xuất và sử dụng thông tin. Điểm khác biệt ở chương này là ta sẽ tự xây dựng các tool tìm kiếm thay vì dùng tool có sẵn.



Hình 5.3: Các loại tìm kiếm: kho dữ liệu bên ngoài/nội bộ, keyword/vector/graph search, và structure-based search. *(PDF gốc tr. 175)*

Có nhiều loại phương pháp tìm kiếm, như minh hoạ ở Hình 5.3. Phương pháp phù hợp thay đổi tuỳ theo đặc điểm dữ liệu và loại câu hỏi. Mỗi cách tiếp cận hiệu quả trong một hoàn cảnh: khi cần khớp chính xác từ ngữ, khi tìm nội dung tương tự về ngữ nghĩa, khi duyệt theo quan hệ, hoặc khi trực tiếp xem xét cấu trúc file. Trong phần này, ta tìm hiểu nguyên lý của bốn phương pháp tìm kiếm chính và khi nào mỗi phương pháp phù hợp nhất.

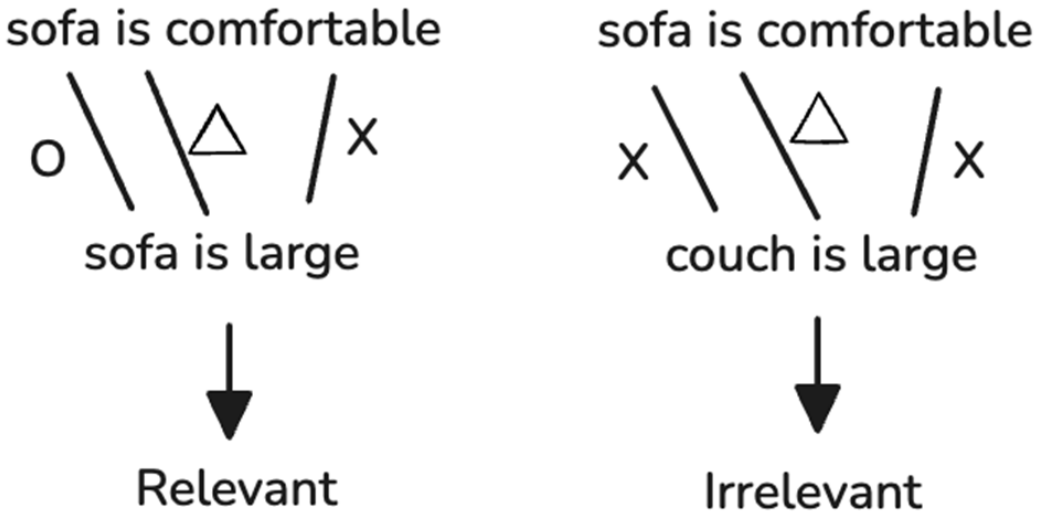
5.2.1 Keyword search

Keyword search là phương pháp tìm kiếm truyền thống nhất, đánh giá độ liên quan bằng cách tính tần suất và mức độ quan trọng của các từ trong tài liệu. Các thuật toán tiêu biểu gồm BM25 và TF-IDF.

TF-IDF (Term Frequency-Inverse Document Frequency) kết hợp hai chỉ số. Nó nhân tần suất xuất hiện của một từ trong một tài liệu (TF) với độ hiếm của từ đó trên toàn bộ tập tài liệu (IDF). Những từ như "the" xuất hiện ở khắp nơi có tầm quan trọng thấp, còn những từ chỉ xuất hiện dày đặc trong một số tài liệu cụ thể có tầm quan trọng cao. BM25 là thuật toán cải tiến từ TF-IDF, cho kết quả chính xác hơn nhờ điều chỉnh theo độ dài tài liệu và phản ánh hiệu ứng bão hoà của tần suất từ.

Keyword search hiệu quả nhất khi cần tìm chính xác một từ hoặc cụm từ. Không có phương pháp nào nhanh và chính xác hơn keyword search khi tìm tên hàm như `get_user_by_id`, tìm mã lỗi `404`, hay kiểm tra một giá trị cấu hình cụ thể như `MAX_CONNECTIONS`. Là công nghệ đã được kiểm chứng qua nhiều thập kỷ, nó đảm bảo tốc độ tìm kiếm nhanh ngay cả trên tập dữ liệu quy mô lớn.

Tuy vậy, keyword search có những giới hạn căn bản. Nó không xử lý được từ đồng nghĩa hay cách diễn đạt tương tự. Hình 5.4 minh hoạ vấn đề này. "Sofa" và "couch" cùng chỉ một món đồ nội thất, nhưng keyword search không biết hai từ này liên quan với nhau. Tìm "sofa" sẽ không ra các tài liệu chứa "couch". Nếu không xây dựng riêng một từ điển đồng nghĩa, mối liên hệ ngữ nghĩa này không thể được nhận ra tự động.



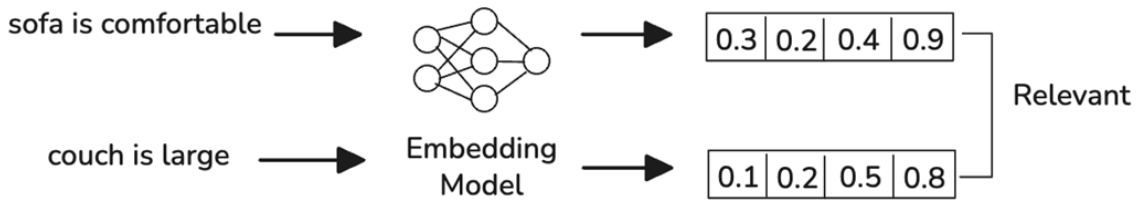
Hình 5.4: Keyword search: dù "sofa" và "couch" là các từ tương tự, keyword search không xác định được độ tương đồng của chúng nếu không có xử lý riêng. Những từ phổ biến như "is" xuất hiện thường xuyên ở mọi nơi không được coi là quan trọng (biểu thị bằng hình tam giác). Ví dụ bên trái được đánh giá là liên quan nhờ trùng từ "sofa", còn ví dụ bên phải bị đánh giá là không liên quan dù có liên quan về ngữ nghĩa. *(PDF gốc tr. 177)*

5.2.2 Vector search

Vector search là phương pháp lượng hoá ý nghĩa của văn bản rồi tính độ tương đồng. Trước hết, nó dùng một embedding model để biến văn bản thành một điểm trong không gian vector nhiều chiều. Trong quá trình này, các văn bản có nghĩa tương tự được ánh xạ tới các vị trí gần nhau, còn văn bản khác nghĩa được ánh xạ tới các vị trí xa nhau.

Embedding là việc biểu diễn từ hoặc câu dưới dạng mảng gồm hàng trăm đến hàng nghìn con số. Ví dụ, từ "cat" có thể trở thành một vector như `[0.2, -0.5, 0.8, ...]`. Điểm mấu chốt là phép biến đổi này giữ được ý nghĩa. "Cat" và "kitten" là hai từ khác nhau nhưng nghĩa gần nhau, nên chúng được đặt ở các vị trí gần nhau trong không gian vector. Ngược lại, "cat" và "car" giống nhau về chính tả nhưng khác nghĩa, nên bị đặt cách xa nhau.

Hình 5.5 cho thấy nguyên lý của vector search. Các tài liệu được chuyển sẵn thành vector và lưu trữ; khi tìm kiếm, query cũng được chuyển thành vector theo cùng cách. Sau đó hệ thống tính khoảng cách giữa vector query và các vector tài liệu đã lưu để tìm và trả về những tài liệu gần nhất. Phép tính khoảng cách thường dùng cosine similarity hoặc Euclidean distance. Cosine similarity là giá trị cosin của góc giữa hai vector: giá trị càng gần 1 thì càng tương đồng, càng gần 0 thì càng ít liên quan. Euclidean distance là khoảng cách theo đường thẳng giữa hai điểm: giá trị càng nhỏ thì càng tương đồng.



Hình 5.5: Phương pháp vector search: xác định độ liên quan bằng cách chuyển dữ liệu thành vector. Với một embedding model được huấn luyện tốt, nó có thể nhận ra "sofa" và "couch" tương tự nhau mà không cần xử lý từ đồng nghĩa. *(PDF gốc tr. 178)*

Ưu điểm lớn nhất của vector search là có thể dùng trực tiếp câu hỏi bằng ngôn ngữ tự nhiên. Nếu bạn hỏi "cách lưu thông tin vào database", đoạn code liên quan về ngữ nghĩa như "persist data to storage" vẫn có thể được tìm ra dù không có đúng cụm từ đó. Vấn đề từ đồng nghĩa từng gây khó cho keyword search cũng được giải quyết một cách tự nhiên, vì embedding model đã học được trong quá trình huấn luyện rằng "sofa" và "couch" được dùng trong những ngữ cảnh tương tự.

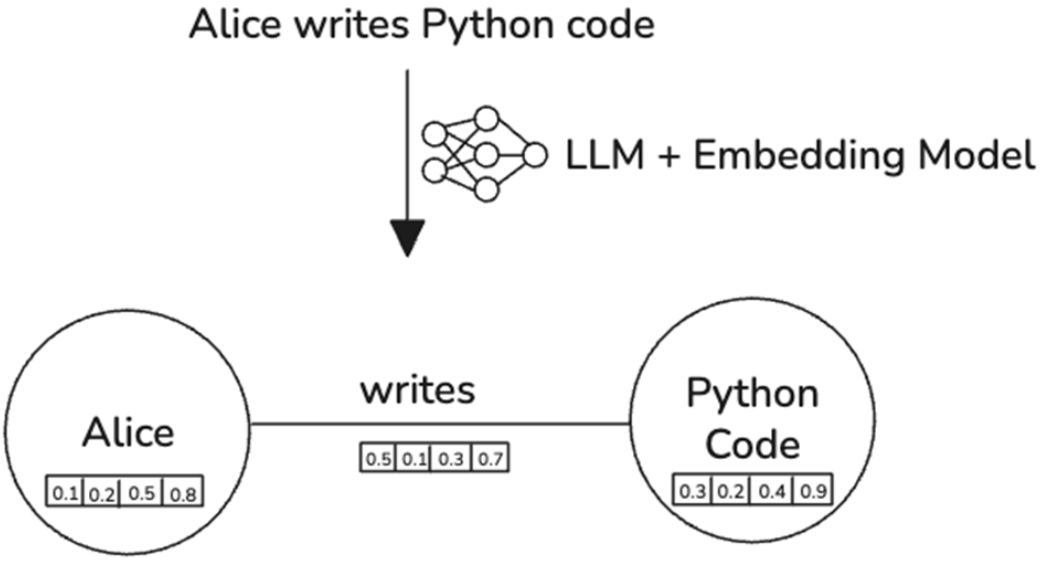
Các embedding model hiện đại còn xét đến ngữ cảnh. Cùng một từ được chuyển thành các vector khác nhau tuỳ tình huống. "Bank" trở thành các vector khác nhau khi chỉ tổ chức tài chính và khi chỉ bờ sông. Đặc tính nhận biết ngữ cảnh này giúp tìm kiếm ngữ nghĩa chính xác hơn.

Tuy nhiên, vector search không phải lúc nào cũng là lựa chọn tốt nhất. Khi cần khớp chính xác từ ngữ, ví dụ tìm tên hàm hay tên biến cụ thể, keyword search hiệu quả hơn. Keyword search cũng có lợi thế khi tìm ký tự đặc biệt hoặc cú pháp code. Vì những lý do đó, hybrid search kết hợp keyword search và vector search được dùng rộng rãi trong thực tế.

Ở mục 5.3, ta sẽ xem chi tiết vector search hoạt động thế nào bằng cách tự cài đặt embedding, chunking và vector database.

5.2.3 Graph search

Graph search là phương pháp biểu diễn và duyệt thông tin dưới dạng cấu trúc đồ thị gồm các node (thực thể) và edge (quan hệ). Hình 5.6 cho một ví dụ đơn giản. Từ câu "Alice writes Python code", "Alice" và "Python code" trở thành các node, còn "writes" trở thành edge nối chúng. Khi đồ thị đã được xây dựng, những câu hỏi như "Alice viết gì?" hay "Ai viết code Python?" có thể được trả lời bằng cách duyệt đồ thị.



Hình 5.6: Phương pháp graph search: trích xuất các thực thể và quan hệ giữa các thực thể từ câu văn rồi biểu diễn dưới dạng đồ thị. Mỗi thực thể và quan hệ đôi khi được chuyển thành embedding vector để dùng đồng thời cả vector search lẫn graph search. *(PDF gốc tr. 179)*

Graph search vượt trội với các câu hỏi phức tạp dựa trên quan hệ. Những truy vấn quan hệ nhiều bước như "mọi đoạn code gọi hàm này", "code khác do các lập trình viên tham gia dự án này viết", hay "các file được sửa gần đây bởi người từng cộng tác với A" có thể xử lý hiệu quả bằng cách duyệt đồ thị. Nó hữu ích cho phân tích phụ thuộc trong code, hiểu cơ cấu tổ chức và khám phá knowledge graph.

Thách thức lớn nhất của graph search là trích xuất thực thể và quan hệ từ văn bản phi cấu trúc. Trước đây việc này cần các luật phức tạp hoặc mô hình machine learning, nhưng tiến bộ gần đây của LLM đã đơn giản hoá quá trình này rất nhiều. Chỉ cần yêu cầu LLM "trích xuất người, tổ chức và địa điểm được nhắc trong tài liệu này và xác định quan hệ giữa chúng", ta có thể tự động xây dựng một knowledge graph. Nhờ sự tự động hoá này, graph search đang được chú ý trở lại.

Tuy nhiên, graph search có nhược điểm là chi phí xây dựng cao và đòi hỏi kiến thức chuyên môn về mô hình hoá đồ thị. Cần cân nhắc những điều như nên trích xuất thực thể và quan hệ nào, thiết kế graph schema ra sao. Sách này không trực tiếp đề cập graph search. Bạn đọc quan tâm có thể tham khảo cuốn *Essential GraphRAG* của Manning (`https://www.manning.com/books/essential-graphrag`).

5.2.4 Structure-based search

Structure-based search là phương pháp tận dụng chính cấu trúc của dữ liệu để khám phá khi dữ liệu đã được tổ chức có hệ thống. Ví dụ tiêu biểu là các kho mã nguồn. Code được tổ chức có hệ thống thành thư mục và file, và mỗi file có các cấu trúc rõ ràng như class, hàm và biến.

Hãy xem một lập trình viên khám phá một codebase mới thế nào. Đầu tiên, họ dùng lệnh `tree` để nắm cấu trúc thư mục tổng thể. Họ xem các thư mục cấp cao nhất như `src/`, `tests/` và `docs/`, rồi xem cấu trúc con của các module quan tâm. Tiếp theo, họ dùng `grep` hoặc chức năng tìm kiếm của IDE để tìm hàm hay class cụ thể. Họ có thể nhanh chóng tìm ra hàm `get_user_by_id` được định nghĩa ở đâu và được gọi ở đâu. Cuối cùng, họ mở các file tìm được và đọc toàn bộ code để hiểu chi tiết cài đặt.

Cách khám phá này hiệu quả vì bản thân cấu trúc code đã chứa thông tin. Tên file (`user_service.py`), cấu trúc thư mục (`/services/auth/`) và tên hàm (`authenticate_user`) đều là metadata có ý nghĩa. Không cần thuật toán tìm kiếm phức tạp, chỉ với thông tin cấu trúc này đã có thể tìm ra đoạn code mong muốn.

Các công cụ lập trình hiện đại như Claude Code, Cursor và Gemini CLI dùng đúng cách tiếp cận này. Vì LLM quen thuộc với các lệnh bash như `tree`, `grep` và `cat`, agent có thể khám phá codebase giống

lập trình viên: trước hết nắm cấu trúc, tìm các file liên quan, đọc những phần cần thiết và dần mở rộng hiểu biết.

Structure-based search cũng phù hợp với các bài toán file zip của GAIA đã bàn ở mục 5.1. Khi giải nén, nhiều file và thư mục hiện ra. Agent đi qua quy trình: trước hết nắm cấu trúc, lấy gợi ý từ tên file, chọn các file liên quan và xác nhận nội dung của chúng. Ta sẽ trực tiếp cài đặt cách tiếp cận này ở mục 5.4.[1]

Hướng dẫn chọn phương pháp tìm kiếm

Bốn phương pháp tìm kiếm vừa xem xét phù hợp với những tình huống khác nhau. Bảng 5.1 tóm tắt nên chọn phương pháp nào cho từng tình huống.

Bảng 5.1: Phương pháp tìm kiếm khuyến nghị theo tình huống

Tình huống	**Phương pháp khuyến nghị**	**Lý do**
Tìm chính xác từ/code	Keyword Search	Cần khớp chính xác cho `get_user_by_id`, mã lỗi `404`, v.v.
Câu hỏi bằng ngôn ngữ tự nhiên	Vector Search	Cần tìm theo ngữ nghĩa cho các query như "cách lưu dữ liệu" ("how to store data")
Câu hỏi dựa trên quan hệ	Graph Search	Cần duyệt quan hệ cho "code gọi hàm này"
Khám phá cấu trúc thư mục/file	Structure-based Search	Khi bản thân cấu trúc là thông tin, như file zip hay codebase
Môi trường production	Hybrid Search	Kết hợp ưu điểm của nhiều phương pháp

Trong thực tế, hiếm khi chỉ dùng một phương pháp duy nhất; hybrid search là phổ biến. Cách tiếp cận tiêu biểu nhất là kết hợp keyword search và vector search. Keyword search bắt được các kết quả khớp chính xác, còn vector search bắt được sự tương đồng ngữ nghĩa. Điểm số từ hai kết quả tìm kiếm được chuẩn hoá rồi tính trung bình có trọng số để quyết định thứ hạng cuối cùng. Nhờ vậy, cả tên chính xác như hàm `get_user_by_id` lẫn câu hỏi ngữ nghĩa như "cách lấy thông tin người dùng" đều được xử lý.

Trong sách này, ta sẽ trực tiếp cài đặt vector search và structure-based search. Vector search là phương pháp tìm kiếm được chú ý nhiều nhất kể từ khi LLM xuất hiện, còn structure-based search đang thu hút nhiều quan tâm, chủ yếu quanh các coding agent, khi hiệu năng LLM và context window ngày càng lớn. Ta chỉ giới thiệu khái niệm keyword search và graph search, nhưng có thể tích hợp thêm chúng vào hệ thống xây dựng trong sách khi cần.

5.3 Thực hành vector search

Cốt lõi của vector search là chuyển văn bản thành vector và tính khoảng cách. Hãy chuyển văn bản thành vector và quan sát sự tương đồng ngữ nghĩa được biểu diễn thành khoảng cách như thế nào. Ta sẽ lần lượt cài đặt embedding, chunking và vector search, rồi kết thúc bằng một bài thực hành kết hợp các kỹ thuật này để trích thông tin liên quan từ kết quả web search.

5.3.1 Embedding: chuyển văn bản thành vector

Embedding là quá trình chuyển văn bản thành mảng số có hàng trăm đến hàng nghìn chiều. Ý tưởng then chốt là các văn bản tương tự về ngữ nghĩa được đặt gần nhau trong không gian vector. Embedding model học cách biểu diễn các quan hệ ngữ nghĩa này thành khoảng cách vector thông qua huấn luyện trên lượng văn bản khổng lồ.

[1] Sách ghi "section 5.4", nhưng phần cài đặt thực tế nằm ở mục 5.3.5 đến 5.3.9.

Hãy cài đặt một hàm chuyển văn bản thành embedding vector bằng model `text-embedding-3-small` của OpenAI. Model này có tỉ lệ giá trên hiệu năng rất tốt, phù hợp cho việc học. Hàm dưới đây bọc chuỗi đơn lẻ vào một list để xử lý, trích từng embedding vector từ phản hồi của API và trả về dưới dạng mảng numpy.

Listing 5.1. Cài đặt hàm `get_embeddings`

```
from openai import OpenAI
import numpy as np

client = OpenAI()

def get_embeddings(texts, model="text-embedding-3-small"):
    """Convert text to embedding vectors."""
    if isinstance(texts, str):
        texts = [texts]  #A
    response = client.embeddings.create(input=texts, model=model)  #B
    return np.array([item.embedding for item in response.data])  #C
```

`#A` Bọc chuỗi đơn lẻ vào list để xử lý thống nhất.
`#B` Gọi embedding API của OpenAI.
`#C` Trích các embedding vector thành mảng numpy.

Giờ hãy kiểm tra xem embedding có thực sự nắm bắt được sự tương đồng ngữ nghĩa không. Ta chuẩn bị ba câu: câu thứ nhất về một con mèo ngủ trên sofa, câu thứ hai về một chú mèo con chơi với đồ chơi, câu thứ ba về một con chó chạy trong công viên. Hàm `cosine_similarity` tính độ tương đồng giữa câu thứ nhất với hai câu còn lại. Cosine similarity là cosin của góc giữa hai vector: giá trị càng gần 1 thì độ tương đồng càng cao, càng gần 0 thì càng thấp.

Listing 5.2. So sánh độ tương đồng embedding

```
from sklearn.metrics.pairwise import cosine_similarity

sentences = [
    "The cat is sleeping on the couch",
    "A kitten is playing with a toy",
    "The dog is running in the park"
]

embeddings = get_embeddings(sentences)

cat_kitten = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]  #A
cat_dog = cosine_similarity([embeddings[0]], [embeddings[2]])[0][0]  #A

print(f"Cat vs Kitten: {cat_kitten:.3f}")
print(f"Cat vs Dog: {cat_dog:.3f}")
```

`#A` Tính cosine similarity giữa các embedding vector.

Kết quả cho thấy độ tương đồng giữa câu "cat" và câu "kitten" cao hơn giữa câu "cat" và câu "dog". Dù "cat" và "kitten" là hai từ khác nhau, chúng liên quan về ngữ nghĩa. Keyword search sẽ coi chúng là những tài liệu hoàn toàn khác nhau. Đây chính là lời giải cho vấn đề "sofa và couch" đã nêu ở trên: vector search tìm được nội dung liên quan về ngữ nghĩa mà không cần khớp chính xác từ ngữ.

5.3.2 Chunking: chia văn bản dài thành các đơn vị tìm kiếm

Embed cả một tài liệu thành một vector duy nhất gây ra nhiều vấn đề. Thứ nhất, nó có thể vượt quá giới hạn token của embedding model. Thứ hai, tài liệu dài chứa nhiều chủ đề, khiến vector bị "trung bình

hoá" và làm giảm độ chính xác tìm kiếm. Ví dụ, nếu một cuốn hướng dẫn gồm "hướng dẫn cài đặt", "xử lý sự cố" và "thông số sản phẩm" bị chuyển thành một vector, tìm "hướng dẫn cài đặt" có thể không đưa tài liệu đó lên các kết quả đầu. Thứ ba, trả về toàn bộ tài liệu làm kết quả tìm kiếm sẽ lãng phí context. Giải pháp là chia tài liệu thành các đơn vị nhỏ (chunk) và embed riêng từng chunk.

Kích thước chunk là một bài toán đánh đổi. Quá nhỏ thì mất thông tin ngữ cảnh: không biết "cái này" chỉ cái gì. Quá lớn thì nhiều chủ đề trộn lẫn, làm giảm độ chính xác tìm kiếm. Thông thường 200 đến 1.000 token là phù hợp, khoảng 150 đến 750 từ tiếng Anh. Kích thước chunk tối ưu phụ thuộc vào đặc điểm tài liệu. Văn bản dày đặc thông tin như tài liệu kỹ thuật hợp với chunk nhỏ hơn, còn văn bản tự sự hợp với chunk lớn hơn để giữ ngữ cảnh.

Một khái niệm quan trọng trong fixed-length chunking là *overlap* (phần chồng lấn). Chỉ đơn giản cắt văn bản mỗi 500 ký tự có thể làm đứt ngữ cảnh ở ranh giới chunk. Ví dụ, nếu câu "This function processes data. The processed result is" bị cắt ở ranh giới chunk, chunk thứ nhất không biết "the processed result" là gì, còn chunk thứ hai không biết "this function" chỉ cái gì. Overlap cho phép các chunk kề nhau dùng chung một phần văn bản, giảm nhẹ vấn đề này. Thông thường overlap được đặt bằng 10–20% kích thước chunk.

Hàm dưới đây cài đặt fixed-length chunking. Vòng `while` cắt văn bản từ vị trí `start` một đoạn dài `chunk_size` để tạo chunk. Phần then chốt là tính vị trí bắt đầu tiếp theo: nếu chưa tới cuối tài liệu, ta đặt nó bằng `end - overlap` để lùi lại một đoạn bằng overlap. Ở cuối tài liệu, ta dùng nguyên `end` để kết thúc vòng lặp.

Listing 5.3. Hàm fixed-length chunking

```
def fixed_length_chunking(text, chunk_size=500, overlap=50):
    """Split text into fixed-length chunks."""
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        start = end - overlap if end < len(text) else end  #A
    return chunks
```

`#A` Lùi lại một đoạn bằng overlap để giữ ngữ cảnh giữa các chunk.

Một phép thử đơn giản cho thấy tác dụng của overlap. Chia một văn bản 283 ký tự với `chunk_size` 100 và `overlap` 20 tạo ra nhiều chunk hơn (4) so với khi không có overlap (3). Nhờ overlap, mỗi chunk dùng chung 20 ký tự với chunk kề bên, giữ được mạch ngữ cảnh.

Listing 5.4. Cài đặt vector search (tiêu đề theo sách gốc; nội dung là phép thử hàm chunking)

```
sample = "A" * 283
chunks = fixed_length_chunking(sample, chunk_size=100, overlap=20)
print(f"Original: {len(sample)} chars -> {len(chunks)} chunks")
```

5.3.3 Cài đặt vector search

Đến đây ta đã biết cách chuyển văn bản thành embedding và chia văn bản dài thành chunk. Giờ hãy cài đặt phần tìm kiếm thực sự. Mục tiêu là tìm các chunk giống query nhất. Logic cốt lõi đơn giản đến bất ngờ.

Hàm dưới đây hoạt động theo bốn bước. Đầu tiên, nó chuyển query thành vector. Sau đó `cosine_similarity` tính độ tương đồng giữa vector query và tất cả vector chunk trong một lần. `argsort()[::-1]` sắp xếp chỉ số theo độ tương đồng giảm dần và chọn `top_k` phần tử đầu. Cuối cùng, hàm trả về các dictionary ghép mỗi chunk với điểm tương đồng của nó.

Listing 5.5. Cài đặt vector search

```
def vector_search(query, chunks, chunk_embeddings, top_k=3):
    """Find the most similar chunks to the query."""
    query_embedding = get_embeddings(query)
    similarities = cosine_similarity(query_embedding, chunk_embeddings)[0]
    top_indices = similarities.argsort()[::-1][:top_k]

    results = []
    for idx in top_indices:
        results.append({
            'chunk': chunks[idx],
            'similarity': similarities[idx]
        })
    return results
```

Hãy kiểm tra hoạt động bằng một phép thử đơn giản. Ta chuẩn bị bốn tài liệu ngắn, sinh embedding cho chúng và tìm "AI and coding".[2] Ta gọi `get_embeddings` để tính trước embedding của mọi tài liệu, rồi truyền query cùng các embedding đó cho `vector_search`.

Listing 5.6. Thử nghiệm vector search

```
documents = [
    "Python is a programming language",
    "Machine learning uses Python extensively",
    "Cats are popular pets",
    "Deep learning is a subset of machine learning"
]

doc_embeddings = get_embeddings(documents)
results = vector_search("Artificial Intelligence", documents, doc_embeddings,
↪  top_k=4)

for r in results:
    print(f"{r['similarity']:.3f}: {r['chunk']}")
```

Output như sau:

```
0.353: Deep learning is a subset of machine learning
0.307: Machine learning uses Python extensively
0.162: Python is a programming language
0.048: Cats are popular pets
```

Tìm "Artificial Intelligence" đưa các tài liệu về "deep learning" và "machine learning" lên đầu. Keyword search sẽ không tìm được những tài liệu thiếu đúng các từ "artificial" hay "intelligence". Đây là bản chất của vector search: tìm kiếm ngữ nghĩa khả thi chỉ với embedding và phép tính độ tương đồng, không cần thuật toán phức tạp.

Vì sao cần vector database

Cách làm ta vừa cài đặt có giới hạn. Vì phải tính độ tương đồng với mọi chunk, độ phức tạp thời gian là $O(n)$. Với một triệu tài liệu, mỗi lần tìm kiếm cần một triệu phép tính. Vector database giải quyết vấn đề này. Chúng hỗ trợ tìm kiếm nhanh bằng các thuật toán ANN (Approximate Nearest Neighbor), lưu trữ embedding hiệu quả và cung cấp khả năng lọc theo metadata. Các vector database phổ biến gồm ChromaDB, Pinecone, Weaviate và Qdrant. Với bài thực hành này, cài đặt trực tiếp của ta là đủ vì chỉ xử lý dữ liệu quy mô nhỏ, và mục tiêu là hiểu nguyên lý.

Các vector database chính và đặc điểm:

[2] Lời văn sách ghi query là "AI and coding", còn code và output dùng "Artificial Intelligence".

- **Chroma**: database gọn nhẹ chạy được cục bộ, hợp cho làm prototype và dự án nhỏ. Không cần dựng server riêng. https://www.trychroma.com/
- **Pinecone**: dịch vụ cloud được quản lý toàn phần, khả năng mở rộng rất tốt nhưng phải trả phí. Chọn khi muốn giảm gánh nặng quản lý hạ tầng trong môi trường production quy mô lớn. https://www.pinecone.io/
- **Weaviate**: mã nguồn mở nhưng có tính năng cấp doanh nghiệp. Hỗ trợ GraphQL API, hybrid search và tích hợp với nhiều embedding model. https://weaviate.io/
- **Qdrant**: viết bằng Rust, hiệu năng rất tốt và khả năng lọc mạnh. Hỗ trợ cả on-premises lẫn cloud. https://qdrant.tech/
- **Milvus**: tối ưu cho xử lý vector quy mô lớn và hỗ trợ tăng tốc GPU. Hợp với hệ thống lớn xử lý hàng tỉ vector. https://milvus.io/ko

5.3.4 Bài tập: tìm thông tin liên quan trong kết quả web search

Hãy kết hợp embedding, chunking và vector search vừa học. Mục tiêu là chỉ tìm những phần liên quan tới một chủ đề cụ thể trong kết quả web search thực tế. Bài tập này cho bạn trực tiếp trải nghiệm khả năng khớp ngữ nghĩa của vector search.

Kịch bản bài tập

Tìm "2025 Nobel Prize" trên web trả về các kết quả lẫn lộn giữa vật lý, hoá học, y học và các lĩnh vực khác. Nếu chỉ muốn tìm nội dung liên quan tới "quantum computing" thì sao? Hãy xem liệu có tìm được nội dung liên quan ngay cả khi cụm từ chính xác "quantum computing" không xuất hiện trong kết quả tìm kiếm.

Bước 1: chuẩn bị kết quả web search

Tìm "2025 Nobel Prize" trên Tavily và thu thập kết quả. Tham số then chốt ở đây là `include_raw_content=True`, giúp lấy toàn bộ nội dung trang thay vì chỉ đoạn trích. Ta duyệt qua các kết quả, chỉ giữ những kết quả có `raw_content`, và lưu tiêu đề, toàn bộ nội dung và URL của từng kết quả.

Listing 5.7. Lấy kết quả web search

```
from tavily import TavilyClient

tavily = TavilyClient()
response = tavily.search(
    "2025 Nobel Prize winners",
    max_results=10,
    include_raw_content=True
)

search_results = []
for result in response['results']:
    if result.get('raw_content'):
        search_results.append({
            'title': result['title'],
            'content': result['raw_content'],
            'url': result['url']
        })
```

Bước 2: kiểm tra số token

Đưa thẳng kết quả web search cho LLM sẽ tốn bao nhiêu token? Ta dùng `encoding_for_model` của tiktoken để lấy encoder cho gpt-5, rồi chuyển văn bản thành danh sách token bằng method `encode` và đo độ dài.

Listing 5.8. Đếm token trong kết quả tìm kiếm

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-5")

full_text = "\n\n".join([
    f"Title: {r['title']}\n{r['content']}"
    for r in search_results
])
total_tokens = len(enc.encode(full_text))

print(f"Total characters: {len(full_text)}")
print(f"Total tokens: {total_tokens}")
```

Output như sau:

```
Total characters: 136448
Total tokens: 37312
```

Nếu số token lên tới vài nghìn, mỗi lần đều đưa hết vào context là kém hiệu quả. Chi phí tăng, và thông tin không cần thiết trở thành nhiễu có thể làm giảm hiệu năng. Chỉ trích các phần liên quan có thể giảm xuống chỉ còn vài trăm token.

Bước 3: chunking và embedding

Chia toàn bộ văn bản thành chunk và sinh embedding cho từng chunk. Dùng `fixed_length_chunking` để tạo các chunk 500 ký tự với overlap 50 ký tự, rồi truyền cả danh sách chunk cho `get_embeddings` để lấy mọi embedding trong một lần gọi API.

Listing 5.9. Chunking và embedding kết quả tìm kiếm

```
all_chunks = []
for result in search_results:
    text = f"Title: {result['title']}\n{result['content']}"
    chunks = fixed_length_chunking(text, chunk_size=500, overlap=50)
    for chunk in chunks:
        all_chunks.append({
            'text': chunk,
            'title': result['title'],
            'url': result['url']
        })

print(f"Total chunks: {len(all_chunks)}")

chunk_texts = [c['text'] for c in all_chunks]
chunk_embeddings = get_embeddings(chunk_texts)
```

Bước 4: chạy vector search

Tìm "quantum computing" để lấy các chunk liên quan. Truyền query, danh sách chunk và chunk embedding cho hàm `vector_search` đã cài đặt trước đó, yêu cầu 3 kết quả đầu.

Listing 5.10. Chạy vector search trên các chunk

```
query = "quantum computing"
results = vector_search(query, chunks, chunk_embeddings, top_k=3)

print(f"Query: '{query}'\n")
print("=" * 60)
for i, r in enumerate(results, 1):
    print(f"\n[{i}] Similarity: {r['similarity']:.3f}")
    print(f"{r['chunk'][:300]}...")
```

Ghi chú

Code trong sách truyền biến `chunks` cho `vector_search`, nhưng sau vòng lặp ở Listing 5.9, `chunks` chỉ còn là các chunk của kết quả tìm kiếm cuối cùng. Danh sách khớp với `chunk_embeddings` là `chunk_texts`. Code ở đây giữ nguyên như sách.

Output cho thấy 3 chunk giống "quantum computing" nhất về ngữ nghĩa:

```
Query: 'quantum computing'

[1] Similarity: 0.596
Content about quantum mechanics as the foundation of digital technology...

[2] Similarity: 0.579
Content about quantum computing's economic potential reaching $2 trillion by
↪  2035...

[3] Similarity: 0.566
Content about Physics laureates working on quantum experiments...
```

Kết quả cho thấy vector search đã nhận ra thành công nội dung liên quan đến lượng tử trong nhiều ngữ cảnh khác nhau: vật lý nền tảng, tác động kinh tế và các nhà khoa học đoạt giải Nobel, tất cả đều có điểm tương đồng trên 0,5.

Bước 5: hiệu quả tiết kiệm token

Hãy xem ta tiết kiệm được bao nhiêu token khi chỉ dùng 3 chunk đầu. Ta nối các chunk được chọn, tính số token và in ra tỉ lệ tiết kiệm so với tổng số token.

Listing 5.11. Tính lượng token tiết kiệm được

```
top_chunks = [r['chunk'] for r in results]
selected_text = "\n\n".join(top_chunks)
selected_tokens = len(enc.encode(selected_text))

print(f"Total tokens: {total_tokens}")
print(f"Selected tokens: {selected_tokens}")
print(f"Savings rate: {(1 - selected_tokens/total_tokens)*100:.1f}%")
```

Output cho thấy lượng token dùng giảm mạnh:

```
Total tokens: 37312
Selected tokens: 304
Savings rate: 99.2%
```

Chỉ trích ba chunk liên quan nhất, ta giảm context từ hơn 37.000 token xuống còn 304, đạt tỉ lệ tiết kiệm 99,2% mà vẫn giữ được thông tin liên quan nhất tới "quantum computing".

Điểm rút ra

Bài tập này xác nhận hai điểm chính. Thứ nhất, vector search tìm được nội dung liên quan về ngữ nghĩa mà không cần khớp chính xác keyword. Thứ hai, chỉ trích những phần liên quan thay vì đưa toàn bộ kết quả web search vào context giúp tiết kiệm đáng kể token.

Trong thực tế, các dịch vụ tìm kiếm tối ưu cho LLM như Tavily đã cài đặt sẵn chức năng tương tự bên trong. Dùng `search_depth="advanced"` cùng tham số `chunks_per_source`, bạn có thể đạt kết quả tương đương mà không phải tự xây pipeline chunking và vector search:

```
response = tavily.search(
    "2025 Nobel Prize winners",
    search_depth="advanced",
    chunks_per_source=3
)
```

Lệnh này trả về trực tiếp từ API nội dung đã được chia chunk sẵn và lọc theo độ liên quan. Tuy vậy, hiểu cơ chế bên dưới như ta đã làm trong bài tập này giúp bạn tuỳ biến cách tiếp cận cho từng trường hợp cụ thể và gỡ rối khi giải pháp có sẵn không đáp ứng được. Ở mục 5.5,[3] ta sẽ xem cách áp dụng kỹ thuật này vào Callback để tự động nén kết quả web search.

5.3.5 Structure-based search

Khi có nhiều file, cần phải khám phá. Giải nén một file nén sẽ lộ ra nhiều thư mục và file, và ta không thể biết trước file nào chứa câu trả lời. Trong tình huống như vậy có một cách tiếp cận hiệu quả mà không cần thuật toán tìm kiếm phức tạp: tận dụng chính cấu trúc của hệ thống file.

Các bài toán GAIA có file zip có thể tiếp cận theo cùng cách. Giải nén archive sẽ thấy nhiều file và thư mục. Agent trước hết xem xét cấu trúc, chọn các file liên quan dựa trên gợi ý từ tên file, rồi kiểm tra nội dung. Hãy trực tiếp cài đặt quy trình này.

Các tool sẽ cài đặt trong phần này gồm:

Tool	Mục đích
`unzip_file`	Giải nén file zip
`list_files`	Xem cấu trúc thư mục
`read_file`	Đọc nhiều định dạng file, gồm text, CSV, Excel
`read_media_file`	Phân tích file ảnh, audio và PDF bằng LLM

Hãy bắt đầu bằng việc tìm hiểu cách chuẩn bị bộ dữ liệu GAIA kèm file đính kèm.

5.3.6 Chuẩn bị bộ dữ liệu GAIA

Một số bài toán của benchmark GAIA có file đính kèm bên cạnh nội dung câu hỏi. Chúng gồm nhiều định dạng như zip, pdf, xlsx, ảnh và audio. Agent phải mở và phân tích các file này để tìm ra đáp án đúng. Trong phần này, ta dùng các bài toán có file đính kèm để thực hành structure-based search.

Ở chương 2, ta đã tải bộ dữ liệu GAIA bằng `load_dataset()`. Tuy nhiên, cách này chỉ lấy dataset và không bao gồm các file đính kèm. Để tải file đính kèm, ta cần dùng `snapshot_download()`.

Đoạn code sau tải bộ dữ liệu GAIA và tải các file đính kèm. `load_dataset()` cung cấp metadata như nội dung câu hỏi và đáp án dưới dạng đối tượng Dataset, còn `snapshot_download()` lưu các file đính kèm xuống máy. Ta đặt `local_dir` là thư mục gốc của dự án để cache có thể dùng lại giữa các chương.

Listing 5.12. Tải bộ dữ liệu GAIA kèm file đính kèm

[3] Sách ghi "section 5.5"; phần này thực tế nằm ở mục 5.4.4.

```
from datasets import load_dataset
from huggingface_hub import snapshot_download
from pathlib import Path

# Load dataset (same as Chapter 2)
dataset = load_dataset("gaia-benchmark/GAIA", "2023_all", split="validation")

# Download attached files
NOTEBOOK_DIR = Path.cwd()
PROJECT_ROOT = NOTEBOOK_DIR.parent
CACHE_DIR = PROJECT_ROOT / "gaia_cache"

snapshot_download(
    repo_id="gaia-benchmark/GAIA",
    repo_type="dataset",
    allow_patterns="2023/validation/*",
    local_dir=CACHE_DIR
)
```

Khi agent giải nén hoặc sửa file, rất khó khôi phục trạng thái ban đầu. Để giải quyết vấn đề này, ta cài đặt một hàm sao chép từ cache sang thư mục làm việc. Gọi hàm này trước mỗi bài toán đảm bảo ta luôn bắt đầu từ cùng một trạng thái ban đầu.

Listing 5.13. Hàm đặt lại workspace

```
import shutil

WORK_DIR = NOTEBOOK_DIR / "gaia_workspace"

def reset_workspace():
    """Restore the workspace to its initial state."""
    shutil.rmtree(WORK_DIR, ignore_errors=True)
    shutil.copytree(CACHE_DIR / "2023/validation", WORK_DIR)
    print(f"Workspace reset: {WORK_DIR}")

reset_workspace()
```

Hãy kiểm tra những bài toán nào có file đính kèm.

Listing 5.14. Tìm các bài toán có file đính kèm

```
problems_with_files = [p for p in dataset if p.get('file_name')]
problem_with_zip = [p for p in problems_with_files if
↪  p['file_name'].endswith('.zip')]

print(f"Total problems: {len(dataset)}")
print(f"Problems with attachments: {len(problems_with_files)}")
print(f"Total problems with zip files: {len(problem_with_zip)}")

problem = problem_with_zip[0]
print(f"Question: {problem['Question'][:100]}...")
print(f"File name: {problem['file_name']}")

file_path = WORK_DIR / problem['file_name']
print(f"File exists: {file_path.exists()}")
```

Trường `file_name` lưu tên file đính kèm. Ghép nó với đường dẫn thư mục làm việc sẽ ra đường dẫn thực tế của file. Giờ hãy cài đặt các tool cho phép agent khám phá và đọc những file này.

5.3.7 Cài đặt các tool hệ thống file

Hãy nghĩ về cách một lập trình viên khám phá codebase mới. Đầu tiên họ dùng lệnh `tree` để hiểu cấu trúc tổng thể. Họ chọn những file có vẻ liên quan dựa trên gợi ý từ tên thư mục và tên file. Họ mở file để xem nội dung và khám phá thêm file khác khi cần. Các coding agent như Claude Code, Cursor và Gemini CLI hoạt động đúng theo cách đó.

unzip_file: giải nén archive

Khi một bài toán GAIA có file zip đính kèm, việc đầu tiên agent phải làm là giải nén. Việc này có thể cài đặt bằng thư viện chuẩn `zipfile`. Nếu không chỉ định đường dẫn giải nén, hàm tạo một thư mục mang tên file zip và trả về danh sách các file đã giải nén để agent lên kế hoạch cho bước tiếp theo.

Listing 5.15. Cài đặt tool `unzip_file`

```
import zipfile
from pathlib import Path

def unzip_file(zip_path: str, extract_to: str = None) -> str:
    """Extract a zip file to the specified directory."""
    zip_path = Path(zip_path)
    if not zip_path.exists():
        return f"File not found: {zip_path}"

    # Default extraction path: create folder with zip filename
    if extract_to is None:
        extract_to = zip_path.parent / zip_path.stem
    else:
        extract_to = Path(extract_to)

    extract_to.mkdir(parents=True, exist_ok=True)

    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        file_list = zip_ref.namelist()
        zip_ref.extractall(extract_to)

    # Format results
    result = f"Extracted {len(file_list)} files to {extract_to}/\n\n"
    result += "Contents:\n"
    for f in file_list[:20]:
        result += f"  - {f}\n"
    if len(file_list) > 20:
        result += f"  ... and {len(file_list) - 20} more files\n"
    return result
```

list_files: xem cấu trúc thư mục

Sau khi giải nén, ta cần xem có những file nào. Tool `list_files` trả về danh sách file và thư mục ở đường dẫn chỉ định. Thư mục được phân biệt bằng hậu tố `/`, còn file kèm thông tin kích thước.[4] File ẩn bị loại bỏ, và thư mục được xếp trước để cấu trúc dễ hiểu hơn.

Listing 5.16. Cài đặt tool `list_files`

```
from pathlib import Path

def list_files(path: str = ".") -> str:
```

[4] Code trong Listing 5.16 thực tế không in kích thước file.

```
    """List files and directories in the given path."""
    path = Path(path)
    if not path.exists():
        return f"Path not found: {path}"
    if not path.is_dir():
        return f"Not a directory: {path}"

    items = []
    for item in sorted(path.iterdir()):
        if item.name.startswith('.'):
            continue
        if item.is_dir():
            items.append(f"{item.name}/")
        else:
            items.append(f"{item.name}")

    # Sort directories first
    dirs = [i for i in items if i.endswith('/')]
    files = [i for i in items if not i.endswith('/')]

    result = f"Directory: {path}\n"
    for item in dirs + files:
        result += f"  {item}\n"
    return result
```

read_file: đọc nhiều định dạng file

Khi đã hiểu cấu trúc file, ta cần đọc nội dung. Các bài toán GAIA có file đính kèm ở nhiều định dạng ngoài file text, gồm CSV, Excel và PDF. Tool `read_file` kiểm tra phần mở rộng và đọc file theo cách phù hợp, trả về nội dung dạng text.

Hàm chính điều hướng tới handler phù hợp dựa trên phần mở rộng của file.

Listing 5.17. Cài đặt tool `read_file`

```
from pathlib import Path

TEXT_EXTENSIONS = ['.txt', '.py', '.js', '.json', '.md', '.html',
                   '.css', '.xml', '.yaml', '.yml', '.log', '.sh']
SPREADSHEET_EXTENSIONS = ['.xlsx', '.xls', '.csv']

def read_file(file_path: str, start_line: int = 1, end_line: int = -1) -> str:
    """Read file content. Supports txt, py, json, md, csv, xlsx."""
    path = Path(file_path)
    if not path.exists():
        return f"File not found: {file_path}"

    ext = path.suffix.lower()

    if ext in TEXT_EXTENSIONS:
        return _read_text_file(file_path, start_line, end_line)
    elif ext == '.csv':
        return _read_csv(file_path)
    elif ext in SPREADSHEET_EXTENSIONS:
        return _read_excel(file_path)
    else:
        return _read_text_file(file_path, start_line, end_line)
```

File text được trả về kèm số dòng. Hai tham số `start_line` và `end_line` cho phép chỉ đọc một khoảng cụ thể, hữu ích khi chỉ cần xem phần cần thiết của file dài.

Listing 5.18. Đọc file text kèm số dòng

```
def _read_text_file(file_path: str, start_line: int, end_line: int) -> str:
    with open(file_path, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    # Adjust line numbers (1-indexed to 0-indexed)
    start_idx = max(0, start_line - 1)
    end_idx = len(lines) if end_line == -1 else min(end_line, len(lines))

    selected_lines = lines[start_idx:end_idx]

    result = []
    for i, line in enumerate(selected_lines, start=start_line):
        result.append(f"{i:4d} | {line.rstrip()}")
    return '\n'.join(result)
```

File CSV và Excel được đọc bằng pandas và chuyển thành bảng markdown. Định dạng markdown giúp LLM dễ hiểu cấu trúc bảng.

Listing 5.19. Đọc file CSV và Excel

```
import pandas as pd

def _read_csv(file_path: str) -> str:
    df = pd.read_csv(file_path)
    return df.to_markdown(index=False)

def _read_excel(file_path: str) -> str:
    df = pd.read_excel(file_path)
    return df.to_markdown(index=False)
```

read_media_file: phân tích ảnh và audio

Một số bài toán GAIA có file đính kèm là ảnh, audio hoặc PDF. Những file này cần LLM phân tích chứ không chỉ trích xuất text đơn giản. Ta tách chức năng này khỏi `read_file` vì phân tích media luôn cần một câu hỏi cụ thể: "Cần tìm gì?"

Hàm chính điều hướng tới handler phù hợp dựa trên phần mở rộng của file. Ta sẽ xem kỹ handler cho PDF vì nó kết hợp trích xuất text với phân tích hình ảnh.

Listing 5.20. Cài đặt tool `read_media_file`

```
import fitz  # pymupdf
import base64
from pathlib import Path
from openai import OpenAI

IMAGE_EXTENSIONS = ['.png', '.jpg', '.jpeg', '.gif', '.webp', '.bmp']
AUDIO_EXTENSIONS = ['.mp3', '.wav', '.m4a', '.flac', '.ogg', '.webm']
PDF_EXTENSIONS = ['.pdf']

def read_media_file(file_path: str, query: str) -> str:
    """Analyze an image, audio, or PDF file using LLM."""
    ext = Path(file_path).suffix.lower()

    if ext in IMAGE_EXTENSIONS:
        return _analyze_image(file_path, query)
    elif ext in AUDIO_EXTENSIONS:
```

```
        return _analyze_audio(file_path, query)
    elif ext in PDF_EXTENSIONS:
        return _analyze_pdf(file_path, query)
    else:
        return f"Unsupported media format: {ext}"

def _analyze_image(file_path: str, query: str) -> str:
    with open(file_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    ext = Path(file_path).suffix.lower().lstrip('.')
    media_type = "image/jpeg" if ext == "jpg" else f"image/{ext}"

    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "image_url", "image_url": {
                    "url": f"data:{media_type}; base64,{image_data}"
                }}
            ]
        }]
    )
    return response.choices[0].message.content

def _analyze_audio(file_path: str, query: str) -> str:
    with open(file_path, "rb") as f:
        audio_data = base64.b64encode(f.read()).decode("utf-8")

    ext = Path(file_path).suffix.lower().lstrip('.')

    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o-audio-preview",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "input_audio", "input_audio": {
                    "data": audio_data,
                    "format": ext
                }}
            ]
        }]
    )
    return response.choices[0].message.content

def _analyze_pdf(file_path: str, query: str) -> str:
    doc = fitz.open(file_path)

    # Extract text for context
    text_content = ""
    for page in doc:
        text_content += page.get_text()

    # Convert pages to images
```

```
    images = []
    for page in doc[:5]:  # First 5 pages
        pix = page.get_pixmap(matrix=fitz.Matrix(2, 2))
        img_bytes = pix.tobytes("png")
        images.append(base64.b64encode(img_bytes).decode('utf-8'))

    # Build content with text and images
    content = [{
        "type": "text",
        "text": f"{query}\n\nExtracted text:\n{text_content[:3000]}"
    }]
    for img_b64 in images:
        content.append({
            "type": "image_url",
            "image_url": {"url": f"{img_b64}"}
        })

    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": content}]
    )
    return response.choices[0].message.content
```

Handler PDF kết hợp hai cách: trích xuất text bằng pymupdf và chuyển trang thành ảnh để phân tích hình ảnh. Chỉ trích xuất text thì không xử lý được bảng, sơ đồ hay tài liệu scan. Bằng cách gửi cả text đã trích xuất lẫn ảnh các trang cho LLM, agent có thể trả lời câu hỏi về mọi nội dung PDF.

Phân tích ảnh dùng gpt-4o với dữ liệu ảnh mã hoá base64, còn phân tích audio dùng gpt-4o-audio-preview, model hỗ trợ đầu vào audio gốc. Các cài đặt này theo cùng khuôn mẫu với phân tích PDF. Xem code đầy đủ trong repository.

5.3.8 Kết nối tool với agent

Giờ hãy kết nối các tool hệ thống file vừa cài đặt với Agent từ chương 4. Ta bọc từng hàm bằng decorator `@tool` và đăng ký chúng cùng tool web search từ chương 3.

Đoạn code sau cho thấy cách đăng ký các tool hệ thống file với agent. Vì khám phá file cần nhiều bước nối tiếp, ta đặt `max_steps` bằng 20.

Listing 5.21. Đăng ký các tool hệ thống file với agent

```
from scratch_agents.tools import tool
from scratch_agents.agent import Agent

# File system tools
@tool
def unzip_file(zip_path: str, extract_to: str = None) -> str:
    """Extract a zip file to the specified directory."""
    # Implementation from Listing 5.15

(...)

# Configure with existing tools
tools = [
    search_web,
    unzip_file,
    list_files,
    read_file,
    read_media_file,
```

```
]

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=tools,
    instruction="You are a helpful assistant that can search the web and "
                "explore files to answer questions.",
    max_steps=20
)
```

Giờ agent có thể khám phá hệ thống file như một lập trình viên. Khi nhận một file zip, nó giải nén archive, xem cấu trúc thư mục, đọc các file liên quan và tìm ra câu trả lời.

5.3.9 Giải các bài toán GAIA có file zip

Hãy giải một bài toán thực tế của benchmark GAIA có file zip đính kèm. Ta sẽ minh hoạ toàn bộ quá trình khám phá, từ giải nén tới tìm ra câu trả lời. Trước hết, ta đặt lại workspace và chọn một bài toán có file zip đính kèm. Ta đưa đường dẫn file vào prompt để agent biết file đính kèm nằm ở đâu.

Listing 5.22. Chuẩn bị bài toán GAIA có file zip

```
# Reset workspace to clean state
reset_workspace()

# Select a problem with zip file attachment
zip_problems = [p for p in problems_with_files if
↪  p['file_name'].endswith('.zip')]
problem = zip_problems[1]
file_path = WORK_DIR / problem['file_name']

# Construct prompt including file location
prompt = f"""{problem['Question']}

The attached file is located at: {file_path}"""

print(prompt)
```

Giờ hãy chạy agent và quan sát quá trình khám phá cấu trúc file.

Listing 5.23. Chạy agent trên bài toán có file zip

```
response = await agent.run(prompt)
print(response)
```

Quá trình khám phá của agent theo một khuôn mẫu có hệ thống. Đầu tiên, nó giải nén file zip bằng `unzip_file`. Sau đó nó gọi `list_files` để xem cấu trúc thư mục. Hai file hiện ra: `Job Listing.pdf` và `Applicants.xlsx`. Chỉ riêng tên file đã cho thấy rõ quan hệ giữa chúng: một file mô tả yêu cầu công việc, file kia liệt kê các ứng viên.

Agent trước hết phân tích file PDF bằng `read_media_file` để trích các yêu cầu về trình độ. Sau đó nó đọc file Excel bằng `read_file` để xem dữ liệu của tất cả ứng viên. Cuối cùng, nó đối chiếu các yêu cầu với từng ứng viên để tìm ra câu trả lời.

Điểm cần chú ý là agent duyệt cấu trúc file mà không cần thuật toán tìm kiếm phức tạp. Tên file và phần mở rộng là những gợi ý tự nhiên. “Job Listing” gợi ý các yêu cầu cần đối chiếu, còn “Applicants” cho biết dữ liệu ứng viên cần lọc. Agent dựa vào các quy ước đặt tên và dấu hiệu định dạng này để nhanh chóng định vị thông tin cần cho bài toán.

5.4 Mở rộng agent bằng callback

Agent cài đặt ở chương 4 cung cấp chức năng cơ bản là thực thi tool và nhận kết quả. Tuy nhiên, khi triển khai thực tế sẽ nảy sinh những tình huống cần xin phép người dùng trước khi thực thi một tool, hoặc cần nén kết quả tìm kiếm để quản lý context hiệu quả. Liệu ta có phải sửa logic lõi của Agent để đáp ứng những yêu cầu này?

May mắn là không. Với Callback pattern, ta có thể mở rộng hành vi của Agent mà không đụng tới code lõi. Trong phần này, ta sẽ thêm cơ chế Callback vào Agent và học cách dùng nó qua hai ví dụ thực tế.

5.4.1 Nhu cầu mở rộng agent

Hãy xét các điểm mở rộng của agent từ góc nhìn Context Engineering. Trong quá trình agent gọi LLM và thực thi tool, có nhiều điểm ta có thể can thiệp.

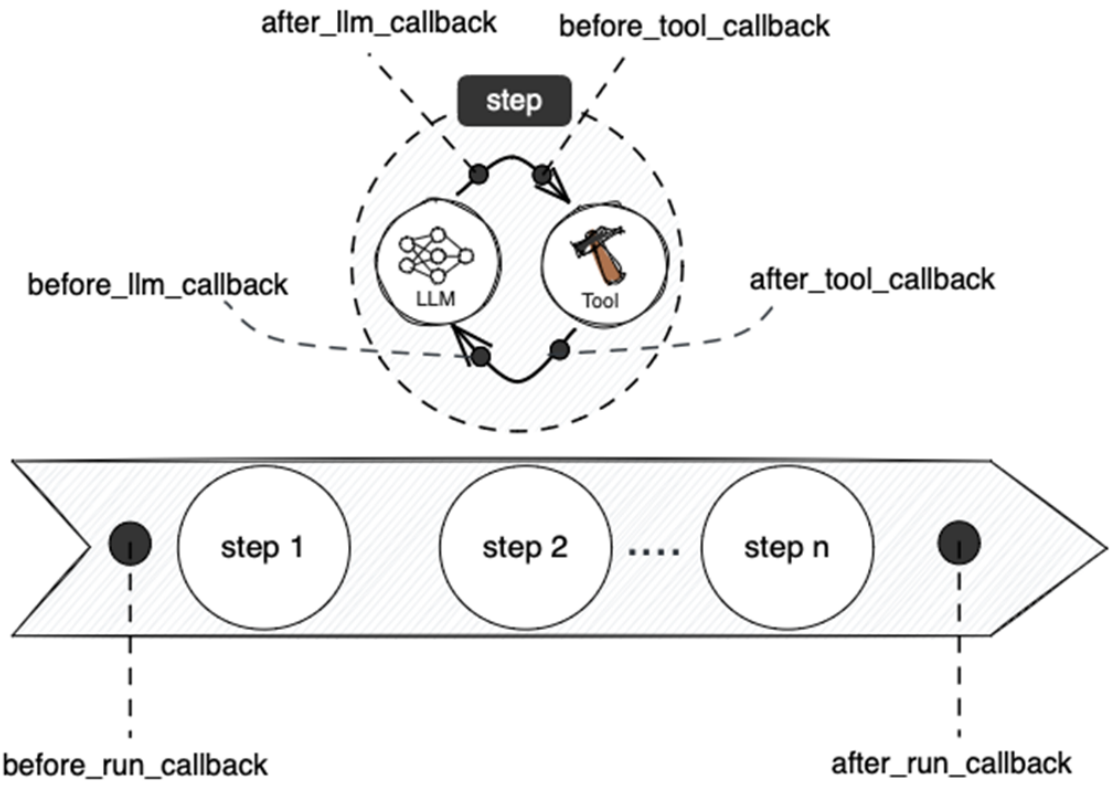
Can thiệp trước khi gọi LLM cho phép ta chèn thêm thông tin vào context, nén các message cũ, hoặc kiểm tra response đã được cache. Sau khi LLM phản hồi, ta có thể kiểm tra hoặc hậu xử lý kết quả. Trước khi thực thi tool, ta có thể yêu cầu người dùng phê duyệt các thao tác nguy hiểm hoặc trả về kết quả đã cache. Sau khi thực thi tool, ta có thể nén kết quả hoặc che thông tin nhạy cảm. Cuối cùng, sau khi toàn bộ lượt chạy kết thúc, ta có thể lưu hội thoại hoặc tạo bản tóm tắt.

Tất cả những điều này đều làm được nhờ Callback. Callback là các hàm do người dùng định nghĩa, được gọi tại những điểm thực thi cụ thể. Chúng cho phép thêm nhiều khả năng mà không phải sửa logic lõi của agent.

Các loại Callback ta sẽ dùng như sau, minh hoạ ở Hình 5.7.

- `before_llm_callback` và `after_llm_callback` chạy trước và sau khi gọi LLM. Chúng dùng để thao tác trên context hoặc chặn response.
- `before_tool_callback` và `after_tool_callback` chạy trước và sau khi thực thi tool. Đây là các callback ta tập trung trong phần này. Chúng dùng để phê duyệt việc thực thi tool hoặc nén kết quả.
- `after_run_callback` chạy sau khi toàn bộ lượt chạy của agent hoàn tất. Nó có thể dùng để lưu nội dung hội thoại vào memory, và ta sẽ dùng nó khi bàn về Memory ở chương 6.

Trong phần này, ta cài đặt và sử dụng các tool callback, vốn liên quan trực tiếp tới chủ đề của chương. Các callback còn lại theo cùng khuôn mẫu, và code đầy đủ có trong GitHub repository.



Hình 5.7: Các điểm chèn Callback trong luồng thực thi của agent. LLM callback được gọi trước và sau `think()`, còn tool callback được gọi trước và sau mỗi lần thực thi tool bên trong `act()`. *(PDF gốc tr. 208)*

5.4.2 Cài đặt tool callback

Giờ hãy thêm hỗ trợ tool callback vào class Agent. Trước hết, ta định nghĩa cách callback hoạt động.

`before_tool_callback(context, tool_call)` được gọi trước khi tool thực thi. Nếu callback này trả về một giá trị, việc thực thi tool thật bị bỏ qua và giá trị đó được dùng làm kết quả. Nếu nó trả về `None`, tool thật sẽ được thực thi.

`after_tool_callback(context, tool_result)` được gọi sau khi tool thực thi. Nếu callback này trả về một giá trị, giá trị đó thay thế kết quả gốc. Nếu nó trả về `None`, kết quả gốc được giữ nguyên.

Khi đăng ký nhiều callback, việc thực thi dừng ở callback đầu tiên trả về giá trị khác `None`. Đây gọi là Early Exit pattern.

Ta sửa constructor của Agent để thêm các tham số callback. Nó nhận hai list: `before_tool_callbacks` và `after_tool_callbacks`. Nếu không truyền callback, chúng được khởi tạo là list rỗng để code phía sau có thể duyệt qua mà không cần kiểm tra `None`.

Listing 5.24. Thêm tham số callback vào Agent

```
class Agent:
    def __init__(
        self,
        model: LlmClient,
        tools: List[BaseTool] = None,
        instructions: str = "",
        max_steps: int = 10,
        output_type: Optional[Type[BaseModel]] = None,
        before_tool_callbacks: List[Callable] = None,
        after_tool_callbacks: List[Callable] = None,
```

```
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.tools = self._setup_tools(tools or [])
        self.output_type = output_type

        # Initialize callback lists
        self.before_tool_callbacks = before_tool_callbacks or []
        self.after_tool_callbacks = after_tool_callbacks or []
```

Tiếp theo, ta refactor method `act()`. Code ban đầu thực thi tool trực tiếp cho từng `tool_call`, giờ nó được chia thành ba giai đoạn.

Ở giai đoạn thứ nhất, ta duyệt qua `before_tool_callbacks` và thực thi từng callback. Nếu một callback trả về giá trị khác `None`, ta lưu giá trị đó vào `tool_response` và thoát vòng lặp. Ta dùng `inspect.isawaitable()` để hỗ trợ cả callback bất đồng bộ.

Ở giai đoạn thứ hai, ta chỉ thực thi tool thật nếu `tool_response` vẫn là `None`. Nếu callback đã cung cấp kết quả, giai đoạn này bị bỏ qua.

Ở giai đoạn thứ ba, ta duyệt qua `after_tool_callbacks`. Nếu một callback trả về `ToolResult` mới, nó thay thế kết quả gốc.

Listing 5.25. Refactor method `act()` với callback

```
import inspect

async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall]
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    results = []

    for tool_call in tool_calls:
        if tool_call.name not in tools_dict:
            raise ValueError(f"Tool '{tool_call.name}' not found")

        tool = tools_dict[tool_call.name]
        tool_response = None
        status = "success"

        # Stage 1: Execute before_tool_callbacks
        for callback in self.before_tool_callbacks:
            result = callback(context, tool_call)
            if inspect.isawaitable(result):
                result = await result
            if result is not None:
                tool_response = result
                break

        # Stage 2: Execute actual tool only if callback didn't provide a result
        if tool_response is None:
            try:
                tool_response = await tool(context, **tool_call.arguments)
            except Exception as e:
                tool_response = str(e)
                status = "error"
```

```
        tool_result = ToolResult(
            tool_call_id=tool_call.tool_call_id,
            name=tool_call.name,
            status=status,
            content=[tool_response],
        )

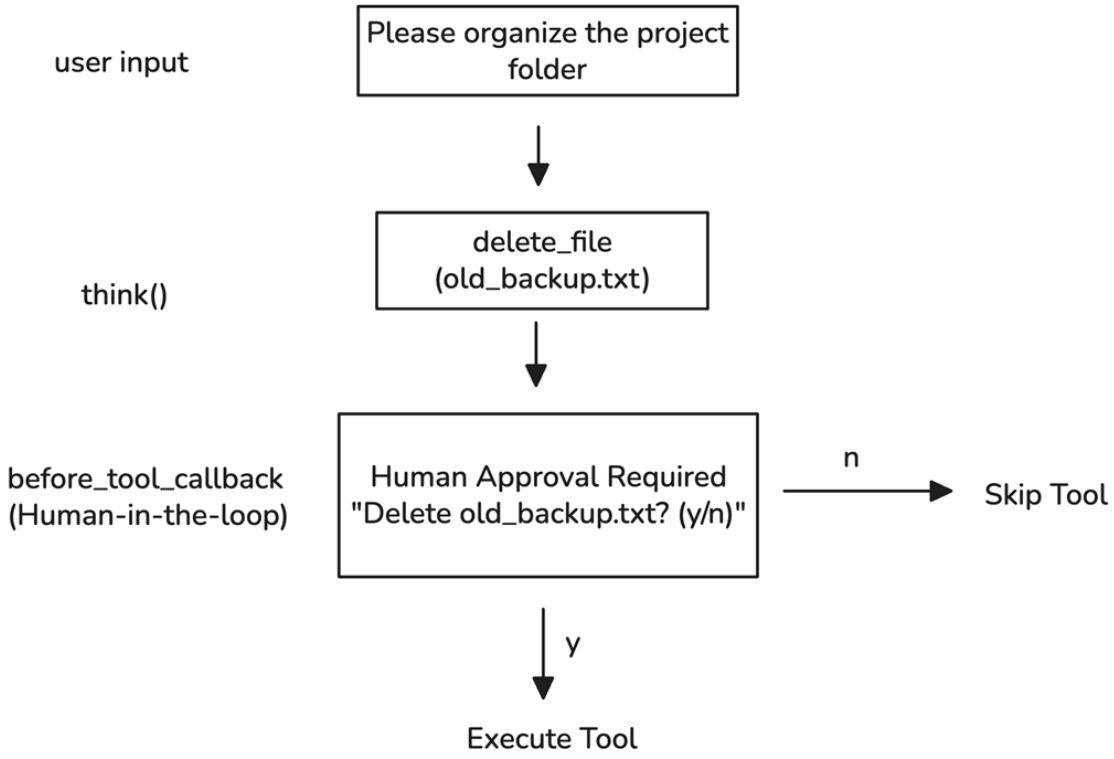
        # Stage 3: Execute after_tool_callbacks
        for callback in self.after_tool_callbacks:
            result = callback(context, tool_result)
            if inspect.isawaitable(result):
                result = await result
            if result is not None:
                tool_result = result
                break

        results.append(tool_result)

    return results
```

5.4.3 Human in the loop: phê duyệt thực thi tool

Để agent tự động thực hiện tác vụ thì hiệu quả, nhưng đôi khi lại rủi ro. Những thao tác nhạy cảm như xoá file, gửi email hay sửa database nên được người dùng xác nhận trước khi thực thi. Human in the Loop pattern được thiết kế đúng cho những tình huống này, như minh hoạ ở Hình 5.8.



Hình 5.8: Luồng phê duyệt thực thi tool trong Human in the Loop pattern. Khi người dùng yêu cầu "Clean up the project folder" (dọn dẹp thư mục dự án), agent tạm dừng lúc định gọi tool `delete_file` và xin người dùng phê duyệt. *(PDF gốc tr. 213)*

Hãy tạo một tool xoá file làm ví dụ cho một tool nguy hiểm cần phê duyệt. Đây là cài đặt giả lập, chỉ

trả về một thông báo chứ không thực sự xoá file.

Listing 5.26. Cài đặt tool `delete_file`

```
@tool
def delete_file(file_path: str) -> str:
    """Deletes a file. This action cannot be undone."""
    # Only returns message instead of actual deletion (for demo)
    return f"File {file_path} has been deleted."
```

Giờ hãy cài đặt một callback chỉ yêu cầu phê duyệt khi các tool nguy hiểm như `delete_file` được gọi. Callback trước hết kiểm tra tên tool có nằm trong danh sách tool cần phê duyệt không. Nếu không, nó trả về `None` để cho thực thi ngay. Nếu có, callback hiển thị thông tin tool và chờ người dùng nhập. Nhập `'y'` sẽ trả về `None` để cho phép thực thi tool thật, còn mọi đầu vào khác trả về một thông báo từ chối để bỏ qua việc thực thi tool.

Listing 5.27. Cài đặt approval callback

```
# List of dangerous tools requiring approval
DANGEROUS_TOOLS = ["delete_file", "send_email", "execute_sql"]

def approval_callback(context: ExecutionContext, tool_call: ToolCall):
    """Requests user approval before executing dangerous tools."""
    # Execute immediately if not a dangerous tool
    if tool_call.name not in DANGEROUS_TOOLS:
        return None

    print(f"\n[WARNING] Dangerous tool execution requested")
    print(f"Tool: {tool_call.name}")
    print(f"Arguments: {tool_call.arguments}")

    response = input("Do you want to execute? (y/n): ").lower().strip()

    if response == 'y':
        print("[OK] Approved. Executing...\n")
        return None  # Proceed with actual tool execution
    else:
        print("[X] Denied. Skipping execution.\n")
        return f"User denied execution of {tool_call.name}"
```

Giá trị trả về của callback là điểm mấu chốt. Trả về `None` khiến method `act()` thực thi tool thật. Trả về một chuỗi thì chuỗi đó được dùng làm kết quả tool, và tool thật không chạy. LLM sẽ thấy thông báo từ chối này ở bước tiếp theo và có thể thử cách khác.

Hãy áp dụng callback này cho một agent. Các tool an toàn như `list_files` hay `read_file` thực thi không cần phê duyệt, còn chỉ khi `delete_file` được gọi thì mới yêu cầu phê duyệt.

Listing 5.28. Tạo agent với approval callback

```
agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[list_files, read_file, delete_file],
    instructions="You are a helpful assistant that can explore and manage
    ↪  files.",
    max_steps=20,
    before_tool_callbacks=[approval_callback],
)
```

Khi chạy, bạn sẽ thấy luồng sau: `list_files` thực thi không cần phê duyệt vì nó không nằm trong danh sách tool nguy hiểm. Khi `delete_file` được gọi, lời nhắc phê duyệt xuất hiện.

5.4.4 Nén kết quả tìm kiếm

Trong bài tập ở mục 5.3.4, ta đã thấy kết quả web search có thể tốn hơn 37.000 token, và chỉ trích các phần liên quan bằng vector search có thể tiết kiệm hơn 99%, giảm xuống còn 304 token. Tuy nhiên, làm thủ công quy trình này mỗi lần thì phiền. Hãy tự động hoá nó ở cấp agent bằng `after_tool_callback`.

Ý tưởng rất đơn giản. Sau khi tool web search thực thi, nếu kết quả quá dài, ta dùng vector search để chỉ trích những phần liên quan tới query. Callback này chỉ áp dụng cho tool tìm kiếm và để nguyên kết quả của các tool khác.

Luồng của callback như sau. Đầu tiên, ta kiểm tra tên tool và trả về `None` nếu không phải `search_web`. Nếu là tool tìm kiếm, ta kiểm tra độ dài kết quả và giữ nguyên nếu dưới 2.000 ký tự. Với kết quả dài, ta tìm query của lần tìm kiếm đó rồi thực hiện chunking và vector search đã cài đặt ở mục 5.3. Cuối cùng, ta trả về một `ToolResult` mới chỉ chứa 3 chunk đầu.

Listing 5.29. Cài đặt callback nén kết quả tìm kiếm

```
def search_compressor(context: ExecutionContext, tool_result: ToolResult):
    """Callback that compresses web search results."""
    # Pass through unchanged if not a search tool
    if tool_result.name != "search_web":
        return None

    original_content = tool_result.content[0]

    # No compression needed if result is short enough
    if len(original_content) < 2000:
        return None

    # Extract search query matching the tool_call_id
    query = _extract_search_query(context, tool_result.tool_call_id)
    if not query:
        return None

    # Use functions implemented in section 5.3
    chunks = fixed_length_chunking(original_content, chunk_size=500, overlap=50)
    embeddings = get_embeddings(chunks)
    results = vector_search(query, chunks, embeddings, top_k=3)

    # Create compressed result
    compressed = "\n\n".join([r['chunk'] for r in results])

    return ToolResult(
        tool_call_id=tool_result.tool_call_id,
        name=tool_result.name,
        status="success",
        content=[compressed]
    )
```

Hàm phụ trợ `_extract_search_query` tìm query của một lần gọi tìm kiếm cụ thể trong `ExecutionContext`. Vì LLM có thể yêu cầu nhiều lần tìm kiếm song song, khớp chính xác theo `tool_call_id` an toàn hơn là chỉ lấy lần tìm kiếm gần nhất. Ta duyệt qua các event, tìm những item có kiểu `ToolCall`, tên là `search_web` và khớp ID, rồi trả về đối số `query`.

Listing 5.30. Trích search query từ context

```
def _extract_search_query(context: ExecutionContext, tool_call_id: str) -> str:
    """Extracts the search query for a specific tool_call_id from context."""
    for event in context.events:
        for item in event.content:
```

```
            if (isinstance(item, ToolCall)
                and item.name == "search_web"
                and item.tool_call_id == tool_call_id):
                return item.arguments.get("query", "")
    return ""
```

Giờ hãy tạo một agent áp dụng cả hai callback cùng lúc. Ta đăng ký approval callback vào `before_tool_callbacks` và callback nén vào `after_tool_callbacks`.

Listing 5.31. Tạo agent với nhiều callback

```
agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[search_web, list_files, read_file, delete_file],
    instructions="You are a helpful assistant that can search the web and "
                 "explore files to answer questions.",
    max_steps=20,
    before_tool_callbacks=[approval_callback],
    after_tool_callbacks=[search_compressor],
)
```

Khi web search thực thi, kết quả được nén tự động. Phần việc ta làm thủ công ở mục 5.3.4 giờ được áp dụng tự động trong lúc agent chạy. Ta đạt cùng lợi ích tiết kiệm token và giảm nhiễu mà hoàn toàn không phải sửa logic lõi của agent.

Điểm rút ra

Trong phần này, ta học được rằng có thể mở rộng hành vi của agent mà không sửa logic lõi nhờ Callback pattern.

- `before_tool_callback` can thiệp trước khi thực thi tool. Nếu nó trả về một giá trị, việc thực thi thật bị bỏ qua và giá trị đó được dùng làm kết quả. Có thể dùng cho phê duyệt Human in the Loop, kiểm tra cache, kiểm tra đầu vào, v.v.
- `after_tool_callback` can thiệp sau khi thực thi tool. Nếu nó trả về một giá trị, giá trị đó thay thế kết quả gốc. Có thể dùng cho nén kết quả tìm kiếm, che thông tin nhạy cảm, chuyển đổi định dạng kết quả, v.v.

Kết hợp nhiều callback cho phép xử lý gọn gàng các yêu cầu phức tạp. Mỗi callback có một trách nhiệm duy nhất và có thể thêm hoặc bỏ khi cần.

Ghi chú

Cài đặt Human in the Loop trong phần này chạy đồng bộ. Toàn bộ chương trình bị chặn trong lúc hàm `input()` chờ người dùng nhập. Cách này ổn khi thử nghiệm ở môi trường cục bộ, nhưng không dùng được trong web service hay API server. Ở chương 6, ta sẽ cài đặt Human in the Loop bất đồng bộ: agent không dừng giữa chừng mà xin phê duyệt qua một response gửi người dùng, rồi tiếp tục thực thi khi phản hồi của người dùng tới.

Ta cũng sẽ dùng `after_run_callback` ở chương 6, để lưu nội dung hội thoại vào Memory sau khi cuộc hội thoại kết thúc. Bạn sẽ tiếp tục thấy callback pattern được dùng như một cơ chế cốt lõi để mở rộng chức năng của agent.

5.5 Tóm tắt

- Dữ liệu nội bộ cần cách tiếp cận khác với web search. Một file đơn lẻ có thể đọc thẳng vào context, nhưng nhiều file thì cần khám phá, và dữ liệu quy mô lớn cần cơ chế tìm kiếm để chỉ trích những phần liên quan.
- Bốn phương pháp tìm kiếm phục vụ các nhu cầu khác nhau: keyword search để khớp chính xác, vector search cho tương đồng ngữ nghĩa, graph search để duyệt quan hệ, và structure-based search để duyệt hệ thống file có tổ chức. Hệ thống production thường kết hợp nhiều phương pháp.
- Vector search chuyển văn bản thành vector số, trong đó tương đồng ngữ nghĩa trở thành khoảng cách hình học. Pipeline gồm chia chunk tài liệu dài, sinh embedding và tính cosine similarity để tìm nội dung liên quan nhất.
- Structure-based search tận dụng chính cách tổ chức hệ thống file làm thông tin. Tên thư mục, tên file và cây thư mục là metadata có ý nghĩa mà agent có thể duyệt bằng các tool đơn giản như `list_files` và `read_file`.
- Callback pattern mở rộng hành vi của agent mà không sửa logic lõi. `before_tool_callback` cho phép phê duyệt Human in the Loop với thao tác nguy hiểm, còn `after_tool_callback` cho phép tự động nén kết quả tìm kiếm.
- Context engineering thông qua tìm kiếm giảm mạnh lượng token. Trong bài tập, chỉ trích các chunk liên quan đã giảm hơn 37.000 token xuống còn 304 token, đạt tỉ lệ tiết kiệm 99% mà vẫn giữ thông tin cần để trả lời query.

Chương 6

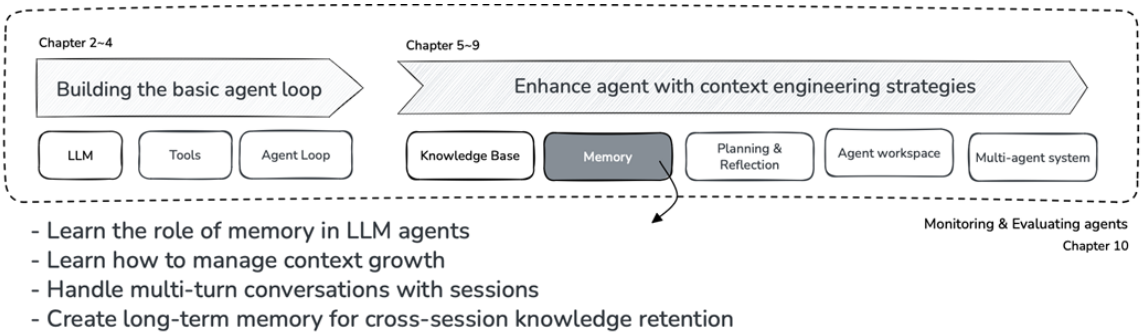
Thêm memory cho agent

Nội dung chương này

- Vai trò của memory trong các LLM agent.
- Kiểm soát sự phình to của context bằng sliding window, compaction và summarization.
- Hiện thực session cho hội thoại nhiều lượt (multi-turn).
- Xây dựng workflow human-in-the-loop bất đồng bộ.
- Tạo long-term memory để giữ lại tri thức xuyên suốt các session.

Memory là thứ phân biệt một công cụ không trạng thái (stateless) với một trợ lý thông minh. Không có memory, agent không thể nhớ lại những gì đã xảy ra trước đó trong cùng một tác vụ, không thể tiếp nối hội thoại từ các session trước, cũng không thể học từ kinh nghiệm cũ. Mỗi lần tương tác đều bắt đầu lại từ đầu: người dùng buộc phải nhắc lại context, còn agent thì không thể tiến bộ theo thời gian.

Chương này xử lý memory theo ba kiểu sử dụng. Thứ nhất, ta hiện thực các chiến lược tối ưu context để tránh hiện tượng bùng nổ context (context explosion) khi giải các bài toán phức tạp. Thứ hai, ta xây dựng `Session` và `SessionManager` để duy trì mạch hội thoại qua nhiều lần tương tác, rồi mở rộng kiến trúc này để hỗ trợ workflow human-in-the-loop bất đồng bộ. Cuối cùng, ta tạo một hệ thống long-term memory có khả năng trích xuất, lưu trữ và truy xuất tri thức vượt qua ranh giới session bằng vector search.



Hình 6.1: Tổng quan cấu trúc cuốn sách: trọng tâm là Chương 6. *(PDF gốc tr. 220)*

6.1 Giải phẫu memory của agent

Memory biến agent từ một công cụ đơn giản thành một trợ lý thông minh biết học hỏi và thích nghi. Thiếu memory, ngay cả agent tinh vi nhất cũng phải bắt đầu lại từ con số không mỗi lần chạy, không thể tham chiếu các cuộc hội thoại trước hay rút kinh nghiệm từ quá khứ.

Trong mục này, ta xem xét các giới hạn của agent hiện tại, phân biệt short-term memory với long-term memory, rồi chia tiếp short-term memory thành hai pattern dựa trên cách agent được sử dụng. Sau đó ta làm rõ framework Context Engineering ở Chương 1 gắn với memory như thế nào.

6.1.1 Giới hạn của kiến trúc memory hiện tại

Agent ta phát triển đến giờ dựa vào một `ExecutionContext` "thô". Cách nó hoạt động rất đơn giản: trong lúc chạy thì nối thêm mọi thứ, chạy xong thì bỏ hết. Cách này ổn với các tác vụ đơn giản, nhưng thiếu một cơ chế để *quản lý* thông tin một cách hiệu quả.

Hãy xét hai tình huống. Thứ nhất là vấn đề về **không gian**: khi giải một bài GAIA phức tạp, kết quả các lần gọi tool tích tụ rất nhanh. Context có thể vượt giới hạn cửa sổ của LLM, khiến agent sập trước khi làm xong. Thứ hai là vấn đề về **thời gian**: khi làm trợ lý cá nhân qua nhiều cuộc hội thoại, agent quên sạch mọi thứ giữa các session. Một người dùng hôm qua đã nói "Tôi là James, tôi làm marketing" thì hôm nay lại phải nhắc lại thông tin đó.

Những giới hạn này bắt nguồn từ việc thiếu quản lý memory. Ta sẽ phân loại các vấn đề này thành ba thách thức riêng biệt và phác thảo giải pháp cho từng cái.

Ba thách thức của kiến trúc memory hiện tại

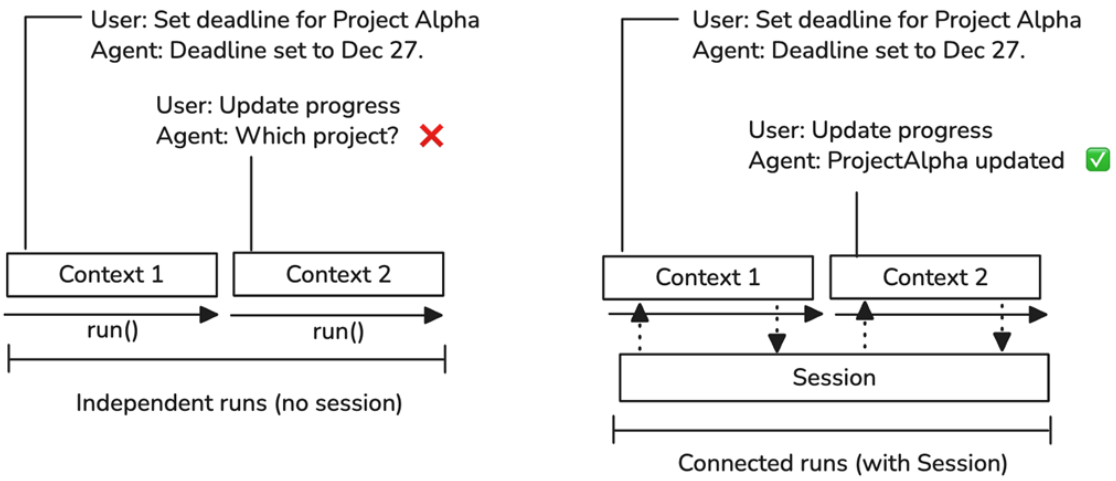
Trước khi đi vào các thách thức, ta thống nhất ba khái niệm làm nền tảng cho mọi thứ sẽ xây dựng trong chương này.

- **Session** là một cuộc hội thoại liên tục, duy nhất giữa người dùng và agent. Hãy hình dung nó như việc mở một cửa sổ chat mới: mọi thứ được trao đổi trong cửa sổ đó thuộc về cùng một session. Đóng cửa sổ và sau đó bắt đầu một cuộc hội thoại mới sẽ tạo ra một session mới.
- **Short-term memory** lưu những gì đã làm cho tới lúc này: lịch sử hội thoại, kết quả gọi tool và các bước suy luận trung gian, tức là mọi thứ được lưu trong `ExecutionContext.events`.
- **Long-term memory** là thông tin vẫn tồn tại sau khi session kết thúc. Khi hôm nay người dùng nói "Tôi làm marketing", long-term memory bảo đảm tuần sau agent vẫn nhớ điều đó, kể cả trong một session hoàn toàn mới.

Thách thức 1: Bùng nổ context. Nhiều agent được dùng theo kiểu chạy một lần. Khi chạy benchmark GAIA, ta gọi `run()` một lần rồi kết thúc. Với những trường hợp như vậy, cách hiện thực hiện tại là đủ. Tuy nhiên, với bài toán phức tạp, nhiều lần gọi tool có thể khiến context tăng rất nhanh. Câu hỏi cốt lõi là: "Làm sao nén context ngay trong lần thực thi hiện tại?"

Thách thức 2: Thiếu tính liên tục của hội thoại. Chuyện gì xảy ra khi agent nhận nhiều yêu cầu từ người dùng, như một chatbot hội thoại, hoặc cần tạm dừng vì một số lần thực thi tool cần người dùng phê duyệt? Làm sao duy trì trạng thái qua nhiều lần gọi `run()`?

Không có quản lý session đúng cách, mỗi lần gọi `run()` hoạt động độc lập và mạch hội thoại bị đứt. Hình 6.2 minh hoạ điều này: bên trái, khi không có session, một yêu cầu tiếp nối sau khi đã hỏi về Project Alpha khiến agent không biết người dùng đang nói tới dự án nào. Bên phải, khi có session, context được giữ lại và agent hiểu đúng điều được nhắc tới.



Hình 6.2: Không có Session (trái), mỗi lần `run()` tạo ra một `ExecutionContext` mới và bị bỏ đi sau khi chạy xong. Có Session (phải), nhiều lần gọi `run()` dùng chung một Session, nhờ đó duy trì được tính liên tục của hội thoại. *(PDF gốc tr. 223)*

Trong các dịch vụ web, server hoạt động không trạng thái. Mỗi HTTP request được xử lý độc lập, và bộ nhớ của server được giải phóng khi request kết thúc. Khi người dùng gửi một tin nhắn khác 10 phút sau, đó là một request mới.

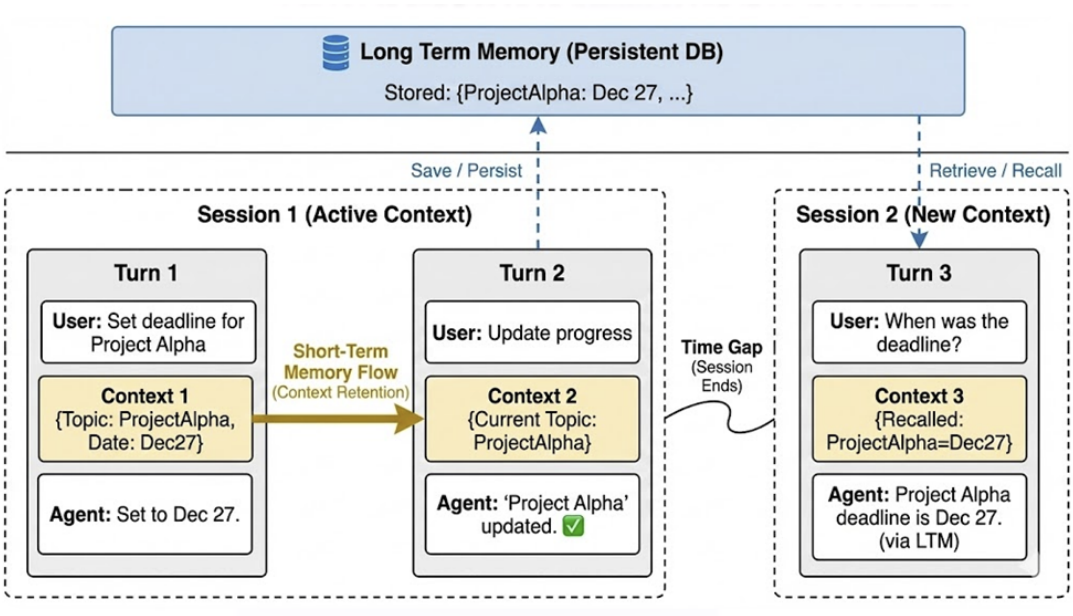
Muốn duy trì context hội thoại thì phải làm việc đó một cách tường minh: gán cho mỗi cuộc hội thoại một `session_id` duy nhất, lưu trạng thái session ra kho lưu trữ bên ngoài khi `run()` kết thúc, và nạp lại trạng thái trước đó khi lần `run()` kế tiếp bắt đầu.

Thách thức 3: Thiếu tri thức bền vững. Hãy xét một trợ lý AI cá nhân hoá. Thông tin như "Tôi làm marketing" được chia sẻ hôm nay cần được nhớ vào tuần sau, kể cả trong một cuộc hội thoại mới. Cái gì nên được giữ lại sau khi session kết thúc, và giữ bằng cách nào?

Điều này khác căn bản với thách thức 2. Tính liên tục của hội thoại duy trì context trong cùng một `session_id`: bạn có thể đóng rồi mở lại cửa sổ chat, và context vẫn còn miễn là cùng một session. Tri thức bền vững thì chia sẻ thông tin giữa các `session_id` khác nhau: agent vẫn nhớ người dùng khi bắt đầu một cuộc hội thoại hoàn toàn mới.

Hình 6.3 cho thấy cơ chế này. Khi session 1 kết thúc, hệ thống xem xét liệu có thông tin nào đáng giữ lại không. Nếu có, tri thức đó được lưu vào long-term memory. Khi một session mới bắt đầu, agent tìm kiếm và truy xuất thông tin liên quan khi cần.

Hình vẽ đơn giản hoá quá trình bằng cách thể hiện việc lưu trữ ở cấp session, nhưng trong thực tế hệ thống hoạt động ở mỗi lần thực thi `run()`: lưu và truy xuất dần dần để kiểm soát chi tiết hơn.



Hình 6.3: Long-term memory cho phép chia sẻ thông tin giữa các session. Mỗi lần `run()` tạo một `ExecutionContext`, nhiều lần chạy hợp thành một Session, và thông tin quan trọng từ các session được trích xuất và lưu lại để dùng về sau. *(PDF gốc tr. 224)*

6.1.2 Context engineering và memory

Ở Chương 1, ta đã định nghĩa Context Engineering là "nghệ thuật cung cấp đúng thông tin vào đúng thời điểm" và giới thiệu năm chiến lược. Chương 6 (Memory) là nơi hội tụ của ba trong số các chiến lược đó (Bảng 6.1).

Bảng 6.1: Các chiến lược context engineering và các chương liên quan

Chiến lược	Mô tả	Chương liên quan
Generation	Xây dựng context bằng output do LLM sinh ra	Chương 7
Retrieval	Đưa thông tin từ nguồn bên ngoài vào context	Chương 3, Chương 5, Chương 6
Write	Lưu từ context ra kho lưu trữ bên ngoài	Chương 6, Chương 8
Reduce	Nén và xoá bớt context	Chương 6
Isolate	Tách tác vụ/tool vào các môi trường riêng biệt	Chương 8, Chương 9

Ba pattern ở trên (Pattern 1, 2, 3, tương ứng với ba thách thức đã nêu) ánh xạ vào các chiến lược context engineering như sau. Pattern 1 và 2 liên quan đến Reduce và Write: để quản lý context đang phình ra, ta thu nhỏ nó bằng các phương pháp như sliding window, compaction và summarization, hoặc tách riêng và lưu nó ra bên ngoài. Pattern 3 lưu thông tin quan trọng ra kho lưu trữ bên ngoài (Write) và truy xuất lại để dùng (Retrieval).

Bảng 6.2: Ánh xạ từng pattern vào các chiến lược

Pattern	Chiến lược chính	Mô tả
Pattern 1, 2	Reduce + Write	Quản lý context lớn bằng sliding window, compaction, summarization
Pattern 3	Write + Retrieval	Trích xuất và lưu memory (Write) → tìm kiếm qua tool ở session sau (Retrieval)

Ta gộp Pattern 1 và 2 với nhau vì cả hai cùng giải quyết một vấn đề: "Làm sao quản lý context đang được gửi tới LLM ngay lúc này?"

- **Pattern 1:** khi context tăng lên bên trong một lần `run()`.
- **Pattern 2:** khi lịch sử hội thoại tích tụ qua nhiều lần gọi `run()`.

Giải pháp cho cả hai cũng dùng chung chiến lược Reduce. Sliding window chỉ giữ N message gần nhất. Compaction xoá các tool call cùng kết quả của chúng, hoặc lưu chúng ra file. Summarization cô đọng các đoạn hội thoại cũ.

Điểm khác nhau nằm ở nơi dữ liệu cư trú. Ở Pattern 1, ta làm việc với `events` bên trong `ExecutionContext`. Ở Pattern 2, `SessionManager` nạp các `events` được lưu trong một `Session`, rồi xử lý chúng theo cùng một cách.

6.2 Quản lý context trong lúc thực thi

Thách thức cốt lõi của Pattern 1 là context tăng rất nhanh khi các tool call và kết quả của chúng tích tụ trong quá trình giải bài toán phức tạp. Ta sẽ hiện thực các chiến lược cụ thể để xử lý vấn đề này.

6.2.1 Tách lưu trữ khỏi trình bày

Nguyên tắc đầu tiên của quản lý context là tách việc lưu trữ (storage) khỏi việc trình bày (presentation). Mọi event được ghi trong `ExecutionContext.events` giữ nguyên, không bị thay đổi; chỉ có `LlmRequest.contents` gửi tới LLM là được xử lý.

Vì sao việc tách này quan trọng

Hãy coi `ExecutionContext` là nguồn sự thật (ground truth): một bản ghi đầy đủ, có cấu trúc về mọi thứ đã xảy ra. `LlmRequest` chỉ là một phép chiếu được tính ra từ đó, có thể tinh chỉnh và tối ưu cho từng lần gọi model cụ thể.

Việc tách này mang lại nhiều lợi ích. Không có lịch sử gốc, việc truy ra nguyên nhân agent thất bại trở nên khó khăn. Để trả lời câu hỏi "Vì sao nó rút ra kết luận sai từ các kết quả tìm kiếm này?", bạn cần chính các kết quả tìm kiếm gốc tại thời điểm đó. Ngoài ra, một số tool có thể tham chiếu kết quả của các thao tác đã thực hiện trước đó; nếu bạn xoá bản gốc, các tool này có thể không hoạt động đúng.

Hãy nghĩ đến việc đăng ảnh lên mạng xã hội: bạn giữ ảnh gốc và chỉ áp filter khi đăng. Vì ảnh gốc vẫn còn, sau này bạn luôn có thể xử lý nó theo cách khác. Cách quản lý context của ta cũng vậy. `ExecutionContext` là kho lưu giữ toàn bộ lịch sử thực thi, còn `LlmRequest` xử lý lịch sử đó để chuyển tới LLM. Bạn có thể thay đổi cách LLM nhìn thấy context mà không cần migrate hay sửa dữ liệu đã lưu.

Hiện thực bằng callback

Ở Chương 4, ta đã giới thiệu `before_llm_callback`. Callback này chạy ngay trước lần gọi LLM, tạo cơ hội để xử lý `LlmRequest`.

Callback nhận cả `ExecutionContext` lẫn `LlmRequest`. Nó đọc trạng thái hiện tại từ `ExecutionContext` và sửa nội dung của `LlmRequest`. Nếu callback trả về `None`, lần gọi LLM diễn ra với request đã được sửa; nếu nó trả về một `LlmResponse`, lần gọi LLM bị bỏ qua và response đó được dùng thay thế. Điểm mấu chốt là `ExecutionContext.events` chỉ được đọc; mọi thay đổi chỉ diễn ra trong `LlmRequest.contents`.

```
before_llm_callback(context: ExecutionContext, request: LlmRequest) ->
↪  Optional[LlmResponse]
```

Để áp dụng các chiến lược tối ưu context một cách linh hoạt, bạn cần điều chỉnh được các giá trị cấu hình như ngưỡng token. Hàm dưới đây giải quyết việc này bằng closure pattern. `create_optimizer_callback` nhận một hàm tối ưu và một ngưỡng làm tham số để sinh ra một hàm callback. Callback được sinh ra chỉ gọi hàm tối ưu khi số token vượt ngưỡng.

Listing 6.1. Tạo optimizer callback có ngưỡng

```
def create_optimizer_callback(apply_optimization, threshold: int = 50000):
    """Factory function that creates a callback applying optimization strategy"""
    async def callback(
        context: ExecutionContext,
        request: LlmRequest
    ) -> Optional[LlmResponse]:
        token_count = count_tokens(request)
        if token_count < threshold:
            return None
        # Support both sync and async functions
        result = apply_optimization(context, request)
        if inspect.isawaitable(result):
            await result
        return None
    return callback
```

Các hàm tối ưu tuân theo chữ ký `(context, request) -> None`. Lý do phải nhận `context` là vì trạng thái thực thi (chẳng hạn quá trình nén đã tiến tới đâu) cần được ghi vào `context.state`. Giờ ta xem các chiến lược tối ưu cụ thể.

6.2.2 Chiến lược sliding window

Cách đơn giản nhất là chỉ giữ N message gần nhất. Bằng cách bỏ các message cũ và chỉ giữ những message mới nhất, bạn giới hạn được kích thước context.

Hiện thực cơ bản

Hàm dưới đây hiện thực chiến lược sliding window qua ba bước:

1. Duyệt qua `contents` để tìm câu hỏi gốc của người dùng.
2. Giữ nguyên vô điều kiện mọi thứ cho tới message của người dùng, bảo đảm agent vẫn nắm được bối cảnh bài toán.
3. Với các phần tử phía sau (message của assistant, tool call, tool result), chỉ giữ `window_size` phần tử gần nhất.

Listing 6.2. Hiện thực chiến lược sliding window

```
def apply_sliding_window(
    context: ExecutionContext,
    request: LlmRequest,
    window_size: int = 20
) -> None:
    """Sliding window that keeps only the most recent N messages"""
    contents = request.contents
    # Find user message position
    user_message_idx = None
    for i, item in enumerate(contents):
        if isinstance(item, Message) and item.role == "user":
```

```
            user_message_idx = i
            break
    if user_message_idx is None:
        return
    # Preserve up to user message
    preserved = contents[:user_message_idx + 1]
    # Keep only the most recent N from remaining items
    remaining = contents[user_message_idx + 1:]
    if len(remaining) > window_size:
        remaining = remaining[-window_size:]
    request.contents = preserved + remaining
```

Để dùng sliding window, truyền nó vào `create_optimizer_callback`.

Listing 6.3. Dùng sliding window với optimizer callback

```
optimizer = create_optimizer_callback(
    apply_optimization=apply_sliding_window,
    threshold=30000
)
agent = Agent(
    model=LlmClient(model="gpt-5"),
    before_llm_callback=optimizer
)
```

Hạn chế

Sliding window dễ hiện thực và có hành vi dễ đoán, nhưng có một số hạn chế. Nếu N nhỏ, bạn có thể mất context quan trọng; nếu N lớn, nó không giải quyết được vấn đề bùng nổ context. Căn bản hơn, số lượng message không phải là thước đo tốt cho kích thước context. Kết quả đọc một file chỉ là một message nhưng có thể chiếm hàng chục nghìn token. Vì vậy, muốn quản lý chính xác hơn thì phải đo trực tiếp số token.

6.2.3 Đếm token

Giới hạn thực sự của LLM được xác định bằng số token, không phải số message. Một kết quả đọc file có thể chiếm 10.000 token, trong khi 20 lượt hội thoại ngắn cộng lại có khi chỉ 500 token. Quản lý context chính xác đòi hỏi đo trực tiếp số token.

Giới thiệu tiktoken

tiktoken là thư viện tokenizer do OpenAI cung cấp. Vì nó giống hệt tokenizer mà các model GPT thực sự dùng, bạn có thể tính chính xác số token. Ngay cả khi dùng model khác (Claude, Gemini, …), nó vẫn dùng được để ước lượng gần đúng.

Hàm dưới đây tính tổng số token của một `LlmRequest`. Mấu chốt là dùng `build_messages` để chuyển request sang đúng định dạng thực sự gửi tới API, rồi mới đếm token. Cách này cho số token chính xác, bao gồm cả instructions, contents và định nghĩa tool.

Listing 6.4. Đếm token bằng tiktoken

```
def count_tokens(request: LlmRequest) -> int:
    """Calculate total token count of LlmRequest"""
    import tiktoken
    from .llm import build_messages
    # Select encoding for model, use default on failure
    try:
```

```
        encoding = tiktoken.encoding_for_model(request.model_id or "gpt-5")
    except KeyError:
        encoding = tiktoken.get_encoding("o200k_base")
    # Convert to API message format then count tokens
    messages = build_messages(request)
    total_tokens = 0
    for message in messages:
        # Per-message overhead (role, separators, etc.)
        total_tokens += 4
        # Content tokens
        if message.get("content"):
            total_tokens += len(encoding.encode(message["content"]))
        # tool_calls tokens
        if message.get("tool_calls"):
            for tool_call in message["tool_calls"]:
                func = tool_call.get("function", {})
                if func.get("name"):
                    total_tokens += len(encoding.encode(func["name"]))
                if func.get("arguments"):
                    total_tokens += len(encoding.encode(func["arguments"]))
    # Tool definition tokens
    if request.tools:
        for tool in request.tools:
            tool_def = tool.tool_definition
            total_tokens += len(encoding.encode(json.dumps(tool_def)))
    return total_tokens
```

Vì sao vẫn cần tối ưu khi còn dưới giới hạn

Phần lớn model hiện đại hỗ trợ context window từ 128K đến hơn 1M token. Bạn có thể nghĩ chỉ cần tối ưu khi sắp chạm giới hạn này. Tuy nhiên, context phình to gây ra vấn đề từ rất lâu trước khi chạm mức tối đa.

Tối ưu context không chỉ để tránh lỗi. Ngay cả khi còn trong giới hạn, context phình to vẫn gây ra ba vấn đề. Thứ nhất, chi phí và độ trễ tăng theo số token: giá model tính theo token, và thời gian tới token đầu tiên (time-to-first-token) tăng theo kích thước context. Thứ hai, thông tin không liên quan làm model xao nhãng khỏi tác vụ trước mắt. Nghiên cứu cho thấy model gặp khó khăn khi tìm thông tin liên quan bị chôn trong context dài, hiện tượng được gọi là "lost in the middle". Thứ ba, bạn cần chừa chỗ cho phản hồi của model.

Các nghiên cứu gần đây và kinh nghiệm vận hành thực tế cho thấy hiệu năng bắt đầu suy giảm từ rất sớm, khi còn cách xa giới hạn context được quảng cáo. Nghiên cứu "Context Rot" của Chroma (2025) phát hiện rằng hiệu năng model ngày càng thiếu ổn định khi độ dài đầu vào tăng, kể cả với các tác vụ đơn giản. Đội kỹ sư của Manus nhận xét rằng "hiệu năng model có xu hướng suy giảm khi vượt quá một độ dài context nhất định, dù về mặt kỹ thuật cửa sổ vẫn hỗ trợ".

Ngưỡng chính xác thay đổi theo model, độ phức tạp của tác vụ và trường hợp sử dụng, nhưng có một đồng thuận rõ ràng: context làm việc tối ưu nhỏ hơn nhiều so với kích thước cửa sổ tối đa. Nguyên tắc chung là thu gọn context về đúng những gì tác vụ thực sự cần.

Trong các ví dụ của sách, ta dùng một giá trị ngưỡng nhỏ (khoảng 20K token) để quan sát rõ hành vi của compaction và quản lý context.

6.2.4 Chiến lược compaction

Ý tưởng cốt lõi của compaction là thay dữ liệu đã được xử lý bằng các tham chiếu gọn nhẹ. Nếu bạn đã đọc một file và phân tích xong, không cần giữ toàn bộ nội dung file trong context nữa. Thay vào đó, bạn để lại một gợi ý kiểu "file này đã được đọc và có thể đọc lại nếu cần".

Cách tiếp cận này theo nguyên tắc "scope by default" (mặc định thu hẹp phạm vi): cung cấp cho agent lượng context tối thiểu cần thiết, và để nó chủ động yêu cầu thêm khi cần. Agent sẽ thấy một tham chiếu như "Search results processed. Re-search if needed." Nếu thực sự cần lại dữ liệu đó, nó sẽ gọi tool.

Compaction chia làm hai loại tuỳ theo đối tượng được nén:

Nén ToolCall: các thao tác ghi

Những tool như `create_file`, vốn lưu dữ liệu ra bên ngoài, chứa dữ liệu lớn trong phần arguments của ToolCall. Ví dụ, lưu một báo cáo 10.000 ký tự ra file sẽ tạo ra ToolCall như sau:

```
ToolCall(
    name="create_file",
    arguments={"path": "report.md", "content": "... 10000 characters of report
    ↪  content ..."})
```

Không cần tiếp tục giữ trong context những nội dung đã được lưu ra file. Hãy bỏ `content` khỏi arguments của ToolCall và thay bằng một message tham chiếu.

Nén ToolResult: các thao tác đọc

Những tool như `read_file` hay `search_web`, vốn lấy dữ liệu từ nguồn bên ngoài, chứa dữ liệu lớn trong phần `content` của ToolResult. Nội dung file hay kết quả tìm kiếm có thể chiếm hàng chục nghìn token.

Nếu agent đã xử lý xong thông tin, chỉ để lại "thông tin gì đã được lấy về" thay cho dữ liệu gốc. Nếu agent cần tham chiếu lại nội dung đó, nó có thể gọi lại tool.

Hiện thực

Đoạn code dưới đây hiện thực compaction. Mấu chốt là xử lý cả ToolCall lẫn ToolResult trong khi duyệt `contents` bằng một vòng `for` duy nhất. Vì ToolCall luôn xuất hiện trước ToolResult tương ứng, khi gặp một ToolCall bạn có thể lưu lại arguments của nó để dùng khi nén ToolResult về sau.

`TOOLCALL_COMPACTION_RULES` định nghĩa những tool có arguments của `ToolCall` cần được nén, còn `TOOLRESULT_COMPACTION_RULES` định nghĩa những tool có `content` của `ToolResult` cần được nén.

Listing 6.5. Hiện thực chiến lược compaction

```
# Tools to compress ToolCall arguments
TOOLCALL_COMPACTION_RULES = {
    "create_file": "[Content saved to file]",
}

# Tools to compress ToolResult content
TOOLRESULT_COMPACTION_RULES = {
    "read_file": "File content from {file_path}. Re-read if needed.",
    "search_web": "Search results processed. Query: {query}. Re-search if
    ↪  needed.",
    "tavily_search": "Search results processed. Query: {query}. Re-search if
    ↪  needed.",
}

def apply_compaction(context: ExecutionContext, request: LlmRequest) -> None:
    """Compress tool calls and results into reference messages"""
    tool_call_args: Dict[str, Dict] = {}
    compacted = []
    for item in request.contents:
```

```
        if isinstance(item, ToolCall):
            # Save arguments (for use when compressing ToolResult later)
            tool_call_args[item.tool_call_id] = item.arguments
            # If the ToolCall itself is a compression target (create_file, etc.)
            if item.name in TOOLCALL_COMPACTION_RULES:
                compressed_args = {
                    k: TOOLCALL_COMPACTION_RULES[item.name] if k == "content"
                    ↪  else v
                    for k, v in item.arguments.items()
                }
                compacted.append(ToolCall(
                    tool_call_id=item.tool_call_id,
                    name=item.name,
                    arguments=compressed_args
                ))
            else:
                compacted.append(item)
        elif isinstance(item, ToolResult):
            # If ToolResult is a compression target (read_file, search_web, etc.)
            if item.name in TOOLRESULT_COMPACTION_RULES:
                args = tool_call_args.get(item.tool_call_id, {})
                template = TOOLRESULT_COMPACTION_RULES[item.name]
                compressed_content = template.format(
                    file_path=args.get("file_path", args.get("path", "unknown")),
                    query=args.get("query", "unknown")
                )
                compacted.append(ToolResult(
                    tool_call_id=item.tool_call_id,
                    name=item.name,
                    status=item.status,
                    content=[compressed_content]
                ))
            else:
                compacted.append(item)
        else:
            compacted.append(item)
    request.contents = compacted
```

Cách này biến "100K token nhiễu trong mọi prompt" thành một nguồn tài nguyên chính xác, chỉ lấy khi cần. Dữ liệu có thể rất lớn, nhưng context window vẫn gọn nhẹ.

Đánh đổi với prompt caching

Có một yếu tố quan trọng cần cân nhắc khi áp dụng compaction: prompt caching.

Prompt caching là tính năng tối ưu chi phí do các LLM API cung cấp. Khi phần đầu của prompt giống hệt nhau giữa các request liên tiếp, API sẽ dùng lại kết quả đã tính trước đó. Điều này có thể giảm chi phí tới 90% và tăng tốc độ phản hồi. OpenAI, Anthropic và Google đều hỗ trợ tính năng này, và nó được áp dụng tự động mà không cần cấu hình gì đặc biệt.

Vấn đề là compaction sửa các message trong quá khứ. Điều này quan trọng vì prompt caching hoạt động bằng cách dùng lại phần tính toán cho các tiền tố (prefix) giống hệt nhau. Hãy xem chuyện gì xảy ra qua hai bước liên tiếp:

```
Step 5: [system][user][msg1][msg2][msg3][msg4][msg5]
Step 6: [system][user][msg1][msg2][msg3][msg4][msg5][msg6]
                                                    ^ only this needs new
                                                    ↪  computation
```

Khi bạn sửa các message ở giữa context (như nén các tool result cũ), mọi thứ phía sau điểm đó đều trở nên khác đi:

```
Step 6 (Compacted): [system][user][msg1][compressed msg2][msg3]...[msg6]
                                        ^ changed -> everything after is cache
                                        ↪  miss
```

Giải pháp là tối ưu dựa trên ngưỡng. Dưới một ngưỡng nhất định, giữ context ổn định để tận dụng lợi ích của caching. Chỉ sửa đổi khi cái giá của việc tràn context lớn hơn cái giá của cache miss. Hàm `create_optimizer_callback` ta đã hiện thực đảm nhận đúng vai trò này.

```
# Below 50000 tokens, don't apply Compaction (maintain caching benefits)
# At 50000 tokens or above, apply Compaction (prevent degradation)
optimizer = create_optimizer_callback(
    apply_optimization=apply_compaction,
    threshold=50000
)
```

6.2.5 Chiến lược summarization

Compaction chỉ nhắm vào một số tool call và kết quả cụ thể. Nếu bản thân hội thoại chung hoặc phần suy luận của agent trở nên dài, chỉ compaction thôi thì không giải quyết được. Trong những trường hợp này, summarization được dùng như biện pháp cuối cùng.

Chọn đối tượng tóm tắt

Summarization chọn các message cũ nhất làm đối tượng tóm tắt trước. Tuy nhiên, câu hỏi gốc của người dùng phải được giữ nguyên: agent không được quên mình đang cố giải quyết việc gì.

Summarization chỉ nhắm vào quá trình trung gian: những tool nào đã được gọi, thông tin gì đã được phát hiện, và tiến độ đã đạt được. Phần thông tin này được tóm tắt rồi thêm vào system prompt.

Thiết kế prompt tóm tắt

Dưới đây là prompt dùng để tóm tắt quá trình trung gian của agent. Yêu cầu output có cấu trúc giúp các bản tóm tắt nhất quán.

```
SUMMARIZATION_PROMPT = """You are summarizing an AI agent's work progress.

Given the following execution history, extract:
1. Key findings: Important information discovered
2. Tools used: List of tools that were called
3. Current status: What has been accomplished and what remains

Be concise. Focus on information that will help the agent continue its work.

Execution History:
{history}

Provide a structured summary."""
```

Hiện thực

Hàm dưới đây hiện thực summarization. Logic cốt lõi gồm bốn bước:

1. Tìm vị trí message của người dùng để đánh dấu điểm bắt đầu của vùng được giữ nguyên.

2. Giữ nguyên dạng gốc `keep_recent` message gần nhất.
3. Dùng LLM tóm tắt quá trình trung gian nằm giữa hai phần trên.
4. Dựng lại kết quả theo dạng "câu hỏi của người dùng + message tóm tắt + các message gần đây".

Việc ghi lại vị trí tóm tắt gần nhất vào `context.state` giúp tránh tóm tắt lại những phần đã được tóm tắt ở các lần gọi summarization tiếp theo.

Listing 6.6. Hiện thực chiến lược summarization

```
async def apply_summarization(
    context: ExecutionContext,
    request: LlmRequest,
    llm_client: LlmClient,
    keep_recent: int = 5
) -> None:
    """Replace old messages with a summary"""
    contents = request.contents
    # Find user message position
    user_idx = None
    for i, item in enumerate(contents):
        if isinstance(item, Message) and item.role == "user":
            user_idx = i
            break
    if user_idx is None:
        return
    # Check previous summary position (skip already-summarized portions)
    last_summary_idx = context.state.get("last_summary_idx", user_idx)
    # Calculate summarization target range
    summary_start = last_summary_idx + 1
    summary_end = len(contents) - keep_recent
    # Overlap prevention: exit if nothing to summarize or range is invalid
    if summary_end <= summary_start:
        return
    # Determine portions to preserve (no overlap)
    preserved_start = contents[:last_summary_idx + 1]
    preserved_end = contents[summary_end:]
    to_summarize = contents[summary_start:summary_end]
    # Generate summary
    history_text = format_history_for_summary(to_summarize)
    summary = await generate_summary(llm_client, history_text)
    # Add summary to instructions
    request.append_instructions(f"[Previous work summary]\n{summary}")
    # Keep only preserved portions in contents
    request.contents = preserved_start + preserved_end
    # Record summary position
    context.state["last_summary_idx"] = len(preserved_start) - 1

def format_history_for_summary(items: List[ContentItem]) -> str:
    """Convert ContentItem list to text for summarization"""
    lines = []
    for item in items:
        if isinstance(item, Message):
            lines.append(f"[{item.role}]: {item.content[:500]}...")
        elif isinstance(item, ToolCall):
            lines.append(f"[Tool Call]: {item.name}({item.arguments})")
        elif isinstance(item, ToolResult):
            content_preview = str(item.content[0])[:200] if item.content else ""
            lines.append(f"[Tool Result]: {item.name} -> {content_preview}...")
```

```
    return "\n".join(lines)

async def generate_summary(llm_client: LlmClient, history: str) -> str:
    """Generate history summary using LLM"""
    request = LlmRequest(
        instructions=[SUMMARIZATION_PROMPT.format(history=history)],
        contents=[Message(role="user", content="Please summarize.")]
    )
    response = await llm_client.generate(request)
    for item in response.content:
        if isinstance(item, Message):
            return item.content
    return ""
```

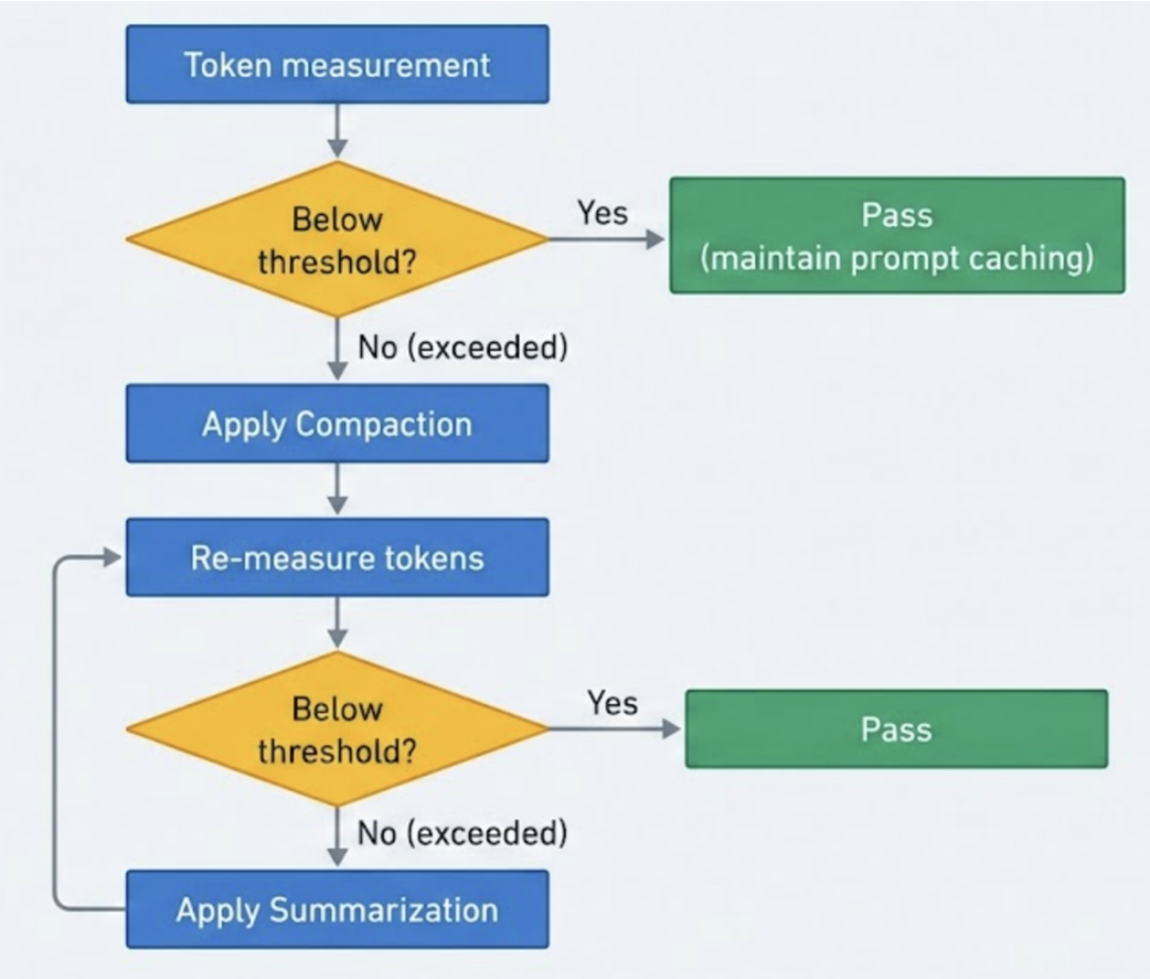
Compaction và summarization

Hai chiến lược này khác nhau về bản chất. Compaction có tính tất định (deterministic): bạn dự đoán được chính xác cái gì sẽ bị nén dựa trên tên tool. Summarization mang tính ngữ nghĩa (semantic): một LLM quyết định điều gì là quan trọng. Các thao tác tất định rẻ hơn, nhanh hơn và dễ đoán hơn, nên ta áp dụng chúng trước.

Còn một khác biệt quan trọng nữa: khả năng đảo ngược. Với compaction, agent có thể khôi phục dữ liệu gốc bằng cách gọi lại tool. Với summarization, phần suy luận gốc bị mất vĩnh viễn. Vì vậy, hãy thử compaction trước và chỉ áp dụng summarization khi compaction không đủ.

6.2.6 Quản lý context phân tầng

Nên áp dụng các chiến lược này theo thứ tự nào? Xét về chi phí và hiệu quả, cách tiếp cận phân tầng minh hoạ trong Hình 6.4 là hợp lý.



Hình 6.4: Luồng quyết định chiến lược quản lý context. *(PDF gốc tr. 243)*

Đầu tiên, đo số token. Nếu dưới ngưỡng thì không làm gì cả. Cách này giữ được lợi ích của prompt caching và tránh xử lý không cần thiết.

Nếu vượt ngưỡng, áp dụng compaction. Compaction miễn phí và nhanh. Trong phần lớn trường hợp, các lần đọc file hay kết quả tìm kiếm là thủ phạm chính làm context tăng, nên chỉ riêng compaction thường đã đủ.

Nếu sau compaction vẫn vượt ngưỡng, áp dụng summarization. Đây là biện pháp cuối cùng. Nó tốn thêm chi phí cho một lần gọi LLM, nhưng vẫn tốt hơn là thất bại vì lỗi context.

Hiện thực tích hợp

Để áp dụng nhiều chiến lược theo trình tự, viết chúng thành một class sẽ gọn gàng hơn. Class dưới đây tích hợp mọi chiến lược. Method `__call__` được đăng ký làm `before_llm_callback` và thực thi tuần tự ba giai đoạn. Sau mỗi giai đoạn, số token được đo lại, và nếu đã dưới ngưỡng thì method trả về ngay.

Listing 6.7. Class ContextOptimizer phân tầng

```
class ContextOptimizer:
    """Hierarchical context optimization strategy"""
    def __init__(
        self,
        llm_client: LlmClient,
        token_threshold: int = 50000,
        enable_compaction: bool = True,
```

```
        enable_summarization: bool = True,
        keep_recent: int = 5
    ):
        self.llm_client = llm_client
        self.token_threshold = token_threshold
        self.enable_compaction = enable_compaction
        self.enable_summarization = enable_summarization
        self.keep_recent = keep_recent

    async def __call__(
        self,
        context: ExecutionContext,
        request: LlmRequest
    ) -> Optional[LlmResponse]:
        """Register as before_llm_callback"""
        # Step 1: Measure tokens
        if count_tokens(request) < self.token_threshold:
            return None
        # Step 2: Apply Compaction
        if self.enable_compaction:
            apply_compaction(context, request)
            if count_tokens(request) < self.token_threshold:
                return None
        # Step 3: Apply Summarization
        if self.enable_summarization:
            await apply_summarization(
                context,
                request,
                self.llm_client,
                self.keep_recent
            )
        return None
```

Đăng ký với agent

Đoạn code sau cho thấy cách đăng ký `ContextOptimizer` làm `before_llm_callback` của agent. LLM dùng cho việc tối ưu là một model rẻ hơn (gpt-5-mini) để giảm chi phí, còn model chính của agent là model mạnh hơn (gpt-5).

```
optimizer = ContextOptimizer(
    llm_client=LlmClient(model="gpt-5-mini"),
    token_threshold=50000,
    enable_compaction=True,
    enable_summarization=True
)
agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[search_web, read_file, create_file],
    instructions="You are a helpful research assistant.",
    max_steps=30,
    before_llm_callback=optimizer
)
```

6.3 Thực thi liên tục: Session và quản lý trạng thái

Ta đã xem thách thức cốt lõi của Pattern 2 (thực thi liên tục): duy trì context hội thoại qua nhiều lần gọi `run()`. Hiện tại, agent của ta tạo một `ExecutionContext` mới ở mỗi lần gọi `run()`, khiến mọi thông tin bị mất khi lần thực thi kết thúc. Nó không nhớ cuộc hội thoại hôm qua, cũng không nhớ cuộc hội thoại vừa diễn ra cách đây ít phút.

`Session` và `SessionManager` mà ta sẽ hiện thực trong mục này giải quyết vấn đề đó. Bằng cách lưu lịch sử hội thoại trong một Session và truy cập lại nó với cùng `session_id`, ta có thể tiếp nối liền mạch các cuộc hội thoại trước. Ví dụ sau minh hoạ khả năng này:

```
# Maintaining conversation continuity with the same session_id
result1 = await agent.run("My name is Alice", session_id="session_1")
# -> "Nice to meet you, Alice!"

result2 = await agent.run("What's my name?", session_id="session_1")
# -> "Your name is Alice."
```

Tuy nhiên, vai trò của Session không dừng ở việc lưu lịch sử chat. Session có thể lưu không chỉ lịch sử hội thoại mà cả trạng thái thực thi trung gian. Tận dụng khả năng này, ta có thể vượt qua giới hạn căn bản của cách hiện thực human-in-the-loop đồng bộ ở Chương 5.

Ở Chương 5, ta hiện thực việc người dùng phê duyệt bằng hàm `input()`. Trước khi chạy một tool nguy hiểm, agent hỏi "Do you approve? (Y/N)" trên terminal và chờ người dùng trả lời. Cách này chạy tốt khi phát triển ở máy local, nhưng không dùng được trong các dịch vụ production.

Giải pháp là tạm dừng việc thực thi của agent khi gặp một tool cần phê duyệt, lưu trạng thái lại, và tiếp tục sau. Khi agent gặp một tool call cần phê duyệt, nó lưu trạng thái thực thi hiện tại vào Session rồi trả về ngay. Sau đó người dùng có thể bấm nút trên trang web, phản hồi một thông báo trên ứng dụng di động, hoặc thậm chí phê duyệt qua email vài ngày sau. Khi có phê duyệt, trạng thái đã lưu được khôi phục và việc thực thi tiếp tục.

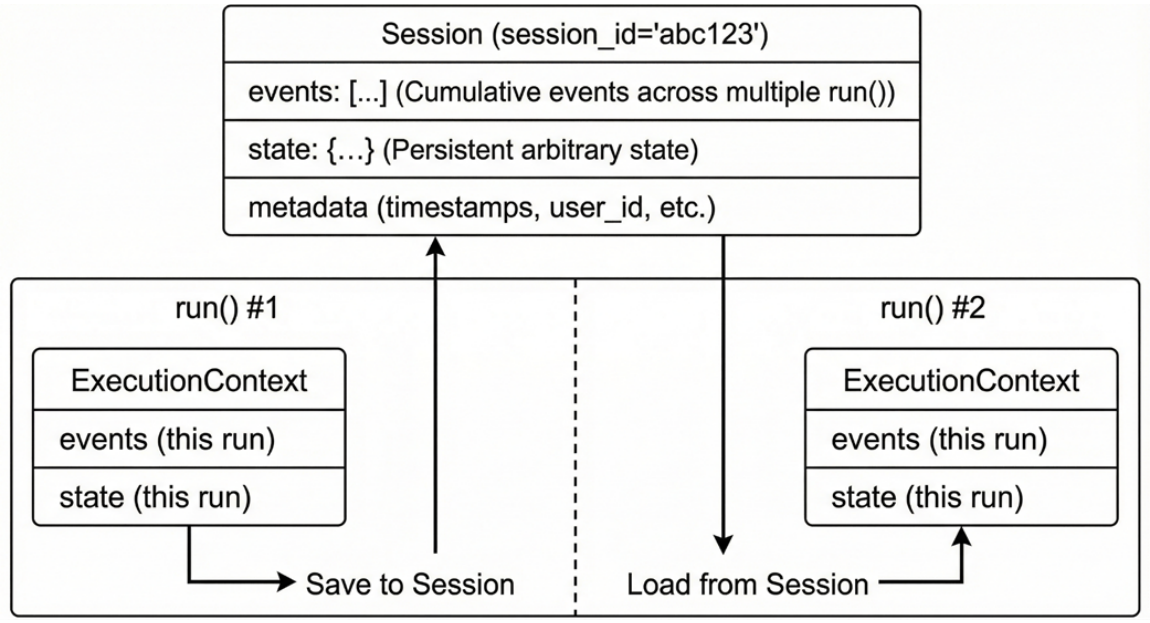
Trong mục này, ta đạt hai mục tiêu:

- **Phần 1: Nền tảng Session.** Ta tìm hiểu cấu trúc của class `Session` và `SessionManager`, rồi tích hợp chúng vào agent để hỗ trợ hội thoại nhiều lượt.
- **Phần 2: Human-in-the-loop.** Ta tận dụng khả năng lưu giữ trạng thái của Session để hiện thực một workflow phê duyệt tool bất đồng bộ. Qua đó, ta sẽ thấy tận mắt rằng Session mang lại giá trị vượt xa việc chỉ lưu lịch sử chat.

6.3.1 Class Session

Session là một container chứa mọi thông tin cần được giữ lại qua nhiều lần gọi `run()`. Nó phục vụ hai mục đích cốt lõi. Thứ nhất, nó lưu lịch sử hội thoại để có thể tiếp nối các cuộc hội thoại trước. Thứ hai, nó lưu trạng thái thực thi tuỳ ý để các tác vụ bị gián đoạn có thể được tiếp tục.

Trước khi thiết kế cấu trúc của Session, ta cần làm rõ quan hệ của nó với `ExecutionContext` đã hiện thực ở Chương 4. Một `ExecutionContext` chứa mọi event và trạng thái phát sinh trong một lần thực thi `run()`. Khi lần thực thi kết thúc, `ExecutionContext` đó không còn được dùng nữa. Ngược lại, một Session tồn tại qua nhiều lần thực thi `run()`. Do đó, Session có vòng đời dài hơn `ExecutionContext`. Pattern hoạt động như sau: khi mỗi lần thực thi `run()` hoàn tất, thông tin cần thiết được lưu vào Session; khi lần thực thi `run()` kế tiếp bắt đầu, thông tin được khôi phục từ Session. Quan hệ này được thể hiện trong Hình 6.5.



Hình 6.5: Quan hệ giữa Session và ExecutionContext. *(PDF gốc tr. 248)*

Giờ ta hiện thực class `Session`. `session_id` định danh duy nhất cho session, còn `user_id` (tuỳ chọn) định danh người dùng. Danh sách `events` lưu mọi event tích luỹ qua nhiều lần gọi `run()`, còn dictionary `state` lưu trạng thái thực thi tuỳ ý. Các timestamp ghi lại thời điểm session được tạo và được cập nhật lần cuối.

Listing 6.8. Thiết kế class Session

```
class Session(BaseModel):
    """Container for persistent conversation state across multiple run()
    ↪  calls."""
    session_id: str
    user_id: str | None = None
    events: list[Event] = Field(default_factory=list)
    state: dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=datetime.now)
    updated_at: datetime = Field(default_factory=datetime.now)
```

Một điểm đáng chú ý trong thiết kế class `Session` là tính linh hoạt của trường `state`. Với kiểu `dict[str, Any]`, nó có thể lưu bất kỳ loại dữ liệu nào. Nó có thể giữ một tool call đang chờ phê duyệt trong kịch bản human-in-the-loop, hoặc lưu tiến độ của một tác vụ chạy lâu. Ta chọn thiết kế tổng quát này để hỗ trợ nhiều kiểu thực thi khác nhau mà không phải sửa bản thân class `Session`.

6.3.2 Quản lý session bằng SessionManager

Class `Session` chỉ là một container dữ liệu; nó không biết cách tự tạo ra hay tự lưu trữ chính nó. Ta cần một thành phần quản lý riêng chịu trách nhiệm tạo, truy xuất và lưu session. Vai trò này thuộc về `SessionManager`.

Các trách nhiệm cốt lõi của `SessionManager` gồm:

- **Create**: tạo một session mới và lưu nó.
- **Get**: tìm và trả về một session có sẵn theo `session_id`.
- **Get or Create**: trả về session có sẵn nếu tìm thấy, nếu không thì tạo mới.

- **Save**: ghi các thay đổi của session trở lại kho lưu trữ.

Đầu tiên ta định nghĩa các trách nhiệm này thành một abstract class, rồi cung cấp các hiện thực cụ thể tuỳ theo loại kho lưu trữ. Chẳng hạn, `InMemorySessionManager` lưu session trong một dictionary của Python, còn môi trường production có thể thay nó bằng một hiện thực dựa trên database.

Định nghĩa abstract class

Trước hết, ta định nghĩa interface mà mọi hiện thực `SessionManager` phải tuân theo. Method `create` tạo session mới, `get` truy xuất session có sẵn, và `save` ghi thay đổi xuống kho lưu trữ. Vì `get_or_create` chỉ là tổ hợp của `get` và `create`, ta hiện thực nó như một method thông thường thay vì abstract method. Các subclass chỉ cần hiện thực `get` và `create`, và `get_or_create` sẽ tự động hoạt động.

Lưu ý rằng mọi method đều được khai báo `async`. Với các hiện thực đơn giản như `InMemorySessionManager`, thao tác bất đồng bộ là không cần thiết. Tuy nhiên, các hiện thực dùng database hay API bên ngoài thì cần đến nó. Bằng cách định nghĩa interface bất đồng bộ ngay trong abstract class, ta có thể hoán đổi các hiện thực mà không phải sửa code gọi.

Listing 6.9. Abstract class BaseSessionManager

```
from abc import ABC, abstractmethod

class BaseSessionManager(ABC):
    """Abstract base class for session management."""

    @abstractmethod
    async def create(
        self,
        session_id: str,
        user_id: str | None = None
    ) -> Session:
        """Create a new session."""
        pass

    @abstractmethod
    async def get(self, session_id: str) -> Session | None:
        """Retrieve a session by ID. Returns None if not found."""
        pass

    @abstractmethod
    async def save(self, session: Session) -> None:
        """Persist session changes to storage."""
        pass

    async def get_or_create(
        self,
        session_id: str,
        user_id: str | None = None
    ) -> Session:
        """Get existing session or create new one."""
        session = await self.get(session_id)
        if session is None:
            session = await self.create(session_id, user_id)
        return session
```

Hiện thực in-memory

Giờ ta hiện thực `InMemorySessionManager` cho môi trường phát triển và kiểm thử. Nó dùng một dictionary của Python làm kho lưu trữ, nghĩa là mọi session sẽ mất khi chương trình kết thúc. Trong

production, bạn sẽ thay nó bằng một hiện thực dựa trên database.

Method `create` khởi tạo một đối tượng `Session` mới và lưu vào dictionary. Nếu đã có session với cùng `session_id`, nó raise `ValueError` để tránh vô tình ghi đè. Method `get` tra session trong dictionary và trả về `None` nếu không tìm thấy. Method `save` lưu session vào dictionary.

Listing 6.10. Hiện thực InMemorySessionManager

```
class InMemorySessionManager(BaseSessionManager):
    """In-memory session storage for development and testing."""

    def __init__(self):
        self._sessions: dict[str, Session] = {}

    async def create(
        self,
        session_id: str,
        user_id: str | None = None
    ) -> Session:
        """Create a new session."""
        if session_id in self._sessions:
            raise ValueError(f"Session {session_id} already exists")
        session = Session(
            session_id=session_id,
            user_id=user_id
        )
        self._sessions[session_id] = session
        return session

    async def get(self, session_id: str) -> Session | None:
        """Retrieve a session by ID."""
        return self._sessions.get(session_id)

    async def save(self, session: Session) -> None:
        """Save session to storage."""
        self._sessions[session.session_id] = session
```

Khi đã có `Session` và `SessionManager`, ta tích hợp chúng vào agent để hỗ trợ hội thoại nhiều lượt.

6.3.3 Tích hợp session vào agent

Giờ ta cần tích hợp các thành phần này vào kiến trúc agent hiện có. Việc tích hợp gồm hai phần: mở rộng `ExecutionContext` để tham chiếu tới một session, và sửa method `run()` của agent để nạp và lưu session.

Mở rộng ExecutionContext

`ExecutionContext` hiện thực ở Chương 4 đóng vai trò container cho mọi trạng thái trong một lần thực thi. Nó lưu lịch sử thực thi trong danh sách `events` và trạng thái runtime tuỳ ý trong dictionary `state`. Khi có session, các trách nhiệm này dịch chuyển: event và state giờ nằm bên trong `Session`, còn `ExecutionContext` giữ một tham chiếu tới session hiện tại.

Nguyên tắc thiết kế then chốt ở đây là tương thích ngược. Code hiện có dùng `context.events` và `context.state` ở rất nhiều chỗ. Ta dùng property để cung cấp quyền truy cập trong suốt, cho phép đoạn code này chạy mà không cần sửa đổi.

Đoạn code dưới đây là `ExecutionContext` đã mở rộng. Các trường mới là `session` và `session_manager`. Trường `session` tham chiếu tới session mà lần thực thi hiện tại thuộc về, còn `session_manager` đảm nhận việc lưu và truy xuất session. Các property `events` và `state` bên trong trả về `session.events` và `session.state`, nên code hiện có vẫn chạy như cũ.

Listing 6.11. ExecutionContext hỗ trợ session

```
@dataclass
class ExecutionContext:
    # Existing attributes...

    # Session support
    session: Session | None = None
    session_manager: BaseSessionManager | None = None

    @property
    def events(self) -> list[Event]:
        if self.session:
            return self.session.events

    @property
    def state(self) -> dict[str, Any]:
        if self.session:
            return self.session.state

    def add_event(self, event: Event):
        """Append an event to the execution history."""
        self.events.append(event)
        # Update session timestamp if available
        if self.session:
            self.session.updated_at = datetime.now()
```

Sửa phần khởi tạo agent

Tiếp theo, ta sửa constructor của class `Agent` để nhận một `SessionManager`. Nếu được truyền vào, agent lưu lại nó; nếu không, mặc định dùng `InMemorySessionManager`.

Listing 6.12. Khởi tạo agent với SessionManager

```
class ToolCallingAgent:
    """Tool-calling agent with session support."""

    def __init__(
        self,
        model: LlmClient,
        tools: list[BaseTool] | None = None,
        instructions: str = "",
        max_steps: int = 10,
        session_manager: BaseSessionManager | None = None,
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.tools = self._setup_tools(tools or [])
        # Use provided session manager or create default
        self.session_manager = session_manager or InMemorySessionManager()
```

Sửa method run()

Những thay đổi quan trọng nhất nằm ở method `run()`. Hỗ trợ session cần ba bổ sung. Thứ nhất, nhận tham số `session_id` để chỉ định lần thực thi diễn ra trong session nào. Thứ hai, nạp hoặc tạo session khi bắt đầu thực thi. Thứ ba, lưu session khi thực thi hoàn tất.

Trong đoạn code dưới đây, hãy chú ý các thay đổi chính. Khi có `session_id`, `get_or_create()` lấy ra session. Nếu session đã tồn tại, lịch sử hội thoại trước đó nằm sẵn trong `session.events`. Khi tạo `ExecutionContext`, ta inject cả `session` lẫn `session_manager`. Sau khi thực thi xong, `save()` lưu session lại.

Listing 6.13. Method run() hỗ trợ session

```
async def run(
    self,
    user_input: str,
    session_id: str | None = None,
    context: ExecutionContext | None = None,
) -> AgentResult:
    """Execute the agent with optional session support."""
    # Load or create session if session_id is provided
    session = None
    if session_id:
        session = await self.session_manager.get_or_create(session_id)

    # Create context with session support
    if context is None:
        context = ExecutionContext(
            session=session,
            session_manager=self.session_manager,
        )

    # Existing run() logic...

    # Save session if available
    if session:
        await self.session_manager.save(session)

    return AgentResult(output=context.final_result, context=context)
```

6.3.4 Ví dụ cơ bản: hội thoại nhiều lượt

Sau khi đã tích hợp `Session` và `SessionManager` vào agent, ta kiểm tra xem nó có hoạt động trong thực tế không. Ta tạo một agent với `InMemorySessionManager`, vốn lưu session trong một dictionary trên bộ nhớ. Hiện thực này phù hợp cho kiểm thử và các ứng dụng chạy một tiến trình.

Listing 6.14. Tạo agent hỗ trợ session

```
agent = Agent(
    model=model,
    instructions="You are a helpful assistant.",
    session_manager=InMemorySessionManager()
)
```

Không có session_id

Chuyện gì xảy ra khi ta không chỉ định `session_id` hoặc dùng giá trị khác nhau cho mỗi lần gọi? Mỗi lần gọi `run()` được coi là một cuộc hội thoại độc lập, không có ký ức về các tương tác trước.

```
result1 = await agent.run("Hi! My name is Alice and I'm a software engineer.")
print(f"Response 1: {result1.output}")

result2 = await agent.run("What's my name and what do I do?")
print(f"Response 2: {result2.output}")
```

Output sẽ tương tự như sau:

```
Response 1: Hello Alice! Nice to meet you. It's great to hear that you're a
↪  software engineer. How can I help you today?
Response 2: I don't have any previous context about you. Could you please tell me
↪  your name and what you do?
```

Ở lần gọi thứ hai, agent không còn nhớ gì về cuộc hội thoại đầu tiên.

Dùng cùng một session_id

Giờ ta dùng cùng một `session_id` cho cả hai lần gọi. Hai lần gọi `run()` dùng chung một Session, nhờ đó context hội thoại được giữ lại.

```
result1 = await agent.run(
    "Hi! My name is Alice and I'm a software engineer.",
    session_id="alice_session"
)
result2 = await agent.run(
    "What's my name and what do I do?",
    session_id="alice_session"
)
```

Output sẽ tương tự như sau:

```
Response 1: Hello Alice! Nice to meet you. It's great to hear that you're a
↪  software engineer. How can I help you today?
Response 2: Your name is Alice, and you're a software engineer.
```

Lần này, agent nhớ đúng thông tin từ lần tương tác trước.

Hiểu luồng xử lý bên trong

Hãy xem điều gì diễn ra bên trong để hiểu sự khác nhau giữa hai kịch bản này.

Khi dùng cùng một session_id:

1. Lần `run()` thứ nhất: `get_or_create("alice_session")` tạo một Session mới.
2. Cuộc hội thoại được lưu trong `session.events`, và `save()` ghi nó xuống kho lưu trữ.
3. Lần `run()` thứ hai: `get_or_create("alice_session")` trả về Session đã có.
4. `ExecutionContext` tham chiếu tới Session này, nên `context.events` chứa cả cuộc hội thoại trước.
5. LLM sinh phản hồi với đầy đủ quyền truy cập vào context trước đó.

Khi không chỉ định session_id hoặc session_id khác nhau:

1. Mỗi lần gọi `run()` tạo một Session mới.
2. Danh sách `events` của Session mới rỗng, nên LLM phản hồi mà không có context trước đó.

Trong thực tế, bạn sẽ dùng session khi xây dựng các ứng dụng hội thoại, nơi người dùng tương tác qua nhiều tin nhắn. Không có session, mỗi lần gọi `run()` hoàn toàn độc lập, điều này phù hợp với các tác vụ chạy một lần như các bài trong benchmark GAIA.

6.3.5 Cấu trúc dữ liệu cho tool confirmation

Giờ ta hiện thực một workflow human-in-the-loop bất đồng bộ (asynchronous). Ví dụ này cho thấy Session có giá trị vượt xa việc lưu lịch sử chat đơn thuần: nó cho phép tạm dừng và tiếp tục quá trình thực thi tại bất kỳ điểm nào, điều thiết yếu với các workflow cần con người phê duyệt.

Để hiện thực workflow này, trước hết ta cần định nghĩa các cấu trúc dữ liệu: các kiểu biểu diễn những tool call nào đang chờ phê duyệt và người dùng đã đưa ra quyết định gì.

PendingToolCall `PendingToolCall` biểu diễn một tool call đang chờ được thực thi. Nó giống `ToolCall` mà ta định nghĩa ở Chương 4, nhưng có thêm thông tin cần cho yêu cầu phê duyệt. Trường `tool_call` giữ nguyên đối tượng `ToolCall` gốc, còn trường `confirmation_message` chứa thông điệp hiển thị cho người dùng, ví dụ: "This action will delete database.db. Do you want to continue?" (Hành động này sẽ xoá `database.db`. Bạn có muốn tiếp tục?).

Listing 6.15. Lớp `PendingToolCall`

```
class PendingToolCall(BaseModel):
    """A tool call awaiting user confirmation."""
    tool_call: ToolCall
    confirmation_message: str
```

ToolConfirmation `ToolConfirmation` biểu diễn quyết định phê duyệt của người dùng. Trường `tool_call_id` xác định phản hồi này ứng với tool call nào, trường `approved` cho biết có được duyệt hay không. `modified_arguments` là trường tuỳ chọn, chứa các giá trị đã sửa nếu người dùng thay đổi tham số. Chẳng hạn người dùng có thể yêu cầu: "Tôi đồng ý xoá file, nhưng đổi file đích thành `temp.txt`."

Listing 6.16. Lớp `ToolConfirmation`

```
class ToolConfirmation(BaseModel):
    """User's decision on a pending tool call."""
    tool_call_id: str
    approved: bool
    modified_arguments: dict | None = None
```

Mở rộng AgentResult `AgentResult` mà ta hiện thực ở Chương 4 là một container chứa kết quả thực thi của agent: trường `output` lưu kết quả cuối cùng, trường `context` lưu `ExecutionContext`. Để hỗ trợ human-in-the-loop, ta cần phân biệt quá trình thực thi đã hoàn tất hay đang tạm dừng để chờ phê duyệt.

Ta thêm trường `status` để biểu diễn điều này: `"complete"` nghĩa là hoàn tất bình thường, `"pending"` nghĩa là đang chờ phê duyệt, `"error"` nghĩa là đã xảy ra lỗi. Khi trạng thái là `"pending"`, trường `pending_tool_calls` chứa danh sách các tool call đang chờ duyệt.

Listing 6.17. `AgentResult` hỗ trợ trạng thái pending

```
from typing import Literal

class AgentResult(BaseModel):
    """Result of agent execution."""
    output: str | BaseModel | None = None
    context: ExecutionContext
    status: Literal["complete", "pending", "error"] = "complete"
    pending_tool_calls: list[PendingToolCall] = Field(default_factory=list)
```

Tuần tự hoá state bằng Pydantic Trường `state` của Session có kiểu `dict[str, Any]`, nên có thể chứa dữ liệu tuỳ ý. Tuy nhiên, dictionary của Python không thể lưu trực tiếp các đối tượng Pydantic model; vấn đề sẽ nảy sinh khi lưu session vào cơ sở dữ liệu hoặc tuần tự hoá (serialize) nó sang JSON.

Hai method `model_dump()` và `model_validate()` của Pydantic giải quyết vấn đề này. `model_dump()` chuyển một đối tượng Pydantic thành dictionary, còn `model_validate()` khôi phục dictionary trở lại thành đối tượng Pydantic. Dùng pattern này, ta có thể lưu và khôi phục an toàn các đối tượng phức tạp trong state của Session.

Đoạn code dưới đây minh hoạ pattern lưu và khôi phục các pending tool call. Khi method `act()` phát hiện một tool cần phê duyệt, nó tuần tự hoá bằng `model_dump()` rồi lưu vào state. Ở lần gọi `run()` kế tiếp, nó khôi phục bằng `model_validate()` và tiếp tục thực thi.

```
# Saving: when pending occurs in act()
pending_calls = [PendingToolCall(tool_call=tc, confirmation_message=msg)]
context.state["pending_tool_calls"] = [p.model_dump() for p in pending_calls]

# Restoring: when resuming run()
raw_pending = context.state.get("pending_tool_calls", [])
pending_calls = [PendingToolCall.model_validate(d) for d in raw_pending]
```

Pattern tuần tự hoá này cho phép lưu đủ loại trạng thái thực thi mà không phải sửa lớp Session. Ngoài kịch bản human-in-the-loop, ta có thể lưu theo cùng cách tiến độ của các tác vụ chạy lâu, dữ liệu checkpoint, các kết quả trung gian mà agent thu thập được, v.v.

Đến đây các cấu trúc dữ liệu đã sẵn sàng. Tiếp theo, ta bổ sung tính năng "yêu cầu phê duyệt" cho chính các tool và hiện thực logic tạm dừng/tiếp tục của agent.

6.3.6 Mở rộng tool để hỗ trợ phê duyệt

Làm sao xác định một tool có cần phê duyệt hay không? Cách trực quan nhất là đưa thông tin này vào chính tool. Bằng cách mở rộng lớp `BaseTool` đã hiện thực ở Chương 4 với các thuộc tính liên quan đến phê duyệt, agent có thể kiểm tra các thuộc tính này trước khi thực thi tool và tạm dừng khi cần.

Mở rộng BaseTool Ta thêm hai thuộc tính vào lớp `BaseTool`: `requires_confirmation`, một giá trị boolean cho biết tool có cần người dùng duyệt trước khi chạy hay không; và `confirmation_message_template`, mẫu (template) cho thông điệp xin phê duyệt hiển thị cho người dùng. Template dùng các placeholder `{name}` và `{arguments}`, được điền động bằng thông tin thực của tool call.

Listing 6.18. `BaseTool` hỗ trợ phê duyệt

```
class BaseTool(ABC):
    # Existing attributes: name, description, parameters, execute()...

    # Confirmation support
    requires_confirmation: bool = False
    confirmation_message_template: str = (
        "The agent wants to execute '{name}' with arguments: {arguments}. "
        "Do you approve?"
    )

    def get_confirmation_message(self, arguments: dict[str, Any]) -> str:
        """Generate a confirmation message for this tool call."""
        return self.confirmation_message_template.format(
            name=self.name,
            arguments=arguments
        )
```

Method `get_confirmation_message()` điền các tham số thực vào template để tạo thông điệp cuối cùng. Khi agent tạm dừng việc thực thi tool, nó đưa thông điệp này vào `PendingToolCall` để trình bày cho người dùng.

Định nghĩa các tool cần phê duyệt Ta cũng sửa decorator `@tool` đã hiện thực ở Chương 4 để nhận thêm các tham số `requires_confirmation` và `confirmation_message`. Code sửa đổi đầy đủ có trong GitHub repository. Dưới đây là ví dụ định nghĩa các tool cần phê duyệt bằng decorator đã cập nhật.

`delete_file` được cấu hình với `requires_confirmation=True`, nên cần người dùng duyệt trước khi chạy. Ngược lại, `get_weather` là thao tác chỉ đọc nên được chạy ngay mà không cần duyệt. Dùng định dạng `{arguments[key]}` trong thông điệp tuỳ biến cho phép đưa giá trị cụ thể của tham số vào thông điệp, giúp người dùng hiểu chính xác hành động nào sẽ được thực hiện trước khi quyết định có duyệt hay không.

Listing 6.19. Các tool có yêu cầu phê duyệt

```
@tool(
    name="delete_file",
    description="Delete a file from the filesystem",
    requires_confirmation=True,
    confirmation_message="[!] The agent wants to delete '{arguments[filename]}'.
    ↪  "
                         "This action cannot be undone. Do you approve?"
)
def delete_file(filename: str) -> str:
    """Delete the specified file."""
    import os
    os.remove(filename)
    return f"Successfully deleted {filename}"


@tool(
    name="get_weather",
    description="Get current weather for a location"
)
def get_weather(location: str) -> str:
    """Get weather information. No confirmation needed."""
    return f"Weather in {location}: Sunny, 22°C"
```

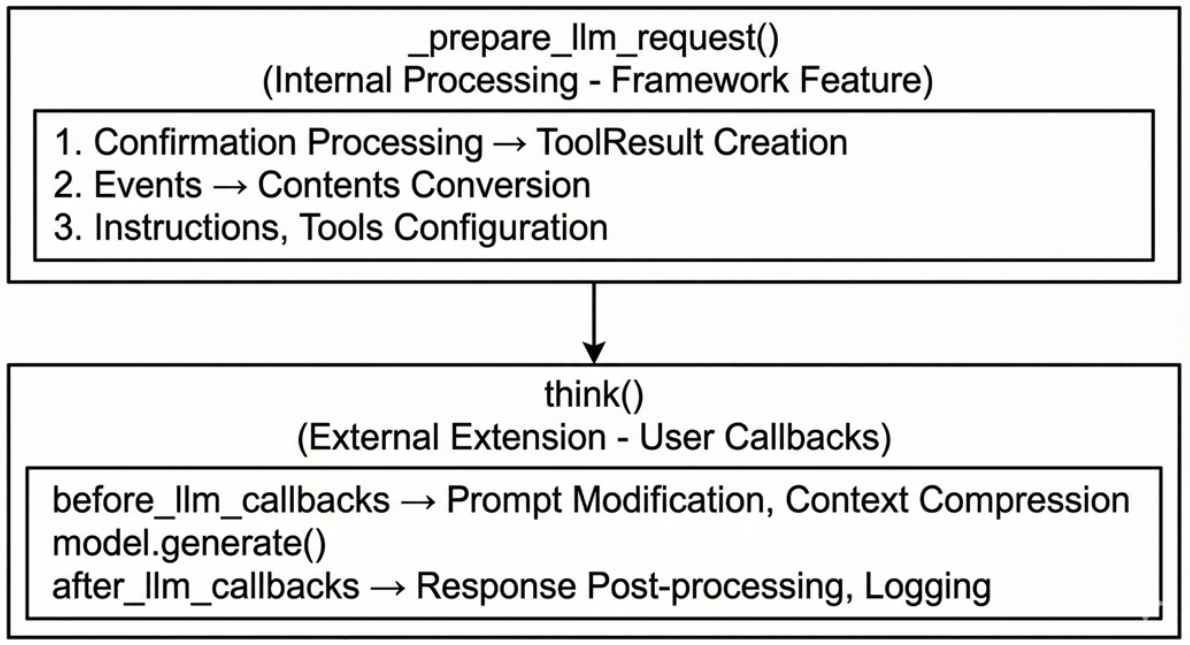
6.3.7 Hiện thực tạm dừng và tiếp tục trong agent

Sau khi đã thêm thuộc tính yêu cầu phê duyệt cho tool, đến lúc hiện thực logic để agent kiểm tra thuộc tính này rồi tạm dừng hoặc tiếp tục thực thi. Ta sẽ sửa các method `act()` và `run()` để hoàn thiện workflow human-in-the-loop.

Tính năng nội tại và phần mở rộng bên ngoài Trước khi đi vào hiện thực, cần bàn một quyết định thiết kế quan trọng. Ta đã thấy cách tuỳ biến hành vi agent bằng callback: sửa prompt bằng `before_llm_callback` và hậu xử lý kết quả tool bằng `after_tool_callback`. Các callback này là những điểm mở rộng bên ngoài (external extension point) do người dùng cài vào.

Tool Confirmation thì khác. Đây không phải tính năng người dùng tuỳ chọn thêm vào, mà là tính năng cốt lõi do chính framework cung cấp. Logic dừng thực thi khi gặp tool cần phê duyệt và tiếp tục khi nhận được phê duyệt là một phần hành vi nội tại của agent.

Vì vậy, việc xử lý Tool Confirmation được hiện thực bên trong method `_prepare_llm_request()` thay vì dưới dạng callback. Method này chịu trách nhiệm cho logic nội bộ dựng request gửi tới LLM; nó chuyển kết quả phê duyệt thành các `ToolResult` để chúng tự nhiên nằm trong dòng hội thoại, như minh hoạ ở Hình 6.6.



Hình 6.6: Tính năng nội tại và phần mở rộng bên ngoài trong kiến trúc agent. *(PDF gốc tr. 265)*

Trong cấu trúc này, `_prepare_llm_request()` lo phần logic thiết yếu của framework, còn callback lo phần tuỳ biến của người dùng. Khi hai tầng được tách bạch rõ ràng, mỗi tầng có thể tập trung vào trách nhiệm của riêng nó.

Nguyên tắc thiết kế Trước khi hiện thực, ta đặt ra hai nguyên tắc thiết kế cốt lõi.

- **Chặn trước khi thực thi (Pre-execution Blocking)**: Tool cần phê duyệt phải được người dùng duyệt *trước* khi chạy. Chạy tool trước, cho xem kết quả rồi mới hỏi "Tôi đã làm việc này, có ổn không?" là vô nghĩa. Những hành động như xoá file hay xử lý thanh toán không thể hoàn tác.
- **Phê duyệt theo whitelist (Whitelist Approval)**: Người dùng có thể chỉ duyệt một phần trong các pending tool call. Khi agent yêu cầu ba tool call, người dùng phải có thể duyệt hai và từ chối một. Chỉ những tool nằm trong danh sách được duyệt mới được chạy; những tool không có trong danh sách tự động bị từ chối.

Sửa method act() Method `act()` ta hiện thực ở Chương 4 chạy ngay mọi tool call mà LLM trả về. Giờ ta sửa nó để kiểm tra thuộc tính `requires_confirmation` và lưu các tool cần phê duyệt vào `context.state` thay vì chạy chúng.

Listing 6.20. Method `act()` có kiểm tra phê duyệt

```
async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall]
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    pending_calls = []

    for tool_call in tool_calls:
        tool = tools_dict.get(tool_call.name)

        # Check if confirmation is required
```

```
        if tool.requires_confirmation:
            pending = PendingToolCall(
                tool_call=tool_call,
                confirmation_message=tool.get_confirmation_message(
                    tool_call.arguments
                )
            )
            pending_calls.append(pending)
            continue  # Do not execute, do not create ToolResult

        # Execute tools that don't require confirmation
        # ... existing tool execution logic ...

    # Store pending calls in session state
    if pending_calls:
        context.state["pending_tool_calls"] = [
            p.model_dump() for p in pending_calls
        ]

    return results
```

Hãy xem các thay đổi chính. Khi duyệt qua các tool call, ta kiểm tra thuộc tính `requires_confirmation` của từng tool. Với tool cần phê duyệt, ta tạo đối tượng `PendingToolCall`, thêm vào danh sách `pending_calls` rồi chuyển sang tool kế tiếp mà không tạo `ToolResult`. Chỉ những tool không cần phê duyệt mới được chạy ngay và trả về kết quả.

Kiểu trả về của `act()` vẫn là `List[ToolResult]` như cũ. Vì thông tin pending tool call được lưu trong `context.state`, nơi gọi có thể kiểm tra `context.state` để biết có đang pending hay không. Cách này thêm tính năng mới mà vẫn giữ tương thích với interface hiện có.

Sửa method run() Method `run()` phải xử lý hai kịch bản. Thứ nhất, khi gặp tool cần phê duyệt trong một lần thực thi mới, nó tạm dừng và trả về với trạng thái pending. Thứ hai, khi tiếp tục một lần thực thi đã tạm dừng trước đó, nó ghi các quyết định phê duyệt của người dùng vào `context.state` rồi tiếp tục chạy.

Listing 6.21. Method `run()` hỗ trợ tạm dừng và tiếp tục

```
async def run(
    self,
    user_input: str | None = None,
    session_id: str | None = None,
    tool_confirmations: List[ToolConfirmation] | None = None,
    context: ExecutionContext | None = None,
) -> AgentResult:
    # Register confirmations for processing in _prepare_llm_request
    if tool_confirmations:
        context.state["tool_confirmations"] = [
            c.model_dump() for c in tool_confirmations
        ]

    # Main agent loop
    while not context.final_result and context.current_step < self.max_steps:
        await self.step(context)

        # Check for pending confirmations after each step
        if context.state.get("pending_tool_calls"):
            pending_calls = [
                PendingToolCall.model_validate(p)
```

```
                for p in context.state["pending_tool_calls"]
            ]
            return AgentResult(
                status="pending_confirmation",
                context=context,
                pending_tool_calls=pending_calls,
            )
```

Các thay đổi chính:

- **Tham số `tool_confirmations`**: nhận danh sách các quyết định phê duyệt của người dùng. Khi tiếp tục từ trạng thái pending trước đó, danh sách này được ghi vào `context.state`. Việc xử lý thực sự diễn ra trong `_prepare_llm_request()` ở lần gọi `step()` tiếp theo.
- **Kiểm tra pending**: sau mỗi lần chạy `step()` trong vòng lặp agent, kiểm tra xem `context.state` có pending tool call không. Nếu có, trả về ngay với trạng thái `pending_confirmation` để tạm dừng thực thi.
- **Tham số `user_input` tuỳ chọn**: khi tiếp tục một lần thực thi đã tạm dừng, chỉ cần `tool_confirmations`; `user_input` giờ là tuỳ chọn.

Sửa method _prepare_llm_request() Method `_prepare_llm_request()` xử lý các quyết định phê duyệt đã được ghi trong `context.state`. Nó chạy các tool được duyệt và ghi kết quả thành event, đồng thời ghi thông điệp từ chối cho các tool bị từ chối. Method này giờ là async vì cần `await` khi chạy các tool được duyệt.

Listing 6.22. `_prepare_llm_request()` sửa đổi để xử lý phê duyệt

```
async def _prepare_llm_request(
    self,
    context: ExecutionContext
) -> LlmRequest:
    """Package context into an LlmRequest, processing confirmations first."""
    # Process pending confirmations if both are present
    if ("pending_tool_calls" in context.state and
        "tool_confirmations" in context.state):
        confirmation_results = await self._process_confirmations(context)

        # Add results as an event so they appear in contents
        if confirmation_results:
            confirmation_event = Event(
                execution_id=context.execution_id,
                author=self.name,
                content=confirmation_results,
            )
            context.add_event(confirmation_event)

        # Clear processed state
        del context.state["pending_tool_calls"]
        del context.state["tool_confirmations"]

    # Flatten events into content items
    flat_contents = []
    for event in context.events:
        flat_contents.extend(event.content)

    # ... existing LlmRequest construction logic ...
```

Thứ tự xử lý rất quan trọng. Trước hết xử lý các quyết định phê duyệt để sinh `ToolResult` và ghi chúng thành event; sau đó mới dựng `contents` gồm cả các event đó. Nhờ vậy LLM thấy được kết quả duyệt/từ chối ở bước tiếp theo.

Thứ tự này được bảo đảm vì `_prepare_llm_request()` là method nội bộ. Nếu ta hiện thực logic này trong `before_llm_callback`, sẽ phát sinh vấn đề về thời điểm: các event mới thêm sẽ không có mặt trong `contents`, vì callback nhận một `LlmRequest` đã được dựng sẵn. Đây là thêm một lý do để các tính năng nội tại của framework nên được xử lý trong method nội bộ thay vì callback.

Hiện thực logic xử lý phê duyệt Method `_process_confirmations()` thực hiện việc xử lý phê duyệt thực sự. Với mỗi pending tool call, nó kiểm tra trạng thái phê duyệt, chạy các tool được duyệt, và ghi thông điệp từ chối cho các trường hợp không được duyệt (bị từ chối rõ ràng hoặc không có trong danh sách phê duyệt).

Listing 6.23. Hiện thực `_process_confirmations()`

```
async def _process_confirmations(
    self,
    context: ExecutionContext
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    results = []

    # Restore pending tool calls from state
    pending_map = {
        p["tool_call"]["tool_call_id"]: PendingToolCall.model_validate(p)
        for p in context.state["pending_tool_calls"]
    }

    # Build confirmation lookup by tool_call_id
    confirmation_map = {
        c["tool_call_id"]: ToolConfirmation.model_validate(c)
        for c in context.state["tool_confirmations"]
    }

    # Process ALL pending tool calls
    for tool_call_id, pending in pending_map.items():
        tool = tools_dict.get(pending.tool_call.name)
        confirmation = confirmation_map.get(tool_call_id)

        if confirmation and confirmation.approved:
            # Merge original arguments with modifications
            arguments = {
                **pending.tool_call.arguments,
                **(confirmation.modified_arguments or {})
            }

            # Execute the approved tool
            try:
                output = await tool(context, **arguments)
                results.append(ToolResult(
                    tool_call_id=tool_call_id,
                    name=pending.tool_call.name,
                    status="success",
                    content=[output],
                ))
            except Exception as e:
                results.append(ToolResult(
                    tool_call_id=tool_call_id,
```

```
                    name=pending.tool_call.name,
                    status="error",
                    content=[str(e)],
                ))
        else:
            # Rejected: either explicitly or not in confirmation list
            if confirmation:
                reason = confirmation.reason or "Tool execution was rejected by
                ↪  user."
            else:
                reason = "Tool execution was not approved."

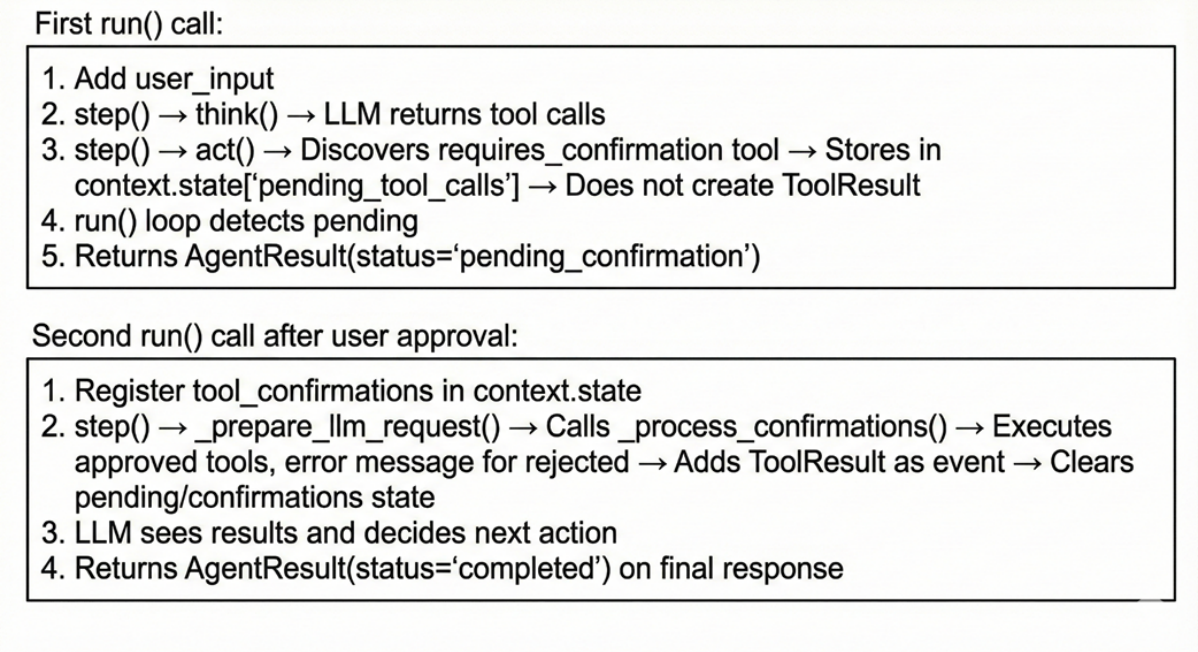
            results.append(ToolResult(
                tool_call_id=tool_call_id,
                name=pending.tool_call.name,
                status="error",
                content=[reason],
            ))

    return results
```

Hiện thực này có ba điểm then chốt:

- **Phê duyệt theo whitelist**: ta duyệt qua *tất cả* pending tool call và chỉ chạy những cái có trong danh sách phê duyệt với `approved=True`. Cái nào không có trong danh sách hoặc có `approved=False` đều bị coi là bị từ chối.
- **Gộp tham số**: khi có `modified_arguments`, ta gộp nó với tham số gốc bằng `{**original, **modified}`. Tham số sửa đổi ghi đè lên tham số gốc, cho phép người dùng sửa chọn lọc từng tham số, chẳng hạn chỉ đổi đường dẫn file.
- **Phản hồi rõ ràng**: tool bị từ chối cũng được ghi thành `ToolResult`. Vì LLM chờ đợi kết quả cho mỗi tool call, ta phải báo rõ cho nó khi một tool bị từ chối để nó phản ứng phù hợp. Ví dụ, nếu việc xoá file bị từ chối, LLM có thể đề xuất cách khác hoặc hỏi lại người dùng cho rõ.

Tóm tắt toàn bộ luồng Hình 6.7 tóm tắt toàn bộ luồng của cơ chế tạm dừng và tiếp tục mà ta vừa hiện thực.



Hình 6.7: Luồng tổng thể của cơ chế tạm dừng và tiếp tục. *(PDF gốc tr. 275)*

6.3.8 Ví dụ hoàn chỉnh: workflow human-in-the-loop

Giờ ta ghép mọi thành phần đã hiện thực để minh hoạ một workflow human-in-the-loop hoàn chỉnh. Kịch bản đơn giản: agent định xoá một file, tạm dừng để chờ người dùng duyệt, rồi tiếp tục thực thi dựa trên quyết định của người dùng.

Thiết lập ví dụ Trước hết ta định nghĩa một tool cần phê duyệt và tạo agent có hỗ trợ session. Tool `delete_file` được cấu hình với `requires_confirmation=True`, nên agent sẽ tạm dừng và chờ người dùng duyệt trước khi chạy nó.

Listing 6.24. Thiết lập tool và agent cho human-in-the-loop

```
from agent import Agent, InMemorySessionManager, ToolConfirmation
from tools import tool


@tool(
    name="delete_file",
    description="Delete a file from the filesystem",
    requires_confirmation=True,
    confirmation_message="[!] Delete '{arguments[filename]}'? This cannot be
    ↪  undone."
)
def delete_file(filename: str) -> str:
    import os
    os.remove(filename)
    return f"Deleted {filename}"


@tool(name="list_files", description="List files in a directory")
def list_files(directory: str = ".") -> str:
    import os
    return "\n".join(os.listdir(directory))
```

```
agent = Agent(
    model=model,
    tools=[delete_file, list_files],
    instructions="You are a helpful file management assistant.",
    session_manager=InMemorySessionManager()
)
```

Bước 1: yêu cầu ban đầu Người dùng yêu cầu agent xoá một file tạm. Khi agent định gọi tool `delete_file`, quá trình thực thi tạm dừng vì tool này cần phê duyệt.

Listing 6.25. Yêu cầu ban đầu kích hoạt trạng thái chờ phê duyệt

```
result = await agent.run(
    "Please delete the file named temp.txt",
    session_id="file_session"
)

print(f"Status: {result.status}")
# Status: pending_confirmation

print(f"Pending tools: {len(result.pending_tool_calls)}")
# Pending tools: 1

for pending in result.pending_tool_calls:
    print(f"Tool: {pending.tool_call.name}")
    print(f"Arguments: {pending.tool_call.arguments}")
    print(f"Message: {pending.confirmation_message}")
# Tool: delete_file
# Arguments: {'filename': 'temp.txt'}
# Message: [!] Delete 'temp.txt'? This cannot be undone.
```

Agent trả về ngay với `status="pending_confirmation"`. Danh sách `pending_tool_calls` chứa chi tiết về tool đang chờ duyệt, chẳng hạn thông điệp xin phê duyệt hiển thị cho người dùng.

Bước 2: người dùng phê duyệt Trong ứng dụng thực tế, người dùng có thể bấm nút trên trang web hoặc phản hồi một thông báo trên điện thoại. Ở đây ta giả lập việc phê duyệt bằng cách tạo một đối tượng `ToolConfirmation` rồi tiếp tục thực thi.

Listing 6.26. Tiếp tục thực thi sau khi người dùng phê duyệt

```
# User approves the deletion
confirmation = ToolConfirmation(
    tool_call_id=result.pending_tool_calls[0].tool_call.tool_call_id,
    approved=True
)

result = await agent.run(
    session_id="file_session",
    tool_confirmations=[confirmation]
)

print(f"Status: {result.status}")
# Status: complete

print(f"Output: {result.output}")
# Output: I've deleted the file temp.txt as requested.
```

Khi gọi lại `run()` với cùng `session_id` kèm quyết định phê duyệt, agent tiếp tục thực thi: nó chạy tool đã được duyệt, nhận kết quả và sinh câu trả lời cuối cùng.

6.4 Long-term memory: tích luỹ tri thức xuyên session

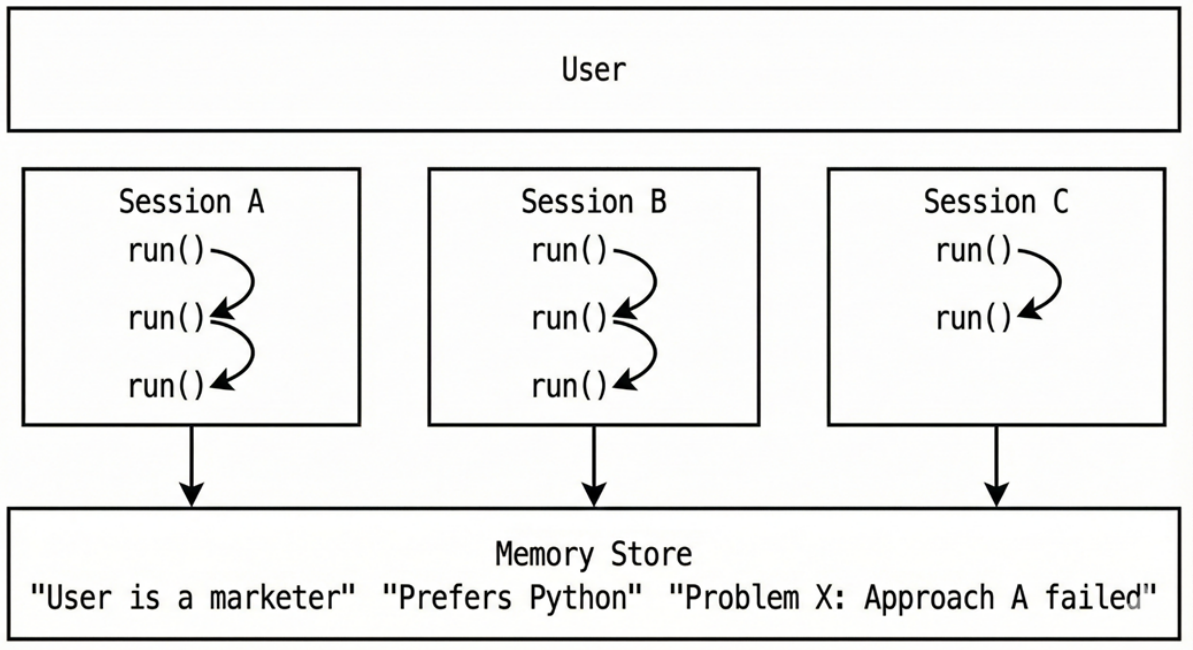
Session và SessionManager cho phép ta tiếp tục hội thoại trong cùng một `session_id`. Tuy nhiên, khi một session kết thúc hẳn, thông tin của nó cũng mất theo. Trong mục này, ta hiện thực long-term memory để tích luỹ và tận dụng tri thức vượt qua ranh giới session.

Long-term memory là ví dụ tiêu biểu cho việc kết hợp hai chiến lược Write (lưu trữ) và Retrieval (truy xuất) của Context Engineering. Sau khi một session kết thúc, thông tin quan trọng được lưu vào kho lưu trữ bên ngoài (Write); ở các session mới, thông tin này được truy xuất và thêm vào context (Retrieval).

6.4.1 Cấu trúc của long-term memory

SessionManager đã hiện thực và MemoryManager giới thiệu ở đây giải quyết hai bài toán khác nhau.

- **SessionManager** duy trì tính liên tục của hội thoại *trong* một session. Khi gọi `run()` nhiều lần với cùng `session_id`, nội dung hội thoại trước được giữ lại. Nhưng nếu bắt đầu hội thoại mới với `session_id` khác, ta không truy cập được nội dung session trước.
- **MemoryManager** lưu thông tin quan trọng ngay cả khi session đã kết thúc, cho phép truy xuất và dùng lại ở các session sau. Ví dụ: "người dùng từng nói họ làm marketing" hay "lần trước giải bài toán này bằng cách nào".



Hình 6.8: Vai trò của MemoryManager. *(PDF gốc tr. 279)*

SessionManager lo tính liên tục theo chiều ngang trong một session (mạch hội thoại), còn MemoryManager lo tính liên tục theo chiều dọc xuyên các session (tích luỹ tri thức).

Hai trụ cột: lưu trữ và truy xuất Hệ thống long-term memory có hai thao tác cốt lõi: lưu trữ (Write) và truy xuất (retrieval). Cách đơn giản nhất là chuyển thẳng văn bản thành vector embedding để lưu, rồi

khi truy xuất thì tìm và trả về các vector giống với văn bản truy vấn. Tuy nhiên, lưu nguyên cả cuộc hội thoại sẽ đưa vào nhiều nhiễu, và thông tin quan trọng có thể bị chìm lấp.

Các thư viện memory gần đây như mem0 và graphiti cải thiện điều này bằng cách dùng LLM để chỉ trích xuất những thông tin đáng giữ lại. Khi lưu thông tin mới, chúng thực hiện một trong bốn thao tác để xử lý xung đột với các memory đã có:

- **ADD**: lưu thông tin mới chưa tồn tại.
- **UPDATE**: sửa các memory đã có.
- **DELETE**: xoá các memory mâu thuẫn.
- **SKIP**: bỏ qua thông tin đã được lưu.

Trong chương này ta chỉ hiện thực ADD và SKIP. Thông tin hồ sơ người dùng (tên, nghề nghiệp, sở thích, …) có thể thay đổi theo thời gian nên cần UPDATE và DELETE, nhưng hồ sơ giải bài thì khác. Ngay cả với cùng một bài toán, các cách tiếp cận hoặc kết quả khác nhau vẫn đáng lưu thành các bản ghi riêng (ADD), còn những lần thử giống hệt nhau thì có thể bỏ qua (SKIP). Không cần sửa hay xoá các bản ghi thất bại trong quá khứ; giữ chúng lại giúp tránh lặp lại cùng sai lầm. Tính bất biến (immutability) này của memory giúp hiện thực của ta đơn giản hơn.

6.4.2 Trích xuất thông tin: structured output

Nếu thông tin cần trích xuất vào memory chỉ là các sự kiện đơn giản thì dạng văn bản là đủ. Những thông tin gói gọn trong một câu, như "người dùng làm marketing" hay "thích Python", có thể lưu và tìm kiếm nguyên dạng.

Đôi khi thông tin cần lưu phức tạp hơn. Xét các bản ghi giải bài GAIA: thay vì chỉ ghi "đã giải bài này", ta cần lưu bài toán là gì, đã dùng cách tiếp cận nào và kết quả ra sao. Thông tin này phải có cấu trúc để khi gặp bài toán tương tự, agent có thể tham khảo cách tiếp cận đã thành công hoặc tránh những cách đã thất bại.

Mở rộng LlmClient Ta thêm hai method vào `LlmClient` để phục vụ trích xuất thông tin và tạo vector embedding. Method `generate()` từ Chương 4 nhận `LlmRequest` làm đầu vào và chuyên cho việc thực thi agent; với việc sinh văn bản cơ bản, ta cần những interface đơn giản hơn.

Method `ask()` nhận một prompt và trả về văn bản hoặc JSON. Khi truyền tham số `response_format`, nó trả về output theo định dạng được yêu cầu.

```
# Simple text generation
answer = await llm_client.ask("What is 2 + 2?")

# JSON structured output
data = await llm_client.ask(prompt, response_format=GaiaOutput)
```

Định nghĩa schema cho memory Ta dùng Pydantic để định nghĩa schema cho bản ghi giải bài GAIA. Schema nắm bắt các thành phần thiết yếu của một lần thử giải: `task_summary` mô tả bài toán yêu cầu gì, `approach` ghi lại phương pháp và tool đã dùng, `final_answer` lưu đáp án agent đã nộp, `is_correct` cho biết đáp án đúng hay sai, và `error_analysis` ghi lại lý do thất bại khi lần thử bị sai.

Listing 6.27. Định nghĩa schema `TaskMemory`

```
class TaskMemory(BaseModel):
    """Structured memory for GAIA problem-solving records"""
    task_summary: str = Field(description="What the problem asked")
    approach: str = Field(description="Methods and tools used to solve it")
    final_answer: str = Field(description="The agent's submitted answer")
```

```
    is_correct: bool = Field(description="Whether the answer was correct")
    error_analysis: str | None = Field(
        default=None,
        description="Why the attempt failed, if it did"
    )

    def to_embedding_text(self) -> str:
        """Generate text for vector search"""
        return f"Task: {self.task_summary}"
```

Method `to_embedding_text()` sinh văn bản dùng cho vector search. Ở đây nó chỉ gồm phần tóm tắt bài toán, để khi truy xuất có thể tìm được các bài toán tương tự. Cách tiếp cận và đáp án được lưu dưới dạng metadata và dùng sau khi truy xuất.

Thiết kế prompt trích xuất Ta thiết kế một prompt để trích xuất memory có cấu trúc từ lịch sử thực thi của agent. Prompt yêu cầu LLM phân tích toàn bộ execution trace và xác định các thành phần chính: bài toán đang giải là gì, đã dùng cách tiếp cận nào, và có thành công hay không.

```
TASK_MEMORY_EXTRACTION_PROMPT = """You are analyzing an agent's problem-solving
↪  record to extract a learning record.

Analyze the following execution history:
<execution_history>
{execution_history}
</execution_history>
"""
```

Lịch sử thực thi chứa quá trình suy luận của agent, các tool call, kết quả trung gian, và phản hồi do người dùng thêm vào cho biết đáp án đúng hay sai. LLM tổng hợp tất cả những thứ này thành một memory có cấu trúc.

Prompt kiểm tra trùng lặp Trước khi lưu một memory mới, ta kiểm tra xem đã có thông tin tương tự hay chưa. Việc này ngăn kho memory bị lấp đầy bởi các bản ghi thừa về cùng một bài toán.

Listing 6.28. Schema `DuplicateCheckResult` và prompt

```
class DuplicateCheckResult(BaseModel):
    """Result of duplicate check"""
    decision: str = Field(description="ADD (new information) or SKIP
    ↪  (duplicate)")
    reason: str = Field(description="Explanation for the decision")


DUPLICATE_CHECK_PROMPT = """Compare the new memory against existing memories to
↪  determine if it's a duplicate.

Existing memories:
{existing_memories}

New memory:
{new_memory}

Respond with one of:
- ADD: This is new information that should be stored
- SKIP: Similar information already exists, no need to store
```

```
Judgment criteria:
- Same problem with different approach or different result counts as new
↪  information
- Same problem with same approach and same result is a duplicate"""
```

Prompt cung cấp các memory đã có làm context và yêu cầu LLM đưa ra quyết định nhị phân. Schema `DuplicateCheckResult` nắm bắt cả quyết định lẫn lý do đằng sau nó.

6.4.3 Xây dựng vector store với ChromaDB

Ở Chương 5, ta đã học nguyên lý của vector search qua việc tự hiện thực: tính cosine similarity và trả về các kết quả đứng đầu. Giờ ta hiện thực việc lưu trữ và truy xuất bằng một vector database. Để thực hành, ta dùng ChromaDB vì nó nhẹ và dễ cấu hình.

Cài đặt ChromaDB và các khái niệm cốt lõi ChromaDB là một vector database gọn nhẹ chạy cục bộ. Chỉ cần cài package Python là dùng được ngay, không phải dựng server riêng, nên rất hợp cho làm prototype và học tập.

```
uv add chromadb
```

Khái niệm cốt lõi trong ChromaDB là Collection. Một Collection gom các tài liệu liên quan lại với nhau, tương tự một bảng trong cơ sở dữ liệu quan hệ. Mỗi Collection lưu cùng nhau văn bản tài liệu, vector embedding và metadata.

Cách dùng cơ bản Đoạn code sau minh hoạ luồng lưu trữ và truy xuất cơ bản của ChromaDB. Ta tạo một client in-memory bằng `chromadb.Client()` và một collection bằng `get_or_create_collection()`. Khi chỉ định một hàm embedding ở tham số `embedding_function`, ChromaDB tự động chuyển văn bản thành vector ở cả lúc lưu lẫn lúc truy xuất.

Listing 6.29. Cách dùng cơ bản ChromaDB

```
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

# Create client (in-memory)
client = chromadb.Client()

# Set up embedding function
embedding_fn = OpenAIEmbeddingFunction(
    model_name="text-embedding-3-small"
)

# Create collection
collection = client.get_or_create_collection(
    name="my_memories",
    embedding_function=embedding_fn
)

# Store documents - just pass text, embeddings are generated automatically
collection.add(
    ids=["mem_001", "mem_002"],
    documents=["The cat is sleeping on the sofa", "The dog is playing in the
    ↪  park"],
    metadatas=[{"category": "cat"}, {"category": "dog"}]
)
```

```
# Search - just pass query text, embeddings are generated automatically
results = collection.query(
    query_texts=["Where is the cat resting"],
    n_results=1
)

print(results["documents"])  # [['The cat is sleeping on the sofa']]
print(results["metadatas"])  # [[{'category': 'cat'}]]
```

Khi đã chỉ định `embedding_function`, văn bản tài liệu được tự động embed khi gọi `add()`, và `query_texts` cũng được tự động embed khi gọi `query()` để thực hiện tìm kiếm theo độ tương đồng. Việc sinh embedding và tính cosine similarity đều được xử lý bên trong.

Dùng metadata Tham số `metadatas` lưu thông tin bổ sung cho từng tài liệu. Ta có thể dùng metadata này trong kết quả tìm kiếm để xem các thông tin phụ như loại tài liệu, thời điểm tạo hay nguồn. Trong kho memory hiện thực ở mục tiếp theo, ta lưu những thông tin như cách tiếp cận và tính đúng/sai dưới dạng metadata.

6.4.4 Hiện thực TaskMemoryManager

Giờ ta hiện thực `TaskMemoryManager`, lớp lưu và truy xuất các bản ghi giải bài GAIA bằng ChromaDB. Lớp này đảm nhận toàn bộ pipeline: trích xuất memory có cấu trúc từ lịch sử thực thi, kiểm tra trùng lặp, và lưu vào ChromaDB.

Khởi tạo lớp Constructor của `TaskMemoryManager` nhận một `LlmClient` qua injection, dùng cho việc trích xuất memory và kiểm tra trùng lặp. ChromaDB client và collection cũng được khởi tạo ở đây.

Listing 6.30. Khởi tạo `TaskMemoryManager`

```
import uuid
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction


class TaskMemoryManager:
    """Memory manager for GAIA problem-solving learning"""

    def __init__(
        self,
        llm_client: LlmClient,
        collection_name: str = "task_memories",
    ):
        self.llm_client = llm_client

        # ChromaDB setup
        self.client = chromadb.Client()
        embedding_fn = OpenAIEmbeddingFunction(
            model_name="text-embedding-3-small"
        )
        self.collection = self.client.get_or_create_collection(
            name=collection_name,
            embedding_function=embedding_fn
        )
```

Các method trích xuất memory Ta hiện thực các method trích xuất memory có cấu trúc từ lịch sử thực thi. `_format_execution_history()` chuyển danh sách event thành định dạng văn bản phù hợp cho LLM: nó duyệt nội dung của từng event và định dạng riêng message, tool call và tool result.

Trong `_extract_memory()`, ta truyền `response_format=TaskMemory` để LLM sinh JSON khớp với schema `TaskMemory`. Vì `LlmClient` tự parse kết quả này và trả về một instance `TaskMemory`, không cần viết thêm logic parse JSON.

Listing 6.31. Các method trích xuất memory

```
class TaskMemoryManager:
    # ... previous code ...

    async def _extract_memory(self, execution_history: str) -> TaskMemory | None:
        """Extract structured memory from execution history"""
        prompt = TASK_MEMORY_EXTRACTION_PROMPT.format(
            execution_history=execution_history
        )
        try:
            return await self.llm_client.ask(
                prompt=prompt,
                response_format=TaskMemory
            )
        except Exception as e:
            print(f"Memory extraction failed: {e}")
            return None

    def _format_execution_history(self, events: list[Event]) -> str:
        """Convert event list to text"""
        lines = []
        for event in events:
            for item in event.content:
                if isinstance(item, Message):
                    lines.append(f"[{item.role}]: {item.content}")
                elif isinstance(item, ToolCall):
                    lines.append(f"[Tool Call]: {item.name}({item.arguments})")
                elif isinstance(item, ToolResult):
                    content_preview = str(item.content[0])[:500] if item.content
                    ↪  else ""
                    lines.append(f"[Tool Result]: {item.name} ->
                    ↪  {content_preview}")
        return "\n".join(lines)
```

Method kiểm tra trùng lặp Method này kiểm tra xem một memory mới có trùng với memory đã có hay không. Nếu không có kết quả tìm kiếm nào sẵn có, nó trả về `False`. Nó chuyển các memory đã có thành văn bản và đưa chúng, cùng với memory mới, vào prompt. Khi nhận phản hồi theo schema `DuplicateCheckResult`, có thể đọc trực tiếp quyết định qua `result.decision`. Nếu quyết định là `"SKIP"`, tức memory này bị trùng và không nên lưu, method trả về `True` để chặn việc lưu.

Listing 6.32. Method kiểm tra trùng lặp

```
class TaskMemoryManager:
    # ... previous code ...

    async def _is_duplicate(
        self,
        new_memory: TaskMemory,
        existing_results: dict
    ) -> bool:
```

```
        """Determine if a new memory duplicates an existing one"""
        if not existing_results["metadatas"] or not
        ↪  existing_results["metadatas"][0]:
            return False

        # Convert existing memories to text
        existing_texts = []
        for meta in existing_results["metadatas"][0]:
            existing_texts.append(
                f"task_summary: {meta.get('task_summary')}, "
                f"- approach: {meta.get('approach')}, "
                f"is_correct: {meta.get('is_correct')}"
            )

        prompt = DUPLICATE_CHECK_PROMPT.format(
            existing_memories="\n".join(existing_texts),
            new_memory=f"task_summary: {new_memory.task_summary}, "
                       f"approach: {new_memory.approach}, "
                       f"is_correct: {new_memory.is_correct}"
        )

        try:
            result = await self.llm_client.ask(
                prompt=prompt,
                response_format=DuplicateCheckResult
            )
            return result.decision == "SKIP"
        except Exception:
            return False  # Proceed with storage if parsing fails
```

Method save() Method `save()` ghép mọi thành phần lại với nhau. Nó chuyển lịch sử thực thi thành văn bản, dùng LLM trích xuất memory có cấu trúc, tìm các memory tương tự đã có để kiểm tra trùng lặp, và lưu memory vào ChromaDB nếu không trùng. Với phần documents, nó sinh văn bản tìm kiếm được bằng `to_embedding_text()`; với phần metadata, nó chuyển toàn bộ các trường của `TaskMemory` thành dictionary bằng `model_dump()` để lưu.

Listing 6.33. Method `save()` của `TaskMemoryManager`

```
class TaskMemoryManager:
    # ... previous code ...

    async def save(self, context: ExecutionContext) -> str | None:
        """
        Extract and save memory from execution context.

        Returns:
            memory_id if saved, None if ignored as duplicate
        """
        # 1. Convert execution history to text
        execution_history = self._format_execution_history(context.events)

        # 2. Extract structured memory using LLM
        memory = await self._extract_memory(execution_history)
        if memory is None:
            return None

        # 3. Duplicate check - search with the same text used for storage
        existing = self.collection.query(
```

```
            query_texts=[memory.to_embedding_text()],
            n_results=3
        )
        if await self._is_duplicate(memory, existing):
            return None

        # 4. Store in ChromaDB
        memory_id = str(uuid.uuid4())
        self.collection.add(
            ids=[memory_id],
            documents=[memory.to_embedding_text()],
            metadatas=[memory.model_dump()]
        )
        return memory_id
```

Method search() Method `search()` truy xuất các memory liên quan tới một truy vấn và trả về chúng dưới dạng danh sách đối tượng `TaskMemory`. Khi gọi `collection.query()`, ChromaDB lo việc sinh embedding, tính độ tương đồng và sắp xếp. Bằng cách chuyển metadata trong kết quả thành `TaskMemory`, ta truy cập các trường `task_summary`, `approach`, `final_answer`, `is_correct` và `error_analysis` một cách type-safe.

Listing 6.34. Method `search()` của `TaskMemoryManager`

```
class TaskMemoryManager:
    # ... previous code ...

    async def search(self, query: str, top_k: int = 5) -> list[TaskMemory]:
        """Search for memories related to the query."""
        results = self.collection.query(
            query_texts=[query],
            n_results=top_k
        )

        if not results["metadatas"] or not results["metadatas"][0]:
            return []

        return [TaskMemory(**meta) for meta in results["metadatas"][0]]
```

Tích hợp MemoryManager vào agent Ta thêm `memory_manager` vào lớp `Agent`. `memory_manager` là tuỳ chọn: với các tác vụ đơn giản không cần long-term memory, có thể để là `None`. Trong method `run()`, ta truyền `memory_manager` vào `ExecutionContext` để các tool truy cập được qua `context.memory_manager`.

Listing 6.35. Thêm `memory_manager` vào `Agent`

```
class Agent:
    def __init__(
        ...
        memory_manager: TaskMemoryManager | None = None,  # Added
    ):
        ...
        self.memory_manager = memory_manager  # Added
```

6.4.5 Truy xuất memory

Cách trực quan nhất để dùng các memory đã lưu là hiện thực việc truy xuất thành một tool mà agent gọi khi cần. Nhưng nếu có những memory luôn phải được truy xuất trước khi LLM nhìn thấy request thì

sao? Ví dụ: lấy thông tin hồ sơ người dùng ở đầu mỗi cuộc hội thoại, hoặc kiểm tra xem trước đây đã giải bài toán tương tự chưa. Ta có thể viết một callback riêng cho việc này, nhưng khi đó logic liên quan tới memory bị chia ra giữa một Tool và một callback. Cách gọn hơn là để chính Tool chèn context vào LLM request.

Truy xuất dựa trên tool Ta bắt đầu với `MemoryTool`, tool truy xuất các bản ghi giải bài trong quá khứ. Agent có thể gọi tool này để kiểm tra các bài toán tương tự. Method `execute()` tìm trong kho memory và định dạng kết quả để LLM hiểu được cách tiếp cận nào đã thành công và cách nào đã thất bại.

Listing 6.36. `MemoryTool` cho truy xuất tường minh

```
class MemoryTool(BaseTool):
    """Tool for retrieving past problem-solving records"""

    def __init__(self):
        super().__init__(
            name="recall_memory",
            description=(
                "Search for past problem-solving records. "
                "Use this to check if similar problems were solved before."
            ),
        )

    async def execute(self, context: ExecutionContext, query: str) -> str:
        """Search memories and return formatted results"""
        memories = await context.memory_manager.search(query, top_k=3)
        if not memories:
            return ""
        return self._format_memories(memories)

    def _format_memories(self, memories: list[TaskMemory]) -> str:
        """Format memories for display"""
        results = []
        for i, mem in enumerate(memories, 1):
            status = "Correct" if mem.is_correct else "Incorrect"
            text = f"""[Record {i}]
- Problem: {mem.task_summary}
- Approach: {mem.approach}
- Answer: {mem.final_answer}
- Result: {status}"""
            if not mem.is_correct and mem.error_analysis:
                text += f"\n- Error analysis: {mem.error_analysis}"
            results.append(text)
        return "\n\n".join(results)
```

Khi đã đăng ký tool này, agent có thể tự quyết định lúc nào cần tìm các kinh nghiệm liên quan.

Tự động chèn bằng process_llm_request Truy xuất dựa trên tool phù hợp khi việc tìm kiếm là tuỳ chọn. Nhưng với một số trường hợp, ta muốn memory được truy xuất tự động trước mỗi lần gọi LLM. Thay vì tạo một callback riêng, hãy override method `process_llm_request()` trong `BaseTool`. Method này chạy trước mỗi LLM request và có thể sửa trực tiếp request.

Điểm mấu chốt là có thể tái sử dụng logic của `execute()` bên trong `process_llm_request()`. Thay vì lặp lại code tìm kiếm và định dạng, `process_llm_request()` chỉ việc gọi `execute()` rồi nối kết quả vào LLM request.

Tool có được LLM gọi hay chỉ dùng để tự động chèn là tuỳ vào `tool_definition`. Nếu `tool_definition` là một dict, LLM có thể gọi tool tường minh. Nếu `tool_definition` là `None`, tool không

xuất hiện trong danh sách tool của LLM, nhưng `process_llm_request()` vẫn chạy trước mỗi lần gọi LLM. Cho đơn giản, ở đây ta dùng cách tự động chèn bằng cách đặt `tool_definition=None`.

Listing 6.37. `MemoryTool` với cơ chế tự động chèn

```
class MemoryTool(BaseTool):
    # ... execute, _format_memories same as above ...

    def __init__(self):
        super().__init__(
            name="recall_memory",
            description="...",
            tool_definition=None,  # Automatic injection only
        )

    async def process_llm_request(
        self,
        context: ExecutionContext,
        request: LlmRequest,
    ) -> None:
        """Inject relevant memories before LLM call"""
        user_msgs = [c for c in request.contents if isinstance(c, Message) and
        ↪  c.role == "user"]
        if not user_msgs:
            return

        result = await self.execute(context, user_msgs[-1].content)
        if not result:
            return

        request.append_instructions(
            f"""The following are records from similar problems solved in the
            ↪  past:
<PAST_EXPERIENCES>
{result}
</PAST_EXPERIENCES>
Reference successful approaches and avoid approaches that led to failures."""
        )
```

Thay đổi duy nhất so với phiên bản trước là thêm `tool_definition=None` trong `__init__`. Điều này khiến LLM không nhìn thấy tool, nhưng method `process_llm_request()` của nó vẫn chạy trước mỗi lần gọi LLM và tự động chèn các memory liên quan. Nếu muốn LLM gọi tool này một cách tường minh, chỉ cần bỏ dòng `tool_definition=None`.

Tích hợp vào agent Để `process_llm_request()` hoạt động, ta sửa method `_prepare_llm_request()` của `Agent`. Ở Chương 4, ta dựng danh sách tool bằng một comprehension đơn giản:

```
tools = [t.tool_definition for t in request.tools]
```

Giờ cần hai thay đổi. Thứ nhất, lọc bỏ các tool có `tool_definition` là `None`, vì một số tool có thể chỉ phục vụ tự động chèn chứ không để LLM gọi. Thứ hai, gọi method `process_llm_request()` của từng tool. Lưu ý method trở thành async vì `process_llm_request()` là bất đồng bộ.

Listing 6.38. `_prepare_llm_request()` sửa đổi để tool chèn context

```
class Agent:
    # ... existing code ...
```

```
    async def _prepare_llm_request(
        self, context: ExecutionContext
    ) -> LlmRequest:
        flat_contents = []
        for event in context.events:
            flat_contents.extend(event.content)

        # Filter tools that should be exposed to the LLM
        llm_tools = [t for t in self.tools if t.tool_definition is not None]

        request = LlmRequest(
            instructions=[self.instructions] if self.instructions else [],
            contents=flat_contents,
            tools=llm_tools,
            tool_choice="auto" if llm_tools else None,
        )

        # Let tools modify the request
        for tool in self.tools:
            await tool.process_llm_request(context, request)

        return request
```

Logic lọc này cho phép một tool tham gia vào `process_llm_request()` mà không xuất hiện trong danh sách tool của LLM. `MemoryTool` dùng đúng pattern này: nó đặt `tool_definition=None` nên LLM không gọi được, nhưng `process_llm_request()` của nó vẫn chạy để tự động chèn memory.

6.5 Tóm tắt

- Memory biến agent từ một công cụ không trạng thái (stateless) thành một trợ lý thông minh biết học hỏi và thích nghi qua các lần tương tác.
- Ba pattern sử dụng giải quyết ba thách thức khác nhau: Pattern 1 (One-off Execution) quản lý context trong một lần chạy, Pattern 2 (Continuous Execution) duy trì hội thoại qua nhiều lần chạy, và Pattern 3 (Long-term Memory) lưu giữ tri thức xuyên các session.
- Các chiến lược quản lý context đi theo cách tiếp cận phân cấp: đo số token trước, áp dụng Compaction cho kết quả tool, và chỉ dùng Summarization như phương án cuối cùng.
- Nguyên tắc then chốt là tách lưu trữ khỏi trình bày: `ExecutionContext` giữ bất biến, còn `LlmRequest` được tối ưu cho từng lần gọi.
- Session và SessionManager cho phép hội thoại nhiều lượt bằng cách lưu bền events và state qua các lần gọi `run()`.
- Cùng kiến trúc đó hỗ trợ các workflow Human-in-the-Loop bất đồng bộ, trong đó quá trình thực thi tạm dừng để chờ người dùng phê duyệt và tiếp tục khi có xác nhận.
- Long-term memory kết hợp hai chiến lược Write và Retrieval: thông tin có cấu trúc được trích xuất từ lịch sử thực thi, kiểm tra trùng lặp, lưu vào vector database và được truy xuất khi liên quan đến tác vụ mới.
- Việc truy xuất memory có thể hiện thực thành một tool tường minh mà agent gọi khi cần, hoặc thành cơ chế tự động chèn qua `process_llm_request()`, thêm context liên quan trước mỗi lần gọi LLM.

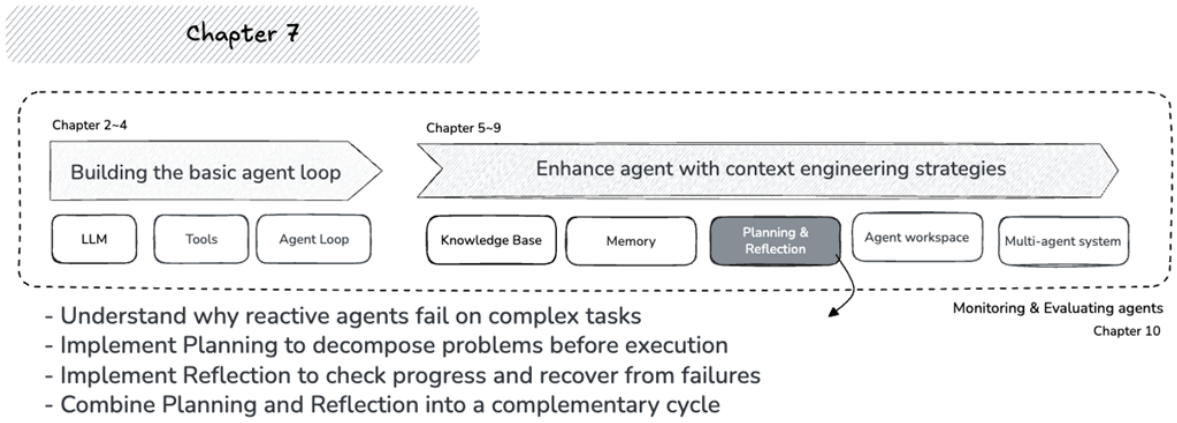
Chương 7

Planning và reflection cho tác vụ phức tạp

Chương này đề cập:

- Giới hạn của agent phản ứng (reactive agent) khi gặp tác vụ phức tạp.
- Tool planning chia nhỏ bài toán thành danh sách tác vụ (task list).
- Tool reflection giúp kiểm tra tiến độ và phục hồi sau thất bại.
- Chu trình bổ trợ lẫn nhau giữa planning và reflection.

Agent ta đã xây dựng quan sát tình hình hiện tại, quyết định gọi tool nào, rồi lặp lại chu trình này. Cách tiếp cận phản ứng (reactive) như vậy chạy tốt với tác vụ đơn giản nhưng gặp khó với bài toán phức tạp cần nhiều bước. Agent mất phương hướng, quên những gì mình đã làm, hoặc không phục hồi được khi gặp lỗi. Planning và reflection khắc phục các giới hạn này bằng cách cho agent "thời gian để suy nghĩ". Planning chia bài toán phức tạp thành các tác vụ dễ quản lý trước khi thực thi. Reflection dừng lại để kiểm tra tiến độ và điều chỉnh hướng đi khi có chuyện không ổn.



Hình 7.1: Vai trò của planning và reflection trong AI agent. *(PDF gốc tr. 302)*

Trước hết ta xem vì sao agent phản ứng thất bại và chuyên gia con người tiếp cận tác vụ phức tạp khác đi thế nào. Sau đó ta cài đặt một tool Planning đơn giản ghi danh sách tác vụ vào context, tiếp theo là một tool reflection cho phép kiểm tra tiến độ và phục hồi sau thất bại. Cuối cùng, ta kết hợp hai chiến lược thành một chu trình bổ trợ và xem chúng giải quyết các kiểu thất bại (failure mode) của agent phản ứng ra sao.

7.1 Cho agent thời gian để suy nghĩ

Planning và reflection là hai chiến lược cốt lõi cho agent thời gian để suy nghĩ. Trước hết ta xem xét các giới hạn của cách tiếp cận ReAct, rồi tìm lời giải từ cách chuyên gia con người làm việc. Ta cũng tìm hiểu nguyên lý "thời gian để suy nghĩ" (đã được kiểm chứng trong nghiên cứu về LLM) áp dụng cho agent như thế nào, và hiểu planning cùng reflection đóng vai trò gì dưới góc nhìn Context Engineering.

7.1.1 Giới hạn của ReAct

Agent ta đã cài đặt hoạt động theo cách tiếp cận ReAct. Nó quan sát tình hình hiện tại, quyết định gọi tool nào, quan sát lại kết quả, rồi quyết định hành động kế tiếp. Điểm mạnh của cách này là khả năng thích ứng. Nếu kết quả tìm kiếm khác kỳ vọng, nó thử từ khoá khác; nếu một tool lỗi, nó tìm phương án thay thế. Lý do là ở mỗi thời điểm nó chọn hành động có vẻ phù hợp nhất.

Tuy nhiên, cách này giống như lái xe không có GPS. Đến mỗi ngã tư, bạn đi theo "hướng nào trông ổn nhất với những gì nhìn thấy lúc này". Cách đó ổn khi đích đến ở gần, nhưng khi cần tìm một lộ trình phức tạp thì rất dễ lạc.

Với bài toán phức tạp, agent ReAct thể hiện một số kiểu thất bại điển hình:

- **Mất phương hướng vì cố làm quá nhiều cùng lúc.** Agent chạy nhiều lượt tìm kiếm web để nhanh chóng lấy thông tin cần cho tác vụ, rồi không tận dụng được kết quả đúng cách.
- **Quên những gì đã làm ở giữa chừng.** Agent không dùng lại thông tin đã tìm thấy trước đó, dẫn tới tìm kiếm lặp lại.
- **Vội vàng tuyên bố "Tôi đã tìm ra đáp án"** nhưng lại cố trả lời chỉ với thông tin dở dang.
- **Không phục hồi được sau thất bại.** Khi lời gọi tool thất bại hoặc đi sai hướng, thay vì phân tích nguyên nhân và tìm phương án khác, agent lặp lại cùng một thao tác cho tới khi chạm `max_steps`.

Các vấn đề này có điểm chung gì? Agent chỉ nghĩ về "việc cần làm ngay bây giờ" mà không cân nhắc "nên tiếp cận tổng thể thế nào" hay "mọi thứ tới giờ có đang tiến triển tốt không".

7.1.2 Cách chuyên gia con người làm việc

Hãy xem một nhà nghiên cứu lành nghề xử lý một câu hỏi phức tạp. Khi được hỏi "Với tốc độ chạy marathon của Kipchoge, phải mất bao nhiêu giờ để tới Mặt Trăng?", họ không bắt tay tìm kiếm ngay.

Trước tiên, họ phân rã câu hỏi: "Kỷ lục marathon của Kipchoge là bao nhiêu? Từ đó mình tính được tốc độ. Khoảng cách tới Mặt Trăng là bao nhiêu? Mình nên dùng khoảng cách lúc gần nhất." Sau đó họ tìm hiểu từng thứ một, vừa làm vừa sắp xếp: "Đã tìm thấy kỷ lục marathon, đổi ra km/h thì được khoảng 21km/h." Khi gặp kết quả ngoài dự kiến, họ điều chỉnh cách tiếp cận: "Khoảng cách tới Mặt Trăng mỗi nguồn ghi một khác. Nên dùng khoảng cách trung bình hay điểm cận địa (perigee)? Câu hỏi nói 'lúc gần nhất' nên mình dùng perigee." Trước khi đưa đáp án cuối cùng, họ kiểm tra xem còn thiếu gì không: "Kỷ lục marathon, tính tốc độ, khoảng cách Mặt Trăng, tính thời gian… xong hết."

Mô thức tương tự cũng xuất hiện trong phát triển phần mềm. Lập trình viên giàu kinh nghiệm không viết code ngay. Họ viết PRD (Product Requirements Document) hoặc tài liệu spec trước, rồi tiến hành cài đặt và kiểm thử trong khi tham chiếu tài liệu đó. Khi có vấn đề phát sinh lúc cài đặt, họ xem lại spec và điều chỉnh cách tiếp cận. Các coding agent như Claude Code và Cursor cũng theo phương pháp này. Khi cài đặt tính năng phức tạp, chúng tạo kế hoạch cài đặt trước, rồi đối chiếu tiến độ với kế hoạch ở từng giai đoạn. Đó chính là planning (viết spec) và reflection (đối chiếu với spec).

Hướng dẫn phát triển agent của Anthropic nhấn mạnh điểm này:

> *"Inspiration for these practices came from knowing what effective software engineers do every day."*
> (Cảm hứng cho các thực hành này đến từ việc hiểu những gì kỹ sư phần mềm giỏi làm hằng

ngày.)
— Anthropic, “Building effective agents.”

Thông điệp ở đây là: hãy thiết kế agent lấy cảm hứng từ cách chuyên gia con người làm việc.

7.1.3 Vì sao thời gian suy nghĩ quan trọng

Một mô thức được nghiên cứu về LLM xác nhận nhiều lần là cho mô hình thời gian suy nghĩ sẽ cải thiện hiệu năng so với đòi trả lời ngay. Chain-of-Thought prompting cải thiện đáng kể độ chính xác ở bài toán toán học chỉ bằng cách thêm câu “Let’s think step by step”. Nó hiệu quả vì buộc mô hình sinh ra các bước suy luận trung gian một cách tường minh. Reasoning Model đưa nguyên lý này đi xa hơn: sinh thinking token trước khi trả lời, nhờ đó nâng cao nhiều khả năng giải bài toán phức tạp.

Planning và reflection áp dụng nguyên lý này cho agent. Planning là thời gian suy nghĩ *trước* khi hành động: “Mình nên làm gì? Theo thứ tự nào?” Reflection là thời gian nhìn lại *sau* khi hành động: “Tới giờ mình đã biết được gì? Mình có đang đi đúng hướng không? Có nên đổi hướng không?” Điều này trao cho agent một dạng siêu nhận thức (metacognition): khả năng tự xem xét quá trình suy nghĩ của chính mình.

Dưới góc nhìn context engineering, planning và reflection có một điểm chung thú vị: cả hai đều không trực tiếp thực hiện tác vụ. Chúng không tìm kiếm web, không tính toán, không đọc file. Thay vào đó, chúng sinh ra các tín hiệu văn bản để điều khiển dòng công việc. Planning thêm “việc cần làm tiếp theo” vào context. Khi đoạn văn bản như “1. Research Kipchoge’s record, 2. Research Moon distance, 3. Calculate time” đi vào context, LLM tham chiếu kế hoạch này ở các bước sau để giữ hướng đi. Reflection thêm “đánh giá và hướng đi tới thời điểm này” vào context. Đoạn văn bản như “Marathon record acquired. Moon distance research: Wikipedia failed, need to switch to web search” sẽ ảnh hưởng đến quyết định kế tiếp của LLM.

Trong năm chiến lược context engineering giới thiệu ở chương 1, Planning và Reflection thuộc chiến lược *generation* (sinh). Khác với retrieval (mang thông tin từ nguồn bên ngoài vào) hay write (lưu thông tin ra bên ngoài), cách này để LLM tự sinh văn bản ngay trong context nhằm điều khiển workflow. Giờ ta xem cách cài đặt hai chiến lược này.

7.2 Planning: Định hướng

Cốt lõi của planning là phân rã bài toán phức tạp thành các đơn vị dễ quản lý. Ngay cả danh sách việc cần làm (to-do list) hằng ngày cũng là một dạng phân rã nhỏ. Chia một việc lớn như “viết báo cáo” thành “thu thập tài liệu → viết bản nháp → rà soát và chỉnh sửa” giúp nó dễ xử lý hơn nhiều.

Lấy bài toán Kipchoge làm ví dụ.

Câu hỏi gốc: “How many thousand hours would it take to reach the Moon at Kipchoge’s marathon pace?” (Với tốc độ marathon của Kipchoge, phải mất bao nhiêu nghìn giờ để tới Mặt Trăng?)

Chuyện gì xảy ra nếu agent nhận câu hỏi này và bắt đầu tìm kiếm ngay? Nó tìm “Kipchoge moon hours”, và khi không thấy kết quả mong muốn, nó thử các từ khoá khác và dễ dàng mất phương hướng trong quá trình đó. Thay vào đó, phân rã câu hỏi sẽ cho ra:

Các tác vụ sau phân rã:

1. Tìm kỷ lục marathon của Kipchoge → tính tốc độ.
2. Tìm khoảng cách tới Mặt Trăng (perigee).
3. Tính thời gian = khoảng cách ÷ tốc độ.
4. Làm tròn tới hàng nghìn giờ.

Với cách phân rã này, mỗi bước trở nên rõ ràng và tiêu chí hoàn thành cũng hiển nhiên. Bạn dễ dàng đánh giá mỗi bước thành công hay chưa bằng các câu hỏi như “Đã tìm thấy kỷ lục marathon chưa?” hay “Đã xác nhận khoảng cách tới Mặt Trăng chưa?”.

7.2.1 Khi nào cần planning?

Không phải bài toán nào cũng cần planning. Vì planning đòi hỏi lời gọi LLM, dùng nó trong tình huống không cần thiết sẽ thành chi phí thừa (overhead).

Khi cần planning:

- Bài toán cần nhiều bước tìm hiểu (ví dụ: bài toán Kipchoge).
- Bài toán cần kết hợp nhiều mẩu thông tin (ví dụ: so sánh giá cổ phiếu của hai công ty).

Khi không cần planning:

- Câu hỏi đơn giản, giải được bằng một lần tìm kiếm (ví dụ: "Thời tiết Seoul hôm nay thế nào?").
- Tác vụ đã có quy trình rõ ràng (ví dụ: "Dịch giúp tôi đoạn văn này").

Lập kế hoạch cho câu "Hôm nay thời tiết thế nào?" là lãng phí. Với bài toán giải được bằng một lần tìm kiếm, tạo kế hoạch kiểu "1. Xác nhận địa điểm, 2. Tìm thời tiết, 3. Sắp xếp kết quả" chỉ tốn token và thời gian vô ích.

Để agent tự phân biệt được điều này, description của tool planning phải nêu rõ "khi nào dùng và khi nào không dùng", như ta sẽ thấy trong phần cài đặt.

7.2.2 Cài đặt tool planning

Ta sẽ tạo tool planning ở dạng đơn giản nhất. Trước khi cài đặt, cần đưa ra vài quyết định thiết kế.

Chỉ quản lý một kế hoạch hay nhiều kế hoạch cùng lúc? Trong hệ thống phức tạp, có thể bạn cần tách kế hoạch chính khỏi kế hoạch con, hoặc theo dõi nhiều mục tiêu cùng lúc. Nhưng trong đa số trường hợp, mỗi câu hỏi một kế hoạch là đủ. Quản lý nhiều kế hoạch đòi hỏi thêm logic để quyết định "tham chiếu kế hoạch nào". Ta bắt đầu đơn giản.

Khi sửa kế hoạch, chỉ cập nhật phần thay đổi hay sinh lại toàn bộ? Hỗ trợ sửa từng phần như "đổi trạng thái tác vụ 3 thành completed" có thể tiết kiệm token. Nhưng cách này cần logic theo dõi chính xác trạng thái hiện tại của kế hoạch, và lỗi do tham chiếu sai chỉ số (index) hoặc mất đồng bộ rất hay gặp. Ngược lại, sinh lại toàn bộ kế hoạch mỗi lần giúp cài đặt đơn giản. LLM chỉ việc viết lại cả kế hoạch, kiểu "trạng thái hiện tại là thế này". Với kế hoạch 5–10 tác vụ, chi phí token phát sinh là rất nhỏ.

Biểu diễn trạng thái tiến độ của từng tác vụ thế nào? Phân biệt tác vụ chưa bắt đầu (`pending`), đang thực hiện (`in_progress`) hay đã hoàn thành (`completed`) giúp LLM dễ xác định "việc gì làm tiếp".

Cấu trúc dữ liệu tác vụ (Task)

Lớp `Task` là một cấu trúc dữ liệu đơn giản lưu nội dung và trạng thái của tác vụ. Method `__str__` áp dụng định dạng checkbox khác nhau cho từng trạng thái, giúp LLM nắm nhanh tiến độ.

Listing 7.1. Định nghĩa cấu trúc dữ liệu Task

```
from typing import Literal, List
from pydantic import BaseModel

class Task(BaseModel):
    content: str
    status: Literal["pending", "in_progress", "completed"]

    def __str__(self):
        if self.status == "pending":
```

```
            return f"[ ] {self.content}"
        elif self.status == "in_progress":
            return f"[>] **{self.content}**"
        elif self.status == "completed":
            return f"[x] ~~{self.content}~~"
        return self.content
```

Tác vụ `pending` hiển thị dưới dạng checkbox trống `[ ]`, tác vụ `in_progress` là dấu tiến độ `[>]` kèm chữ đậm, còn tác vụ `completed` là dấu hoàn thành `[x]` kèm gạch ngang. Định dạng này là mẫu quen thuộc của to-do list nên LLM hiểu dễ dàng.

Tool `create_tasks`

Giờ ta cài đặt tool planning. Có thể tạo nó đơn giản bằng decorator `@tool` đã học ở chương 4. Tool nhận một danh sách tác vụ và trả về dưới dạng chuỗi, kết quả này được ghi vào context dưới dạng `ToolResult`.

Description của tool là thông tin then chốt LLM dùng để phán đoán "khi nào dùng tool này". Description nêu rõ:

- **Khi nào dùng:** câu hỏi phức tạp cần nhiều bước tìm hiểu hoặc cần kết hợp thông tin từ nhiều nguồn khác nhau.
- **Khi nào không dùng:** câu hỏi đơn giản giải được bằng một lần tìm kiếm, tác vụ có quy trình hiển nhiên.
- **Cách dùng:** đánh dấu tác vụ đã xong là `completed`, đặt tác vụ kế tiếp thành `in_progress`.

Viết description chi tiết giúp ngăn LLM gọi tool planning trong những tình huống không cần thiết.

Listing 7.2. Cài đặt tool planning

```
from typing import List
from scratch_agents.tools import tool

@tool
def create_tasks(tasks: List[Task]) -> str:
    """Create or update a task plan.
    WHEN TO USE:
    - Complex queries requiring multiple steps of research
    - Questions that need to combine information from different sources
    WHEN NOT TO USE:
    - Simple questions answerable with a single search
    - Tasks with obvious, straightforward procedures
    HOW TO USE:
    - Regenerate the entire task list with updated statuses
    - Mark completed tasks as 'completed'
    - Mark the next task to work on as 'in_progress'
    - Keep future tasks as 'pending'
    """
    result = []
    for task in tasks:
        result.append(str(task))
    return "\n".join(result)
```

Bản thân phần cài đặt tool rất đơn giản: nhận danh sách tác vụ, chuyển thành chuỗi và trả về. Chuỗi trả về được ghi vào context dưới dạng `ToolResult`, cho phép LLM tham chiếu kế hoạch ở các bước sau.

Vì sao cài đặt đơn giản như vậy? LLM vốn đã có năng lực hiểu và sinh ngôn ngữ mạnh. Thay vì viết logic phức tạp bằng code, tận dụng năng lực của LLM hiệu quả hơn. Tool chỉ ghi kế hoạch vào context, còn việc lập kế hoạch và quản lý trạng thái thực sự do LLM đảm nhận.

7.2.3 Ví dụ sử dụng tool planning

Giờ ta thêm tool planning vào một agent và xem nó hoạt động thực tế ra sao.

Listing 7.3. Chạy agent có tool Planning

```
from scratch_agents import Agent, LlmClient
from scratch_agents.tools import search_web

agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[create_tasks, search_web],
    instructions="You are a helpful assistant that plans before acting on complex
    ↪  queries.",
    max_steps=10,
)

result = await agent.run(
    "If Eliud Kipchoge could maintain his marathon pace indefinitely, "
    "How many thousand hours would it take him to reach the Moon?")
```

Khi nhận câu hỏi này, agent gọi `create_tasks` trước để lập kế hoạch. Giá trị trả về của tool được ghi vào context dưới dạng `ToolResult`.

```
Step 1: create_tasks call
Result:
[ ] Find Kipchoge marathon world record time
[ ] Calculate pace in km/h
[ ] Find Earth-Moon distance at perigee
[ ] Calculate time and convert to thousands of hours
```

Sau đó, nó cập nhật trạng thái khi đi qua từng bước. Khi tác vụ đầu tiên xong, nó đánh dấu `completed` và đặt tác vụ kế tiếp thành `in_progress`.

```
Step 2: search_web("Kipchoge marathon world record") call
Step 3: create_tasks call (state update)
Result:
[x] ~~Find Kipchoge marathon world record time~~
[>] **Calculate pace in km/h**
[ ] Find Earth-Moon distance at perigee
[ ] Calculate time and convert to thousands of hours
```

Khi kế hoạch được ghi vào context, LLM tham chiếu nó ở các bước sau, giống như người ta nhìn vào to-do list để quyết định làm gì tiếp. Nhờ định dạng checkbox, chỉ cần liếc qua là thấy tác vụ nào đã xong, tác vụ nào đang làm và tác vụ nào còn lại.

Đánh đổi về độ chi tiết của kế hoạch

Nếu tác vụ quá vụn, số lần gọi `create_tasks` để cập nhật trạng thái sẽ quá nhiều, làm tăng thời gian thực thi. Thay vì tách riêng từng thao tác đơn giản, nhóm tác vụ thành các đơn vị có ý nghĩa sẽ hiệu quả hơn. Luôn có sự đánh đổi giữa độ chi tiết (granularity) của kế hoạch và độ trễ phản hồi của hệ thống.

7.2.4 Hướng mở rộng

Tool planning ta cài đặt tới giờ được cố ý giữ đơn giản. Nó đủ cho đa số trường hợp, nhưng nếu cần xử lý tác vụ phức tạp hơn, bạn có thể mở rộng theo các hướng sau:

- **Thêm tiêu chí hoàn thành:** định nghĩa tường minh khi nào mỗi tác vụ được coi là xong. Thay vì "Research Kipchoge's record", đưa vào điều kiện hoàn thành cụ thể như "Xác nhận thời gian và cự ly kỷ lục marathon của Kipchoge" giúp LLM đánh giá chính xác hơn tác vụ đã xong hay chưa.
- **Cấu trúc phân cấp (Task/Action):** chia tác vụ lớn (Task) thành các hành động nhỏ hơn (Action). Ví dụ, dưới Task "Nghiên cứu thị trường" có thể có các Action "Tìm đối thủ A", "Tìm đối thủ B" và "So sánh kết quả".
- **Quản lý phụ thuộc (deps):** biểu diễn quan hệ thứ tự giữa các tác vụ. "Tìm kỷ lục Kipchoge" và "Tìm khoảng cách Mặt Trăng" độc lập với nhau và có thể làm đồng thời, nhưng "Tính thời gian" chỉ làm được sau khi cả hai tác vụ kia hoàn thành. Làm rõ các phụ thuộc như vậy cho phép thực thi song song.

Tuy nhiên, các mở rộng này làm tăng độ phức tạp đáng kể. Cần có code để quản lý đồ thị phụ thuộc, theo dõi cấu trúc phân cấp và điều phối thực thi song song. Tốt nhất là chỉ thêm dần chúng khi tool planning đơn giản không giải quyết được vấn đề.

Điều cốt lõi là câu hỏi, không phải cấu trúc

Quan trọng hơn một đồ thị phụ thuộc phức tạp là hướng LLM tự đặt ra đúng câu hỏi. Đưa các câu hỏi như sau vào description của tool sẽ hiệu quả:

- Mình cần biết gì để giải bài toán này?
- Tiến hành theo thứ tự nào thì hiệu quả?
- Làm sao biết mỗi bước đã hoàn thành?

7.3 Reflection: Kiểm tra và điều chỉnh

Nếu planning là "lập kế hoạch trước khi hành động" thì reflection là "dừng lại giữa chừng để kiểm tra". Một nhà nghiên cứu lành nghề tự nhiên sẽ dừng lại khi kết quả tìm kiếm khác kỳ vọng, khi tổng hợp nhiều nguồn, hoặc khi kiểm tra xem còn sót gì trước khi đưa đáp án cuối. Họ không chỉ tìm kiếm, tìm lại ngay rồi vội trả lời. Trên đường đi, họ tự hỏi "Tới giờ mình đã tìm được gì?" và "Mình có đang đi đúng hướng không?".

Agent ReAct thiếu thói quen kiểm tra này. Ở mỗi thời điểm, chúng chỉ nghĩ "làm gì tiếp" mà không hỏi "tới giờ mọi việc có ổn không". Ngay cả khi kết quả tìm kiếm có vẻ lạ, chúng vẫn chuyển sang bước tiếp; ngay cả khi thông tin mâu thuẫn, chúng vẫn cứ tiếp tục.

Reflection khiến agent tự hỏi những câu sau:

- Tới giờ mình đã biết được gì?
- Mình còn cách mục tiêu ban đầu bao xa?
- Cách tiếp cận hiện tại có hiệu quả không?
- Có điều gì cần thay đổi không?

Khi câu trả lời cho các câu hỏi này được ghi vào context, LLM sẽ tham chiếu chúng trong các quyết định sau. Nếu planning thêm "việc cần làm tiếp theo" vào context thì reflection thêm "đánh giá và hướng đi tới thời điểm này" vào context.

7.3.1 Khi nào cần reflection?

Hãy nghĩ về những thời điểm con người tự nhiên dừng lại khi làm việc. Đó chính là những lúc agent cũng cần reflection.

Khi vừa hoàn thành một bước có ý nghĩa. Con người tự nhiên dừng lại khi xong một bước: "Xong bước một. Sắp xếp lại đã rồi mới đi tiếp." Agent cũng có thể kiểm tra tiến độ và chuẩn bị cho bước sau khi hoàn thành một giai đoạn của kế hoạch. Đây là điểm kiểm tra (checkpoint) dễ dự đoán nhất.

Khi tool gặp lỗi. Khi tìm kiếm thất bại hay API không phản hồi, con người nghĩ: "Sao không chạy nhỉ? Phải tìm cách khác thôi." Không có reflection, agent dễ lặp lại cùng một thất bại cho tới khi chạm `max_steps`. Lỗi là tín hiệu rõ ràng, nên đây là thời điểm đặc biệt quan trọng để kiểm tra.

Khi tổng hợp nhiều mẩu thông tin. Khi đã thu thập được nhiều tài liệu, con người dừng lại: "Khoan, sắp xếp lại cái đã." Việc kiểm tra càng quan trọng khi có thông tin mâu thuẫn. Khi tìm khoảng cách tới Mặt Trăng trong bài toán Kipchoge, bạn có thể gặp các con số khác nhau như "khoảng cách trung bình 384,400km" và "perigee 356,500km, apogee 406,700km". Để quyết định dùng giá trị nào, bạn cần dừng lại suy nghĩ: "Đề bài hỏi 'lúc gần nhất' nên mình dùng perigee."

Khi cần rà soát trước đáp án cuối cùng. Ngay trước khi trả lời, con người kiểm tra "Xem lại cho chắc không sót gì." Để ngăn agent vội tuyên bố "Tôi đã tìm ra đáp án", cần có bước tự kiểm chứng (self-verification) trước khi hoàn tất. Đây là checkpoint quan trọng nhất nhưng cũng dễ bị bỏ qua nhất.

Khi không cần reflection. Không phải lúc nào cũng cần kiểm tra. Làm reflection sau mỗi lời gọi tool chỉ làm tăng overhead. Trong các thao tác đơn giản, thẳng thắn, nó trở thành độ trễ không cần thiết. Nếu mọi thứ diễn ra như dự kiến, cứ tiếp tục sẽ hiệu quả hơn.

Khác biệt với summarization

Summarization (đã học ở chương 6) khác với reflection. Summarization là việc nén, tự động kích hoạt khi độ dài context vượt ngưỡng; mục đích của nó là giảm số token. Ngược lại, reflection được LLM chủ động thực hiện một cách có chọn lọc khi phán đoán rằng "cần kiểm tra"; mục đích của nó là kiểm tra hướng đi và điều chỉnh khi cần.

7.3.2 Cài đặt tool reflection

Giống tool planning, ta cài đặt tool reflection thật đơn giản. Thay vì viết logic phức tạp bằng code, ta tận dụng khả năng phán đoán của LLM.

Quyết định thiết kế. Tham số reflection (`analysis`) là văn bản tự do. Vì mỗi tình huống cần nội dung khác nhau (phân tích lỗi, tổng hợp thông tin, kiểm tra tiến độ, …), ép vào một cấu trúc cố định sẽ làm giảm tính linh hoạt.

Cờ `need_replan` là tín hiệu phối hợp với tool planning. Khi phán đoán rằng "kế hoạch hiện tại không còn hợp lệ", đặt cờ này thành `True` sẽ thúc đẩy việc sửa kế hoạch ở bước kế tiếp.

Tool reflection

Description của tool nêu bốn tình huống sử dụng: PROGRESS REVIEW (khi hoàn thành một bước), ERROR ANALYSIS (khi tool thất bại), RESULT SYNTHESIS (khi tổng hợp thông tin) và SELF CHECK (trước đáp án cuối cùng). Đưa ví dụ cụ thể cho từng tình huống sẽ hướng LLM gọi tool vào đúng thời điểm.

Listing 7.4. Cài đặt tool reflection

```
from scratch_agents.tools import tool

@tool
def reflection(analysis: str, need_replan: bool = False) -> str:
    """Pause and analyze progress before continuing.
    WHEN TO USE:
    1. PROGRESS REVIEW - After completing a meaningful step
        "Kipchoge's record found: 2:01:09. Moving to moon distance research."
    2. ERROR ANALYSIS - When a tool fails or returns unexpected results
        "Wikipedia tool failed. Cause: service unavailable. Alternative: use web
        ↪  search."
    3. RESULT SYNTHESIS - When combining information from multiple sources
        "Two different moon distances found. Problem asks for closest approach,
        ↪  so using perigee: 356,500km."
    4. SELF CHECK - Before providing final answer
        "Have all required data: marathon pace 20.81km/h, moon distance
        ↪  356,500km. Ready to calculate."
    WHEN NOT TO USE:
    - After every single tool call (excessive overhead)
    - During simple, straightforward operations
    - When everything is proceeding as expected
    Args:
        analysis: Your assessment of current situation and next direction
        need_replan: Set True if the current plan needs modification
    """
    if need_replan:
        return f"Reflection recorded (REPLAN NEEDED): {analysis}"
    return f"Reflection recorded: {analysis}"
```

Bản thân phần cài đặt tool rất đơn giản: nhận nội dung phân tích và trả về dưới dạng chuỗi. Kết quả này được ghi vào context dưới dạng `ToolResult`, và LLM tham chiếu nó trong các quyết định sau.

Nếu tool planning ghi "việc cần làm tiếp theo" vào context thì tool reflection ghi "đánh giá tới thời điểm này" vào context. Cả hai đều là chiến lược generation: thêm văn bản do LLM sinh ra vào context để tác động lên các quyết định sau.

7.3.3 Giá trị thực sự của reflection: Phục hồi sau thất bại

Giá trị của reflection lộ rõ không phải khi mọi thứ suôn sẻ mà khi có vấn đề phát sinh. Hãy cố ý tạo một tình huống thất bại để xem tác dụng của reflection.

Tạo một tool luôn thất bại. Tạo một tool Wikipedia luôn luôn lỗi. Trong thực tế, API bên ngoài có thể lỗi vì nhiều lý do: server quá tải, sự cố mạng, API thay đổi, v.v.

Listing 7.5. Tool Wikipedia luôn thất bại

```
@tool
def get_wikipedia_page(title: str) -> str:
    """Get a Wikipedia page by title."""
    raise RuntimeError("Wikipedia service is temporarily unavailable")
```

Chạy kèm reflection. Thêm tool reflection và tool Wikipedia vào agent rồi chạy.

Listing 7.6. Chạy agent có reflection để phục hồi lỗi

```
agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[get_wikipedia_page, search_web, reflection],
    instructions="Always try Wikipedia first when researching topics.",
    max_steps=5,
)

result = await agent.run(
    "What is the distance from Earth to Moon at perigee?"
)
```

Có reflection, agent phân tích thất bại và tìm ra phương án thay thế.

7.3.4 Chạy agent dùng reflection để tổng hợp kết quả nghiên cứu

Reflection hữu ích không chỉ cho phục hồi sau thất bại mà cả khi tổng hợp nhiều mẩu thông tin. Dưới đây là ví dụ sắp xếp kết quả tìm kiếm.

Listing 7.7. Chạy agent dùng reflection để tổng hợp nghiên cứu (sách gốc ghi nhầm tiêu đề là "Task data structure definition")

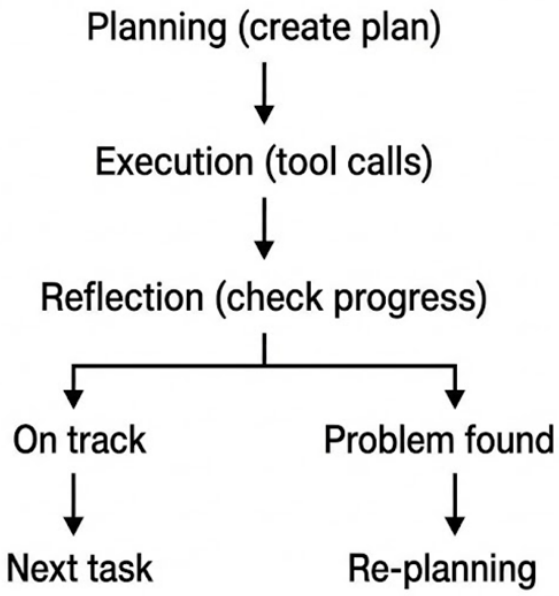
```
agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[search_web, reflection],
    instructions="You are a research assistant.",
)

result = await agent.run(
    "Research recent developments in quantum computing."
)
```

Sau khi agent thực hiện nhiều lượt tìm kiếm, nó tổng hợp kết quả bằng reflection. Nếu không có reflection, agent có thể cứ tiếp tục tìm thêm, hoặc trả lời mà không sắp xếp thông tin đã thu thập cho đàng hoàng. Nhờ reflection, phán đoán "mình đã thu thập đủ thông tin" được ghi tường minh vào context, và điều này dẫn bước tiếp theo hướng tới việc sinh câu trả lời.

7.4 Kết hợp planning và reflection

Planning và reflection không hoạt động tách rời mà tạo thành một chu trình duy nhất, như Hình 7.2. Ta xem hai chiến lược này kết nối với nhau thế nào và tóm lại cách chúng giải quyết các giới hạn của ReAct.



Hình 7.2: Chu trình Planning–Reflection trong quá trình thực thi agent. *(PDF gốc tr. 320)*

Đầu tiên, agent phân rã tác vụ và sắp thứ tự bằng planning. Sau đó nó thực hiện công việc bằng cách gọi tool theo kế hoạch. Sau khi thực thi, nó kiểm tra tiến độ bằng reflection. Nếu kết quả kiểm tra cho thấy mọi việc ổn, nó chuyển sang tác vụ kế tiếp; nếu có vấn đề, nó sửa kế hoạch.

Tham số `need_replan` của tool reflection đã cài đặt chính là mắt xích nối hai bước này. Khi reflection xác định "kế hoạch hiện tại không còn hợp lệ", nó đặt `need_replan=True`. Khi tín hiệu này được ghi vào context, LLM gọi lại `create_tasks` ở bước sau để sửa kế hoạch.

Ví dụ, xét trường hợp tool Wikipedia thất bại trong bài toán Kipchoge. Trong reflection, có thể phân tích "Wikipedia service unavailable, need to switch to web search" nhưng vẫn đặt `need_replan=False`. Lý do là bản thân kế hoạch (tìm khoảng cách tới Mặt Trăng) vẫn còn hợp lệ; chỉ cách làm cần thay đổi. Ngược lại, nếu kết quả tìm kiếm cho thấy "Kipchoge giữ kỷ lục half-marathon chứ không phải marathon", bạn nên đặt `need_replan=True`. Vì tiền đề của chính câu hỏi đã thay đổi, cần xem xét lại toàn bộ kế hoạch.

Như vậy, planning và reflection không phải các tính năng độc lập mà tạo thành một chu trình bổ trợ. Planning cung cấp hướng đi, reflection kiểm tra hướng đi, và khi cần thì quay lại planning.

7.4.1 Các kiểu thất bại và cách giải quyết

Kiểu thất bại thứ nhất là "mất phương hướng vì cố làm quá nhiều cùng lúc". Nó xảy ra khi agent chạy nhiều lượt tìm kiếm đồng thời rồi không tận dụng được kết quả đúng cách. Planning giải quyết vấn đề này: nó phân rã tác vụ thành các đơn vị rõ ràng và cung cấp cấu trúc để làm từng việc một. Khi đã phân rã thành "1. Research Kipchoge's record, 2. Research Moon distance, 3. Calculate", agent chỉ chuyển sang tác vụ thứ hai sau khi xong tác vụ thứ nhất.

Kiểu thất bại thứ hai là "vội vàng tuyên bố đã tìm ra đáp án". Agent cố trả lời chỉ với thông tin dở dang trước khi thu thập đủ các chi tiết cần thiết. Planning giải quyết bằng cách nêu rõ tiêu chí hoàn thành, còn bước SELF CHECK của reflection kiểm chứng "Mình đã có đủ thông tin chưa?". Thời điểm sử dụng "Before providing final answer" ghi trong description của tool reflection đảm nhận đúng vai trò này.

Kiểu thất bại thứ ba là "quên những gì đã làm ở giữa chừng": agent không tận dụng được thông tin đã tìm thấy và lặp lại cùng các lượt tìm kiếm khi đi qua nhiều bước. Planning giảm thiểu vấn đề này bằng cách ghi trạng thái từng tác vụ trong kế hoạch để theo dõi "đã đi được tới đâu". Reflection ghi tường minh "tới giờ đã biết được gì" vào context qua PROGRESS REVIEW. Khi các đoạn văn bản này còn trong context, LLM tham chiếu chúng trong các quyết định sau.

Kiểu thất bại thứ tư là "không phục hồi được sau thất bại". Khi lời gọi tool thất bại hoặc đi sai hướng, agent lặp lại cùng một thao tác cho tới khi chạm `max_steps` thay vì phân tích nguyên nhân và tìm phương án khác. Đây là giá trị cốt lõi của Reflection. ERROR ANALYSIS phân tích nguyên nhân thất bại và đề xuất phương án thay thế. Khi một phân tích như "Wikipedia service unavailable. Cause: server overload. Alternative: use web search" được ghi vào context, LLM sẽ thử phương án khác thay vì lặp lại cùng một thất bại.

Bảng 7.1 tóm tắt nội dung trên.

Bảng 7.1: Các kiểu thất bại của ReAct và cách giải quyết

Kiểu thất bại	**Giải pháp của Planning**	**Giải pháp của Reflection**
Cố làm quá nhiều cùng lúc	Phân rã tác vụ, làm từng việc một	–
Tuyên bố hoàn thành quá sớm	Nêu rõ tiêu chí hoàn thành	Tự kiểm chứng (SELF CHECK)
Mất phương hướng / quên việc đã làm	Ghi trạng thái trong kế hoạch	Kiểm tra tiến độ (PROGRESS REVIEW)
Không phục hồi được sau thất bại	–	Phân tích nguyên nhân, đề xuất phương án thay thế (ERROR ANALYSIS)

Planning chủ yếu giải quyết các vấn đề "nhìn về phía trước", còn Reflection giải quyết các vấn đề "nhìn lại phía sau". Một số vấn đề được giải quyết nhờ cả hai phối hợp. Một nhà nghiên cứu lành nghề không tìm kiếm ngay mà trước hết phân rã câu hỏi (planning), tìm hiểu từng thứ một và sắp xếp dọc đường (reflection), rồi điều chỉnh cách tiếp cận khi gặp kết quả ngoài dự kiến (re-planning). Kết hợp planning và reflection chính là đưa phong cách làm việc tự nhiên này vào agent.

Ở chương tiếp theo, ta sẽ tìm hiểu cách trao cho agent khả năng thực thi code thông qua môi trường sandbox. Code là cách diễn đạt quy trình thực hiện tác vụ bằng ngôn ngữ lập trình, nên về bản chất nó có những đặc điểm tương tự planning mà ta đã học trong chương này. Sandbox trở thành không gian làm việc mới của agent, giống như cách con người làm việc qua máy tính.

7.5 Tóm tắt

- Planning và reflection cho agent "thời gian để suy nghĩ". Thay vì phản ứng theo từng khoảnh khắc, agent lập kế hoạch trước khi hành động và kiểm tra sau khi hành động. Điều này trao cho agent siêu nhận thức (metacognition): khả năng tự xem xét quá trình của chính mình.
- Planning phân rã bài toán phức tạp thành các đơn vị rõ ràng, dễ quản lý. Khi "1. Research Kipchoge's record, 2. Research Moon distance, 3. Calculate time" được ghi vào context, LLM tham chiếu kế hoạch này để giữ hướng đi qua nhiều bước.
- Reflection có giá trị khi có vấn đề phát sinh, không phải khi mọi thứ suôn sẻ. Khi tool thất bại hoặc kết quả ngoài dự kiến, Reflection cho phép phân tích nguyên nhân và đưa ra chiến lược thay thế thay vì lặp lại cùng một thất bại.
- Planning và reflection tạo thành một chu trình bổ trợ. Planning cung cấp hướng đi, reflection kiểm tra hướng đi, và khi cần thì kích hoạt re-planning. Không cái nào hoạt động tách rời.
- Dưới góc nhìn context engineering, cả hai đều là chiến lược generation. Planning thêm "việc cần làm tiếp theo" vào context, còn reflection thêm "đánh giá và hướng đi tới thời điểm này". Các đoạn văn bản này tác động lên các quyết định sau của LLM.